Julia As Your First Programming Language: A Book for Scientists

(Version October 19, 2026)

Martin Alfaro
PhD in Economics

REMARK!

The book is still work in progress in terms of writing, content, and subjects covered. I'll continue adding new content as I go. If you find mistakes or have any suggestions, open an issue on the **book's GitHub page**. Your feedback is greatly appreciated!

This PDF is an adapted version of the **the online content**. As such, many of its features (e.g., gifs) are only accessible there.

WHY JULIA?

Scientific computing demands both an intuitive design and execution speed. Julia is built to deliver both.

SIMPLE AND EXPRESSIVE: Julia's syntax is concise and close to mathematical notation, so you can translate models into readable code that's easy to maintain.

INTERACTIVE PROTOTYPING: rapidly build and explore models in real-time, enabling quick iteration and immediate result inspection.

HIGH PERFORMANCE: Julia is engineered for speed. It's capable of achieving speeds comparable to C or Fortran, without forcing you to rewrite performance-critical parts in another language. Moreover, it offers seamless support for parallel computing, including native multithreading and GPU acceleration.

A UNIFIED COMPUTATIONAL PLATFORM: No need to switch between multiple languages. In Julia, you can preprocess data, perform statistical analysis, generate graphical representations, and implement numerical models. Furthermore, Julia interoperates with other languages and tools, allowing you to leverage the strengths of Python, R, or C++. Its ecosystem includes mature packages for data analysis, plotting, and computational modeling.

A QUICK OVERVIEW OF THE BOOK

AUDIENCE: The book is intended **for an audience with little or no background in programming**. This doesn't mean that we solely cover basic topics. Rather, it defines the book's approach of starting from elementary concepts, **gradually introducing more advanced concepts as we progress**.

APPROACH: Throughout the book, **I've made a conscious effort to distinguish between what's essential and what's ancillary**, with the latter clearly labelled as optional. My goal is that you don't become bogged down in particular details, while still having the possibility of exploring topics further if you wish.

TOPICS: The book focuses on the foundational concepts of the language, without pursuing an exhaustive examination of all its features. My philosophy is that **you can easily incorporate additional features if you grasp the logic of the language**.

TABLE OF CONTENTS

PART I: BASICS OF JULIA

INTRODUCTION

1. Installation and Preliminaries

1a. Installation and Resources 1
1b. Running Julia 3
1c. VS Code (OPTIONAL) 6
1d. A Minimal Set of Good Practices 8

2. Variables

2a. Overview and Goals 12
2b. Variables, Types, and Operators 14
2c. Numbers 20
2d. Strings 23
2e. Arrays (Vectors and Matrices) 25
2f. Tuples 31

CORE CONCEPTS

3. Functions

3a. Overview and Goals 34
3b. Function Calls and Packages 35
3c. Defining Your Own Functions 41
3d. Variable Scope & Relevance of Functions 52
3e. Map and Broadcasting 57

4. Control Flow

4a. Overview and Goals 75
4b. Conditions 77
4c. Conditional Statements 86
4d. For-Loops 91

USING JULIA

5. Vectors: Indexing and Mutations

5a. Overview and Goals 105

5b. Mutable and Immutable Objects 106
5c. Assignments vs Mutations 109
5d. Vector Creation and Initialization 115
5e. Slices: Copies vs Views 129
5f. Array Indexing 134
5g. In-Place Operations 141
5h. In-Place Functions 152

6. Working with Collections

6a. Overview and Goals 159
6b. Named Tuples and Dictionaries 160
6c. Chaining Operations 178
6d. Useful Functions for Vectors 188
6e. Illustration - Johnny, the YouTuber 201

PART II: HIGH PERFORMANCE

TYPE STABILITY AND MEMORY ALLOCATIONS

7. Introduction to Performance

7a. Overview and Goals 213
7b. When To Optimize Code? 215
7c. Benchmarking Execution Time 217
7d. Preliminaries on Types 225
7e. Functions: Type Inference and Multiple Dispatch 239

8. Type Stability

8a. Overview and Goals 248
8b. Defining Type Stability 249
8c. Type Stability with Scalars and Vectors 255
8d. Type Stability with Global Variables 262
8e. Barrier Functions 269
8f. Type Stability with Tuples 274
8g. Type Stability with Higher-Order Functions 283
8h. Type-Stability Gotchas 288

9. Reducing Memory Allocations

9a. Overview and Goals 306
9b. Stack vs Heap 308
9c. Objects Allocating Memory 310
9d. Slice Views 314

9e. Reductions 317
9f. Lazy Operations 331
9g. Lazy Broadcasting and Loop Fusion 336
9h. Mutations and Pre-Allocations 346
9i. Static Vectors for Small Collections 359

PARALLELIZATION

10. Vectorization (SIMD)

10a. Overview and Goals 373
10b. Macros as a Means for Optimizations 375
10c. Introduction to SIMD 382
10d. SIMD: Independence of Iterations 386
10e. SIMD: Contiguous Access and Unit Strides 392
10f. SIMD: Branchless Code 406
10g. SIMD Packages 427

11. Multithreading

11a. Overview and Goals 439
11b. Introduction to Multithreading 441
11c. Task-Based Parallelism: @spawn 454
11d. Thread-Safe Operations 460
11e. Parallel For-Loops 469
11f. Parallelization in Practice 476
11g. Multithreading Packages 494

1a. Installation and Resources

Martin Alfaro
PhD in Economics

INTRODUCTION

We start by covering the essential steps to install Julia and VS Code. The latter is a code editor to write and execute code in multiple languages. We also provide some valuable online resources for Julia users.

INSTALLING JULIA

> **Remark**
> All the links mentioned in the book are included in **Links**, located in the left navigation bar.

To download Julia, visit **Julia's official website**. Note that the installation process depends on your computer's operating system.

There are also two official resources for learning Julia:

1. **Julia's official documentation**. Written by Julia's developers.
2. **Julia Discourse**. Official forum for asking questions.

INSTALLING VS CODE

Once Julia is installed, you'll need an editor to write scripts and visualize outputs. There are numerous alternatives in this respect. **Our book supposes you use Visual Studio Code (aka VS Code)**, which is free, officially supported by Julia, and runs on any operating system (i.e., Windows, macOS, and Linux). One of the key benefits of VS Code is the possibility of installing plugins to extend the editor's capabilities. In fact, you'll need to add the *Julia Language Support* plugin for running Julia.

> **Privacy-Oriented Version of VS Code**
> VS Code is open-source software created and maintained by Microsoft. If you want a more private alternative that disables telemetry and tracking, **VSCodium** is a rebuild of VS Code.

Links to other popular editors can be found on **Useful Links**, including Vim, Emacs, NotePad, and Sublime. These editors are officially supported by Julia (except Sublime). I strongly recommend getting proficient in either VS Code or one of these alternatives. This will allow you **to master a single tool for coding in multiple programming languages**.

> ⚠️ **Warning!**
>
> Avoid getting used to specialized editors built for one particular language, such as RStudio for R (or its newer version **Posit**). The editors I mentioned were designed for coding, regardless of the programming language you choose. Mastering a general code editor will enhance your coding skills—you'll be able to apply the same tools and keyboard shortcuts to every language you work with.

INSTALLING R AND PYTHON (*OPTIONAL*)

Julia offers a seamless integration with other programming languages like R and Python. This allows you to export data from Julia, perform specific operations, and then import the results back into Julia. Such an interoperability is particularly useful when a desired function is only available in one of these languages.

For those familiar with R and Python, **this note** outlines some noteworthy differences with respect to Julia. Additionally, this **cheat sheet** provides a quick reference on syntax differences among Matlab, Python, and Julia.

1b. Running Julia

Martin Alfaro
PhD in Economics

INTRODUCTION

In the following, we cover the basic steps for getting started with Julia. As we haven't introduced any programming tools (e.g., functions), we'll keep the discussion to a bare minimum. Specifically, we'll limit ourselves to setting up Julia in VS Code and presenting methods to add comments and file paths.

USING JULIA IN VS CODE

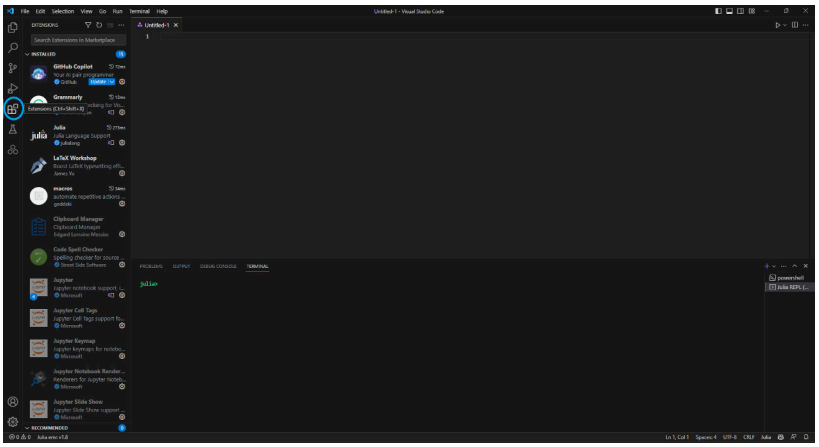
The REPL (Read-Eval-Print Loop) is an interactive programming environment. It lets users input commands and immediately obtain outputs through a command-line interface. When you run `julia`, the REPL is automatically activated and displays the `julia>` prompt, where you can enter commands.

▼ Screenshot



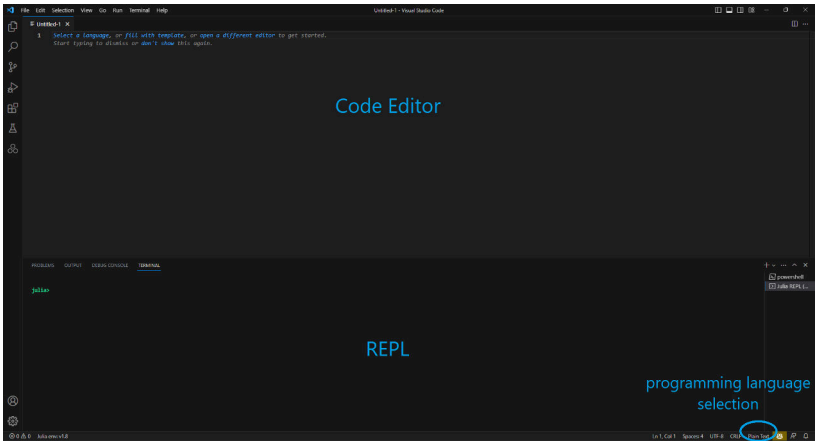
Throughout this book, we'll assume that you're working with a code editor, rather than interacting directly with the REPL. In particular, **VS Code** will be our code editor of choice, including its privacy-focused alternative **VS Codium**. To get started, you'll need to install the Julia extension of VS Code. This can be found by navigating to the Extensions tab, as indicated below by the blue circle.

▼ Screenshot



The layout of VS Code displays the REPL at the bottom of the screen, with code written in the area above it. To execute code, you'll also need to specify the programming language you're using. This can be achieved by clicking on the language option located at the bottom corner of the screen, or by using the keyboard shortcut Ctrl+k + M and typing "julia". All this is demonstrated in the screenshot below.

▼ Screenshot



ADDING COMMENTS IN A SCRIPT

Comments are text annotations ignored during execution, serving as a means to document your code. To add a *single-line comment*, simply precede the text with the # symbol. This symbol can be placed anywhere on a line, with any text that follows disregarded by Julia. Alternatively, you can add *multi-line comments* by delimiting the text with #= at the beginning and =# at the end.

ADDING COMMENTS

```
# This is an example of a comment

x = 2   # `x=2` is run, but anything after `#` won't

#=  This is an example of a longer comment.
    It can be split into several lines, and
    can have any length. =#
```

PATHS OF FILES AND FOLDERS

File management systems vary across operating systems, determining that the syntax for file paths also differs. To accommodate this, Julia provides two approaches. The first one provides an operating system-specific syntax. Below, we illustrate its application for a file `C:\user\file.jl` on Windows and `/user/file.jl` on Linux/macOS. There's also a platform-agnostic alternative to make your code more portable, provided by the `joinpath` function. This is the preferred option, as it can be used with any operating system.

SPECIFYING PATHS

```
# On Windows (note the double \\)
"C:\\user\\file.jl"

# On Unix-based systems (e.g., macOS or Linux)
"/user/file.jl"

# on any operating system
joinpath("/", "user", "file.jl")
```

Two special paths have convenient shortcuts worth mentioning:

- `@__DIR__` identifies the directory where your script is saved.
  For instance, if your script is in `C:\user\julia`, then `joinpath(@__DIR__, "graphs")` refers to `C:/user/julia/graphs`.

- `homedir()` indicates the user's home directory.
  This refers to `C:\Users\username` on Windows (where "username" is your actual user), and is the equivalent of `~` on Linux. For instance, you could access your Google Drive's folder located on either `C:\Users\username\GoogleDrive` or `\home\username\GoogleDrive` by the command `joinpath(homedir(), "GoogleDrive")`.

EXECUTING CODE FROM A FILE

Julia also allows you to work **non-interactively**, by executing code from a script stored in a file. The following example illustrates its implementation, running a file located at `C:\user\julia\graphs.jl` on Windows and at `/users/julia/graphs.jl` on macOS/Linux systems.

EXECUTING CODE FROM A FILE

```
include(joinpath("/", "user", "julia", "graphs.jl"))
```

1c. VS Code (OPTIONAL)

Martin Alfaro
PhD in Economics

FEATURES AND KEYBOARD SHORTCUTS

We present a few keyboard shortcuts and some handy features for **VS Code**. They also apply to its privacy-focused alternative **VS Codium**. Remarkably, these features are largely language-agnostic, holding regardless of the programming language you're working with.

For visual illustration, each feature discussed is accompanied by a GIF. To view these GIFs, simply click "Example", or alternatively press Alt+↑ or Alt+↓ to open and close all of them simultaneously.

For more information about how to use Julia with VS Code, see the **official guide**.

TO RUN A SCRIPT

Select the script to be executed and press Ctrl+Enter

▶ Example

TO FORMAT EXPRESSIONS AND MAKE THEM MORE LEGIBLE

Select the script to be formatted and press Ctrl+k + Ctrl+f. Sometimes, you must run the command twice to activate the tool.

▶ Example

TO ALIGN EQUAL SIGNS

This feature requires a VS Code Extension called "Better Align". It aligns consecutive lines by using the equals sign and other symbols as a reference. It's implemented by pressing Alt + a.

▶ Example

See also the extension "Cursor Align", which aligns code according to the position indicated on each line.

TO EXTEND THE CURSOR VERTICALLY

Hold down Alt+Ctrl and press ↑ or ↓

▶ Example

TO SEE THE DOCUMENTATION OF A FUNCTION

It requires hovering over the function.

▶ Example

Alternatively, go to the REPL, press ?, and then type the function's name you want to search for.

▶ Example

TO AUTOCOMPLETE A WORD

Start typing a word and press Tab when you see the option list.

▶ Example

TO INTRODUCE UNICODE CHARACTERS (TAB COMPLETION)

Type a unicode character, press Ctrl + Space to open an option list, choose an option, and press Tab .

▶ Example

Unicode characters include Greek letters and math symbols. They can be introduced by using the same syntax as in Latex. To add them, you need to start the word with \ (e.g., \eq for ≠) and use Tab completion.

TO SELECT THE SAME WORD MULTIPLE TIMES

Select the word and then press Ctrl+d for selecting the next occurrence of the word.

▶ Example

TO HIDE PART OF THE SCRIPT

Enclose a code block by adding `#region` at the beginning and `#endregion` at the end.

▶ Example

When you have several lines indented, VS Code allows you to hide the block automatically. The following example shows this possibility with a function.

▶ Example

TO TURN MULTIPLE LINES INTO A COMMENT

Select all the lines you want to define as a comment rather than code. Then, press Ctrl + /.

▶ Example

1d. A Minimal Set of Good Practices

Martin Alfaro
PhD in Economics

REMARKS

We conclude this chapter by reviewing various principles to write code. They represent a minimum set of good practices that apply regardless of the programming language used. By adhering to these guidelines, you'll be able to write clear and maintainable code. I suggest incorporating these suggestions into your workflow from the very beginning, as it'll render the learning process smoother.

Several of the suggestions we present might seem inconsequential to you at this point, or give the impression that their importance is exaggerated. For small projects, there's some truth to this—they won't have a substantial impact. However, as projects grow in size and complexity, following these principles becomes crucial. [1] It's not uncommon to revisit your own code after a few months (or even days!) and struggle to understand it. When this occurs, extending the code becomes a daunting task, often resulting in non-reusable code.

As usual, the devil is in the details: the challenge here lies in interpreting and implementing these suggestions effectively. Many of them rely on the reader's judgment, as they require a subjective assessment of when and how to apply them. For example, one suggestion we'll present is to use clear and descriptive names. However, determining what constitutes "clear" or "unclear" is ultimately a matter of personal interpretation. Hopefully, the implementation of the suggestions will become apparent as we move forward and apply these concepts.

WRITE EASY-TO-READ CODE

Code is read more often than it's written. I can't stress enough the importance of this statement. It has a stark implication: write code that is easy to read, even if this requires additional effort or some extra verbosity.

If you end up coding extensively in your future career, you'll likely learn this lesson the hard way. I certainly did. One of the first times I had to reuse an old script, I was completely clueless about my own code. As a consequence, I had to rewrite the entire script from scratch, as making sense of the old code would've taken longer.

> **Remark**
>
> If you're concerned that more readable code requires excessive typing, remember that you can use Tab Completion to autocomplete names. Additionally, AI tools like GitHub Copilot will suggest code while you type, thereby also mitigating the inconvenience.

To illustrate this point, suppose you're reading a script that cleans some data. Imagine in particular that you come across a line that has two possible expressions: `na.rm=TRUE` and `dropmissing=true`. Even if you're unfamiliar with the language's syntax or the concept of missing data, you could likely infer the meaning of `dropmissing=true`: discard entries with no values provided. On the contrary, `na.rm=TRUE` offers no clue. Although this example may appear somewhat abstract, it actually highlights how to discard missing observations in R and Julia: `na.rm=TRUE` corresponds to R and `dropmissing=true` to Julia.[2]

The example also reveals why typing `na.rm=TRUE` might be tempting: it's short and requires less typing. However, it's essential to weigh the long-term benefits of readable code. Although typing more might seem inconvenient in the short term, it represents a minimal effort compared to the future costs of ambiguous code. Moreover, you may feel confident that you'll remember what you intended to write, but it's common to be puzzled by code you wrote just days before.

The benefits of clear code become apparent when you read a script written in an unfamiliar programming language: if the code is well-written and clearly structured, you might grasp the logic and tasks being performed.[3]

Several tips arise as a consequence of this. We list them below.

USE NAMES WITH A CLEAR MEANING

Clear names don't only refer to variables and functions, but files as well. In particular, you should avoid abbreviating. Code editors can be very helpful in this regard, by offering word auto-completion. This feature requires typing the first letters of each word and then pressing Tab.[4]

Avoiding abbreviations has the additional benefit of making it easier to substitute expressions. For instance, suppose you name a variable `re`, and later decide to replace it with a different name. Then, the substitution process becomes more challenging, as the search will also capture functions like `replace` and `repeat`.

Finally, using descriptive names reduces the need for comments. If the code is self-explanatory, comments become only necessary for exceptionally complex code or clarifications that go beyond what's written.

INDENT AND ALIGN YOUR CODE

The implementation details of this suggestion have already been covered in the **previous section**. For further details, please refer to that section.

When writing code sequentially, VS Code automatically provides indentation. You can also format a selected portion of code by pressing Ctrl + k, followed by f. Alternatively, to format the entire script, use the shortcut Alt + Shift + L.[5]

To illustrate how this feature improves readability, consider the following (somewhat exaggerated) example.

NO INDENTATION

```
if x>0 display("x is a positive number") else display("x is a non-positive number") end

function example(a,b)
x=a/10#rescaling x
output=2*b+x
return output
end
```

INDENTED CODE

```
if x > 0
    display("x is a positive number")
else
    display("x is a non-positive number")
end

function example(a, b)
    x      = a / 10               # rescaling x
    output = 2 * b +  x

    return output
end
```

To further improve readability, I suggest also aligning code blocks. Several plugins in VS Code can assist with this task, such as "Better Align" and "Cursor Align". Their use is demonstrated below.

NOT ALIGNED

```
this_is_a_variable = 1
x = 3
another_var = 2

computations_here = x + another_var
more_calcs = this_is_a_variable * another_var
```

ALIGNED

```
this_is_a_variable = 1
x                  = 3
another_var        = 2

computations_here  = x + another_var
more_calcs         = this_is_a_variable * another_var
```

FOOTNOTES

[1.] For real-world examples, read "Brief Story" on this link or the perspective of a former worker from Oracle.

[2.] Python also tends to employ abbreviations that can hinder readability. For instance, to count the number of characters of a variable `x`, Python calls `len(str(x))` while Julia calls `length(string(x))`.

[3.] One way to learn how to write clear code is through AI chatbots, which are pretty good at providing highly readable examples.

[4.] You could eventually use the option "find and replace", whereby you substitute abbreviations for their full name. However, this is error-prone, and you may end up replacing unrelated expressions by substituting all words at once.

[5.] Unlike Python, Julia only uses indentation for readability purposes. It doesn't affect how code is executed.

2a. Overview and Goals

Martin Alfaro
PhD in Economics

> **Remark**
>
> Throughout the book, I made some deliberate choices regarding whether and when to introduce certain subjects. Considering this, I'll include a section called "Overview and Goals" prior to each chapter, which elucidates my rationale for these choices. The goal is to contextualize the book's approach, offering readers some guidance on the best way to engage with the material.

The current chapter deals with **variables and types**. Variables in Julia act as labels of objects, which in turn hold values associated with a specific type. The type determines the information the computer needs to manipulate those values. Their definitions and general properties are presented in Section 2b.

We then explore several types of objects, distinguishing between single-element collections (**scalars**) and multiple-element objects (**collections**). Numbers are a primary example of the former, including integers and floating-point values. They're introduced in Section 2c. In this group, we also discuss characters, which are scalars representing individual symbols. They lead to the coverage of **strings** in Section 2d, which are a sequence of characters that together represent text.

For collections, we focus on two of the most widely used constructs in Julia: **vectors** (presented in Section 2e) and **tuples** (discussed in Section 2f). These structures allow us to group multiple values into a single object, enabling more complex data organization and manipulation.

At this early stage, **we only scratch the surface of the topics**. In particular, the chapter doesn't cover any object in depth, and even excludes important ones. The reason is pedagogical: I didn't want to overwhelm readers with details about objects or types, considering that core programmatic concepts like functions and for-loops haven't yet been introduced.

In light of this, Chapter 2 should be understood as an introduction to objects, providing enough background to progress into the basics of working programmatically. The main skills you should gain from this chapter are:

- Familiarizing yourself with Julia's syntax
- Distinguishing between scalars (single-element objects) and collections.

2b. Variables, Types, and Operators

Martin Alfaro
PhD in Economics

INTRODUCTION

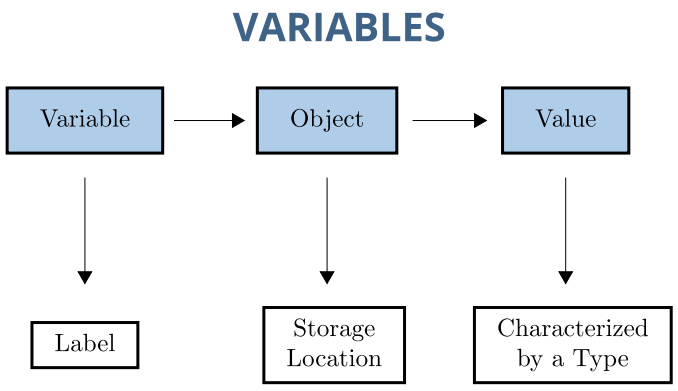
This section introduces the concepts of **variables** and **types**. We'll also present the notion of **operators**, focusing on their syntax. To ensure a smooth learning experience, I've minimized the reliance on objects that we haven't covered yet. The only one introduced is vectors, whose elements are enclosed in brackets (e.g., `[1,2,3]`).

VARIABLES

When a program is executed, the computer stores data in RAM (Random Access Memory). Each piece of data in RAM is referred to as an **object** and assigned a unique memory address. These addresses are typically represented in hexadecimal format (e.g., `0x00007e0966dc0dd0`).

Additionally, every object is associated with a value and a type. **Values** represent the actual data contained within the object. In turn, **types** define the nature of the data stored, providing the computer with critical information for handling the object internally.

Since directly referencing memory addresses would be impractical, we instead define **variables**. They act as human-readable **labels** for objects, simplifying our interaction with the data. Linking objects with a variable relies on the so-called **assignment operator** `=`, which creates a binding between the variable name and the object's memory location. [1] This allows developers to interact with data through symbolic identifiers, rather than raw memory locations.



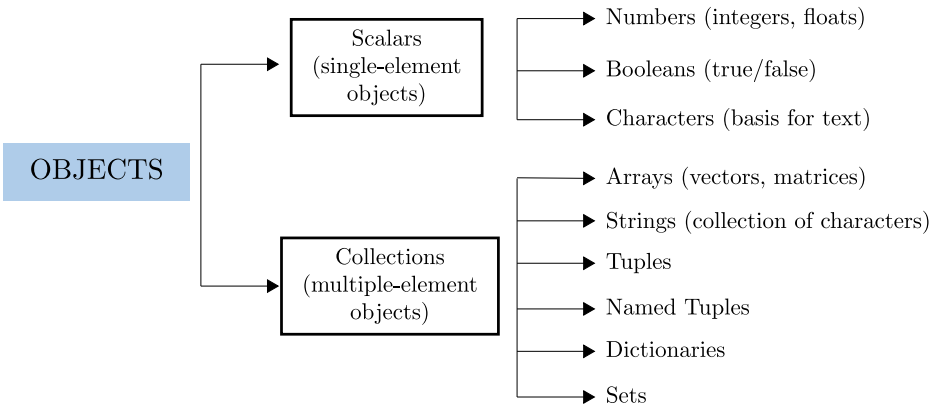
To illustrate this process, let's consider the command `x = "Hello"`. When this is run, several actions take place. First, the computer reserves a memory location to store the object (e.g., at address `0x1234`). This object is then assigned the specific value `"Hello"`, which in Julia is an instance of a type

known as `String`.

At the same time, we're assigning the label `x` to this object, so that `x` points to the memory address `0x1234`. This means that, every time we use `x` in our code, we're actually accessing the object stored at memory address `0x1234`. It's important to note that `x` isn't the object or the value itself, but rather a reference to the memory allocation `0x1234`. This explains why we can define multiple labels to reference exactly the same object (i.e., the same memory address).

CLASSIFICATION OF OBJECTS

Objects are typically characterized according to the number of elements they contain. Specifically, **scalars** refer to single-element objects, while **collections** refer to objects containing multiple elements. Below, we outline some objects encompassed in each category.



NAMES FOR VARIABLES

Variable names in Julia can be defined using Unicode characters, thus offering a wide range of possibilities. This feature enables you to use Greek letters, Chinese characters, symbols, and even emoticons. [2] Underscores `_` are also permitted, which can be helpful for separating words within variable names (e.g., `intermediate_result`). [3] Importantly, names are case-sensitive, so that `bar` and `Bar` are treated as two distinct variables.

VARIABLE NAMES

```
a           = 2
A           = 2       # variable `A` is different from `a`

new_value = 2         # underscores allowed

β           = 2       # Greek letters allowed

中國         = 2       # Chinese characters allowed

x̄           = 2       # decorations allowed
x₁          = 2
ẋ           = 2

🐒          = 2       # emoticons allowed
```

> ⚠️ **Warning!**
>
> **Julia doesn't let you delete variables**. Once a variable is created, it remains in memory until the program terminates. If a variable is taking up too much memory, you can free up space by reassigning it to a smaller object.

> ❗ **Notation for Variable Names**
>
> Julia's developers adopt the convention of using **snake-case notation for variable names**. This format consists of lowercase letters and numbers, with words separated by underscores. (e.g., `snake_case_var1`). Note that this is only a convention, not a language's requirement.

UPDATING VARIABLES

It's possible to assign a new value to a variable using the variable itself. This approach is referred to as **updating a variable**.

UPDATING A VARIABLE

```
x  = 2

x  = x + 3    # 'x' now equals 5
```

Julia offers a concise syntax for updating values, based on the so-called **update operators**. They're implemented by prefixing the assignment operator `=` with the operator to be applied, as demonstrated below.

UPDATE OPERATORS

```
x  = 2

x  = x + 3
x += 3        # equivalent

x  = x * 3
x *= 3        # equivalent

x  = x - 3
x -= 3        # equivalent
```

TYPES

Before diving into the intricacies of Julia, it's essential to familiarize yourself with the basics of its type system. The initial overview we'll present will focus on core definitions and notation, thus providing the minimum necessary for the upcoming chapters. A comprehensive treatment of types, including their role in performance optimization, will be deferred to Part II of this book.

> **Notation for Types**
>
> Julia's developers adopt the convention of **CamelCase** notation for types, where every first letter is capitalized (e.g., `MyType`). Note that this is only a convention, not a language's requirement.

As previously mentioned, types define the nature of values, specifying all the information the computer needs for their storage and manipulation. To better illustrate types, let's split the discussion in terms of scalars and collections.

Common numeric types for scalars include `Int64` for integers, `Float64` for decimal numbers, and `Bool` for binary values (`true` and `false` values). [4] Likewise, the type `Char` represents individual characters, serving as the building block for the `String` type. `String` is the standard type in Julia for representing text, and its values consist of sequences of characters.

Collections, on the other hand, often require **type parameters** for a full characterization of their types. These parameters can be incorporated into any type, and have the goal of providing additional information about its contents.

Type parameters are denoted using `{}` after the type's name. For instance, the type `Vector{Int64}` indicates that the collection represents a vector exclusively containing elements of type `Int64` (e.g., `[2, 4, 6]`). Here, `Int64` serves as a type parameter. Note that type parameters are optional and therefore can be omitted when not needed. Indeed, this is the case with the types for scalars mentioned above.

> **Type Annotations**
>
> You can explicitly declare the type of a variable by using **type annotations**, via the `::` operator. For example, `x::String` ensures that `x` can only store string values throughout the program, resulting in an error if you attempt to reassign `x` with a value of a different type.

CONCRETE TYPES AND ABSTRACT TYPES

In Julia, **types are organized hierarchically**, creating relations of supertypes and subtypes. In particular, it gives rise to the notions of abstract and concrete types.

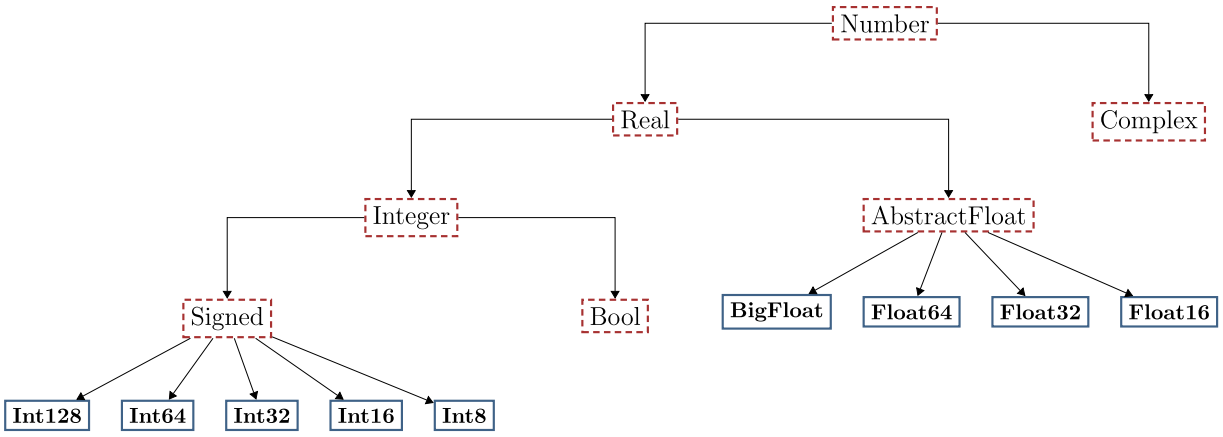
An **abstract type** is a set of types that serve as a parent to other types. The `Any` type in Julia is a prime example of abstract type. It acts as the root of the hierarchy, thus comprising all possible subtypes. In other words, every type in Julia is a subtype of `Any`.

In contrast, a **concrete type** is an irreducible unit, representing a terminal node in the hierarchy and therefore lacking subtypes. Concrete types include in particular primitive types, which represent the most fundamental types that computers use to perform calculations. Examples of primitive types are `Int64` and `Float64`, which directly map to low-level hardware representations.

Abstract types provide great flexibility for writing code. For example, the abstract type `Number` defined in Julia encompasses all possible numeric types (e.g., `Float64`, `Int64`, `Float32`). By declaring a variable as `Number`, programmers avoid unnecessarily constraining their programs to specific numeric representations or precision.

To demonstrate this hierarchy, we consider the concrete types comprised by `Number`. The names included in the table match the exact names in Julia. Note, nonetheless, that the full subtype hierarchy of `Number` is broader than the simplified representation presented. [5]

EXAMPLE OF THE ABSTRACT TYPE "NUMBER"



Note: Dashed red borders indicate abstract types, while solid blue borders indicate concrete types.

OPERATORS

In programming, **operators** are symbols that represent operations performed on objects. They can be thought of as syntactic sugar for functions, as we'll see in the next chapters. In fact, almost all operators in Julia can be employed as functions.

For instance, the symbol `+` in `x + y` is an operator that performs the addition of `x` and `y`. Likewise, the symbols `x` and `y` are referred to as the **operands**, representing the operator's inputs to perform its calculation. *Operators follow specific syntax rules based on the number of operands they require*. Understanding this syntax will prove useful for several topics covered later in the book. Next, we define and illustrate the syntax through several examples. At this point, just focus on how operators are written, even if their specific functions are not yet clear.

- **Unary operators**: They take *one operand*, with the operator written to the left of it. [6] Formally, their syntax is `<operator>x`, such as `√x` or `-x`.

- **Binary operators**: They take *two operands*, and the operator is written between them. [7] Formally, their syntax is `x <operator> y`, such as `x + y` or `x^y` for x^y.

- **Ternary operators**: They take *three operands*. Formally, their syntax is `x <operator1> y <operator2> z`. Ternary operators are rare, which is why the specific operator `x ? y : z` is directly referred to as *the* ternary operator. We'll see that this operator performs a conditional evaluation, returning `y` if `x` is true and `z` returned otherwise.

FOOTNOTES

[1.] While it's common to say that "a variable has a specific type", this is an informal shorthand. Technically, it's the value of the variable that has a specific type, not the variable itself.

[2.] You can insert Unicode characters by copying and pasting them from a list like this one. Alternatively, you can use tab completion with the commands listed in the Julia documentation.

[3.] Not all symbols are allowed. For instance, names with common mathematical symbols like `x^` or `%x` aren't permitted. Additionally, numbers are allowed, but they can't be included as the first character (e.g., `2x` is invalid).

[4.] The suffix `64` in these types represents the precision of the number. This represents the maximum number of significant digits or decimals that a type can accurately represent.

[5.] The subtype `Signed` from `Integers` represents positive or negative integers. Although not included in the graph, there's also a type called `Unsigned`, which only accepts positive integers.

[6.] Operators to the left of the operand are known as **prefix operators**. Conversely, operators written to the right of the operand are known as **postfix operators**, and Julia has a few of them (e.g., `'` to transpose a vector or matrix `x`, which is written as `x'`). Despite this, we won't use postfix operators in this book.

[7.] Operators with this syntax are called **infix operators**.

2c. Numbers

Martin Alfaro
PhD in Economics

INTRODUCTION

The previous section introduced the concept of variables, distinguishing between those containing a single element (scalars) and collections. This section expands on scalars, exclusively focusing on those holding numeric values.

NUMBERS

Computers store numbers in various formats, treating integers and decimal numbers as separate entities. Even within each category of numbers, multiple representations emerge depending on the intended level of precision. This precision is determined by the number of bits allocated to store values in memory, which in turn defines the maximum range of values that a data type supports. [1] The representation just described extends well beyond Julia, and is intrinsic to how computers operate at a fundamental level.

In modern computers, numbers typically have a default size of 64 bits. In particular, the default types in Julia for numbers are:

- `Int64` for integers.
- `Float64` for decimal numbers. [2]

> **Remark**
>
> Julia provides the type `Int` as a more versatile option than `Int64`. This adapts to your computer's architecture: `Int` defaults to `Int64` on 64-bit systems and `Int32` on 32-bit systems. Since most modern machines operate on a 64-bit architecture, `Int` typically defaults to `Int64`. Note that there's no equivalent type `Float` for floating-point numbers, with Julia always defaulting to `Float64`.

It's worth emphasizing that `Int64` and `Float64` are two different data types. Thus, while `1` is a value with type `Int64`, the same value becomes `1.0` as a `Float64` type. Importantly, both values are treated differently by the computer.

NUMBERS

```
x = 1       # `Int64`

y = 1.0     # `Float64`
z = 1.      # alternative notation for `1.0`
```

> **Remark**
>
> To enhance code readability, you can break up long numbers by inserting underscores `_`.
>
> **NOTATION FOR NUMBERS**
>
> ```
> x = 1000000
> x = 1_000_000         # equivalent but more readable
>
> y = 1000000.24
> y = 1_000_000.24      # _ can also be used with decimal numbers
> ```

The type `Float64` encompasses not only decimal numbers, but also two special values: `Inf` for infinity and `NaN` for indeterminate expressions such as 0/0 (`NaN` stands for "not a number"). Considering this, all the following variables have type `Float64`.

FLOAT64

```
x = 2.5

y = 10/0

z = 0/0
```

```
julia> x
2.5
julia> y
Inf
julia> z
NaN
```

BOOLEAN VARIABLES

A distinct numeric type is `Bool`, which facilitates the representation of **Boolean variables**. These variables can only take on the values `true` and `false`. Internally, they're implemented as integers, with `true` corresponding to `1` and `false` to `0`. Because of this implementation, Julia accepts `1` and `0` interchangeably with `true` and `false`.

Boolean expressions come into play when evaluating conditions, such as when checking if a number exceeds a certain value or a string matches a specific pattern. These conditional evaluations can then be employed to control the flow of the program, determining different paths of execution depending on whether the condition is true or false. Some examples of Boolean values are presented below.

BOOLEAN VARIABLES

```
x = 2
y = 1

z = (x > y)       # is `x` greater than `y` ?
z = x > y         # equivalent (don't interpret it as 'z = x')
```

```
julia> z
true
```

ARITHMETIC OPERATORS

Numbers can be manipulated through a variety of **arithmetic operators**. These operators are represented by symbols akin to those in other programming languages.

Julia's Arithmetic Operator	Meaning
`x + y`	addition
`x - y`	subtraction
`x * y`	product
`x / y`	division
`x^y`	power (x^y)

It's worth noting that all the operators presented above are *binary*. Consequently, they adhere to the syntax `x <symbol> y`, as indicated in our discussion on operators.

FOOTNOTES

[1.] For instance, 8-bit integers can only represent values from -128 to 127. Likewise, 32-bit floating-point numbers, used for decimal numbers, can represent up to 7 significant digits of precision.

[2.] The term "Float" stands for "floating point" and is how computers represent decimal numbers.

2d. Strings

Martin Alfaro
PhD in Economics

INTRODUCTION

This section presents types for text representation, distinguishing between characters and strings. The coverage will be concise, as our book won't focus on string analysis. However, a minimal treatment is necessary for tasks like specifying paths, displaying messages, and documenting functions.

CHARACTERS

The `Char` type is employed to represent individual characters. They're written by enclosing them in single quotes, as in `'x'` for the character `x`. Given Julia's support for Unicode characters, `Char` encompasses not only numbers and letters, but also a wide range of symbols.

UNICODE CHARACTERS

```
# x equals the character 'a'
x = 'a'

# 'Char' allows for Unicode characters
x = 'β'
y = '🐒'
```

Notice that characters must be enclosed in single quotes `' '` even for symbols like 🐒. Otherwise, Julia will interpret the expression as a variable.

EMOJIS

```
# any character is allowed to define a variable
🐒 = 2      # 🐒 represents a variable, just like if we had defined x = 2

y  = 🐒     # y equals 2, 🐒's value at that moment (not 🐒 itself)
z  = '🐒'   # z equals the character 🐒 (entirely independent of the 🐒 variable )
```

STRINGS

We'll rarely use the type `Char` directly. Instead, we'll work with the so-called type `String`. This is an ordered collection of characters, permitting the representation of text.

Strings can be defined through either double quotes `" "` or triple quotes `""" """`. The latter is particularly convenient for handling newlines, such as when the text has to span multiple lines. [1]

STRING VARIABLES

```
x = "Hello, beautiful world"

x = """Hello, beautiful world"""
```

STRING INTERPOLATION

String interpolation lets you embed Julia code directly inside a string. The embedded expression is then evaluated and replaced in the string with its value.

To interpolate an expression, the string must be prefixed with the `$` symbol. If the expression contains spaces, it must be enclosed in curly braces, as in `$()`. Both cases are exemplified below.

INTERPOLATING A VARIABLE

```
number_students = 10


output_text     = "There are $(number_students) students in the course"
```

```
julia> output_text
"There are 10 students in the course"
```

INTERPOLATING AN EXPRESSION

```
number_matches  = 50
goals_per_match = 2

output_text     = "Last year, Messi scored $(number_matches * goals_per_match) goals"
```

```
julia> output_text
"Last year, Messi scored 100 goals"
```

FOOTNOTES

[1] For more on the differences between double and triple quotes, see here

2e. Arrays (Vectors and Matrices)

Martin Alfaro
PhD in Economics

INTRODUCTION

So far, we've explored scalar variables, which store single-element objects. Now, we'll shift our focus to **collections**, which correspond to **variables comprising multiple elements**. Julia provides several forms of collections, including:

- Arrays (including vectors and matrices)
- Tuples and Named Tuples
- Dictionaries
- Sets

Arrays represent one of the most common data structures for collections. Their distinctive feature is that all their elements share a **homogeneous type**. Formally, arrays are objects with type `Array{T,d}`, where `d` is the number of dimensions and `T` is the element type (e.g., `Int64` or `Float64`). Two special categories of arrays are:

- **Vectors**: 1-dimensional arrays. They're represented by the type `Vector{T}`, which is an alias for `Array{T,1}`.
- **Matrices**: 2-dimensional arrays. They're represented by the type `Matrix{T}`, which is an alias for `Array{T,2}`.

Although we provide a subsection about matrices at the end, this is labeled as optional. The reason is that vectors are sufficient for conveying the topics covered in this book.

> **Remark**
>
> **Julia uses 1 as an array's first index**. This contrasts with many other languages (e.g., Python), where 0 is the first index.

VECTORS

Vectors in Julia are defined as *column-vectors*, and their elements are separated by a comma or a semicolon.

ARRAYS

```
x = [1, 2, 3]          #= column-vector (defined using commas or semicolons)
                           Vector{Int64} (alias for Array{Int64, 1}) =#

x = [1; 2; 3]          # equivalent notation to define `x`
```

```
julia> x
3-element Vector{Int64}:
 1
 2
 3
```

Note that, as previously indicated, arrays are defined so that all elements share a common type. This requirement, however, doesn't prevent arrays from holding elements of different kinds, such as numbers and strings. The reason is that the shared type can be an **abstract type**, effectively providing a unifying type for the collection.

For example, defining the vector `[1, 2.5, "Hello"]` is valid, because its elements will be assigned the abstract type `Any`. Recall that `Any` sits at the top of Julia's type hierarchy, encompassing all concrete types supported by the language.

Although arrays that mix element types are possible, they're strongly discouraged for several reasons, including performance.

ACCESSING VECTOR ELEMENTS

Given a vector `x`, we can access its *i*-th element with `x[i]` and retrieve all its elements with `x[:]`.

VECTOR ACCESS

```
x = [4, 5, 6]
```

```
julia> x
3-element Vector{Int64}:
 4
 5
 6

julia> x[2]
5

julia> x[:]
3-element Vector{Int64}:
 4
 5
 6
```

It's also possible to access a subset of elements within `x`. There are several approaches to achieve this, and we'll only present two basic ones at this point. The simplest method involves setting the indices **via a vector**, using the syntax `x[<vector>]`.

VECTOR

```
x = [4, 5, 6, 7, 8]
```

```
julia> x
5-element Vector{Int64}:
 4
 5
 6
 7
 8

julia> x[[1,3]] # elements of 'x' with indices 1 and 3
2-element Vector{Int64}:
 4
 6

julia> x[1,3] # be careful! this is the notation used for matrices, indicating 'x[row 1, column 3]'
ERROR: BoundsError: attempt to access 5-element Vector{Int64} at index [1, 3]
```

The second approach sets the indices **via ranges**. These are denoted as `<first>:<steps>:<last>`, with Julia assuming unit increments when omitting `<steps>`. To express the first and last index, Julia provides the keywords `begin` and `end`.

RANGES

```
x = [4, 5, 6, 7, 8]
```

```
julia> x
5-element Vector{Int64}:
 4
 5
 6
 7
 8


julia> x[1:2] # steps with unit increments (default increments)
2-element Vector{Int64}:
 4
 5

julia> x[1:2:5] # steps with increments of 2
3-element Vector{Int64}:
 4
 6
 8

julia> x[begin:end] # all elements, equivalent to 'x[:]' or 'x[1:end]'
5-element Vector{Int64}:
 4
 5
 6
 7
 8
```

MATRICES (*OPTIONAL*)

Matrices can be defined as collections of row or column vectors. When constructing a matrix from multiple row vectors, each row must be separated by a semicolon `;`. Conversely, if created using multiple column vectors, their elements must be separated by spaces.

Note that row vectors are treated as special cases of matrices, with their elements separated by a space. In this form, they're matrices consisting of a single row and multiple columns.

MATRICES

```
X = [1 2 ; 3 4]       #= matrix as a collection of row-vectors, separated by semicolons
                         Matrix{Int64} (alias for Array{Int64, 2})=#

X = [ [1,3] [2,4] ]    # identical to `X`, but defined through a collection of column-vectors

Y = [1 2 3]           #= row-vector (defined without commas)
                         Matrix{Int64} (alias for Array{Int64, 2}) =#
```

```
julia> X
2×2 Matrix{Int64}:
 1  2
 3  4
julia> Y
1×3 Matrix{Int64}:
 1  2  3
```

ACCESSING MATRIX ELEMENTS

Given a matrix `X`, we can access the element at row `r` and column `c` by `X[r,c]`. Moreover, `X[r,:]` selects all elements across the row `r`, while `X[:,c]` selects all elements of column `c`. For a row vector `Y`, its *i*-th element can be accessed with `Y[i]`.[1]

ACCESSING MATRIX ELEMENTS

```
X = [5 6 ; 7 8] # matrix

Y = [4 5 6]     # row-vector
```

```
julia> X
2×2 Matrix{Int64}:
 5  6
 7  8


julia> X[2,1]
7


julia> X[1,:]
2-element Vector{Int64}:
 5
 6
julia> X[:,2]
2-element Vector{Int64}:
 6
 8
julia> Y[2]
5
```

To access a subset of elements within a matrix, we can use the same methods employed for vectors. The only difference is that these methods must be applied to rows or columns.

ACCESSING MATRIX ELEMENTS

```
X = [5 6 ; 7 8]
```

```
julia> X
2×2 Matrix{Int64}:
 5  6
 7  8


julia> X[[1,2],1]
2-element Vector{Int64}:
 5
 7


julia> X[1:2,1]
2-element Vector{Int64}:
 5
 7
julia> X[begin:end,1]
2-element Vector{Int64}:
 5
 7
```

FOOTNOTES

[1] Matrices also support linear indexing. For example, a 3×3 matrix `X` accepts indices from 1 to 9. However, unless you intend to iterate over all elements, the two-dimensional notation `X[r,c]` is generally clearer and easier to interpret.

2f. Tuples

Martin Alfaro
PhD in Economics

INTRODUCTION

We continue our exploration of *collections*, defined as objects storing multiple elements. Having previously focused on arrays, we'll now turn our attention to another form of collection known as tuples.

Tuples differ from arrays in several respects. First, they can comprise elements with heterogeneous types, without incurring any performance penalty. Instead, as we'll see in Part II of the book, arrays mixing disparate types (such as integers and strings) perform poorly. Another key distinction is that tuples have a **fixed size** and are **immutable**. This implies that, once a tuple is created, its elements can't be added, removed, or modified. Finally, tuples are only suitable for **storing a small number of elements**. In contrast, large tuples will result in slow computations at best, or directly trigger fatal errors. For this reason, vectors remain the appropriate choice for large collections.

At this stage, we'll limit our discussion to the basic features of tuples, introducing only what's necessary to develop the fundamentals of Julia. A more in-depth exploration will follow in Part II, after we've developed the necessary tools to appreciate their role in high-performance applications.

DEFINITION OF TUPLES

Tuples are defined by enclosing their elements within parentheses `()`, unlike the square brackets `[]` used in vectors. When a tuple contains more than one element, the parentheses are optional and often omitted.

Single-element tuples have stricter syntax rules, requiring parentheses `()` and a trailing comma `,` after the element. For example, a tuple with the single element `10` is represented as `(10,)`. The notation distinguishes a tuple from the expression `(10)`, which would simply be interpreted as the number `10`.

As with vectors, `x[i]` is the syntax for accessing the *i*-th element of a tuple `x`.

TUPLE

```
x = (4,5,6)
x =  4,5,6           #alternative notation
```

```
julia> x
(4, 5, 6)
julia> x[1]
4
```

SINGLE-ELEMENT TUPLE

```
x = (10,)    # not x = (10) (that's just x = 10)
```

```
julia> x
(10,)
julia> x[1]
10
```

TUPLES FOR ASSIGNMENTS

Tuples can be used to assign values to multiple variables at once. This operation is commonly referred to as **unpacking** or **destructuring**. It's implemented by placing both a tuple on the left-hand side of `=` and a collection on the right-hand side, where the latter may be either another tuple or a vector.

A TUPLE ON THE RIGHT-HAND SIDE

```
(x,y) = (4,5)
 x,y  =  4,5       #alternative notation
```

```
julia> x
4
julia> y
5
```

A VECTOR ON THE RIGHT-HAND SIDE

```
(x,y) = [4,5]
 x,y  = [4,5]      # alternative notation
```

```
julia> x
4
julia> y
5
```

The technique is commonly employed when a function returns multiple values, providing convenient syntax to unpack outputs into individual variables.

3a. Overview and Goals

Martin Alfaro
PhD in Economics

The upcoming Chapters 3 and 4 will cover three core tools for programming: functions, conditional statements, and for-loops. Chapter 3 in particular focuses on **functions**, which constitute the backbone of Julia programming. As they're tightly tied to achieving high performance, we'll dedicate considerable time to discussing their usage.

Our coverage of functions will be organized into three categories, based on who defines them:

i) built-in functions,

ii) third-party functions, and

iii) user-defined functions.

The first two types of functions become available in the workspace via packages, which may be loaded implicitly or explicitly. This connection between packages and functions leads us into exploring the concepts together in Section 3b. Instead, user-defined functions are left for Section 3c.

A firm grasp of functions requires understanding variable scope, including the distinction between global and local variables. By establishing this difference, we'll frame functions as self-contained mini-programs designed to perform a specific task. Both subjects are presented together in Section 3d. This perspective on functions will lead to the identification of good practices for using functions, which will have significant implications for the structure of code as we progress. At this point, nonetheless, it suffices if you start becoming familiar with this view.

Finally, we'll introduce the concept of broadcasting in Section 3e. Mastering this technique is crucial, as it lets you seamlessly apply the same function to each element in a collection. Broadcasting is a widely used technique not only in Julia, but also in other programming languages like Python.

3b. Function Calls and Packages

Martin Alfaro
PhD in Economics

INTRODUCTION

Functions can be broadly categorized into three main categories:

- ***i)*** built-in functions, which are provided as part of the core Julia language.
- ***ii)*** third-party functions, typically obtained from external packages.
- ***iii)*** user-defined functions, created by the programmer to perform specific tasks.

This section focuses on i) and ii), with user-defined functions being addressed in the following section.

> **Notation for Functions**
>
> Julia's developers suggest the **snake-case convention for function names**. This style consists of lowercase letters, numbers, and possibly underscores to separate words (e.g., `snake_case123`). Keep in mind that this is just a recommendation, not a language's requirement.

PACKAGES

When a new Julia session is initiated, only a handful of very basic functions are available (e.g., those for sums, products, and subtractions). This is a deliberate choice made by Julia developers, who rely on **packages** to incorporate functions into the workspace. In fact, both built-in and third-party functions are contained in packages, with the only difference that the former are loaded by default.

The approach isn't unique to Julia. However, Julia embraces this philosophy more deeply than other programming languages. Thus, it doesn't even include standard functions such as averages or standard deviations, which are instead relegated to a package called `Statistics`. [1]

This design philosophy is rooted in a programming principle known as **modularity**. The principle promotes the development of small reusable modules, rather than large intertwined code. The main advantage of modularity is to let packages evolve independently, without bugs and deprecations spreading across the entire Julia ecosystem. In practice, it also implies that users must load several packages during each session, even to perform simple tasks.

LOADING PACKAGES AND CALLING FUNCTIONS

The concept of packages in Julia is closely tied to that of modules. Formally, **modules** are self-contained blocks of code, each providing its own workspace and exporting a defined set of functions. In fact, when a Julia session starts, all code is implicitly executed within a default module called `Main`.

For their part, **packages** are a special type of module that additionally include information about their **dependencies**. These are defined as the necessary packages that must be loaded to run the package itself. This dependency information ensures that Julia can automatically manage and load the required components when the package is used.

To get access to the functions provided by a package, we must load it through either the `import` or `using` keyword. While both mechanisms bring the package into scope, they differ in how functions are subsequently invoked. With `import`, functions must be explicitly qualified by prefixing them with the package name. In contrast, `using` makes the exported functions directly accessible, allowing them to be called without any prefix.

Below, we demonstrate each approach by calling the function `mean` from the package `Statistics`. This package isn't loaded by default, but it comes pre-installed with Julia.

"IMPORT" PACKAGE

```
x = [1,2,3]

import Statistics   #getting access to its functions will require the prefix `Statistics.`
Statistics.mean(x)
```

"USING" PACKAGE

```
x = [1,2,3]

using Statistics      #no need to add the prefix `Statistics.` to call its functions (although it's possible to do so)
mean(x)
```

BUILT-IN FUNCTIONS

Julia's built-in functions are formally organized into two packages called `Core` and `Base`. These packages are automatically loaded at the start of each session. The access to their functions replicates the behavior of `using Core` and `using Base`. As a result, most of their associated functions are directly accessible without requiring a package prefix. [2]

Below, we show the syntax for common built-in mathematical functions.

Function in Julia	Meaning
`log(x)`	$\ln(x)$

Function in Julia	Meaning
`exp(x)`	e^x
`sqrt(x)`	$\sqrt{x}$
`abs(x)`	$\|x\|$
`sin(x)`	$\sin(x)$
`cos(x)`	$\cos(x)$
`tan(x)`	$\tan(x)$

Operators as Functions

Most of the symbols employed as operators in Julia are also available as functions. This is illustrated below for several arithmetic operators:

ARITHMETIC OPERATORS AS FUNCTIONS

```
+(2,3)      # same as 2 + 3
-(2,3)      # same as 2 - 3
*(2,3)      # same as 2 * 3
/(2,3)      # same as 2 / 3
^(2,3)      # same as 2 ^ 3
```

WHY USING "IMPORT" IF IT'S MORE VERBOSE?

When functions share the same name across multiple packages, at least one of the packages must be loaded via `import` to prevent naming conflicts. For instance, if both the package `Statistics` and another one called `MyPackage` contain a function called `mean`, Julia will throw an error unless one of them is loaded with `import`.[3]

Beyond resolving naming conflicts, `import` can also enhance code clarity by reducing ambiguity in the meaning of a function. For instance, consider a function called `rank`. This name could reference a wide range of concepts depending on the context (e.g., the rank of a matrix, the order in a list). Explicitly specifying the package when the function is called could clarify its intended purpose.

Remark

`import` may also be beneficial when defining custom functions that will be reused across several projects. For example, imagine defining a function like `table_in_pdf` to export Julia tables to PDF format. While the function name clearly conveys its purpose, someone reading your

> code might reasonably wonder whether this function comes from a standard package. To avoid this ambiguity, you could place the function in a package called `userDefined` and load it with `import UserDefined`. This approach makes it immediately clear that `UserDefined.table_in_pdf` is a custom implementation, rather than part of a standard package.

APPROACHES TO LOADING PACKAGES AND CALLING FUNCTIONS

The concepts discussed so far are sufficient for using packages in Julia effectively. Still, there are a few additional features worth highlighting, since they can further enhance how you interact with packages.

First, users have the possibility of loading only a subset of functions from a package. This is particularly valuable when working with heavy packages which may suffer from significant loading times. For instance, if we only need the function `mean` from `Statistics`, the following two approaches achieve the same result.

"IMPORT" PACKAGE

```
x = [1,2,3]

import Statistics: mean
mean(x)                    # no prefix needed
```

"USING" PACKAGE

```
x = [1,2,3]

using Statistics: mean
mean(x)
```

Note that, in both cases, the function `mean` can be called without prefixing it with the package name. This holds even when the package is loaded via `import`.

Another valuable feature is the ability to assign custom names to packages or functions. This becomes particularly useful when names are lengthy, since aliases can shorten them for convenience while maintaining readability in the code.

CHANGING PACKAGE'S NAME

```
x = [1,2,3]

import Statistics as st
st.mean(x)
```

CHANGING FUNCTION'S NAME

```
x = [1,2,3]

import Statistics: mean as average
average(x)                    # no prefix needed

using Statistics: mean as average
average(x)
```

In this case as well, the function name can be called without any prefix, even if it's been brought into scope via `import`.

MACROS

Macros are ubiquitous in Julia, automating tasks that would otherwise be tedious and time-consuming. We'll only cover how to apply macros, without exploring how to define them. The reason for this is that creating macros requires knowledge of Julia's metaprogramming capabilities, which lies beyond the scope of this book.

Although the benefits of macros may not be immediately obvious at this point, their utility will become evident once we apply them in subsequent sections.

APPLYING MACROS

Macros and functions share important similarities: both operate on inputs and produce outputs. Their key distinction lies in the objects taken as inputs and returned as outputs. Functions evaluate data values such as variables or expressions and return a result. Macros, by contrast, manipulate the syntax of code itself, rewriting statements or expressions before they're executed.

Formally, macros are denoted by prefixing the symbol `@` to their name. They take an entire code expression as their argument and transform it at the syntactic level. For instance, a macro might take the expression `x = some_function(y)` as input and then alter its structure: it could modify each individual component (`x`, `=`, or `some_function(y)`), insert new code, or reorganize the statement entirely. The final output is a modified version of the original expression, which is then integrated into the program during execution.

The primary role of macros is to automate code transformations that would be repetitive or error-prone if written manually. A clear example is Julia's `@.` macro, which appends a dot `.` to every operator and function call within a statement. While the semantic consequences of dot notation will be discussed in upcoming sections, the key point here is that macros enable systematic rewriting of entire code blocks.

MACRO @.

```
# both are equivalent
   z .= foo.(x .+ y)
@. z  = foo(x  + y)          # @. adds . to =, foo, and +
```

> ⚠ **Warning!** - Caution with Macro Usage
>
> While powerful, macros should be applied with considerable care. Because they operate by transforming code before runtime, they can behave as "black boxes", making debugging challenging and potentially introducing subtle errors. Indeed, macros are frequently a source of unexpected behavior in programs. Make sure you understand which part of the expression a macro modifies and how that transformation is carried out.

FOOTNOTES

[1.] The extent to which Julia advocates for this principle is evident in `Statistics` itself, where functions for computing distributions are included in another package called `Distributions`.

[2.] Some built-in functions may require a prefix. For instance, the function `isgreater` must be called via `Base.isgreater`. Furthermore, some submodules are preloaded by default. For instance, the function `Base.Iterators.accumulate` belongs to the `Iterators` submodule of `Base`, and can be directly called using `Iterators.accumulate`.

[3.] Defining a function that shares the name of another package's function isn't necessarily an oversight. For instance, developers could implement their own version of `mean` within a package like `MyPackage`, where averages are computed with a different algorithm.

3c. Defining Your Own Functions

Martin Alfaro
PhD in Economics

INTRODUCTION

Recall that functions can be classified into *i)* built-in functions, *ii)* third-party functions, and *iii)* user-defined functions. The previous section has covered the first two, and **we now focus on iii)**.

USER-DEFINED FUNCTIONS

The first step in creating your own functions is assigning them names. Function names adhere to similar rules as variable names. In particular, they accept Unicode characters, allowing us to specify functions like `∑(x)`. Once defined, functions can be directly called, without any prefix. Thus, a function named `foo` can be simply called by `foo(x)`. [1]

There are two approaches to defining functions. We'll refer to each as the **standard form** and the **compact form**. The standard form is the most general and allows you to write both short and long functions. On the other hand, the compact form is employed for single-line functions and is reminiscent of mathematical definitions.

To illustrate each form, consider a function `foo` that sums the values of the variables `x` and `y`.

STANDARD FORM

```
function foo(x,y)
    x + y
end
```

COMPACT FORM

```
foo(x,y) = x + y
```

In the compact form, the value of the single expression is automatically returned as the output. By contrast, the standard form returns the result of the last line by default. If greater control is needed, the output can be explicitly specified using the keyword `return`.

It's also possible to return multiple values by defining a collection as the output. Among the available options, tuples are generally preferred when the number of outputs is small. As we'll show in Part II of the book, they provide a lightweight structure that avoids the overhead associated with vectors, making them more efficient in both performance and memory usage. This efficiency explains why tuples are commonly adopted in practice for returning compact sets of results.

Below, we illustrate all this.

EXPLICIT OUTPUT

```
function foo(x,y)
    term1 = x + y
    term2 = x * y

    return term2
end
```

```
julia> foo(10,2)
2
```

IMPLICIT OUTPUT

```
function foo(x,y)
    term1 = x + y
    term2 = x * y          # output returned
end
```

```
julia> foo(10,2)
2
```

MULTIPLE OUTPUTS

```
function foo(x,y)
    term1 = x + y
    term2 = x * y

    return term1, term2    # a tuple (notation that omits the parentheses)
end
```

```
julia> foo(10,2)
(3, 2)
```

AN EXPRESSION AS OUTPUT

```
function foo(x,y)
    term1 = x + y
    term2 = x * y

    return term1 + term2
end
```

```
julia> foo(10,2)
5
```

> **Functions without Inputs**
>
> It's possible to define functions that don't require any input arguments, as shown below.

FUNCTION WITHOUT ARGUMENTS

```
function foo()
    a = 1
    b = 1
    return a + b
end
```

The Order In Which Functions Are Defined is Irrelevant

A function can be defined at any point in the code. In fact, it's possible to define a function that invokes another function, even if the latter hasn't yet been introduced. To illustrate, consider the following two code snippets, which are functionally equivalent.

CODE SNIPPET 1

```
foo1(x) = 2 + foo2(x)

foo2(x) = 1 + x
```

```
julia> foo1(2)
5
```

CODE SNIPPET 2

```
foo2(x) = 1 + x

foo1(x) = 2 + foo2(x)
```

```
julia> foo1(2)
5
```

FUNCTIONS AS OPERATORS

Most built-in operators in Julia are also available as functions. For example, the expression `2 + 3` that uses `+` as an operator can be equivalently written as `+(2, 3)`, where `+` is employed as a function.

The same principle extends to user-defined functions when their names consist of certain symbols. In such cases, Julia automatically generates a corresponding operator. As an illustration, let's define a function whose name is `⊕`, where `⊕(x, y)` returns the sum of logarithms for scalar inputs `x` and `y`.

FUNCTIONS AS OPERATORS

```
x = 1
y = 1

⊕(x,y) = log(x) + log(y)
```

```
julia> ⊕(x,y)
0.0
julia> x ⊕ y
0.0
```

Note that not all symbols can be used for this purpose. While the list of allowed symbols isn't officially documented, it can be found in the Julia source code.

POSITIONAL AND KEYWORD ARGUMENTS

Up to this point, we've been defining and calling functions using the syntax `foo(x,y)`. A key characteristic of this style is that arguments are passed in a fixed order. Thus, the call `foo(2,4)` assigns the first argument to `x` and the second to `y`. This convention is referred to as **positional arguments**.

A major drawback of positional arguments is their susceptibility to silent errors: if we accidentally swap the positions of the arguments, the function may still execute and return a wrong result. As the number of arguments grows, the likelihood of introducing such bugs grows, while simultaneously noticing them becomes more difficult.

To circumvent this issue, we can rely on **keyword arguments**. With this approach, each argument must be explicitly named in the function call, rendering their order irrelevant. For example, both `foo(x=2,y=4)` and `foo(y=4,x=2)` would then be valid and equivalent.

The following examples illustrate the difference between positional and keyword arguments when defining and calling functions. In particular, note that positional arguments necessarily require a semicolon during function definitions, but accept either a semicolon or a comma during function calls. Additionally, we show that both approaches can be combined within the same function.

POSITIONAL ARGUMENTS

```
foo(x, y) = x + y
```

```
julia> foo(1,2)
3
```

KEYWORD ARGUMENTS

```
foo(; x, y) = x + y
```

```
julia> foo(x=1, y=1)
2
julia> foo(; x=1, y=1) # alternative notation (only for calling 'foo')
2
```

POSITIONAL AND KEYWORD ARGUMENTS

```
foo(x; y) = x + y
```

```
julia> foo(1 ; y=1)
2
julia> foo(1 , y=1) # alternative notation
2
```

KEYWORD ARGUMENTS WITH DEFAULT VALUES

An additional advantage of keyword arguments is that they can be given default values. This implies that certain arguments may be omitted when the function is called, in which case the omitted arguments automatically assume their predefined defaults. The following examples illustrate this behavior in practice.

POSITIONAL AND KEYWORD ARGUMENTS

```
foo(x; y=1) = x + y
```

```
julia> foo(1) # equivalent to foo(1, y=1)
2
```

OMITTING POSITIONAL ARGUMENTS

```
foo(; x=1, y=1) = x + y
```

```
julia> foo() # equivalent to foo(x=1, y=1)
2
julia> foo(x=2) # equivalent to foo(x=2, y=1)
3
```

Default values can also be defined in terms of earlier arguments. For example, in a function `foo(; x, y)`, the default value of `y` can be set based on the value of `x`. This works because, when a function is called, its arguments are evaluated sequentially from left to right.

PRIOR ARGUMENTS AS DEFAULT VALUES

```
foo(; x, y = x+1) = x + y
```

```
julia> foo(x=2) #function run with implicit value 'y=3'
5
```

SPLATTING

Given a function `foo(x, y)`, it's possible to supply the values of `x` and `y` through a single collection `z`. This is achieved using the splat operator `...`, which unpacks the elements of a collection and passes them as individual arguments to the function.

TUPLE SPLATTING

```
foo(x,y) = x + y

z        = (2,3)
```

```
julia> foo(z...)
5
```

VECTOR SPLATTING

```
foo(x,y) = x + y

z        = [2,3]
```

```
julia> foo(z...)
5
```

ANONYMOUS FUNCTIONS

Anonymous functions offer a third way to define functions. Unlike the standard or compact forms, they're commonly introduced for a different purpose: to serve as inputs to other functions.[2] Functions that call another function as an argument are referred to as **higher-order functions**, and will be studied in Part II of the book.

As the name suggests, anonymous functions aren't referenced by a name. Instead, their definition relies entirely on syntax, which resembles the arrow notation from mathematics (e.g. $x \mapsto \sqrt{x}$). In particular, single-argument anonymous functions are expressed as `x -> <body of the function>`, whereas those with two or more arguments are expressed as `(x,y) -> <body of the function>`.

To illustrate, let's consider the built-in function `map(<function>, <collection>)`. This applies `<function>` element-wise to each element of `<collection>`. For example, given the function `add_two(a) = a + 2`, the call `map(add_two, x)` applies `add_two` to each element of `x = [1,2,3]`,

thus yielding `[3,4,5]`.

The downside of using `map` in this way is that `add_two` must be defined beforehand, unnecessarily cluttering the namespace if `add_two` won't be reused. Anonymous functions provide an elegant alternative by embedding the operation directly within `map`.

VIA DEFINED FUNCTION

```
x          = [1, 2, 3]
add_two(a) = a + 2

output     = map(add_two, x)
```

```
julia> output
3-element Vector{Int64}:
 3
 4
 5
```

VIA ANONYMOUS FUNCTION

```
x          = [1, 2, 3]


output     = map(a -> a + 2, x)
```

```
julia> output
3-element Vector{Int64}:
 3
 4
 5
```

The function `map` can also be employed with multiple arguments, in which case the syntax becomes `map(<function>, <array1>, <array2>)`. For instance, `map(+, [1,2], [2,4])` provides the sum of each pair of numbers, producing `[3,6]`. Using this form, let's apply `a + b` element-wise to each pair of `x` and `y`. Below, we show how this operation can be implemented with an anonymous function.

VIA DEFINED FUNCTION

```
x              = [1,2,3]
y              = [4,5,6]
add_values(a,b) = a + b

output         = map(add_values, x, y)
```

```
julia> output
3-element Vector{Int64}:
 5
 7
 9
```

VIA ANONYMOUS FUNCTION

```
x               = [1,2,3]
y               = [4,5,6]


output          = map((a,b) -> a + b, x, y)
```

```
julia> output
3-element Vector{Int64}:
 5
 7
 9
```

THE "DO-BLOCK" SYNTAX

When working with higher-order functions, anonymous functions prevent unnecessary pollution of the namespace. However, when the function body extends beyond a single line, their use can become cumbersome and harder to read.

To address this limitation, we can employ a **do-block**. This groups a sequence of expressions into a block that can be passed as an argument to a function. While this construct isn't limited to higher-order functions, it's particularly valuable when a function expects another function as its first argument. In such situations, the do-block provides a way to define a multiline anonymous function, without sacrificing readability.

For example, suppose a function of the form `foo(<inner function>, <vector>)`. Instead of embedding a lengthy anonymous function, we can call `foo` through a do-block using the following syntax:

DO-BLOCK SYNTAX

```
foo(<vector>) do <arguments of inner function>
    # body of inner function
    end
```

To illustrate this notation with a concrete scenario, let's revisit the example with the `map` function and rewrite it using a do-block.

VIA DEFINED FUNCTION

```
x          = [1, 2, 3]
add_two(a) = a + 2

output     = map(add_two, x)
```

```
julia> output
3-element Vector{Int64}:
 3
 4
 5
```

VIA ANONYMOUS FUNCTION

```
x          = [1, 2, 3]


output     = map(a -> a + 2, x)
```

```
julia> output
3-element Vector{Int64}:
 3
 4
 5
```

VIA DO-BLOCK SYNTAX

```
x          = [1, 2, 3]

output     = map(x) do a
               a + 2
               end
```

```
julia> output
3-element Vector{Int64}:
 3
 4
 5
```

Do-blocks also accept anonymous functions with multiple arguments, as shown below.

VIA DEFINED FUNCTION

```
x               = [1,2,3]
y               = [4,5,6]
add_values(a,b) = a + b

output          = map(add_values, x, y)
```

```
julia> output
3-element Vector{Int64}:
 5
 7
 9
```

VIA ANONYMOUS FUNCTION

```
x               = [1,2,3]
y               = [4,5,6]


output          = map((a,b) -> a + b, x, y)
```

```
julia> output
3-element Vector{Int64}:
 5
 7
 9
```

VIA DO-BLOCK SYNTAX

```
x               = [1,2,3]
y               = [4,5,6]

output          = map(x,y) do a,b    # not (a,b)
                             a + b
                           end
```

```
julia> output
3-element Vector{Int64}:
 5
 7
 9
```

FUNCTION DOCUMENTATION

To conclude this section, let's discuss how to document functions. A function's documentation, often called a **docstring**, provides a concise description of what the function does, how it should be used, and any important details about its inputs and outputs.

A docstring is added by placing a string expression immediately before the function definition. Once defined, it can be accessed in the same way as the documentation for built-in functions: type the function's name in the REPL after pressing `?`, or directly hover over the function name if you're using VS Code.[3]

STANDARD FORM

```
"This function is written in a standard way. It takes a number and adds two to it."
function add_two(a)
   a + 2
end
```

COMPACT FORM

```
"This function is written in a compact form. It takes a number and adds two to it."
add_two(a) = a + 2
```

For further details about docstrings, see the corresponding section in the official documentation.

FOOTNOTES

[1.] The method to call a function actually depends on the **module** in which it's defined, and whether this module has been "imported" or "used". We won't cover modules in this book. However, they're essential when working on large projects, as each module operates as an independent workspace with its own variables. In fact, every new session in Julia defines a module called `Main` in which you're writing code.

[2.] Anonymous functions are also known as *lambda functions* in other programming languages.

[3.] See here for an example in VS Code.

3d. Variable Scope & Relevance of Functions

Martin Alfaro
PhD in Economics

INTRODUCTION

Variable scope refers to the code block in which a variable is accessible. The concept allows us to distinguish between **global variables**, which are accessible in any part of the code, and **local variables**, which are confined to specific blocks like functions or loops. The existence of scopes determines that the same variable `x` could refer to different objects, depending on where it's called.

When it comes to functions, Julia adheres to specific rules for variable scope. Specifically, given a variable `x` defined outside a function:

- If a new variable `x` is defined inside a function or is passed to a function as an argument, this `x` is considered *local* to that function. Moreover, any reference to `x` within the function refers to this local variable, with no connection to the `x` defined outside the function.
- If a function neither defines a new `x` nor `x` is a function argument, references to `x` inside the function point to the variable defined outside the function (i.e., the global `x`).

In this section, we'll show how these rules work in practice.

GLOBAL AND LOCAL VARIABLES

A variable that's local to a function exists solely within that function's scope. This means that, once the function finishes executing, these variables cease to exist. Consequently, any attempt to reference them outside the function will raise an error.

Variables local to a function encompass:

1. the function arguments,
2. the variables defined in the function body.

Any other variable in a function that's not *i)* or *ii)* necessarily refers to a global variable.

Recognizing whether a variable is local or global is crucial for predicting how a program behaves. Indeed, a local variable may share the same name as a global one, without them being related. The following examples help clarify the differences between global and local variables in practice.

LOCAL VARIABLES

```
x = "hello"

function foo(x)         # 'x' is local, unrelated to 'x = hello' above
    y = x + 2           # 'y' is local, 'x' refers to the function argument

    return x,y
end
```

```
julia> foo(1)
1                   # local x
3                   # local y
julia> x
"hello"
julia> y
ERROR: UndefVarError: y not defined
```

GLOBAL VARIABLES

```
z = 2

function foo(x)
    y = x + z           # 'x' refers to the function argument, 'z' refers to the global

    return x,y,z
end
```

```
julia> foo(1)
1                   # local x
3                   # local y
2                   # global z
julia> x
ERROR: UndefVarError: x not defined
julia> z
2
```

THE ROLE OF FUNCTIONS

In good programming practice, **functions** are best conceived **as self-contained mini-programs**, each designed to perform a specific and well-defined **task**. When a function successfully captures the implementation of that task, it becomes reusable: we can apply the same operation to different inputs or objects, thus avoiding code duplication. This reusability is one of the key reasons functions are such powerful abstractions. By defining a task once, we can rely on it consistently throughout a program, which not only reduces redundancy but also makes the overall structure easier to maintain and reason about.

To achieve this, a function must clearly express both the logic required to perform the task and the data necessary to accomplish it. Within this perspective, local variables simply act as temporary placeholders that help articulate the mechanics of that task. [1] Since their meaning is tied exclusively to the function's internal process, it's natural that local variables can't be accessed from outside. Moreover, this ensures that the function only interacts with the rest of the program through its inputs and outputs, thereby preserving its integrity as a self-contained well-defined task.

To illustrate this view of functions, consider a variable `x`, along with another variable `y` computed by transforming `x` through a function `f`. In particular, assume a transformation that doubles `x`, so that `y = 2 * x`. The following are two approaches to calculating `y`.

APPROACH 1

```
x          = 3

double()   = 2 * x
y          = double()
```

APPROACH 2

```
x          = 3

double(x)  = 2 * x
y          = double(x)
```

APPROACH 2 - EQUIVALENT

```
x          = 3

double(🐒) = 2 * 🐒
y          = double(x)
```

The function in Approach 1 relies on the global variable `x`. This practice is highly discouraged for several reasons. Firstly, it prevents the reusability of the function, as it's specifically designed to double the global variable `x`, rather than acting as a mini-program that doubles *any* variable.

Second, the inclusion of the global variable `x` compromises the function's self-containment, as the function's output depends on the value of `x` at the moment of execution. If you work on a long project, this will turn the code prone to bugs.

Lastly, global variables have a detrimental impact on performance, a topic we'll study later in the book. In fact, global variables in Julia are a direct performance killer.

In contrast, Approach 2 refers to `x` as a local variable. This `x` is unrelated to the global variable `x`—it simply serves as a label to identify the variable to be doubled. Indeed, we could've replaced `x` with any other label, as demonstrated in Approach 3 through the monkey emoji, `🐒`.

By avoiding referencing any variable outside its scope, Approach 2 makes the function self-contained. This allows users to easily anticipate the consequence of executing `double` through a simple inspection of the function, eliminating the need to review the entire codebase. Thus, Approach 2 aligns with the interpretation of a function as a self-contained mini-program: the function embodies the task of doubling a variable, turning the function reusable and applicable to any variable. In this context, applying `double` to the global variable `x` becomes just one possible application.

RECOMMENDATIONS FOR THE USE OF FUNCTIONS

Structuring code around functions offers numerous advantages. However, to fully realize these benefits, users must adhere to certain principles when writing code. This section outlines a few of them and should be considered as a mere introduction to the subject. The topic will be investigated further when we explore high performance.

AVOID GLOBAL VARIABLES IN FUNCTIONS

Global variables are strongly discouraged. This is not only due to the reasons mentioned previously, but also because they have a devastating impact on performance in Julia. The easiest solution to this issue is to pass global variables as function arguments. This practice will actually become second nature once you start viewing functions as self-contained mini-programs. Specifically, by adopting this perspective, you'll conceive local variables as labels to describe a task, rather than references to global variables. This shift in mindset can help you write more efficient and maintainable code.

AVOID REDEFINING VARIABLES WITHIN FUNCTIONS

The suggestion applies to both local variables and function arguments. Redefining these variables can have several disadvantages, including reduced code readability and potential performance degradation. Therefore, it's recommended that you define new variables instead of redefining existing ones. This approach is demonstrated in the following example.

NOT RECOMMENDED

```
function foo(x)
   x      = 2 + x          # redefines the argument

   y      = 2 * x
   y      = x + y          # redefines a local variable
end
```

PREFERRED

```
function foo(x)
   z      = 2 + x           # new variable

   y      = 2 * x
   output = z + y           # new variable
end
```

(OPTIONAL) - Another Issue of Redefining Variables

MODULARITY

We've emphasized the importance of viewing functions as self-contained mini-programs, designed to perform specific tasks. This perspective leads us to highlight the importance of **modularity**: the practice of breaking down a program into multiple small functions, each with its own distinct purpose, inputs, and outputs.

The primary benefit of modularity is the ability to work with independent code blocks. By keeping these blocks separate, we can decompose complex problems into multiple manageable tasks, making it easier to test and debug code. Additionally, modularity makes it possible to eventually improve or substitute parts of the code, without breaking the entire program.

A helpful way to understand this principle is by considering the analogy of building a Lego minifigure. In the first step, multiple blocks are created independently, each representing a specific part of the figure, such as the legs, torso, arms, and head. Then, in the second stage, these individual blocks are brought together and assembled into an integrated minifigure.

This two-step approach offers several advantages. By focusing on each block individually, we can concentrate and refine each part without worrying about the entire structure. Additionally, it provides great flexibility: since each block is created independently, we can modify specific blocks without having to rebuild the entire figure. For instance, if we want to change the figure's head, we can simply swap out the corresponding block, without starting from scratch.

The principle of modularity is closely tied to the suggestion of writing short functions. Some proponents even argue that functions should be limited to fewer than five lines of code Indeed, entire books have been written based on this principle. Although this viewpoint may be considered rather extreme, it clearly emphasizes the advantages of avoiding lengthy functions.

(OPTIONAL) - Example of Modularity

FOOTNOTES

[1] Local variables play a similar role to integration variables in math. Formally, $\mathrm{d}t$ in $\int f\,(t)$ for some function f is simply a symbol indicating over which variable we're integrating. The integral could be equivalently expressed using any other integration variable, such as x in $\int f\,(x)\,\mathrm{d}x$.

3e. Map and Broadcasting

Martin Alfaro
PhD in Economics

INTRODUCTION

This section introduces operations on **iterable collections**, a class of data structures whose elements can be accessed sequentially. Common examples include vectors, tuples, and ranges.

A general way to process iterable collections is by using for-loops, which we'll study later. For now, our focus is on techniques that allow us to apply operations element-wise, without explicitly writing for-loops.

The first approach covered is the `map` function. This applies a given function to each element of a collection, producing a new collection with the transformed values. After this, we'll shift our focus to a fundamental technique in Julia known as **broadcasting**. Its distinctive syntax, which involves appending a dot `.` to the function/operator, makes it easily identifiable throughout the code.

Broadcasting enables the application of functions and operators element-wise. When two collections of the same size are involved, it pairs corresponding elements and applies the operation to each pair. Broadcasting also supports combinations of scalars and same-size collections, where the scalar is expanded to match the collection's size.

> **Broadcasting vs Vectorization**
>
> The terms **broadcasting** and **vectorization** will be used interchangeably throughout the book. Strictly speaking, they're not equivalent: the term vectorization applies when arrays have the same size, while broadcasting is an extension that allows for scalars.

> ⚠ **Warning!**
>
> Later in the book, we'll explore **for-loops** as an alternative approach to transforming arrays. Several languages strongly recommend vectorizing operations to improve speed, instead highly discouraging for-loops. **Such advice does not apply to Julia**. In fact, when it comes to optimizing code, for-loops are often the key to achieving high performance.
>
> Considering this, the main advantage of vectorization in Julia is to streamline code without sacrificing speed.

THE "MAP" FUNCTION

The `map` function is available in most programming languages, although with different names. It's designed to transform collections by applying a function to each of their elements.

Depending on the number of inputs required, `map` can be applied in two different ways. In its simplest form, it takes a single-argument function `foo` and a collection `x`. The syntax is then `map(foo,x)`, returning a new collection with `foo(x[i])` as *i*-th element. A common practice is to use anonymous functions in place of `foo`, as illustrated below.

SINGLE-ARGUMENT FUNCTION

```
x          = [1, 2, 3]

output     = map(log, x)
equivalent = [log(x[1]), log(x[2]), log(x[3])]
```

```
julia> output
3-element Vector{Float64}:
 0.0
 0.693147
 1.09861
julia> equivalent
3-element Vector{Float64}:
 0.0
 0.693147
 1.09861
```

SINGLE-ARGUMENT ANONYMOUS FUNCTION

```
x          = [1, 2, 3]

output     = map(a -> 2 * a, x)
equivalent = [2*x[1], 2*x[2], 2*x[3]]
```

```
julia> output
3-element Vector{Int64}:
 2
 4
 6
julia> equivalent
3-element Vector{Int64}:
 2
 4
 6
```

The second way to apply `map` arises when the function `foo` takes multiple arguments. When this is the case, the syntax is `map(foo, x, y)`, returning a new collection whose *i*-th element is `foo(x[i], y[i])`. Importantly, if the collections `x` and `y` have different sizes, `foo` **is applied element-wise**

until the shortest collection is exhausted. This rule applies even when `x` or `y` are scalars, in which case `map` would return a single element.

To demonstrate its use, let's consider the sum operation. Recall that `+` denotes both an operator (e.g., `2 + 3`) and a function (e.g., `+(2, 3)`). By using `+` in particular as a function, `map` can perform element-wise additions across multiple collections.

VECTORS OF SAME SIZE

```
x          = [ 1, 2, 3]
y          = [-1,-2,-3]

output     = map(+, x, y)        # `+` exists as both operator and function
equivalent = [+(x[1],y[1]), +(x[2],y[2]), +(x[3],y[3])]
```

```
julia> output
3-element Vector{Int64}:
 0
 0
 0

julia> equivalent
3-element Vector{Int64}:
 0
 0
 0
```

ANONYMOUS FUNCTION (EQUIVALENT)

```
x          = [ 1, 2, 3]
y          = [-1,-2,-3]

output     = map((a,b) -> a+b, x, y)
equivalent = [x[1]+y[1], x[2]+y[2], x[3]+y[3]]
```

```
julia> output
3-element Vector{Int64}:
 0
 0
 0

julia> equivalent
3-element Vector{Int64}:
 0
 0
 0
```

VECTORS OF DIFFERENT SIZE

```
x          = [ 1, 2, 3]
y          = [-1,-2]

output     = map(+, x, y)        # `+` exists as both operator and function
equivalent = [+(x[1],y[1]), +(x[2],y[2])]
```

```
julia> output
2-element Vector{Int64}:
 0
 0
julia> equivalent
2-element Vector{Int64}:
 0
 0
```

A VECTOR AND A SCALAR

```
x          = [ 1, 2, 3]
y          = -1

output     = map(+, x, y)        # `+` exists as both operator and function
equivalent = [+(x[1],y[1])]
```

```
julia> output
1-element Vector{Int64}:
 0
julia> equivalent
1-element Vector{Int64}:
 0
```

BROADCASTING

The function `map` can rapidly become unwieldy when dealing with complex functions or multiple arguments. This is where broadcasting comes into play, offering a more streamlined syntax.

Next, we'll explore the concept of broadcasting in a step-by-step manner. First, we'll show how it applies to collections of equal size, covering both functions and operators. After this, we'll demonstrate that broadcasting supports operations that combine scalars and collections, despite not supporting operations with collections of different sizes. In such cases, the scalar is treated as a vector that matches the size of the corresponding collections.

Unlike other programming languages, **broadcasting is an intrinsic feature of Julia**. This means that broadcasting is applicable to *any* function or operator, including user-defined ones.

BROADCASTING FUNCTIONS

Broadcasting expands the versatility of functions, allowing them to be applied element-wise to a collection. This feature is implemented by appending a dot ***after*** the name of the function, as in `foo.(x)`.

Remarkably, **any function `foo` has a broadcasting counterpart `foo.`**. This entails that broadcasting is automatically available for user-defined functions. Furthermore, it determines that broadcasting isn't restricted to numeric collections, but to any type of collection.

Similarly to `map`, broadcasting can be applied to both single- and multiple-argument functions. Each case warrants separate consideration.

As for single-argument functions, broadcasting `foo` over a collection `x` returns a new collection with `foo(x[i])` as its *i*-th element. The following examples demonstrate this.

THIRD-PARTY FUNCTION

```
# `log(a)` only applies to a scalar `a`
x          = [1,2,3]

output     = log.(x)
equivalent = [log(x[1]), log(x[2]), log(x[3])]
```

```
julia> output
3-element Vector{Float64}:
 0.0
 0.693147
 1.09861
julia> equivalent
3-element Vector{Float64}:
 0.0
 0.693147
 1.09861
```

USER-DEFINED FUNCTION

```
square(a)  = a^2     #user-defined function for scalar 'a'
x          = [1,2,3]

output     = square.(x)
equivalent = [square(x[1]), square(x[2]), square(x[3])]
```

```
julia> output
3-element Vector{Int64}:
 1
 4
 9
julia> equivalent
3-element Vector{Int64}:
 1
 4
 9
```

As for multiple-argument functions, suppose a function `foo` and collections `x` and `y`. Then, `foo.(x,y)` returns a new collection with `foo(x[i],y[i])` as its *i*-th element.

Importantly, **collections with different sizes aren't allowed**, establishing a clear contrast between broadcasting and `map`. The sole exception to this rule is when one of the objects is a scalar, as we'll see later.

Below, we provide several examples. The first example in particular makes use of the built-in function `max`, which provides the maximum value among its scalar arguments.

THIRD-PARTY FUNCTION

```
# 'max(a,b)' returns 'a' if 'a>b', and 'b' otherwise
x          = [0, 4, 0]
y          = [2, 0, 8]

output     = max.(x,y)
equivalent = [max(x[1],y[1]), max(x[2],y[2]), max(x[3],y[3])]
```

```
julia> output
3-element Vector{Int64}:
 2
 4
 8
julia> equivalent
3-element Vector{Int64}:
 2
 4
 8
```

USER-DEFINED FUNCTION

```
foo(a,b)   = a + b        # user-defined function for scalars 'a' and 'b'
x          = [-2, -4, -10]
y          = [ 2,  4,  10]

output     = foo.(x,y)
equivalent = [foo(x[1],y[1]), foo(x[2],y[2]), foo(x[3],y[3])]
```

```
julia> output
3-element Vector{Int64}:
 0
 0
 0
julia> equivalent
3-element Vector{Int64}:
 0
 0
 0
```

> **Broadcasting Applies to Any Function**

Broadcasting can be used not only with numeric functions, but functions that take other types as inputs. To illustrate, consider the built-in function `string`, which concatenates its arguments to form a sentence (e.g., `string("hello ","world")` returns `"hello world"`).

BROADCASTING WITH STRING COLLECTIONS

```
country = ["France"  , "Canada"]
is_in   = [" is in " , " is in "]
region  = ["Europe"  , "North America"]

output  = string.(country, is_in, region)
```

```
julia> output
2-element Vector{String}:
 "France is in Europe"
 "Canada is in North America"
```

Broadcasting Applies to Any Iterable Object

Broadcasting isn't exclusive to vectors. Indeed, it can be applied to any iterable collection, such as tuples and ranges.

TUPLES

```
x = (1, 2, 3)     # or simply x = 1, 2, 3
```

```
julia> log.(x)
(0.0, 0.693147, 1.09861)
julia> x .+ x
(2, 4, 6)
```

RANGES

```
x = 1:3
```

```
julia> log.(x)
3-element Vector{Float64}:
 0.0
 0.6931471805599453
 1.0986122886681098
julia> x .+ x
2:2:6
```

TUPLES AND RANGES

```
x = (1, 2, 3)     # or simply x = 1, 2, 3
y = 1:3
```

```
julia> x .+ y
3-element Vector{Int64}:
 2
 4
 6
```

BROADCASTING OPERATORS

It's also possible to **broadcast operators**, in which case they apply them element-wise across collections. Its use requires prepending a dot ***before*** the operator.

Classifying operators by the number of operands helps apply broadcasting, since it directly determines its syntax. Specifically, recall that *unary operators* are written as `<symbol>x`. Thus, for example, `.√x` broadcasts the operator `√`, while `.-x` broadcasts the operator `-`. Likewise, the syntax for *binary operators* is `x <symbol> y`. For instance, `x .+ y` computes the element-wise sum of vectors `x` and `y`, resulting in `[x[1]+y[1], x[2]+y[2], ...]`.

BROADCASTING UNARY OPERATORS

```
x      = [1, 2, 3]


output = .-x
```

```
julia> output
3-element Vector{Int64}:
 -1
 -2
 -3
```

BROADCASTING BINARY OPERATORS

```
x      = [ 1,  2,  3]
y      = [-1, -2, -3]

output = x .+ y
```

```
julia> output
3-element Vector{Int64}:
 0
 0
 0
```

BROADCASTING OPERATORS WITH SCALARS

Broadcasting thus far was applied with inputs of the same size. This is because collections of dissimilar size, such as `x = [1,2]` and `y=[3,4,5]`, aren't in general allowed.

One exception to this rule is when broadcasting applies to vectors of equal size combined with scalars. In such cases, scalars are treated as objects having the same size as the vectors, with all entries equal to the scalar. For example, given `x = [1,2,3]` and `y = 2`, the expression `x .+ y` produces the same result as defining `y = [2,2,2]` and then executing `x .+ y`. This is demonstrated below.

BROADCASTING

```
x      = [0,10,20]
y      = 5

output = x .+ y
```

```
julia> output
3-element Vector{Int64}:
  5
 15
 25
```

EQUIVALENT

```
x      = [0,10,20]
y      = [5, 5, 5]

output = x .+ y
```

```
julia> output
3-element Vector{Int64}:
  5
 15
 25
```

Broadcasting With a Single-Element String

The example based on strings presented above can be rewritten as follows.

BROADCASTING WITH STRINGS

```
country = ["France", "Canada"]
is_in   = " is in "
region  = ["Europe", "North America"]

output  = string.(country, is_in, region)
```

```
julia> output
2-element Vector{String}:
 "France is in Europe"
 "Canada is in North America"
```

COMBINING BROADCAST OPERATIONS

It's possible to simultaneously broadcast operators and functions. Given the pervasiveness of such operations, Julia provides the macro `@.` for an effortless application. The macro should be added at the beginning of the statement, and has the effect of automatically adding a "dot" to each operator and function found.

To demonstrate its use, consider adding two vectors element-wise, which we then transform by squaring the elements of the resulting vector.

BROADCASTING OPERATORS

```
x         = [1, 0, 2]
y         = [1, 2, 0]

temp      = x .+ y
output    = temp .^ 2
```

```
julia> temp
3-element Vector{Int64}:
 2
 2
 2

julia> output
3-element Vector{Int64}:
 4
 4
 4
```

BROADCASTING FUNCTIONS AND OPERATORS

```
x         = [1, 0, 2]
y         = [1, 2, 0]

square(x) = x^2
output    = square.(x .+ y)
```

```
julia> output
3-element Vector{Int64}:
 4
 4
 4
```

BROADCASTING VIA MACRO @.

```
x         = [1, 0, 2]
y         = [1, 2, 0]

square(x) = x^2
output    = @. square(x + y)
```

```
julia> output
3-element Vector{Int64}:
 4
 4
 4
```

BROADCASTING FUNCTIONS VS BROADCASTING OPERATORS

We've demonstrated that both functions and operators can be broadcast. This lets us implement operations in two distinct ways: either broadcast a function that operates on a single element or define a function that directly performs the broadcast operation.

The examples below demonstrate that the same output is obtained using either approach. For the illustration, suppose that the goal is to square each element of `x`.

FUNCTIONS WITH SINGLE-ELEMENT ARGUMENTS

```
x                = [1, 2, 3]

number_squared(a) = a ^ 2              # function for scalar 'a'
output            = number_squared.(x)
```

```
julia> output
3-element Vector{Int64}:
 1
 4
 9
```

FUNCTIONS WITH VECTOR AS ARGUMENT

```
x                = [1, 2, 3]

vector_squared(x) = x .^ 2              # function for a vector 'x'
output            = vector_squared(x)   # '.' not needed (it'd be redundant)
```

```
julia> output
3-element Vector{Int64}:
 1
 4
 9
```

While both approaches yield the same output, **defining a function that operates on a scalar is generally the better choice**. There are two main reasons for this claim.

First, a scalar function such as `number_squared(a)` can be flexibly applied to both individual values and collections: we can simply call the function directly for scalars or rely on its broadcast form for collections. This means the function itself remains general-purpose, without being tied to a specific type of input.

Second, the broadcasting syntax makes the programmer's intent explicit. Continuing with the example, writing `number_squared.(x)` clearly indicates that the operation is applied element-wise across the collection. By contrast, a function like `vector_squared(x)` hides this detail, leaving the reader to infer that the computation is performed element-wise.

BROADCASTING OVER ONLY ONE ARGUMENT

When we broadcast a function or operator over some vectors `x` and `y`, both objects are simultaneously iterated. Yet, there are instances where we want only one argument to vary while keeping the other fixed.

A typical scenario arises when checking whether elements from `x` match any values in a predefined list `y`. To illustrate this, let's first introduce the function `in(a, list)`, which determines whether the scalar `a` equals some element in the vector `list`. For instance, `in(2, [1,2,3])` evaluates to `true`, since `2` is contained in `[1,2,3]`.

Suppose now that, instead of a scalar `a`, we have a vector `x` and the goal is to verify whether *each* of the elements in `x` is present in `list = [1,2,3]`. Specifically, our aim is to verify if `1` belongs to `[1,2,3]`, if `2` belongs to `[1,2,3]`, and if `3` belongs to `[1,2,3]`. Below, we show that this operation can't be directly implemented via a simple broadcast version of `in`.

WRONG IMPLEMENTATION 1

```
x    = [1, 2]
list = [1, 2, 3]
```

```
julia> in.(x, list)
ERROR: DimensionMismatch: arrays could not be broadcast to a common size; got a dimension
with lengths 2 and 3
```

WRONG IMPLEMENTATION 2

```
x    = [1, 2, 4]
list = [1, 2, 3]
```

```
julia> in.(x, list)
3-element BitVector:
 1
 1
 0
```

In the first example, `in.(x, list)` errors because `x` and `list` should either have the same length or one of them be a scalar. In the second example, although the expression yields an output, it's not the intended result: it performs pairwise comparisons, thus checking whether `1==1`, `2==2`, and `4==3`.

Intuitively, we need a mechanism to inform Julia that `list` should be treated as a single fixed argument while iterating over `x`. This can be accomplished in two different ways. First, we can enclose `list` in a collection (e.g., a vector or a tuple).

While it's possible to use any collection to wrap `list`, we'll see in Part II of the book that there's some performance penalty involved when creating vectors. Consequently, **you should prefer tuples when implementing this approach**. This requires inserting `(list,)` as the function argument, which defines a tuple whose only element is `list`. [1] Alternatively, we can rely on the function `Ref`. This requires expressing the function argument as `Ref(list)`, which makes `list` be treated as a single element.

Below, we demonstrate each approach. For completeness, we also show the case where `list` is wrapped in a vector.

VECTOR AS SINGLE ELEMENT

```
x      = [2, 4, 6]
list   = [1, 2, 3]        # 'x[1]' equals the element 2 in 'list'

output = in.(x, [list])
```

```
julia> output
3-element BitVector:
 1
 0
 0
```

TUPLE AS SINGLE ELEMENT

```
x      = [2, 4, 6]
list   = [1, 2, 3]        # 'x[1]' equals the element 2 in 'list'

output = in.(x, (list,))
```

```
julia> output
3-element BitVector:
 1
 0
 0
```

REF

```
x      = [2, 4, 6]
list   = [1, 2, 3]        # 'x[1]' equals the element 2 in 'list'

output = in.(x, Ref(list))
```

```
julia> output
3-element BitVector:
 1
 0
 0
```

The vector returned in each case has a type `BitVector`, where `1` corresponds to `true` and `0` to `false`. The result could equivalently be expressed as `[true, false, false]`. This reflects that `x[1]` equals `2`, where `2` belongs to `list`, whereas `x[2]` and `x[3]` don't equal any element in `list`.

While the previous example focused on functions, the same principle extends to broadcast operators. This can be illustrated through the `∈` operator, which serves a similar purpose to the `in` function: it determines whether a particular element exists within a collection.[2]

TUPLE AS SINGLE ELEMENT

```
x      = [2, 4, 6]
list   = [1, 2, 3]

output = x .∈ (list,)       # only 'x[1]' equals an element in 'list'
```

```
julia> output
3-element BitVector:
 1
 0
 0
```

REF

```
x      = [2, 4, 6]
list   = [1, 2, 3]

output = x .∈ Ref(list)     # only 'x[1]' equals an element in 'list'
```

```
julia> output
3-element BitVector:
 1
 0
 0
```

CURRYING AND FIXING ARGUMENTS (*OPTIONAL*)

Currying is a technique that transforms the evaluation of a function with multiple arguments into a sequence of functions, each evaluated with a single argument. [3] For instance, the curried form of `f(x,y)` would be `f(x)(y)`, providing an identical output.

Our interest in currying lies in its ability to simplify broadcasting: it enables the treatment of an argument as a single object, without the need to use `Ref` or encapsulate objects as vectors/tuples. The technique could seem confusing for new users. In particular, it requires a good understanding of functions as first-class objects, entailing that functions can be treated as variables themselves. My primary goal is that you can at least recognize the syntax of currying, and thus be able to read code that applies the technique.

We start by illustrating how currying can be applied in general.

NO CURRYING

```
addition(x,y)  = 2 * x + y
```

```
julia> addition(2,1)
5
```

CURRYING

```
addition(x,y)  = 2 * x + y

# the following are equivalent
curried1(x)    = (y -> addition(x,y))
curried2       = x -> (y -> addition(x,y))
```

```
julia> curried1(2)(1)
5
julia> curried2(2)(1)
5
```

FIXING AN ARGUMENT

```
addition(x,y)  = 2 * x + y
curried(x)     = (y -> addition(x,y))

# the following are equivalent
f              = curried(2)          # function of 'y', with 'x' fixed to 2
g(y)           = addition(2,y)
```

```
julia> f(1)
5
julia> g(1)
5
```

The key to understanding the syntax is that `curried(x)` is a function itself, with `y` as its argument. The second tab illustrates this clearly through the equivalence between `f = curried(2)` and `addition(2,y)`. These functions help us understand the logic behind curry, but are only useful for the specific case of `x=2`. Instead, `curried(x)` allows the user to call the function through `curried(x)(y)`, and so be used for any `x`.

As for broadcasting, any function `foo` in Julia can be broadcast through `f.`. And we've determined that `curried(x)` is a function just like any other. Therefore, `curried(x)` plays the same role as `foo`, and so we can broadcast over `y` for a fixed `x` through `curried(x).(y)`.

BROADCASTING WITH CURRYING

```
a              = 2
b              = [1,2,3]

addition(x,y) = 2 * x + y
curried(x)    = (y -> addition(x,y))   # 'curried(x)' is a function, and 'y' its argument
```

```
julia> curried(a).(b)
3-element Vector{Int64}:
 5
 6
 7
```

BROADCASTING WITH A FIXED ARGUMENT

```
a             = 2
b             = [1,2,3]

addition(x,y) = 2 * x + y
curried(x)    = (y -> addition(x,y))

#the following are equivalent
f             = curried(a)            # 'foo1' is a function, and 'y' its argument
g(y)          = addition(2,y)
```

```
julia> f.(b)
3-element Vector{Int64}:
 5
 6
 7
julia> g.(b)
3-element Vector{Int64}:
 5
 6
 7
```

Let's now explore how the currying technique can help treat a vector as a single element in broadcasting. To illustrate this, consider the function `in` used previously. This function has a built-in curried version, which can be applied through `in(list).(x)` for vectors `list` and `x`. To better demonstrate its usage, the following example compares an implementation with `Ref`, the built-in curried `in`, and our own curry implementation.

BROADCASTING WITH REF

```
x    = [2, 4, 6]
list = [1, 2, 3]
```

```
julia> n.(x,Ref(list))
3-element BitVector:
 1
 0
 0
```

BROADCASTING WITH CURRY (USER-DEFINED)

```
x    = [2, 4, 6]
list = [1, 2, 3]

our_in(list_elements) = (x -> in(x,list_elements))     # 'our_in(list_elements)' is a
function
```

```
julia> our_in(list).(x) # it broadcasts only over 'x'
3-element BitVector:
 1
 0
 0
```

BROADCASTING WITH CURRY (BASE JULIA)

```
x    = [2, 4, 6]
list = [1, 2, 3]
```

```
julia> in(list).(x) # similar to 'our_in'
3-element BitVector:
 1
 0
 0
```

FOOTNOTES

[1.] Recall that tuples with a single element must be written with a trailing comma, as in `(list,)`. Instead, `(list)` is interpreted as the variable `list`, and hence treated as a vector.

[2.] `∈` can also be applied as a function, with its syntax mirroring that of `in`. Thus, `∈(a, list)` for a scalar `a` yields the same results as `in(a, list)`.

[3.] The term honors the mathematician Haskell Curry, not the spice!

4a. Overview and Goals

Martin Alfaro
PhD in Economics

Chapter 4 continues our analysis of core programming tools. After studying functions in Chapter 3, we now turn to **conditional statements** and **for-loops**.

Conditional statements require first understanding conditions. These are formally defined as Boolean values, which can only take on the values 1 or 0 (equivalently, true or false). We do this in Section 4b, where we present the logical apparatus to define conditions.

With this foundation in place, Section 4c explores conditional statements themselves. They give you control over program flow, letting you execute specific blocks of instructions only when a condition is satisfied. The most common form is the if-then structure: the program runs a particular block of code if a condition is true, and skips that block if it's false.

In Section 4d, we introduce for-loops. They let you execute the same block of code repeatedly, treating each element in a list as a different input. For-loops in Julia are key to unlocking high performance, explaining why they'll become increasingly important as you progress through Part II of this textbook. This behavior contrasts with other programming languages used in scientific computing, such as MATLAB, Python, or R. For-loops there are treated as a performance bottleneck, encouraging users to instead apply vectorized operations.

The goal of Chapter 4 is to introduce foundational concepts and syntax. We won't yet explore the most effective ways to apply these tools or optimize their use. Similarly, we'll defer the analysis of advanced techniques that combine functions, conditions, and for-loops together. In Chapter 5, you'll see how combining these three elements gives rise to powerful patterns in Julia such as in-place functions.

Overall, your focus at this point should be on learning the approaches and syntax for expressing conditions and for-loops.

4b. Conditions

Martin Alfaro
PhD in Economics

INTRODUCTION

This section lays the basics for incorporating conditions into our programs. Formally, a **condition** is any expression built from functions or operators that evaluates to either true or false. For instance, the comparison `x > y` is a condition that checks whether `x` is greater than `y`.

To get the most out of this section, it'll help keep in mind the classification of operators discussed here. This establishes that operators can be categorized according to their number of operands. Specifically, **unary operators** act on a single operand and precede it (i.e. `<operator>x`), whereas **binary operators** take two operands and are placed between them (i.e. `x <operator> y`).

DEFINING CONDITIONS

Conditions are represented as values with type `Bool`, evaluating to either `true` or `false`. These values are internally represented as integers restricted to `1` and `0`.

The representation of Boolean values in the REPL varies depending on their dimension: scalar `Bool` values are displayed as `true` and `false`, while `Bool` vectors use `1` and `0`. This is illustrated below.

BOOLEAN VARIABLE

```
x = 2

#`y` equals `true` or `false`
y = (x > 0)
```

```
julia> y
true
```

BOOLEAN VECTOR

```
x = 2

#'z' element equals 'true' or 'false', represented by 1 or 0
z = [x > 0, x < 0]
```

```
julia> z
2-element Vector{Bool}:
 1
 0
```

> ⚠ **Warning!**
>
> Parentheses are optional when writing single conditions, allowing us to write `y = x > 0` rather than `y = (x > 0)`. Nonetheless, the former syntax is somewhat ambiguous, with the risk of being potentially misinterpreted as `(y = x) > 0`. To avoid confusion, it's a good practice to always include parentheses. This is especially true when working with multiple conditions, where outcomes can be drastically altered.

The condition in the previous example was defined via the operator `>`. More generally, conditions accept **comparison operators**, which are *binary operators* that compare values of various types (e.g., numbers and strings). The next list defines the most common ones.

Comparison Operator	Meaning
`x == y`	equal
`x ≠ y` or `x != y`	not equal
`x < y`	lower than
`x ≤ y` or `x <= y`	lower or equal than
`x > y`	greater than
`x ≥ y` or `x >= y`	greater or equal than

> **Notation**
>
> The non-standard characters appearing in the table can be written using tab completion:
>
> - `≠` via `\ne`, which stands for "not equal",
> - `≥` via `\ge`, which stands for "greater or equal",
> - `≤` via `\le`, which stands for "lower or equal".

> **Comparison operators Are Also Functions**
>
> As we showed here, several symbols can be used as both functions and operators. This applies in particular to comparison operators. For instance, the following expressions are all valid:

OPERATORS AS FUNCTIONS

```
==(1,2)      # same as 1 == 2
 ≠(1,2)      # same as 1 ≠ 2
 ≥(1,2)      # same as 1 ≥ 2
>=(1,2)      # same as 1 ≥ 2
 >(1,2)      # same as 1 > 2
```

LOGICAL OPERATORS

Logical operators allow us to combine multiple conditions into a single one. Formally, they take `Bool` expressions as their operands, and return another `Bool` as their output. The following are the main logical operators used in Julia.

Logical Operator	Meaning
`x && y`	`x` and `y`
`x || y`	`x` or `y`
`!x`	negation of `x`

Notice that `&&` and `||` follow the syntax rules of *binary* operators.

&& AND || OPERATORS

```
x = 2
y = 3

# are both variables positive?
z1 = (x > 0) && (y > 0)

# is either `x` or `y` (or both) positive?
z2 = (x > 0) || (y > 0)
```

```
julia> z1
true
julia> z2
true
```

The "not" operator, represented by `!`, is another operator taking conditions as their operands. This is a unary operator that inverts a condition's value, changing `true` to `false` and vice versa. To use it, you simply place `!` at the start of the condition, before the parentheses.

As an illustration, the variables `y1` and `y2` below become equivalent via `!`.

! OPERATOR

```
x = 2

# is `x` positive?
y1 = (x > 0)

# is `x` not lower than zero nor equal to zero? (equivalent)
y2 = !(x ≤ 0)
```

```
julia> y1 #identical output as 'y2'
true
julia> y2
true
```

LOGICAL OPERATORS AS SHORT-CIRCUIT OPERATORS

A key feature of `&&` and `||` is that they're **short-circuit operators**. This means that, once an operand is evaluated, the remaining operands are evaluated only if the previous operands didn't establish the truth or falseness of the expression. Specifically:

- `(x > 0) || (y > 0)`
  This expression is true when *at least one condition* is satisfied. Thus, Julia begins by analyzing `x > 0`. If this expression is true, it immediately returns `true`, without evaluating any subsequent expression. Only when `x > 0` is false will Julia evaluate `y > 0`.

- `(x > 0) && (y > 0)`
  This expression is true if *both conditions* are satisfied. Thus, Julia begins by analyzing `x > 0`. If this expression is false, it immediately returns `false`, without evaluating any subsequent expression. Only when `x > 0` is true will Julia evaluate `y > 0`.

Since not all operands are always evaluated, it's possible to get a result even if some operands contain invalid expressions. This is shown in the next example, where we include invalid Julia code as a condition.

&&

```
x = 10
```

```
julia> (x < 0) && (this-is-not-even-legitimate-code)
false
julia> (x > 0) && (this-is-not-even-legitimate-code)
ERROR: UndefVarError: `this` not defined
```

||

```
x = 10
```

```
julia> (x > 0) || (this-is-not-even-legitimate-code)
true
julia> (x < 0) || (this-is-not-even-legitimate-code)
ERROR: UndefVarError: `this` not defined
```

PARENTHESES IN MULTIPLE CONDITIONS

The inclusion of parentheses isn't crucial when working with only two conditions. This is because expressions like `(x > 0) && (y > 0)` can be safely written as `x > 0 && y > 0`, without much risk of confusion.

On the contrary, when dealing with three or more conditions, the lack of parentheses can drastically impact the expected behavior of an expression. The following example illustrates this point.

WITHOUT PARENTHESES

```
x = 5
y = 0
```

```
julia> x < 0 && y > 4 || y < 2
true
```

WITH PARENTHESES

```
x = 5
y = 0
```

```
julia> (x < 0) && (y > 4 || y < 2)
false
```

WITH PARENTHESES

```
x = 5
y = 0
```

```
julia> (x < 0 && y > 4) || (y < 2)
true
```

In the example, the expression without parenthesis is equivalent to the last tab's, since `&&` **has higher precedence than** `||` in Julia: when both `&&` and `||` are used, `&&` will be evaluated first.

To avoid confusion when more than two conditions are incorporated, **we'll always add parentheses**. This improves readability and spares us the need to memorize specific rules. The next optional subsection covers Julia's precedence rules in more detail. However, if you'll consistently enclose

conditions in parentheses, you can safely skip it.

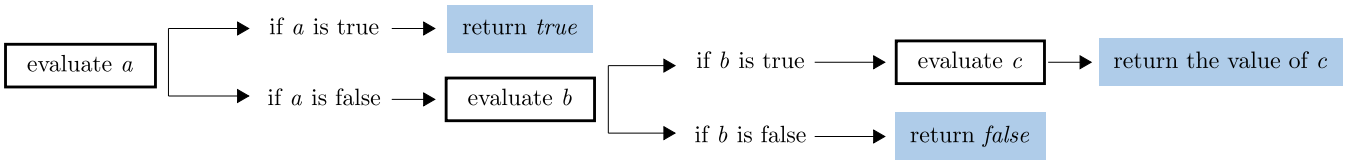
MULTIPLE CONDITIONS WITHOUT PARENTHESES (*OPTIONAL*)

To simplify the explanation, let's focus on cases with three conditions. These conditions will be represented through `Bool` variables `a`, `b`, and `c`, with each variable possibly representing expressions like `x > 0`.

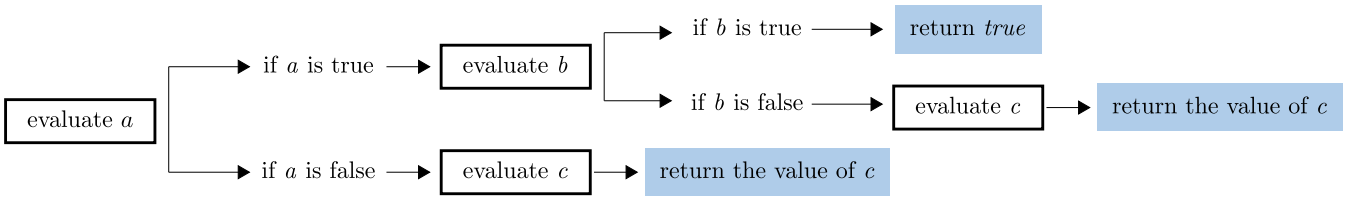
To understand how Julia groups three conditions without parentheses, there are two rules you need to know. First, `&&` has higher precedence than `||`. This means that `a && b || c` is equivalent to `(a && b) || c`, whereas `a || b && c` is equivalent to `a || (b && c)`. Second, `&&` and `||` are short-circuit operators. Thus, `a && b` immediately returns `false` if its first operand `a` is false, without evaluating the second operand `b`. Likewise, `a || b` returns `true` if the first operand `a` is true, without evaluating the second operand `b`.

The following diagrams describe the process for evaluating `a && b || c` and `a || b && c`, based on these two rules.

CASE 1: `a || b && c` is equivalent to `a || (b && c)`



CASE 2: `a && b || c` is equivalent to `(a && b) || c`



To illustrate the rules in practice, let's go through several examples that combine true/false values for `a`, `b`, and `c`. In these examples, we'll use the invalid expression `does-not-matter`. This is to emphasize that some conditions aren't necessarily evaluated thanks to the short-circuit behavior of `&&` and `||`.

APPLICATIONS OF CASE 1

```
julia> false || true && true
true
julia> false || true && false
false
julia> true || does-not-matter
true
```

APPLICATIONS OF CASE 2

```
julia> true && false || true
true
julia> true && false || false
false
julia> false && does-not-matter || true
true
```

FUNCTIONS TO CHECK VECTOR CONDITIONS: "ALL" AND "ANY"

Julia provides two built-in functions called `all` and `any` to evaluate multiple conditions in a collection. The function `all` returns `true` if *every condition* is true, whereas `any` returns `true` if *at least one condition* is true. The functions **require either directly specifying the conditions through a Boolean vector or defining the condition to check through a function**. Next, we cover each case separately.

VECTORS REPRESENTING MULTIPLE CONDITIONS

In the following, we demonstrate the syntax of `all` and `any` when they take a Boolean vector as their argument.

FUNCTIONS 'ALL' AND 'ANY'

```
a                = 1
b                = -1

# function indicating whether all elements satisfy the condition
are_all_positive = all([a > 0, b > 0])

# function indicating whether at least one element satisfies the condition
is_one_positive  = any([a > 0, b > 0])
```

```
julia> are_all_positive
false
julia> is_one_positive
true
```

The function `all` returns `true` only when all the conditions are satisfied, thus requiring that each vector's entry is positive. This doesn't hold in the example, since `b = -1`. Conversely, `any` returns `true` when at least one of the conditions holds, thus requiring at least one element in the vector to be positive. This is satisfied in the example, since `a = 1`.

As we indicated, `all` and `any` do not support passing multiple conditions as separate arguments. This entails that expressions like `all(a > 0, b > 0)` aren't allowed. Nevertheless, this restriction actually makes the functions more flexible, as they **enable the use of broadcasting operations for checking multiple conditions**. For example, the following code snippet implements the same operations as above, but through a vector `x`.

BROADCASTING WITH 'ALL' AND 'ANY'

```
x               = [1, -1]

are_all_positive = all(x .> 0)
is_one_positive  = any(x .> 0)
```

```
julia> are_all_positive
false
julia> is_one_positive
true
```

FUNCTIONS TO REPRESENT MULTIPLE CONDITIONS

In addition to expressing conditions through vectors, `all` and `any` allow **passing a function to represent the condition to check**. The syntax for each case is `all(<function>, <array>)` and `any(<function>, <array>)`, where `<function>` can be an anonymous function. The following examples demonstrate how to implement `all(x .> 0)` and `any(x .> 0)` using this approach.

'ALL' AND 'ANY' WITH FUNCTION AS ARGUMENT

```
x               = [1, -1]

are_all_positive = all(i -> i > 0, x)
is_one_positive  = any(i -> i > 0, x)
```

```
julia> are_all_positive
false
julia> is_one_positive
true
```

By passing a function as an argument, `all` and `any` can additionally be employed **to evaluate the same condition across multiple vectors**. This is achieved by broadcasting `all` and `any`.

BROADCASTING 'ALL' AND 'ANY'

```
x                = [1, -1]
y                = [1,  1]

are_all_positive = all.(i -> i > 0, [x,y])
is_one_positive  = any.(i -> i > 0, [x,y])
```

```
julia> are_all_positive # all elements in 'y' are positive, but not in 'x'
2-element BitVector:
 0
 1
julia> is_one_positive # at least one element of 'x' or 'y' is positive
2-element BitVector:
 1
 1
```

4c. Conditional Statements

Martin Alfaro
PhD in Economics

INTRODUCTION

Programs routinely must choose between alternative operations depending on how their execution unfolds. To handle these possibilities, programs rely on conditional statements, which enable the execution of specific code blocks only when certain conditions are satisfied.

Each code block within a conditional statement is referred to as a **branch**. Based on the number of branches, there are three types of conditional statements:

- **if-then statements**, which consist of a **single branch**. They run a specific operation only if a condition is met, with no operation performed otherwise.
- **if-else statements**, which consist of **two branches**. They run a specific operation if a condition is met, and another if the condition isn't satisfied.
- **if-else-if statements**, which consist of **three or more branches**. They comprise a series of conditions, with each branch executing a different code block.

Next, we cover each case in depth. The presentation builds heavily on the operators introduced in the previous section. If you haven't read it, I strongly recommend doing so before continuing.

IF-THEN STATEMENTS

If-then statements execute an operation only when a condition is met, doing nothing instead when the condition isn't satisfied. These statements can be constructed via:

- the `if` keyword,
- the logical operator `&&`,
- the logical operator `||`.

The approach via `if` keyword is self-explanatory. As for the logical operators, `&&` executes an operation if the condition is true, whereas `||` does it when the condition is *not* satisfied. In fact, `||` is equivalent to `&&` with its condition negated.

Below, we illustrate the syntax for each form. The examples rely on the `println` function, which displays the text passed as argument in the REPL.

IF

```
x = 5

if x > 0
    println("x is positive")
end
```

```
x is positive
```

&&

```
x = 5

(x > 0) && (println("x is positive"))
```

```
x is positive
```

||

```
x = 5

(x ≤ 0) || (println("x is positive"))
```

```
x is positive
```

Note that if-then statements imply that no action would've been taken if, for instance, we had used `x = -1` as a condition. It's only when `x > 0` that `println` is executed.

The `if` approach offers the most flexibility, making it ideal for complex conditional statements. However, it's somewhat verbose for simple conditional statements. For these cases, `&&` and `||` are preferred, as they help us keep the code streamlined.

A common application of `||` is in conjunction with the function `error` to handle errors. This construct immediately interrupts the script's execution when the condition isn't satisfied, displaying the provided message as the argument of `error`. For instance, consider a function `foo(x)` that requires non-negative values for `x`. To enforce this, you could include `x > 0 || error("x must be positive")` at the beginning of the function. If `foo(x)` is then called with a non-positive `x`, it'll immediately halt its execution and print the error message "x must be positive" in the REPL.

> **Remark**
>
> Note that `&&` and `||` behave like if-then statements when they combine a condition with an operation. This is different from using them exclusively with conditions, where all operands would be `Bool` values.

IF-ELSE STATEMENTS

If-else statements execute an operation when a condition is true and another operation when the condition is false. There are two forms to write these statements.

The first one is the most flexible and uses the `if` keyword in combination with `else`. The second method relies on the so-called **ternary operator**, which requires the keywords `?` and `:` via the syntax `<condition> ? <operation if true> : <operation if false>`. This is referred to as *the* ternary operator because it's the only operator in most programming languages that takes three arguments.

We illustrate the syntax of both approaches below.

'IF' AND 'ELSE' KEYWORDS

```
x = 5

if x > 0
    println("x is positive")
else
    println("x is not positive")
end
```

```
x is positive
```

TERNARY OPERATOR

```
x = 5

x > 0 ? println("x is positive") : println("x is not positive")
```

```
x is positive
```

The function `ifelse` offers an alternative for constructing if-else expressions. This function takes three arguments: a condition, an expression to be evaluated if the condition is true, and another one if false. [1]

One advantage of using a function to express an if-else statement is the possibility of broadcasting. This is particularly helpful when creating vectors whose elements vary according to a condition, as demonstrated below.

BROADCASTING "IFELSE"

```
x                   = [4, 2, -6]

x_absolute_value    = ifelse.(x .≥ 0, x, -x)
```

```
julia> x_absolute_value
3-element Vector{Int64}:
 4
 2
 6
```

> **Remark**
>
> Broadcasting `ifelse` **requires broadcasting both** `ifelse` **and the condition**. For instance, the example would throw an error if we execute `ifelse.(x>0, true, false)`. The reason is that `x > 0` checks if the *entire vector* is positive, which is an operation undefined in Julia.

IF-ELSE-IF STATEMENTS

So far, we've analyzed conditional statements that handle only two possibilities: one when the condition is met, and another if it isn't. This binary structure can be limiting when multiple alternatives need to be considered. Basically, it forces you to nest several `if` and `else` statements to manage additional conditions.

To simplify this process, we can use the `elseif` keyword to extend the `if` and `else` approach. This is illustrated below.

NON-EXHAUSTIVE CONDITIONS

```
x = -10

if x > 0
    println("x is positive")
elseif x == 0
    println("x is zero")
end
```

EXHAUSTIVE CONDITIONS

```
x = -10

if x > 0
    println("x is positive")
elseif x == 0
    println("x is zero")
else
    println("x is negative")
end
```

```
x is negative
```

The first examples showcase the benefits provided by the approach. Specifically, `elseif` eliminates the need to explicitly specify actions for every possible scenario. Instead, it performs an action if `x` is positive, another action if `x` is zero, but it does nothing otherwise. In contrast, using `if` and `else` would require an exhaustive approach, where all possible contingents must be accounted for.

Likewise, the second example demonstrates that combinations of the `if`, `else`, and `elseif` keywords are possible.

FOOTNOTES

[1] The function `ifelse` does *not* behave as a **short-circuit operator**. This means that all the operations are computed, despite that only one of them will ultimately be returned as output.

4d. For-Loops

Martin Alfaro
PhD in Economics

INTRODUCTION

A key feature of programming is its ability to automate repetitive tasks, rendering **for-loops** crucial in coding. They let you execute the same block of code multiple times, treating each element in a list as a different input.

Although all languages provide some form of looping, their performance characteristics and idiomatic use vary widely across them. For instance, many high-level languages designed for scientific computing (e.g., MATLAB, Python, and R), programmers are typically encouraged to avoid explicit for-loops when performance matters. Instead, these languages favor vectorized operations (which apply a single operation to an entire array at once) or specialized library calls (which use pre-optimized routines). The design reflects a practical reality: for-loops in those languages are often slow, as their internal design lacks the necessary mechanisms to optimize them automatically.

In contrast, for-loops in Julia aren't only idiomatic but also fast, often matching or surpassing the performance of vectorized alternatives. As a result, mastering for-loops isn't just a matter of stylistic preference. Rather, it's an essential skill for writing efficient programs in Julia.

Part II of this book will examine how for-loops contribute to high performance. For now, our focus is on understanding the construct itself: its syntax, its variations, and the most common patterns for iterating over data structures.

SYNTAX

For-loops delimit their scope via the keywords `for` and `end`. To illustrate the syntax, consider the function `println(a)`, which evaluates `a` and displays its output in the REPL. In case `a` is a string, `println(a)` simply displays the word stored in `a`. The following script repeatedly applies `println` to display each word contained in a collection.

FOR-LOOP SYNTAX

```
for x in ["hello", "beautiful", "world"]
    println(x)
end
```

```
hello
beautiful
world
```

Remark

The keyword `in` can be replaced by `∈` or `=`, where `∈` can be written through tab completion using the command `\in`. Considering this, the following constructions are all equivalent.

IN

```
for x in ["hello", "beautiful", "world"]
    println(x)
end
```

∈

```
for x ∈ ["hello", "beautiful", "world"]
    println(x)
end
```

=

```
for x = ["hello", "beautiful", "world"]
    println(x)
end
```

Furthermore, we can employ any character or term to describe the iteration variable. For instance, we iterate below using `word`.

ALTERNATIVE NAME FOR ITERATION VARIABLE

```
for word in ["hello", "beautiful", "world"]
    println(word)
end
```

```
hello
beautiful
world
```

Based on this example, we can identify three components that characterize a for-loop:

- **A code block to be executed**: represented in the example by `println(x)`, which shows the value of `x` in the REPL.
- **A list of elements**: represented in the example by `["hello","beautiful","world"]`. This specifies the elements over which we'll apply the code block. The list can contain elements with any data type (e.g., strings, numbers, and even functions). The only requirement is that the list must be an **iterable object**, defined as a collection whose elements can be accessed individually. An example of iterable object is vectors, as in the example. However, we'll also introduce others that are most commonly used, such as ranges.

- **An iteration variable**: represented in the example by `x`. This serves as a label that holds the value of each element in the list during iteration. Iteration variables are local variables, with no significance outside the for-loop. Their sole purpose is to provide a convenient way to access and manipulate the elements of the list.

In the following sections, we'll explore different collections that are iterable and therefore can serve as lists. Furthermore, we'll show that these lists can comprise elements not immediately obvious. A typical example is functions, making it possible to apply different functions to the same object.

> **Always Wrap For-Loops in Functions**
>
> At this stage, we're still introducing fundamental concepts. Thus, we're presenting subjects in their simplest form for learning purposes. In particular, this explains why for-loops will be primarily written in the global scope.
>
> However, **you should always wrap for-loops in functions**. Executing for-loops outside a function severely degrades performance, and is additionally subject to different rules regarding variable scope. [1]

ITERATING OVER INDICES

So far, we've considered a simple list like `["hello", "beautiful", "world"]` to demonstrate how for-loops work. In real applications, however, manually specifying each element in a collection is impractical. Fortunately, when a list follows a predictable pattern (e.g., a sequence of numbers), we can simply describe the pattern that generates those elements.

Building on this insight, we'll next explore how to define ranges. They let users define a sequence of numbers, which is particularly useful to access elements of a collection through their indices.

RANGES

Ranges in Julia are defined via the syntax `<begin>:<steps>:<end>`, where `<begin>` represents the starting index and `<end>` the ending index. Likewise, `<steps>` sets the increment between values, defaulting to one when the term is omitted. We can also reverse the order of the sequence, by providing a negative value for `<steps>`. All this is demonstrated below.

RANGE WITH STEPS GIVEN

```
for i in 1:2:5
    println(i)
end
```

```
1
3
5
```

RANGE WITH REVERSE ORDER

```
for i in 3:-1:1
    println(i)
end
```

```
3
2
1
```

> **Remark**
>
> The application of ranges isn't limited to for-loops. They can also be used to define vectors by combining them with the `collect` function.

CREATING A VECTOR FROM A RANGE

```
x = collect(4:6)
```

```
3-element Vector{Int64}:
 4
 5
 6
```

ITERATING OVER INDICES OF AN ARRAY

Ranges provide a straightforward mechanism for traversing the elements of a collection. When used in combination with a for-loop, they let you access each element of a vector by its index. There are several ways to construct such ranges.

A straightforward method is to write `1:length(x)`, which generates all valid indices for the vector `x`. Since `length(x)` returns the number of elements, this range covers every position from the first to the last. While this approach works, it relies on the assumption of linear indexing starting at 1, which isn't always guaranteed. As a result, it can be fragile when applied to collections with different indexing schemes.

A more robust practice is to use `eachindex(x)`. This function produces an iterator that's optimized to the specific structure of the collection. Furthermore, it ensures that you're iterating over the correct set of indices, regardless of the underlying data type. This makes your code more general, more

efficient, and less error-prone, especially when working with other iterable objects that may not use standard indexing.

1:LENGTH(X)

```
x = [4, 6, 8]

for i in 1:length(x)
    println(x[i])
end
```

```
4
6
8
```

EACHINDEX

```
x = [4, 6, 8]

for i in eachindex(x)
    println(x[i])
end
```

```
4
6
8
```

Julia provides other methods to iterate over all indices of a collection. Two constructions worth mentioning are `LinearIndices(x)` and `firstindex(x):lastindex(x)`. Just like the previous methods defined, they create a range that goes from the first to the last index of `x`.

All these approaches can be used interchangeably, as shown below.

EACHINDEX

```
x = [4, 6, 8]

for i in eachindex(x)
    println(x[i])
end
```

```
4
6
8
```

1:LENGTH(X)

```
x = [4, 6, 8]

for i in 1:length(x)
    println(x[i])
end
```

```
4
6
8
```

LINEARINDICES

```
x = [4, 6, 8]

for i in LinearIndices(x)
    println(x[i])
end
```

```
4
6
8
```

FIRSTINDEX(X):LASTINDEX(X)

```
x = [4, 6, 8]

for i in firstindex(x):lastindex(x)
    println(x[i])
end
```

```
4
6
8
```

The multiplicity of methods to implement the same functionality is necessary to handle non-standard indices. For instance, the `OffsetArrays.jl` package sets the first index of arrays to 0, a common convention in many programming languages. When this package is loaded, using `1:length(x)` would break portability, while `firstindex(x):lastindex(x)` adapts seamlessly to the modified indexing scheme.

Unless you're developing a package for other users, you don't need to worry much about which approach to implement.

VARIABLE SCOPE IN FOR-LOOPS

For-loops introduce a new variable scope, which is largely governed by the same rules as functions. Specifically, given a variable named `x`, the rules of variable scope are as follows:

- If `x` is the variable of iteration, then `x` is always local to the for-loop. This holds regardless of whether there's a variable `x` defined outside the for-loop.
- If there's a variable named `x` outside the for-loop and no `x` is defined within the for-loop, `x` refers to the global variable. Moreover, the for-loop can mutate the value of `x`.
- If there's a variable `x` defined within the for-loop but no `x` defined outside it, `x` is local and won't be accessible outside the for-loop.

The following code snippets illustrate these rules. The third example is particularly noteworthy, as it highlights **a common beginner mistake**: defining a variable inside a for-loop and then attempting to use it outside the for-loop, only to discover that it's no longer accessible.

ITERATION VARIABLES ARE LOCAL

```
x = 2

for x in ["hello"]          # this 'x' is local, not related to 'x = 2'
    println(x)
end
```

```
hello
```

NEW VARIABLES ARE LOCAL

```
#no `x` defined outside the for-loop

for word in ["hello"]
    x = word                # `x` is local to the for-loop, not available outside it
end
```

```
julia> x
ERROR: UndefVarError: x not defined
```

X REFERS TO GLOBAL X

```
x = [2, 4, 6]

for i in eachindex(x)
    x[i] * 10               # refers to the `x` outside of the for-loop
end
```

```
julia> x
3-element Vector{Int64}:
 2
 4
 6
```

GLOBAL X MUTATED

```
x = [2, 4, 6]

for i in eachindex(x)
    x[i] = x[i] * 10        # mutates the `x` defined outside the for-loop
end
```

```
julia> x
3-element Vector{Int64}:
 20
 40
 60
```

> ⚠ **Warning!**
>
> The scope rule that we'll present next always applies, except in one edge case. Since this case primarily results from discouraged coding practices, we briefly outline it without further elaboration.
>
> Specifically, the rule won't apply when three conditions occur simultaneously: *i)* the for-loop is *not* inside a function, *ii)* a local variable shares the same name as a global variable, and iii) the script is run non-interactively (i.e., executing code through a script file). [2] When these conditions are met, Julia prevents reassignment of variables inside for-loops.
>
> **You can safely ignore this exception** for two reasons. First, due to performance reasons that we'll explore later in this book, you should always write for-loops within functions. Second, even if this scenario occurs, Julia will issue a warning in the REPL, alerting you that the code may not behave as intended.

The last scope rule applies when for-loops appear inside functions or when code is run interactively:

- if there's a variable `x` defined both within the for-loop and outside it, `x` will be reassigned to the value given inside the for-loop.

The following example demonstrates this rule, where we enclose the for-loop in a function.

REASSIGNMENT OF X

```
function foo()
    x = [2, 4, 6]

    for word in ["hello"]
        x = word              # `x` is reassigned
    end

    return x
end
```

```
julia> foo()
"hello"
```

ARRAY COMPREHENSIONS

Julia provides **array comprehensions** as a concise and expressive way to generate new arrays via for-loops. The general syntax is `[<expression> for... if...]`, where `<expression>` denotes either an operation or a function.

To illustrate, suppose we want to define a new vector `y`, whose elements are the squares of the corresponding entries in `x`. Array comprehensions make this task straightforward. The following code snippets demonstrate the approach by using a direct expression and by calling a function.

ARRAY COMPREHENSION WITH OPERATION

```
x      = [1,2,3]


y      = [a^2 for a in x]
z      = [x[i]^2 for i in eachindex(x)]
```

```
julia> y
3-element Vector{Int64}:
 1
 4
 9
julia> z
3-element Vector{Int64}:
 1
 4
 9
```

ARRAY COMPREHENSION WITH FUNCTION CALL

```
x      = [1,2,3]
foo(a) = a^2

y      = [foo(a) for a in x]
z      = [foo(x[i]) for i in eachindex(x)]
```

```
julia> y
3-element Vector{Int64}:
 1
 4
 9
julia> z
3-element Vector{Int64}:
 1
 4
 9
```

Array comprehensions can also incorporate conditions, allowing you to filter elements as they're generated. In such cases, the condition must be placed at the end of the comprehension.

ARRAY COMPREHENSION THAT ADDS CONDITION

```
x = [1, 2, 3, 4]

y = [a for a in x if a ≤ 2]
z = [x[i] for i in eachindex(x) if x[i] ≤ 2]
```

```
julia> y
2-element Vector{Int64}:
 1
 2
julia> z
2-element Vector{Int64}:
 1
 2
```

Array Comprehensions for Defining Matrices

Array comprehensions can also be used to construct matrices. In this case, the syntax requires a comma to separate the iteration variables for each dimension.

ARRAY COMPREHENSION FOR MATRICES

```
X = [i * j for i in 1:2, j in 1:2]
```

```
julia> X
2×2 Matrix{Int64}:
 1  2
 2  4
```

ITERATING OVER MULTIPLE OBJECTS

For-loops can handle more than just single-value iterations, also supporting **simultaneous iteration over multiple values**.

We'll focus on two common scenarios: simultaneously iterating over two iterable collections, and iterating over both the indices and values of a single collection. In each case, we'll explore how to implement the iteration using both plain for-loops and array comprehensions, highlighting the differences in syntax and use cases.

SIMULTANEOUSLY ITERATING OVER TWO COLLECTIONS

Consider two iterable objects `x` and `y`. There are two strategies for combining their elements, depending on the desired outcome.

First, the function `Iterators.product(x,y)` generates all the possible combinations of elements from `x` and `y`. Thus, it produces a pair `(x[i], y[j])`, where `i` and `j` take all possible indices. The `Iterators.product` function is part of the package `Iterators`, imported by default in each Julia session.

Alternatively, it's possible to iterate over all ordered pairs of `x` and `y`. This is implemented through the function `zip(x,y)`, which provides the pair of *i*-th elements from `x` and `y` in the *i*-th iteration. Thus, compared to `Iterators.product`, it restricts the iterations to the pairs `(x[i], y[i])`.

PAIRS

```
list1 = ["A","B"]
list2 = [ 1 , 2 ]

for (a,b) in zip(list1,list2)                    #it takes pairs with the same index
    println((a,b))
end
```

```
("A", 1)
("B", 2)
```

ALL COMBINATIONS

```
list1 = ["A","B"]
list2 = [ 1 , 2 ]

for (a,b) in Iterators.product(list1,list2)    #it takes all possible combinations
    println((a,b))
end
```

```
("A", 1)
("B", 1)
("A", 2)
("B", 2)
```

SIMULTANEOUSLY ITERATING OVER INDICES AND VALUES

When iterating over a collection, there are scenarios where you need access to both the value and index of an element, rather than handling each separately. For example, when analyzing data, you may want to flag outliers not only by their magnitude, but also by their position in the dataset.

Julia provides a direct way to achieve this through the `enumerate` function. This transforms a collection into an iterator that yields pairs of indices and values, thus providing access to both pieces of information during each iteration.

FOR-LOOPS

```
x = ["hello", "world"]

for (index,value) in enumerate(x)
    println("$index $value")
end
```

```
1 hello
2 world
```

ARRAY COMPREHENSION

```
x = [2, 3]


y = [(x[index] - value) for (index,value) in enumerate(x)]    # `x[index]` equals `value`
```

```
2-element Vector{Int64}:
 0
 0
```

ITERATING OBJECTS TO CREATE VECTORS

To generate vectors that iterate over multiple values, we can employ array comprehension. In particular, to construct a vector that exhausts all combinations between two iterable collections, Julia offers a special syntax: simply repeat the `for` keyword to specify the iteration range of each variable.

Note that the `for` clauses must be written sequentially, without commas. A comma changes the comprehension's behavior, instructing Julia to build a matrix instead of a vector, as demonstrated before.

We illustrate these cases, along with the vectors obtained with `zip` and `Iterators.product` for this purpose.

ALL COMBINATIONS

```
list1 = ["A","B"]
list2 = [ 1 , 2 ]

x     = [(i,j) for i in list1 for j in list2]
```

```
julia> x
4-element Vector{Tuple{String, Int64}}:
 ("A", 1)
 ("A", 2)
 ("B", 1)
 ("B", 2)
```

NOTATION FOR MATRICES

```
list1 = ["A","B"]
list2 = [ 1 , 2 ]

X     = [(i,j) for i in list1, j in list2]                    # this defines a matrix
```

```
julia> x
2×2 Matrix{Tuple{String, Int64}}:
 ("A", 1)  ("A", 2)
 ("B", 1)  ("B", 2)
```

ZIP

```
list1 = ["A","B"]
list2 = [ 1 , 2 ]

x     = [(i,j) for (i,j) in zip(list1,list2)]
```

```
julia> x
2-element Vector{Tuple{String, Int64}}:
 ("A", 1)
 ("B", 2)
```

ITERATORS.PRODUCT

```
list1 = ["A","B"]
list2 = [ 1 , 2 ]

X     = [(i,j) for (i,j) in Iterators.product(list1,list2)]    # this defines a matrix
```

```
julia> X
2×2 Matrix{Tuple{String, Int64}}:
 ("A", 1)  ("A", 2)
 ("B", 1)  ("B", 2)
```

ITERATING OVER FUNCTIONS

Functions in Julia are **first-class objects**, also referred to as **first-class citizens**. This means that functions can be manipulated just like any other data type, such as strings and numbers. In particular, this property makes it possible to store functions in a vector and apply them sequentially to an object. The following example illustrates this by computing descriptive statistics of a vector `x`.

ITERATION OVER FUNCTIONS

```
x                       = [0, 5, 10]
list_functions          = [minimum, maximum]

descriptive(vector,list) = [foo(vector) for foo in list]
```

```
julia> descriptive(x, list_functions)
2-element Vector{Int64}:
  0
 10
```

FOOTNOTES

1. In fact, older versions of Julia were restricting the use of for-loops in the global scope.

2. There are two methods to run code. The first method is the one used thus far, where we work interactively with Julia. This includes both running commands in the REPL and executing lines through a code editor. The second method consists of executing files containing scripts, which is done with the function `include`.

5a. Overview and Goals

Martin Alfaro
PhD in Economics

Thus far, we've laid the groundwork by introducing the fundamentals of Julia. We've presented typical objects used in Julia, distinguishing between scalars and collections, and introduced core programming tools such as functions, conditions, and for-loops. At this initial stage, nonetheless, the emphasis was primarily on familiarizing with the core approaches and their syntax. Thus, we didn't delve into how the tools can be applied and combined.

Equipped with a foundational knowledge of the concepts, we can now begin exploring techniques for working with collections. In particular, Chapter 5 focuses on methods for manipulating **mutable collections**, which are collections whose elements can be modified. Our primary example will be vectors, the most common mutable collections in Julia.

Section 5b introduces **the concept of mutability** in a formal way and identifies which objects possess this property. Once we understand what it means for an object to be mutable, Section 5c establishes **a clear distinction between assignments and mutations**. The difference determines whether we're creating a new object or modifying an existing one. While the distinction may seem unimportant, it'll be crucial for high performance.

The remaining sections concentrate entirely on vectors. Section 5d shows **how to initialize vectors**, Section 5e explains **how to define subsets of elements** (called slices), and Section 5f covers selecting individual elements via **indexing**.

The final two sections deal with operations that modify vectors, giving rise to the concept of **in-place operations**. Section 5g introduces ways to perform these operations. Meanwhile, Section 5h concentrates on mutations carried out through functions. They lead to the notion of **in-place functions**, defined as functions that update the values of at least one of their arguments. These functions are easily recognized in Julia, since developers follow the convention of appending the symbol `!` to their names. For example, if we want to sort the elements of a vector `x`, we can either create a new sorted vector by writing `y = sort(x)` or directly modify `x` by writing `sort!(x)`.

As we begin to integrate these tools, it may take some time to become fully comfortable with their use. In fact, you may occasionally need to revisit the earlier sections on functions, conditions, and for-loops.

5b. Mutable and Immutable Objects

Martin Alfaro
PhD in Economics

INTRODUCTION

Objects in programming can be broadly classified into two categories: mutable and immutable. **Mutable objects** permit modification of their internal state after creation. This means that their elements can be modified, appended, or removed at will, thus providing a high degree of flexibility. A prime example is vectors.

In contrast, **immutable objects** can't be altered after their creation: they prevent additions, removals, or modifications of their elements. A common example of immutable object in Julia is **tuples**. Immutability effectively locks variables into a read-only state, safeguarding against unintended changes. Simultaneously, it can result in performance gains for small objects, as we'll show in Part II of this book.

This section will be relatively brief, focusing solely on the distinctions between mutable and immutable objects. Subsequent sections will expand on their uses and properties.

> **Remark**
>
> A popular package called `StaticArrays` provides an implementation of **immutable vectors**. We'll explore them in the context of high performance, as they can greatly speed up computations involving small vectors. They're created using tuples under the hood.

EXAMPLES OF MUTABILITY AND IMMUTABILITY

To illustrate, the following examples attempt to modify existing elements of a collection. The examples rely on vectors as an example of a mutable object and tuples for immutable ones. Additionally, we present the case of strings as another example of immutable object, which are essentially sequences of characters.

MUTABILITY OF VECTORS

```
x = [3,4,5]
```

```
julia> x[1] = 0

julia> x
3-element Vector{Int64}:
 0
 4
 5
```

IMMUTABILITY OF TUPLES

```
x = (3,4,5)
```

```
julia> x[1] = 0
ERROR: MethodError: no method matching setindex!(::Tuple{Int64, Int64, Int64}, ::Int64,
::Int64)
```

IMMUTABILITY OF STRINGS

```
x = "hello"
```

```
julia> x
'h': ASCII/Unicode U+0068 (category Ll: Letter, lowercase)

julia> x[1] = 'a'
ERROR: MethodError: no method matching setindex!(::String, ::Char, ::Int64)
```

The key characteristic of mutable objects is their ability to modify existing elements. Additionally, it commonly allows for the dynamic addition and removal of elements. In a subsequent section, we'll present various methods for implementing this last functionality. For now, we simply demonstrate the capability by using the functions `push!` and `pop!`, which respectively add and remove an element at the end of a collection.

ADD (MUTABLE OBJECT)

```
x = [3,4]

push!(x, 5)       # add element 5 at the end
```

```
julia> x
3-element Vector{Int64}:
 3
 4
 5
```

REMOVE (MUTABLE OBJECT)

```
x = [3,4,5]

pop!(x)           # delete last element
```

```
julia> x
2-element Vector{Int64}:
 3
 4
```

ADD/REMOVE (IMMUTABLE OBJECT)

```
x = (3,4,5)

pop!(x)           # ERROR
push!(x,6)        # ERROR
```

```
ERROR: MethodError: no method matching pop!(::Tuple{Int64, Int64, Int64})
```

5c. Assignments vs Mutations

Martin Alfaro
PhD in Economics

INTRODUCTION

The upcoming sections will be entirely devoted to **vector mutation**, a process that modifies the contents of a vector. However, to properly cover this subject, we first need to introduce some preliminary concepts, including:

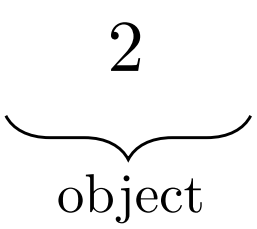
- the distinction between assignment and mutation,
- methods for initializing arrays to eventually mutate them, and
- techniques for vector indexing to select elements.

The current section in particular focuses on **the distinction between assignment and mutation of variables**. The difference between these operations can easily go unnoticed by new users, as both use the operator `=` despite being fundamentally different. Clearly delineating them is important not only for Julia, but also for other programming languages.

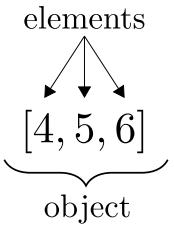
SOME BACKGROUND

Recall that **variables** serve as labels for **objects**, where objects in turn hold **values**. Moreover, objects can be classified according to the number of **elements** contained, ranging from scalars (single-element objects such as integers and floating-point numbers) to collections (e.g. vectors).

NUMBER



VECTOR



The distinction between objects and their elements is crucial for the remainder of the section. This is because *assignments apply to objects, whereas mutations apply to elements*. More specifically, assignments rebind variables to new objects, while mutations simply modify existing elements of an existing object.

At first glance, this difference might feel like an implementation detail, without practical implications regarding how you must write code. In Part II of this book, we'll show that performance is heavily influenced by the choice between reassignments and mutations. Updating the elements of an existing vector is typically far more efficient than constructing a brand-new vector with updated values.

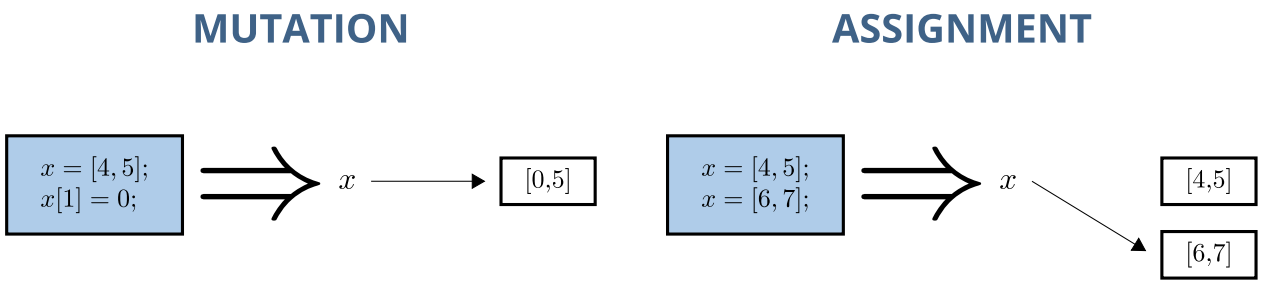
DEFINING ASSIGNMENTS AND MUTATIONS

Assignments establish a binding between a variable and an object through the `=` operator. For instance, `x = 3` and `x = [1,2,3]` are examples of assignments, in which the objects `3` and `[1,2,3]` are respectively bound to `x`.

Mutations, by contrast, **modify the elements of an object, without creating a new one**. These operations also rely on the `=` operator. An example of mutation is given by `x[1] = 0`, which modifies the first element of `x`.

Despite sharing the same operator `=`, assignments and mutations are conceptually distinct. The difference becomes clearer when we **consider objects as residing at specific memory addresses**. Under this view, an assignment for `x` involves two steps: *i*) finding a memory location to store the object, and *ii*) labeling the memory address as `x` for easy access. A mutation, in contrast, modifies the data stored at an existing memory address, leaving the address itself unchanged.

To illustrate, if `x = [6,7]` is run, `x` becomes associated with an object containing `[6,7]`. This constitutes an *assignment*. However, if `x[1] = 0` is executed afterward, the operation modifies the original object `[6,7]`, now becoming `[6,0]`. This operation constitutes a *mutation*: `x` continues to reference the same memory address, even though its content has changed.



> **Mutating All Elements vs Assignment**
>
> Mutating all the elements of `x` doesn't imply a new assignment. For example, modifying `x` using `x[:]` is a mutation.

MUTATING ALL ELEMENTS

```
x    = [4,5]

x[:] = [0,0]
```

```
julia> x
2-element Vector{Int64}:
 0
 0
```

ALIAS VS COPY

Since both assignments and mutations employ the `=` operator, a natural question that arises is when `=` results in an assignment or a mutation.

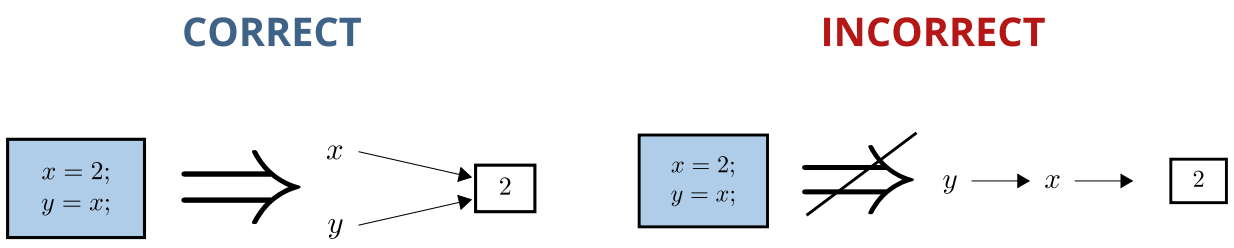
Next, we focus on **cases where entire objects appear on both sides of** `=`, as in `y = x`. Other scenarios are left for upcoming sections, after we introduce the concept of slices (i.e, subsets of elements from a vector).

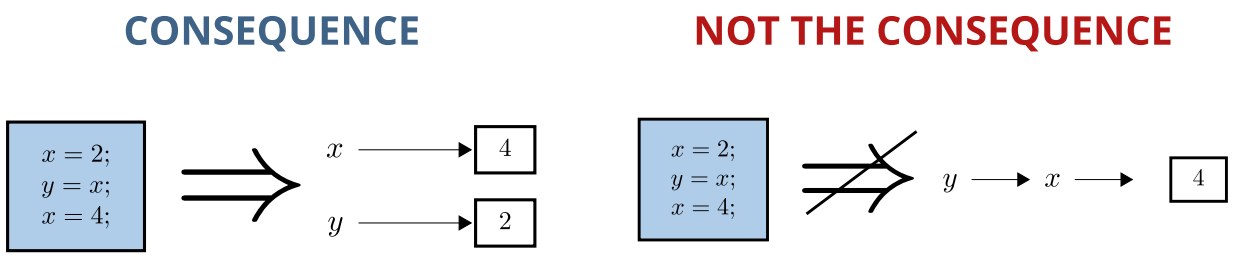
When `y = x` is executed, `y` becomes another name for the object referenced by `x`. In other words, `x` and `y` become different labels for the same underlying object. Formally, we say that `y` is an **alias** of `x`.

It's important to stress that `y = x` doesn't bind `y` to `x` itself. Rather, `y` becomes another label for the object that `x` references. This subtle distinction carries a significant practical implication: reassigning `x` to a new object won't affect `y`'s reference.

To clarify this further, let's consider an example where we first execute `x = 2` and then `y = x`. At this point, both `x` and `y` reference the same object, which holds the value `2`. If we eventually execute `x = 4`, the variable `x` will start pointing to a new object holding the value `4`. However, this won't affect the original object that `x` was originally referencing. As a result, `y` will still point to the original object with value `2`.

This behavior is illustrated visually below.





The same conclusion can be observed by examining code execution.

ALIAS VS COPY

```
x = 2   #'x' points to an object with value 2
y = x   #'y' points to the same object as 'x' (do not interpret it as 'y' pointing to 'x')

x = 4   #'x' now points to another object (but 'y' still points to the object holding 2)
```

```
julia> x
4
julia> y
2
```

Two variables may contain identical elements and yet refer to different objects

The claim can be demonstrated using the operators `==` and `===`, which capture two different notions of equality. The expression `x == y` checks whether `x` and `y` have the same *values*, regardless of whether they reference the same object. In contrast, `x === y` checks whether both `x` and `y` point to the same *object*, thus verifying if they point to the same memory address.

By applying these operators, the following example illustrates that objects with identical elements aren't necessarily referencing the same object.

SAME OBJECT

```
x = [4,5]

y = x
```

```
julia> x == y
true
julia> x === y
true
```

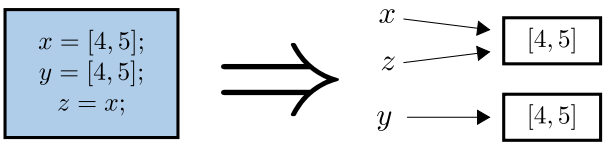
SAME ELEMENTS BUT DIFFERENT OBJECT

```
x = [4,5]

y = [4,5]
```

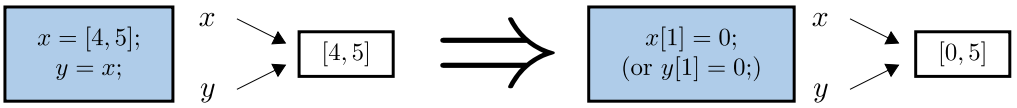
```
julia> x == y
true
julia> x === y
false
```

GRAPHICAL REPRESENTATION



We've indicated that the operation `y = x` creates a new alias for `x`, turning `y` and `x` two different labels for the same object. This implies that **modifying the elements of either `x` or `y` will necessarily change the elements referenced by both**. The following diagram illustrates this.

GRAPHICAL REPRESENTATION



The corresponding code snippet captures this case.

VARIABLE 'Y' AS AN ALIAS

```
x    = [4,5]
y    = x

x[1] = 0
```

```
julia> x
2-element Vector{Int64}:
 0
 5
julia> y
2-element Vector{Int64}:
 0
 5
```

If you instead aim to treat `x` and `y` as pointing to separate objects, you must use the `copy` function. This creates a *new object* with the same elements as the original. Consequently, any modification to the new object won't affect the original one, allowing you to work with `x` and `y` independently.

VARIABLE 'Y' AS A COPY

```
x    = [4,5]
y    = copy(x)

x[1] = 0
```

```
julia> x
2-element Vector{Int64}:
 0
 5
julia> y
2-element Vector{Int64}:
 4
 5
```

5d. Vector Creation and Initialization

Martin Alfaro
PhD in Economics

INTRODUCTION

We continue presenting preliminary concepts for introducing the concept of mutation. The previous section distinguished between the use of `=` for assignments and mutations. Now, we'll deal with approaches to creating vectors.

Our presentation starts by outlining the process of initializing vectors, where memory is reserved without assigning initial values. We'll then discuss how to create vectors holding predefined values, such as zeros or ones. Finally, we show how to concatenate multiple vectors into new ones, through the `vcat` function.

INITIALIZING VECTORS

Creating an array involves two steps: reserving memory for holding its content and assigning initial values to its elements. When you don't intend to populate the array with values right away, it's more efficient to perform only the allocation step. This means reserving memory space, but without setting any initial values.

Technically, initializing entails creating an array filled with `undef` values. These values represent arbitrary content in memory at the moment of allocation. This is why, while `undef` displays concrete numbers when you output the array's content, they're meaningless and vary every time you initialize a new array.

There are two methods for creating vectors with `undef` values. The first one requires explicitly specifying the type and length of the array, which is accomplished via `Vector{<elements' type>}(undef, <length>)`. The second approach is based on the function `similar(x)`, which creates a vector with the same type and dimensions as an existing vector `x`.

'UNDEF' VALUES

```
x_length = 3

x            = Vector{Int64}(undef, x_length)   # `x` can hold `Int64` values, and is
initialized with 3 undefined elements
```

```
julia> x
3-element Vector{Int64}:
 0
 0
 0
```

'SIMILAR'

```
y        = [3,4,5]

x          = similar(y)                          # `x` has the same type as `y`, which is
Vector{Int64}(undef, 3)
```

```
julia> x
3-element Vector{Int64}:
 126781934511632
     42949673047
 126781933981968
```

The example demonstrates that `undef` values don't follow any particular pattern. Moreover, these values vary in each execution, as they reflect any content held in RAM at the moment of allocation. In fact, a more descriptive way to call `undef` values would be **uninitialized values**.

CREATING VECTORS WITH GIVEN VALUES

In the following, we present several approaches to creating arrays holding predefined values.

VECTORS COMPRISING A RANGE

To generate a sequence of values, we can make use of ranges. Recall that the syntax for defining ranges is `<start>: <steps>: <stop>`, where `<steps>` specifies the increment between consecutive values.

In particular, we need the expression `collect(<range>)`. The function `collect` is necessary, since ranges describe the values to be generated, without materializing them. This behavior is consistent with a broader concept known as lazy operations, which will be presented later in this book. [1]

A VECTOR FILLED USING A SEQUENCE

```
some_range = 2:5

x          = collect(some_range)
```

```
julia> x
4-element Vector{Int64}:
 2
 3
 4
 5
```

When a range is created, `<steps>` dictates the number of elements to be generated. Considering this, we can alternatively specify the number of elements to be stored through the syntax `<start> : 1/<number of elements> : <end>`. A more direct way is via the `range` function, whose syntax is `range(<start>, <end>, <number of elements>)`.

The following code snippet demonstrates the use of the `range` function, by generating five evenly spaced elements between 0 and 1.

RANGE

```
x = range(0, 1, 5)
```

```
julia> x
0.0:0.25:1.0
```

RANGE (KEYWORD ARGUMENTS)

```
x = range(start=0, stop=1, length=5)
```

```
julia> x
0.0:0.25:1.0
```

RANGE (KEYWORD ARGUMENTS)

```
x = range(start=0, length=5, stop=1)     # any order for keyword arguments
```

```
julia> x
0.0:0.25:1.0
```

VECTORS WITH SPECIFIC VALUES REPEATED

We can also build vectors of a given length, where every element holds the same value. Two common examples are `zeros` and `ones`, which respectively produce vectors filled with zeros and ones. By default, both functions create vectors whose elements have type `Float64`. You can override this behavior by providing the desired element type as the first argument.

ZEROS

```
length_vector = 3

x             = zeros(length_vector)
```

```
julia> x
3-element Vector{Float64}:
 0.0
 0.0
 0.0
```

ZEROS WITH TYPE

```
length_vector = 3

x             = zeros(Int, length_vector)
```

```
julia> x
3-element Vector{Int64}:
 0
 0
 0
```

ONES

```
length_vector = 3

x             = ones(length_vector)
```

```
julia> x
3-element Vector{Float64}:
 1.0
 1.0
 1.0
```

ONES WITH TYPE

```
length_vector = 3

x             = ones(Int, length_vector)
```

```
julia> x
3-element Vector{Int64}:
 1
 1
 1
```

To create vectors with Boolean values, Julia provides two convenient functions called `trues` and `falses`.

TRUES

```
length_vector = 3

x             = trues(length_vector)
```

```
julia> x
3-element BitVector:
 1
 1
 1
```

FALSES

```
length_vector = 3

x             = falses(length_vector)
```

```
julia> x
3-element BitVector:
 0
 0
 0
```

VECTORS WITH AN ARBITRARY VALUE REPEATED

We can define vectors whose elements all share the same specified value, without restricting this value to special cases like one or zero. This is achieved with the `fill` function.

FILL (SCALAR)

```
length_vector  = 3
filling_object = 1

x              = fill(filling_object, length_vector)
```

```
julia> x
3-element Vector{Int64}:
 1
 1
 1
```

FILL (VECTOR)

```
length_vector  = 3
filling_object = [1,2]

x              = fill(filling_object, length_vector)
```

```
julia> x
3-element Vector{Vector{Int64}}:
 [1, 2]
 [1, 2]
 [1, 2]
```

FILL (ONE-ELEMENT VECTOR)

```
length_vector  = 3
filling_object = [1]

x              = fill(filling_object, length_vector)
```

```
julia> x
3-element Vector{Vector{Int64}}:
 [1]
 [1]
 [1]
```

VECTORS FORMED WITH CONCATENATED VECTORS

There are several ways to construct a vector `z` that combines all elements of two vectors `x` and `y`. A straightforward way is by using `z = [x; y]`. While this works well for concatenating a small number of vectors, it becomes cumbersome when many vectors must be merged. Moreover, it's directly not viable when the number of vectors is unknown in advance.

In such cases, we can employ the `vcat` function. This concatenates all of its arguments into a single vector. When paired with the splat operator `...`, it can also operate on a single argument containing multiple vectors.

VCAT

```
x = [3,4,5]
y = [6,7,8]


z = vcat(x,y)
```

```
julia> x
6-element Vector{Int64}:
 3
 4
 ⋮
 7
 8
```

VCAT AND SPLATTING OPERATOR

```
x = [3,4,5]
y = [6,7,8]

A = [x, y]
z = vcat(A...)
```

```
julia> x
6-element Vector{Int64}:
 3
 4
 ⋮
 7
 8
```

VECTORS WITH REPEATED ELEMENTS FROM AN OBJECT

Closely related to `fill` is the `repeat` function. This takes a *collection* as an input, concatenating their elements repeatedly a specified number of times. The `repeat` function **necessarily requires an array as its input**, throwing an error if a scalar is passed.

REPEAT (VECTOR)

```
nr_repetitions     = 3
elements_to_repeat = [1,2]

x                  = repeat(elements_to_repeat, nr_repetitions)
```

```
julia> x
6-element Vector{Int64}:
 1
 2
 ⋮
 1
 2
```

REPEAT (VECTOR - SINGLE ELEMENT)

```
nr_repetitions     = 3
elements_to_repeat = [1]

x                  = repeat(elements_to_repeat, nr_repetitions)
```

```
julia> x
3-element Vector{Int64}:
 1
 1
 1
```

REPEAT (SCALAR - NOT ALLOWED)

```
nr_repetitions   = 3
vector_to_repeat = 1

x                = repeat(vector_to_repeat, nr_repetitions)
```

```
ERROR: MethodError: no method matching repeat(::Int64, ::Int64)
```

Note that the behavior of `repeat` differs from `fill` function, since the latter repeats the same object without concatenating its elements. In fact, the output of `repeat` is equivalent to creating a vector with `fill` and then using `vcat` to concatenate its elements.

'REPEAT' WITH VECTOR INPUT

```
nr_repetitions     = 3
elements_to_repeat = [1,2]

x                  = repeat(elements_to_repeat, nr_repetitions)
```

```
julia> x
6-element Vector{Int64}:
 1
 2
 ⋮
 1
 2
```

'FILL' WITH VECTOR INPUT

```
length_vector      = 3
filling_object     = [1,2]

x                  = fill(filling_object, length_vector)
```

```
julia> x
3-element Vector{Vector{Int64}}:
 [1, 2]
 [1, 2]
 [1, 2]
```

'FILL' + 'VCAT'

```
length_vector      = 3
filling_object     = [1,2]

temp               = fill(filling_object, length_vector)
x                  = vcat(temp...)
```

```
julia> x
6-element Vector{Int64}:
 1
 2
 ⋮
 1
 2
```

ADDING, REMOVING, AND REPLACING ELEMENTS (*OPTIONAL*)

> ⚠️ **Warning!**
>
> This subsection requires knowledge of a few **concepts we haven't discussed yet**. As such, it's marked as optional.
>
> One such concept is that of **in-place functions**, which will be introduced later. These functions are identified by the symbol `!` appended to the function's name. The symbol is simply a convention chosen by developers, not a syntax requirement. It indicates that the function modifies at least one of its arguments.
>
> Another concept introduced is that of **pairs**, which will be thoroughly examined in a future section too. For the purposes of this subsection, it's sufficient to know that pairs are written in the form `a => b`. In our applications, `a` will represent a given value and `b` denote its corresponding replacement value.

Next, we demonstrate how to add, remove, and replace elements within a vector. We begin by presenting methods for inserting elements.

ADD SCALAR AT END

```
x                = [3,4,5]
element_to_insert = 0


push!(x, element_to_insert)                # add 0 as last element - faster
```

```
julia> x
4-element Vector{Int64}:
 3
 4
 5
 0
```

ADD ONE ELEMENT AT START

```
x                = [3,4,5]
element_to_insert = 0


pushfirst!(x, element_to_insert)           # add 0 as first element - slower
```

```
julia> x
4-element Vector{Int64}:
 0
 3
 4
 5
```

ADD ONE ELEMENT AT SPECIFIC INDEX

```
x                = [3,4,5]
element_to_insert = 0
at_index         = 2

insert!(x, at_index, element_to_insert)    # add 0 at index 2
```

```
julia> x
4-element Vector{Int64}:
 3
 0
 4
 5
```

ADD MULTIPLE ELEMENTS AT END

```
x                 = [3,4,5]
vector_to_insert  = [6,7]


append!(x, vector_to_insert)                # add 6 and 7 as last elements
```

```
julia> x
5-element Vector{Int64}:
 3
 4
 5
 6
 7
```

The function `push!` is particularly helpful to collect outputs in a vector. Since it doesn't require prior knowledge of how many elements will be stored, the vector can grow dynamically as new results are added. Notice that adding elements at the end with `push!` is generally faster than inserting them at the beginning with `pushfirst!`.

Analogous functions exist to remove elements, as shown below.

REMOVE LAST ELEMENT

```
x                 = [5,6,7]


pop!(x)                           # delete last element
```

```
julia> x
2-element Vector{Int64}:
 5
 6
```

REMOVE FIRST ELEMENT

```
x                 = [5,6,7]


popfirst!(x)                      # delete first element
```

```
julia> x
2-element Vector{Int64}:
 6
 7
```

REMOVE ONE ELEMENT AT INDEX

```
x                 = [5,6,7]
index_of_removal  = 2

deleteat!(x, index_of_removal)     # delete element at index 2
```

```
julia> x
2-element Vector{Int64}:
 5
 7
```

REMOVE MULTIPLE ELEMENTS AT INDEX

```
x                 = [5,6,7]
indices_of_removal = [1,3]

deleteat!(x, indices_of_removal)   # delete elements at indices 1 and 3
```

```
julia> x
1-element Vector{Int64}:
 6
```

By analogy with the behavior of `deleteat!`, it's also possible to specify which elements should be retained.

KEEP SINGLE ELEMENT

```
x              = [5,6,7]
index_to_keep  = 2

keepat!(x, index_to_keep)
```

```
julia> x
1-element Vector{Int64}:
 6
```

KEEP MULTIPLE ELEMENTS

```
x              = [5,6,7]
indices_to_keep = [2,3]

keepat!(x, indices_to_keep)
```

```
julia> x
2-element Vector{Int64}:
 6
 7
```

Finally, specific values can be replaced with new ones. This can be done by either creating a new copy using `replace` or by updating the original vector with `replace!`.

Both functions make use of pairs `a => b`, where `a` denotes the value to be replaced and `b` specifies its replacement. Note that these functions perform substitutions based on values, not on indices.

REPLACE ONE VALUE (UPDATE)

```
x = [3,3,5]

replace!(x, 3 => 0)              # in-place (it updates x)
```

```
julia> x
3-element Vector{Int64}:
 0
 0
 5
```

REPLACE MULTIPLE VALUES (UPDATE)

```
x = [3,3,5]

replace!(x, 3 => 0, 5 => 1)      # in-place (it updates x)
```

```
julia> x
3-element Vector{Int64}:
 0
 0
 1
```

REPLACE ONE VALUE (COPY)

```
x = [3,3,5]

y = replace(x, 3 => 0)           # new copy
```

```
julia> y
3-element Vector{Int64}:
 0
 0
 5
```

REPLACE MULTIPLE VALUES (COPY)

```
x = [3,3,5]

y = replace(x, 3 => 0, 5 => 1)   # new copy
```

```
julia> y
3-element Vector{Int64}:
 0
 0
 1
```

FOOTNOTES

[1] Lazy operations specify how a computation should proceed, but postpone producing concrete results until they're explicitly required (either because you request them or because another computation depends on them.) They become especially valuable when combined with other operations, since it can fuse operations and therefore eliminate the need to store intermediate results altogether.

5e. Slices: Copies vs Views

Martin Alfaro
PhD in Economics

INTRODUCTION

This section concludes the introduction of preliminary concepts for the study of mutations. The attention is now shifted to the concept of vector **slices**: subsets of elements drawn from a vector, written as `x[<indices>]`. In particular, the focus will be on the critical distinction between whether slices act as:

- **copies** of the original vector, thus creating a new object at a new memory address.
- **views** of the original vector, where the original object and the slice share the same memory address.

Whether one or the other arises depends on where the slice appears within an expression.

The difference is central to implementing mutations correctly, as such operations are only possible when working with a view. If a slice is instead a copy, the parent object and the slice become entirely independent entities. Therefore, modifications to the slice have no impact on the original object.

In Part II of this book, we'll also see that the distinction between copies and views has implications for performance. Essentially, creating a copy requires allocating new memory, which introduces computational overhead. Views, by contrast, avoid this cost by reusing the memory of the original vector.

Because of its broad significance, we'll examine the topic of copies and views independently of mutations. In particular, we'll identify how slices behave in different cases. Moreover, we'll explain how to instruct Julia to treat slices as views or copies.

SLICES AND THE ASSIGNMENT OPERATOR

Slices in assignments behave differently depending on their position within the expression. Specifically, **slices on the left-hand (LHS) side of `=` act as views**. In this role, slices directly reference the original elements, enabling mutation of the parent object. In contrast, **slices on the right-hand side (RHS) of `=` create a copy**. Since copies point to a new object in memory, any modification to the slice won't affect the original object.

The following code snippet demonstrates these contrasting behaviors.

A SLICE ON THE LHS

```
x    = [4,5]


x[1] = 0        # 'x[1]' is a view, line mutates 'x'
```

```
julia> x
2-element Vector{Int64}:
 0
 5
```

A SLICE ON THE RHS

```
x    = [4,5]
y    = x[1]     # 'y' is unrelated to 'x' because 'x[1]' is a copy

x[1] = 0        # it mutates 'x' but does NOT modify 'y'
```

```
julia> y
4
```

Aliasing vs Copying

Note the difference in behavior. **Slices** on the RHS of `=`, such as in statements `y = x[<indices>]`, are treated as copies. Instead, if we insert the whole object `x` on the RHS of `=`, as in `y = x`, the operation creates an alias. In that case, `y` and `x` will reference the same object, so that any modification made to `y` will also be reflected in `x`.

OBJECT ON THE RHS (ALIASING)

```
x    = [4,5]
y    = x        # the whole object (a view)

x[1] = 0        # it DOES modify 'y'
```

```
julia> y
2-element Vector{Int64}:
 0
 5
```

ALL ELEMENTS ON THE RHS (COPY)

```
x    = [4,5]
y    = x[:]     # a slice of the whole object (a copy)

x[1] = 0        # it does NOT modify 'y'
```

```
julia> y
2-element Vector{Int64}:
 4
 5
```

THE FUNCTION 'VIEW'

Beyond assignments, we must also distinguish whether slices represent copies or views within other expressions. As a rule of thumb, **slices typically default to creating copies**. Such behavior arises when, for instance, a slice is passed as a function argument or incorporated into a computation. Several of the scenarios are illustrated below.

SLICES AS COPIES

```
x = [3,4,5]

#the following slices are all copies
log.(x[1:2])

x[1:2] .+ 2

[sum(x[:]) * a for a in 1:3]

(sum(x[1:2]) > 0) && true
```

In all these cases, if you instead want to work with views, you must indicate it explicitly. Several methods exist for achieving this, with the most straightforward being the function `view`. Its syntax is `view(x, <indices>)`, where `<indices>` specify the indices that define the slice. To demonstrate its usage, we revisit the previous examples.

SLICES AS VIEWS

```
x = [3,4,5]

#we make explicit that we want views
log.(view(x,1:2))

view(x,1:2) .+ 2

[sum(view(x,:)) * a for a in 1:3]

(sum(view(x,:)) > 0) && true
```

SLICES AS COPIES

```
x = [3,4,5]

#the following slices are all copies
log.(x[1:2])

x[1:2] .+ 2

[sum(x[:]) * a for a in 1:3]

(sum(x[1:2]) > 0) && true
```

The examples reveal the potential verbosity involved when `view` isn't used sparingly. To mitigate this issue, Julia provides the `@view` and `@views` macros.

The `@view` macro is equivalent to `view`, allowing you to write `@view x[1:2]` instead of `view(x, 1:2)`. Its benefits, however, are somewhat limited: it saves only a few characters and still requires parentheses when multiple slices appear in the same expression (e.g., `@view(x[1:2]) .+ @view(x[2:3])`). By contrast, the `@views` macro significantly streamlines notation, converting *every* slice within an expression into a view.

MACROS FOR VIEWS

```
x = [4,5,6]

# the following are all equivalent
       y = view(x, 1:2)  .+ view(x, 2:3)
       y = @view(x[1:2]) .+ @view(x[2:3])
@views y = x[1:2] .+ x[2:3]
```

One of the most notable applications of `@views` arises in functions. When placed at the beginning of a function definition, `@views` ensures that every slice appearing in the function body is treated as a view.

@VIEWS AT A FUNCTION DEFINITION

```
@views function foo(x)
  y = x[1:2] .+ x[2:3]
  z = sum(x[:]) .+ sum(y)

  return z
end
```

EQUIVALENT WITH @VIEW

```
function foo(x)
  y = @view(x[1:2]) .+ @view(x[2:3])
  z = sum(@view x[:]) .+ sum(y)

  return z
end
```

5f. Array Indexing

Martin Alfaro
PhD in Economics

INTRODUCTION

To mutate elements in a vector, you must first identify the ones you wish to modify. This selection process is known as **vector indexing**.

We've already covered common indexing methods, including index vectors (e.g., `x[[1,2,3]]`) and ranges (e.g., `x[1:3]`). While these approaches are effective for simple selections, they fall short when you need more expressive control. For example, they're limited when it comes to selections based on conditions.

This section broadens the set of tools available for indexing. In particular, we'll present techniques that build on broadcasting Boolean operations.

LOGICAL INDEXING

Logical indexing (also known as *Boolean indexing* or *masking*) lets you select elements based on conditions. The strategy is based on the creation of a Boolean vector `y` of the same length as `x`, which acts as a filter. This means that `x[y]` keeps elements where `y` is `true` and excludes those where `y` is `false`.

LOGICAL INDEXING

```
x = [1,2,3]
y = [true, false, true]
```

```
julia> x[y]
2-element Vector{Int64}:
 1
 3
```

OPERATORS AND FUNCTIONS FOR LOGICAL INDEXING

Logical indexing becomes a powerful tool when leveraging broadcasting operations, allowing you to easily specify conditions via Boolean vectors. For instance, to select all the elements of `x` lower than 10, you can either broadcast a comparison operator or broadcast an auxiliary function.

INDEXING VIA BROADCASTING OPERATOR

```
x           = [1, 2, 3, 100, 200]

y           = x[x .< 10]
```

```
julia> y
3-element Vector{Int64}:
 1
 2
 3
```

INDEXING VIA BROADCASTING FUNCTION

```
x           = [1, 2, 3, 100, 200]

condition(a) = (a < 10)             #function to eventually broadcast
y           = x[condition.(x)]
```

```
julia> y
3-element Vector{Int64}:
 1
 2
 3
```

When dealing with multiple conditions, they must be combined using the **logical operators** `&&` and `||`. The following example illustrates the syntax for doing this. Note that *all* operators must be broadcast, since `&&` and `||` only work with scalar values. The repeated use of dots in the expression may result in verbose code, so we also show an alternative based on the macro `@.`.

INDEXING VIA BROADCAST OPERATOR

```
x           = [3, 6, 8, 100]

# numbers greater than 5, lower than 10, but not including 8
y           = x[(x .> 5) .&& (x .< 10) .&& (x .≠ 8)]
```

```
julia> y
1-element Vector{Int64}:
 6
```

INDEXING VIA @.

```
x           = [3, 6, 8, 100]

# numbers greater than 5, lower than 10, but not including 8
y           = x[@. (x > 5) && (x < 10) && (x ≠ 8)]
```

```
julia> y
```

INDEXING VIA BROADCAST FUNCTION

```
x           = [3, 6, 7, 8, 100]

# numbers greater than 5, lower than 10, but not including 8
condition(a) = (a > 5) && (a < 10) && (a ≠ 8)          #function to eventually broadcast
y           = x[condition.(x)]
```

```
julia> y
2-element Vector{Int64}:
 6
 7
```

LOGICAL INDEXING VIA `IN` AND `∈`

> **Remark**
>
> The symbols `∈` and `∉` used in this section can be inserted via tab completion:
>
> - `∈` by typing `\in`
> - `∉` by typing `\notin`

Another approach to selecting elements through logical indexing involves `in` and `∈`. Both are available as a function and an operator, although only `∈` supports broadcasting in its operating form. Considering this, the examples below use `in` when referring to the function form and `∈` for the operator form.

At its core, `in(a, list)` and `a ∈ list` check whether the scalar `a` matches any element in the vector `list`. For example, `in(2, [1, 2, 3])` and `2 ∈ [1, 2, 3]` both evaluate to `true`, because `2` is an element of `[1,2,3]`.

Leveraging `in` and `∈` for logical indexing requires replacing `a` by a collection `x`, while using its broadcast form. Note that a correct application necessitates that `list` is treated as a single object during broadcasting. Several techniques accomplish this, as discussed in **a previous section**. In particular, we'll consider the use of `Ref(list)`.[1]

The examples below demonstrate this approach. It constructs a vector `y` that contains the minimum and maximum of a vector `x`.

FUNCTIONS 'IN' AND '∈'

```
x            = [-100, 2, 4, 100]
list         = [minimum(x), maximum(x)]

# logical indexing (both versions are equivalent)
bool_indices = in.(x, Ref(list))    #`Ref(list)` or `(list,)`
bool_indices = (∈).(x,Ref(list))

y            = x[bool_indices]
```

```
julia> bool_indices
4-element BitVector:
 1
 0
 0
 1
julia> y
2-element Vector{Int64}:
 -100
  100
```

OPERATOR '∈'

```
x            = [-100, 2, 4, 100]
list         = [minimum(x), maximum(x)]

# logical indexing
bool_indices = x .∈ Ref(list)            # ∈ is the only option, not possible to broadcast
`in`


y            = x[bool_indices]
```

```
julia> bool_indices
4-element BitVector:
 1
 0
 0
 1
julia> y
2-element Vector{Int64}:
 -100
  100
```

Remark

The `in` function has an alternative **curried version**, allowing the user to directly broadcast `in` while treating `list` as a single element. The syntax for doing this is `in(list).(x)`, as shown in the example below.

CURRIED 'IN'

```
x            = [-100, 2, 4, 100]
list         = [minimum(x), maximum(x)]

#logical indexing
bool_indices = in(list).(x)    #no need to use `Ref(list)` or
`(list,)`
y            = x[bool_indices]
```

```
julia> bool_indices
4-element BitVector:
 1
 0
 0
 1
julia> y
2-element Vector{Int64}:
 -100
  100
```

Remark

`in` and `∈` have negated counterparts `!in` and `∉`, which select elements *not* belonging to a collection.

Below, we apply them to retain the elements of `x` that are neither its minimum nor its maximum.

FUNCTIONS '!IN' AND '∉'

```
x            = [-100, 2, 4, 100]
list         = [minimum(x), maximum(x)]

#identical vectors for logical indexing
bool_indices = (!in).(x, Ref(list))
bool_indices = (∉).(x, Ref(list))              #or `(!∈).(x,
Ref(list))`
y            = x[bool_indices]
```

```
julia> bool_indices
4-element BitVector:
 0
 1
 1
 0
julia> y
2-element Vector{Int64}:
 2
 4
```

OPERATORS '!IN' AND '∉'

```
x            = [-100, 2, 4, 100]
list         = [minimum(x), maximum(x)]

#vector for logical indexing
bool_indices = x .∉ Ref(list)

y            = x[bool_indices]
```

```
julia> bool_indices
4-element BitVector:
 0
 1
 1
 0
julia> y
2-element Vector{Int64}:
 2
 4
```

THE FUNCTIONS 'FINDALL' AND 'FILTER'

We close this section by presenting two additional methods for element selection: the functions `filter` and `findall`.

The function `filter` returns the *elements* of a vector `x` that satisfy a given condition. This condition must be expressed exclusively as a function that takes an element of `x` and returns a Boolean value. Note that, despite what the name might suggest, `filter` retains the elements that meet the condition, rather than discarding those that don't.

'FILTER'

```
x = [5, 6, 7, 8, 9]

y = filter(a -> a < 7, x)
```

```
julia> y
2-element Vector{Int64}:
 5
 6
```

The `findall` function behaves similarly to `filter`, but instead of returning the matching elements, it returns the *indices* within `x`. In addition, `findall` allows the condition to be specified in two forms: either as a Boolean-valued function (just like `filter`), or as a Boolean vector whose length matches that of `x`.

'FINDALL' - FUNCTION AS CONDITION

```
x = [5, 6, 7, 8, 9]

y = findall(a -> a < 7, x)
z = x[findall(a -> a < 7, x)]
```

```
julia> y
2-element Vector{Int64}:
 1
 2

julia> z
2-element Vector{Int64}:
 5
 6
```

'FINDALL' - BOOLEAN VECTOR AS CONDITION

```
x = [5, 6, 7, 8, 9]

y = findall(x .< 7)
z = x[findall(x .< 7)]
```

```
julia> y
2-element Vector{Int64}:
 1
 2

julia> z
2-element Vector{Int64}:
 5
 6
```

FOOTNOTES

[1] Executing `in.(x, list)` or `x .∈ list` would either produce incorrect results or directly raise an error. The expression would simultaneously iterate over each pair of `x` and `list`, when our goal is actually to compare each element of `x` against the entire `list`.

5g. In-Place Operations

Martin Alfaro
PhD in Economics

INTRODUCTION

This section focuses on **in-place operations**, where the contents of an existing object are directly modified. Unlike operations that generate new objects, in-place operations are characterized by the reuse of existing objects, giving rise to the expression *modifying values in place*.

Distinguishing between mutations and the creation of new copies is essential. If an operation mutates an object, any other variable that references the same object will also reflect the change. This behavior can be intended if you seek to update data, but it can introduce subtle bugs if you expected the original to remain unchanged. In-place modifications are also relevant for performance, as they reduce the memory overhead introduced when new objects are created. This aspect will be explored in Part II of the book.

At the heart of in-place operations is the concept of slices, introduced in a previous section. Before proceeding, I recommend reviewing that section before moving forward.

A **slice** of a vector `x` is defined as a subset of its elements, selected through the syntax `x[<indices>]`. Importantly, a slice can behave in two distinct ways:

- as a **copy**, which creates a new object with its own memory address.
- as a **view**, which references the original memory of `x`.

This distinction determines whether changes to the slice affect the original vector.

In what follows, we'll explore three approaches to mutating vectors in place. First, we'll examine mutations by assigning new collections to slices. After this, we'll cover the traditional approach of using for-loops to modify elements one at a time. Finally, we'll introduce the broadcasting assignment operator `.=`, which provides a concise construct for implementing in-place updates.

MUTATIONS VIA COLLECTIONS

The most straightforward approach to mutating a vector is to replace an entire slice with another collection. This is achieved through statements of the form `x[<indices>] = <expression>`, where `<expression>` must match the length of `x[<indices>]`. Because slices on the left-hand side of `=` act as views, the assignment effectively modifies the original vector `x`, rather than creating a new one.

SLICE VIA RANGE INDEXING

```
x         = [1, 2, 3]

x[2:end]  = [20, 30]
```

```
julia> x
3-element Vector{Int64}:
  1
 20
 30
```

SLICE VIA LOGICAL INDEXING

```
x         = [1, 2, 3]

x[x .≥ 2] = [20, 30]
```

```
julia> x
3-element Vector{Int64}:
  1
 20
 30
```

A common use case is when `<expression>` depends on either elements of the original vector or on the slice being modified itself. This allows for self-referential updates, where new values are computed from old ones.

UPDATING WITH OWN VECTOR

```
x         = [1, 2, 3]

x[2:end]  = [x[i] * 10 for i in 2:length(x)]
```

```
julia> x
3-element Vector{Int64}:
  1
 20
 30
```

UPDATING WITH OWN SLICE

```
x         = [1, 2, 3]

x[x .≥ 2] = x[x .≥ 2] .* 10
```

```
julia> x
3-element Vector{Int64}:
  1
 20
 30
```

Importantly, when the left-hand side is a single-element slice, the right-hand side of `=` accepts a scalar instead of a single-element vector. This property will be particularly relevant when we present mutations via for-loops.

SINGLE-ELEMENT SLICE

```
x         = [1, 2, 3]

x[3]      = 30
```

```
julia> x
3-element Vector{Int64}:
  1
  2
 30
```

⚠ **Warning!** - Vectors can only be mutated by objects of the same type

When a vector is created, the type of its elements is implicitly defined. Consequently, attempting to replace elements with an incompatible type will result in an error. For instance, a vector of type `Int64` can only be mutated with other `Int64` values or `Float64` values that can be converted into it. This is shown below.

MUTATION NOT ALLOWED

```
x         = [1, 2, 3]    # Vector{Int64}

x[2:3]    = [3.5, 4]     # 3.5 is Float64
```

```
ERROR: InexactError: Int64(3.5)
```

MUTATION ALLOWED (AUTOMATIC CONVERSION)

```
x         = [1, 2, 3]    # Vector{Int64}

x[2:3]      = [3.0, 4]        # 3.0 is Float64, but accepts
conversion to Int64
```

```
julia> x
3-element Vector{Int64}:
 1
 3
 4
```

MUTATIONS VIA FOR-LOOPS

Previously, we indicated that single-element slices on the left-hand side of `=` permit seamless mutations with scalar values. Thus, statements like `x[i] = 0` directly updates the element at position `i`, without requiring a collection on the right-hand side. Extending this idea, multiple elements of a vector can be updated within a for-loop.

A common use case of this approach arises when populating a vector with values. Typically, this involves first initializing a vector, whose initial contents are irrelevant, and then iterating over its elements with a for-loop to assign the desired values. The strategy is especially prevalent when storing outputs generated during a computation.

SINGLE-ELEMENT SLICE

```
x    = Vector{Int64}(undef, 3)  # `x` is initialized with 3 undefined elements

x[1] = 0
x[2] = 0
x[3] = 0
```

```
julia> x
3-element Vector{Int64}:
 0
 0
 0
```

MUTATION VIA FOR-LOOP

```
x    = Vector{Int64}(undef, 3)  # `x` is initialized with 3 undefined elements

for i in eachindex(x)
    x[i] = 0
end
```

```
julia> x
3-element Vector{Int64}:
 0
 0
 0
```

The approach presented above relies on `x[i]` on the left-hand side of `=`, ensuring each element is treated as a view. An alternative strategy is to use the function `view`. This enables the creation of a variable containing all the elements to be modified. Once that view is constructed, you can perform the mutation on the view itself, rather than repeatedly indexing into the original array.

In the following, we illustrate the technique by mutating a vector initialized with zeros. Note that the function `zeros` defaults to zeros with type `Float64`, explaining why `1` is automatically converted to `1.0`.

MUTATION VIA FOR-LOOP + SLICE

```
x     = zeros(3)


for i in 2:3
    x[i] = 1
end
```

```
julia> x
3-element Vector{Float64}:
 0.0
 1.0
 1.0
```

MUTATION VIA FOR-LOOP + 'VIEW'

```
x     = zeros(3)
slice = view(x, 2:3)

for i in eachindex(slice)
    slice[i] = 1
end
```

```
julia> x
3-element Vector{Float64}:
 0.0
 1.0
 1.0
```

> ⚠️ **Warning!** - For-Loops Should Always be Wrapped in Functions
>
> In the example above, we left the for-loop outside a function. The goal was to highlight the mutating strategy. In practice, however, placing for-loops in the global scope is highly discouraged: it not only severely hurts performance, but also introduces different variable-scope rules. In fact, earlier versions of Julia completely disallowed mutations in the global scope through for-loops.
>
> To perform mutations of global variables safely and efficiently, the for-loop should be placed inside a function. This requires introducing the idea of mutating functions, which we develop in the next section.

MUTATIONS VIA .=

Broadcasting provides a streamlined alternative to for-loops, and can be used for mutations as well. The implementation is based on the broadcasting of the assignment operator `=`, denoted as `.=`. Specifically, the syntax is `x[<indices>] .= <expression>`, where `<expression>` can be either a *vector* or a *scalar*.

When in particular `x[<indices>]` appears on the left-hand side of `.=` and `<expression>` is a vector, the `.=` operator produces the same outcome as using `=`. In fact, using `=` rather than `.=` tends to be more performant in those cases.

VECTOR AS REPLACEMENT

```
x       = [3, 4, 5]

x[1:2]  = x[1:2] .* 10
```

```
julia> x
3-element Vector{Int64}:
 30
 40
  5
```

VECTOR AS REPLACEMENT (EQUIVALENT)

```
x       = [3, 4, 5]

x[1:2] .= x[1:2] .* 10    # identical output (less performant)
```

```
julia> x
3-element Vector{Int64}:
 30
 40
  5
```

Considering this, the main cases where `.=` is useful for mutating `x` are:

- `x[<indices>].= <scalar>`,
- `x .= <expression>`, and
- `y .= <expression>` where `y` is a view of `x`.

Next, we analyze each case separately.

SCALARS ON THE RIGHT-HAND SIDE OF .=

A common scenario where mutations arise is when multiple elements need to be replaced with the *same* scalar value. Implementing this operation with `=` requires providing a collection on the right-hand side, whose length must match the number of elements on the left. This not only introduces unnecessary boilerplate, but also assumes prior knowledge of the elements being replaced.

The broadcasting assignment operator `.=` makes these operations simpler, simply requiring the execution of `x[<indices>] .= <scalar>`. The following code snippet employs this strategy to replace every negative value in `x` with zero.

SCALAR REPLACEMENT

```
x          = [-2, -1, 1]

x[x .< 0] .= 0
```

```
julia> x
3-element Vector{Int64}:
 0
 0
 1
```

OBJECT ITSELF ON THE LEFT-HAND SIDE OF .=

We've already shown that the inclusion of terms like `x[indices]` on the left-hand side of `=` results in mutations. Now, let's turn to cases where an entire object appears on the left-hand side. Here, the focus is on scenarios where the object is `x` itself. Instead, scenarios with slices constructed via `view` will be deferred until the next subsection.

When an object appears on the left-hand side, we need to carefully **distinguish between in-place operations and reassignments**. Whether one or the other operation is implemented depends on whether `.=` or `=` is employed: it's only with `.=` that a mutation takes place. Instead, `=` will perform a reassignment, creating a new object at a new memory address. While the distinction seems irrelevant, since `x` will ultimately hold the new values in both cases, we'll see in Part II of the book that the distinction actually matters for performance.

To illustrate, suppose our goal is to modify *all* the elements of a vector `x`. All the following approaches determine that `x` ends up holding the new intended values, but only the last two achieve this via mutation of `x`.

NEW OBJECT (ASSIGNMENT)

```
x    = [1, 2, 3]

x    = x .* 10
```

```
julia> x
3-element Vector{Int64}:
 10
 20
 30
```

MUTATION VIA .=

```
x    = [1, 2, 3]

x   .= x .* 10
```

```
julia> x
3-element Vector{Int64}:
 10
 20
 30
```

MUTATION VIA SLICE

```
x    = [1, 2, 3]

x[:] = x .* 10
```

```
julia> x
3-element Vector{Int64}:
 10
 20
 30
```

This risk of mixing up `.=` and `=` becomes even greater when using the `@.` macro, rather than manually inserting dots into each operator. In this case, the placement of `@.` relative to `=` determines whether the operation performs a reassignment or a mutation. Specifically:

- If `@.` appears *before* `=`, `x` is mutated since `.=` is used.
- If `@.` instead appears after `=`, only the right-hand side is broadcast, while the assignment is performed with `=`. This results in a reassignment, rather than a mutation.

MUTATION (INTENDED OPERATION)

```
x    = [1, 2, 3]

x   .= x .* 10
```

```
julia> x
3-element Vector{Int64}:
 10
 20
 30
```

MUTATION (EQUIVALENT VIA @.)

```
x    = [1, 2, 3]

@. x = x  * 10
```

```
julia> x
3-element Vector{Int64}:
 10
 20
 30
```

REASSIGNMENT USING @.

```
x    = [1, 2, 3]

x    = @. x * 10
```

```
julia> x
3-element Vector{Int64}:
 10
 20
 30
```

VIEW ALIASES ON THE LEFT-HAND SIDE OF .=

Let's continue with our analysis of entire objects on the left-hand side of `.=`. Our focus now shifts to **view aliases**: variables such as `slices` defined by `slices = view(x[<indices>])`. They allow us to work directly with `slice` rather than `x[<indices>]`.

The introduction of view aliases is especially convenient when performing multiple operations on the same slice. It avoids repeated references to `x[<indices>]`, which would be inefficient, error-prone, and tedious.

As before, it's crucial to distinguish between using `.=` and `=`. In particular, only `.=` will perform a mutation, while `=` will result in a reassignment. With view aliases, however, additional care is required since we must first define a slice (an assignment over a view) and then mutate that slice. This structure determines that two possible wrong uses emerge:

- the initial assignment is performed over a copy of `x[<indices>]`, rather than a view of `x[indices]`.
- the second step performs a reassignment (`=`), rather than a mutation (`.=`).

Below, we consider that the goal is to replace all negative values in `x` with zero. We illustrate the correct usage for implementing this operation, followed by two incorrect patterns.

MUTATION

```
x      = [-2, -1, 1]

slice  = view(x, x .< 0)
slice .= 0
```

```
julia> x
3-element Vector{Int64}:
 0
 0
 1
```

WRONG USE (COPY)

```
x      = [-2, -1, 1]

slice  = x[x .< 0]          # 'slice' is a copy
slice .= 0                  # this does NOT modify `x`
```

```
julia> x
3-element Vector{Int64}:
 -2
 -1
  1
```

WRONG USE (REASSIGNMENT)

```
x      = [-2, -1, 1]

slice  = view(x, x .< 0)
slice  = 0                  # this does NOT modify `x`
```

```
julia> x
3-element Vector{Int64}:
 -2
 -1
  1
```

Note that mutations with view aliases also allow `slice` to be included on the right-hand side of `=`. Below, we provide again the correct implementation, along with two incorrect ones.

MUTATION

```
x      = [1, 2, 3]

slice  = view(x, x .≥ 2)
slice .= slice .* 10        # same as 'x[x .≥ 2] = x[x .≥ 2] .* 10'
```

```
julia> x
3-element Vector{Int64}:
  1
 20
 30
```

WRONG USE (COPY)

```
x      = [1, 2, 3]

slice  = x[x .≥ 2]          # 'slice' is a copy
slice  = slice .* 10        # this does NOT modify `x`
```

```
julia> x
3-element Vector{Int64}:
 1
 2
 3
```

WRONG USE (REASSIGNMENT)

```
x      = [1, 2, 3]

slice  = view(x, x .≥ 2)
slice  = slice .* 10        # this does NOT modify `x`
```

```
julia> x
3-element Vector{Int64}:
 1
 2
 3
```

5h. In-Place Functions

Martin Alfaro
PhD in Economics

INTRODUCTION

This section continues our exploration of **techniques for mutating vectors**. The emphasis is now on **in-place functions**, defined as functions that mutate at least one of their arguments.

Many built-in functions in Julia have an in-place counterpart, which can be easily recognized by the `!` suffix in their names. These versions store the output in one of the function arguments, thereby avoiding the creation of a new object. In practice, it means that one or more of the arguments will be updated immediately after executing the function. For example, given a vector `x`, the call `sort(x)` produces a new vector with ordered elements, but without altering the original `x`. In contrast, the in-place version `sort!(x)` overwrites the content of `x`.

> **Appending ! To A Function Has No Impact on the Code**
>
> **Appending `!` to a function's name doesn't change the function's behavior**. It simply signals to users that the function performs a mutating operation. The goal is to make side effects explicit, helping programmers avoid unintended modifications of objects.

The benefits of in-place functions will become evident in Part II, when discussing high-performance computing. Essentially, by reusing existing objects, in-place functions eliminate the memory overhead associated with creating new objects.

DEFINING IN-PLACE FUNCTIONS

In-place functions, also known as **mutating functions**, are characterized by their ability to modify at least one of their arguments. For example, all the following are considered in-place functions. Following the convention in Julia, we denote them by appending `!` to the function's name.

IN-PLACE FUNCTION

```
x = [1,1]


function foo!(x)
    x[1] = 0
end
```

```
julia> x
2-element Vector{Int64}:
 1
 1
julia> foo!(x) #it mutates 'x'
julia> x
2-element Vector{Int64}:
 0
 1
```

IN-PLACE FUNCTION

```
y = [1,1]


function foo!(x)
    x[1] = 0
end
```

```
julia> y
2-element Vector{Int64}:
 1
 1
julia> foo!(y) #it mutates 'y'
julia> y
2-element Vector{Int64}:
 0
 1
```

IN-PLACE FUNCTION

```
x = [1,1]
y = [2,2]

function foo!(x,y)
    x[1] = 0
end
```

```
julia> x
2-element Vector{Int64}:
 1
 1
julia> foo!(x,y) #it mutates one of the arguments (just x)
julia> x
2-element Vector{Int64}:
 0
 1
```

Functions Can't Reassign Variables

While functions are capable of mutating values, they **can't reassign variables defined outside their scope**. Any attempt to redefine such variable will be interpreted as the creation of a new local variable.

The following code illustrates this behavior by redefining a function argument and a global variable. The output reflects that `foo` in each example treats the redefined `x` as a new local variable, only existing within `foo`'s scope.

FUNCTION ARGUMENT X

```
x = 2

function foo(x)
    x = 3
end
```

```
julia> x
2
julia> foo(x)
julia> x #functions can't redefine global variables, only
mutate them
2
```

GLOBAL VARIABLE X

```
x = [1,2]

function foo()
    x = [0,0]
end
```

```
julia> x
2-element Vector{Int64}:
 1
 2

julia> foo()

julia> x #functions can't redefine variables globally, only mutate them
2-element Vector{Int64}:
 1
 2
```

Strictly speaking, it's possible to reassign a variable by denoting a variable with the `global` keyword. However, its use is typically discouraged, explaining why we won't cover it.

BUILT-IN IN-PLACE FUNCTIONS

In Julia, many built-in functions that operate on vectors are available in two forms: a standard version that returns a new object and an in-place version that mutates its argument. To distinguish them, Julia's developers follow the naming convention that **any function ending with `!` corresponds to an in-place function**.

An example is given by the functions `sort` and `sort!`. Both arrange the elements of a vector in ascending order, with the option `rev=true` implementing a descending order. In its standard form, `sort(x)` creates a new vector containing `x`'s elements sorted, but leaving the original vector `x` unchanged. In contrast, the in-place version `sort!(x)` directly updates the original vector `x`, overwriting its contents with the sorted values. Both functions perform the same conceptual task, but they differ in whether they allocate new memory or reuse existing storage.

SORT - STANDARD VERSION

```
x      = [2, 1, 3]

output = sort(x)
```

```
julia> x
3-element Vector{Int64}:
 2
 1
 3
julia> output
3-element Vector{Int64}:
 1
 2
 3
```

SORT - IN-PLACE VERSION

```
x      = [2, 1, 3]

sort!(x)
```

```
julia> x
3-element Vector{Int64}:
 1
 2
 3
```

It's also common to have in-place functions that accept arguments only serving as a destination for their output. This argument isn't involved in the computation itself. Instead, it provides pre-allocated storage for the function to write its results. By doing so, we avoid allocating a new array every time the function runs.

For instance, `map(foo, x)` applies the function `foo` to each element of `x` and returns a freshly allocated vector. In contrast, `map!(foo, output, x)` performs the same element-wise transformation, but writes directly into the existing vector `output`.

MAP

```
x      = [1, 2, 3]


output = map(a -> a^2, x)
```

```
julia> x
3-element Vector{Int64}:
 1
 2
 3

julia> output
3-element Vector{Int64}:
 1
 4
 9
```

MAP!

```
x      = [1, 2, 3]
output = similar(x)             # we initialize `output`

map!(a -> a^2, output, x)       # we update `output`
```

```
julia> x
3-element Vector{Int64}:
 1
 2
 3

julia> output
3-element Vector{Int64}:
 1
 4
 9
```

FOR-LOOP MUTATIONS VIA IN-PLACE FUNCTIONS

In Julia, any performance-critical code should live inside functions. This not only prevents issues with variable scope, but it's also key for performance. The subject will be discussed extensively in Part II of the book. Similarly, for-loops often provide the most direct path to high performance in Julia.

When both aspects are combined, the ability of functions to mutate their arguments via for-loops is crucial: it determines an efficient computational approach that can reuse existing storage, instead of allocating new arrays repeatedly.

A typical strategy to implement these operations is to initialize vectors with `undef` values, pass them to a function, and fill them element by element via a for-loop. The examples below illustrate this approach.

FOR-LOOP MUTATION

```
x = [3,4,5]

function foo!(x)
    for i in 1:2
        x[i] = 0
    end
end
```

```
julia> foo!(x)

julia> x
3-element Vector{Int64}:
 0
 0
 5
```

POPULATING VECTOR WITH FOR-LOOP

```
x = Vector{Int64}(undef, 3)          # initialize a vector with 3 elements

function foo!(x)
    for i in eachindex(x)
        x[i] = 0
    end
end
```

```
julia> foo!(x)

julia> x
3-element Vector{Int64}:
 0
 0
 0
```

6a. Overview and Goals

Martin Alfaro
PhD in Economics

The previous chapter equipped us with techniques for manipulating vectors. We now expand our toolkit by introducing other data structures and common functions for data analysis.

Specifically, Section 6b **introduces additional types for collections**, including dictionaries and named tuples. Building on our grasp of tuples and vectors, we're now well-positioned to understand the unique features of these alternatives and understand when they're more suitable.

Meanwhile, Section 6c **expands on tools for streamlining code**, which are worthwhile for a daily use of Julia. These tools will make your coding experience smoother, by reducing **boilerplate code** and improving syntax readability. One notable example is the use of pipes.

Additionally, Section 6d introduces **standard functions for data manipulation**, such as removing duplicates, sorting elements, and rounding numbers.

To conclude the chapter, Section 6e **puts into practice the tools we've covered**. This will be done by analyzing data from a hypothetical scenario involving a YouTuber's earnings. While data analysis is generally carried out by using specialized packages such as `DataFrames`, the example will prepare you for a seamless transition to those powerful libraries.

6b. Named Tuples and Dictionaries

Martin Alfaro
PhD in Economics

INTRODUCTION

Our previous discussions of collections have centered around vectors. We also introduced tuples, although without exploring them in much depth. The current section fills that gap y offering a more comprehensive analysis of tuples.

We then broaden the picture by introducing two additional collection types: **named tuples** and **dictionaries**. Along the way, we'll develop a general perspective on collections in terms of their keys and values. Additionally, we'll discuss common operations for manipulating collections, and examine techniques for transforming one collection into another.

KEYS AND VALUES

Most collections in Julia are characterized by **keys**. They serve as unique identifiers for their elements and are paired with a corresponding **value**.[1] For instance, the vector `x = [2, 4, 6]` has the indices `1, 2, 3` as its keys, and `2, 4, 6` as their respective values.

Keys are more general than indices: while indices are limited to integer identifiers, keys can be any valid Julia object (e.g., strings, numbers, or other objects).

To extract the keys and values of a collection, Julia provides the functions `keys` and `values`. The following examples demonstrate their behavior with vectors and tuples, whose keys are represented by indices. Note that neither `keys` nor `values` return a vector, requiring the `collect` function to obtain a vector representation.

VECTOR

```
x        = [4, 5, 6]

x_keys   = collect(keys(x))
x_values = collect(values(x))
```

```
julia> x_keys
3-element Vector{Int64}:
 1
 2
 3
julia> x_values
3-element Vector{Int64}:
 4
 5
 6
```

TUPLE

```
x        = (4, 5, 6)

x_keys   = collect(keys(x))
x_values = collect(values(x))
```

```
julia> x_keys
3-element Vector{Int64}:
 1
 2
 3
julia> x_values
3-element Vector{Int64}:
 4
 5
 6
```

THE TYPE `PAIR`

Collections of key-value pairs in Julia are represented by the type `Pair{<key type>, <value type>}`. Although we'll rarely work with objects of this type in this book, they form the basis for constructing other collections, such as dictionaries and named tuples.

A key-value pair can be created by using the operator `=>`, as in `<key> => <value>`. For instance, `"a" => 1` constructs a pair, where `a` is the key and `1` the corresponding value. Alternatively, pairs can be created using the function `Pair(<key>, <value>)`, where `Pair("a",1)` is equivalent to `"a" => 1`.

Given a pair `x`, we can access its key via either `x[1]` or `x.first`. Likewise, its value can be obtained using either `x[2]` or `x.second`. All this is demonstrated below.

KEY OF A PAIR

```
some_pair = ("a" => 1)      # or simply 'some_pair = "a" => 1'

some_pair = Pair("a", 1)    # equivalent
```

```
julia> some_pair
"a" => 1
julia> some_pair[1]
"a"
julia> some_pair.first
"a"
```

VALUE OF A PAIR

```
some_pair = ("a" => 1)      # or simply 'some_pair = "a" => 1'

some_pair = Pair("a", 1)    # equivalent
```

```
julia> some_pair
"a" => 1
julia> some_pair[2]
1
julia> some_pair.second
1
```

THE TYPE `SYMBOL`

The type used to represent keys vary across collections. One particularly common choice is `Symbol`, which offers an efficient way to encode string-based identifiers. A symbol named `x` is written as `:x`, and can be constructed from a string using the function `Symbol(<string>)`.[2]

A VECTOR OF SYMBOLS

```
vector_symbols = [:x, :y]
```

```
julia> vector_symbols
2-element Vector{Symbol}:
 :x
 :y
```

A VECTOR OF SYMBOLS - EQUIVALENT

```
vector_symbols = [Symbol("x"), Symbol("y")]
```

```
julia> vector_symbols
2-element Vector{Symbol}:
 :x
 :y
```

NAMED TUPLES

> ⚠ **Warning!**
>
> As we'll show later in the book, **tuples and named tuples are only suitable for small collections**. Using them with large collections can result in poor performance or directly fatal errors. For large collections, arrays remain the preferred choice.
>
> Defining what qualifies as *small* is challenging, and unfortunately there's no definitive answer. We can only indicate that collections with fewer than 10 elements are certainly small, while those exceeding 100 elements clearly exceed the definition.

Named tuples share several properties with regular tuples, including their **immutability**. However, they also exhibit some notable differences. An important one is that **the keys of named tuples are objects of type** `Symbol`, in contrast to numerical indices as in regular tuples.

Named tuples also differ syntactically, requiring being enclosed in parentheses `()`. Omitting them isn't possible, unlike with regular tuples. Furthermore, when creating a single-element named tuple, the syntax requires either a trailing comma `,` after the element (similar to regular tuples) or a leading semicolon `;` before the element.[3]

To construct a named tuple, each element must be specified in the format `<key> = <value>`, such as `a = 10`. Alternatively, we can use a pair `<key with Symbol type> => <value>`, as in `:a => 10`. Once a named tuple `nt` is created, the element `a` can be accessed either by key lookup `nt[:a]` or by dot syntax `nt.a`.

The following examples illustrate these concepts.

NAMED TUPLE

```
# all 'nt' are equivalent
nt = (  a=10, b=20)
nt = (; a=10, b=20)
nt = (  :a => 10, :b => 20)
nt = (; :a => 10, :b => 20)
```

```
julia> nt
(a = 10, b = 20)

julia> nt.a
10

julia> nt[:a]
10
```

SINGLE-ELEMENT NAMED TUPLE

```
# all 'nt' are equivalent
nt = (  a=10,)
nt = (; a=10 )
nt = (  :a => 10,)
nt = (; :a => 10 )

#not 'nt =  (a = 10)'  -> this is interpreted as 'nt = a = 10'
#not 'nt = (:a => 10)' -> this is interpreted as a pair
```

```
julia> nt
(a = 10,)
julia> nt.a
10
julia> nt[:a]
10
```

DISTINCTION BETWEEN THE CREATION OF TUPLES AND NAMED TUPLES

It's possible to create named tuples from existing variables. For instance, given variables `x = 10` and `y = 20`, one can define `nt = (; x, y)`. This creates a named tuple with keys `x` and `y`, whose corresponding values are `10` and `20`.

The semicolon `;` plays a crucial role in this construction, as it distinguishes named tuples from regular tuples. Omitting it, as in `nt = (x, y)`, would instead result in a regular tuple.

MULTIPLE ELEMENTS

```
x   = 10
y   = 20

nt  = (; x, y)
tup = (x, y)
```

```
julia> nt
(x = 10, y = 20)
julia> tup
(10, 20)
```

SINGLE ELEMENT

```
x   = 10


nt  = (; x)
tup = (x, )
```

```
julia> nt
(x = 10,)
julia> tup
(10,)
```

DICTIONARIES

Dictionaries are collections of key-value pairs, exhibiting three distinctive features:

- **Dictionary keys can be any object**: strings, numbers, and other objects are possible.
- **Dictionaries are mutable**: elements can be modified, added, and removed after creation.
- **Dictionaries are unordered**: keys have no inherent order.

The function `Dict` can be used to create dictionaries, where each argument is a key-value pair written in the form `<key> => <value>`.

NUMBER

```
some_dict = Dict(3 => 10, 4 => 20)
```

```
julia> some_dict
Dict{Int64, Int64} with 2 entries:
  4 => 20
  3 => 10
julia> some_dict[1]
10
```

STRING

```
some_dict = Dict("a" => 10, "b" => 20)
```

```
julia> some_dict
Dict{String, Int64} with 2 entries:
  "b" => 20
  "a" => 10
julia> some_dict["a"]
10
```

SYMBOL

```
some_dict = Dict(:a => 10, :b => 20)
```

```
julia> some_dict
Dict{Symbol, Int64} with 2 entries:
  :a => 10
  :b => 20
julia> some_dict[:a]
10
```

TUPLE

```
some_dict = Dict((1,1) => 10, (1,2) => 20)
```

```
julia> some_dict
10
julia> some_dict[(1,1)]
10
```

Regular dictionaries are inherently unordered, meaning that the access to their elements doesn't follow any pattern. The following example illustrates this, by collecting the dictionary keys into a vector.[4]

NUMBER

```
some_dict      = Dict(3 => 10, 4 => 20)

keys_from_dict = collect(keys(some_dict))
```

```
julia> keys_from_dict
2-element Vector{Int64}:
 4
 3
```

STRING

```
some_dict      = Dict("a" => 10, "b" => 20)

keys_from_dict = collect(keys(some_dict))
```

```
julia> keys_from_dict
2-element Vector{String}:
 "b"
 "a"
```

SYMBOL

```
some_dict      = Dict(:a => 10, :b => 20)

keys_from_dict = collect(keys(some_dict))
```

```
julia> keys_from_dict
2-element Vector{Symbol}:
 :a
 :b
```

TUPLE

```
some_dict      = Dict((1,1) => 10, (1,2) => 20)

keys_from_dict = collect(keys(some_dict))
```

```
julia> keys_from_dict
2-element Vector{Tuple{Int64, Int64}}:
 (1, 2)
 (1, 1)
```

CREATING TUPLES, NAMED TUPLES, AND DICTIONARIES

Tuples, named tuples, and dictionaries can be constructed from other collections. The only requirement is that the source collection possesses a key-value structure.

To demonstrate this feature, we begin by creating **dictionaries** from a variety of collections.

VECTOR

```
vector = [10, 20] # or tupl = (10,20)



dict = Dict(pairs(vector))
```

```
julia> dict
Dict{Int64, Int64} with 2 entries:
  2 => 20
  1 => 10
```

VECTORS

```
keys_for_dict   = [:a, :b]
values_for_dict = [10, 20]


dict = Dict(zip(keys_for_dict, values_for_dict))
```

```
julia> dict
Dict{Symbol, Int64} with 2 entries:
  :a => 10
  :b => 20
```

TUPLES

```
keys_for_dict   = (:a, :b)
values_for_dict = (10, 20)


dict = Dict(zip(keys_for_dict, values_for_dict))
```

```
julia> dict
Dict{Symbol, Int64} with 2 entries:
  :a => 10
  :b => 20
```

NAMED TUPLE

```
nt_for_dict = (a = 10, b = 20)



dict = Dict(pairs(nt_for_dict))
```

```
julia> dict
Dict{Symbol, Int64} with 2 entries:
  :a => 10
  :b => 20
```

COMPREHENSION

```
keys_for_dict      = (:a, :b)
values_for_dict    = (10, 20)
vector_keys_values     =     [(keys_for_dict[i],     values_for_dict[i])     for     i     in
eachindex(keys_for_dict)]

dict = Dict(vector_keys_values)
```

```
julia> dict
Dict{Symbol, Int64} with 2 entries:
  :a => 10
  :b => 20
```

Likewise, we can define a **tuple** from other collections, as shown below.

VARIABLES

```
a               = 10
b               = 20

tup             = (a, b)
```

```
julia> tup
(10, 20)
```

VECTORS 1

```
values_for_tup = [10, 20]

tup             = (values_for_tup... ,)
```

```
julia> tup
(10, 20)
```

VECTORS 2

```
values_for_tup = [10, 20]

tup             = Tuple(values_for_tup)
```

```
julia> tup
(10, 20)
```

Finally, **named tuples** can also be constructed from other collections.

VARIABLES

```
a = 10
b = 20


nt = (; a, b)
```

```
julia> nt
(a = 10, b = 20)
```

VECTORS 1

```
keys_for_nt   = [:a, :b]
values_for_nt = [10, 20]


nt = (; zip(keys_for_nt, values_for_nt)...)
```

```
julia> nt
(a = 10, b = 20)
```

VECTORS 2

```
keys_for_nt   = [:a, :b]
values_for_nt = [10, 20]


nt = NamedTuple(zip(keys_for_nt, values_for_nt))
```

```
julia> nt
(a = 10, b = 20)
```

TUPLES

```
keys_for_nt   = (:a, :b)
values_for_nt = (10, 20)


nt = NamedTuple(zip(keys_for_nt, values_for_nt))
```

```
julia> nt
(a = 10, b = 20)
```

COMPREHENSION

```
keys_for_nt        = [:a, :b]
values_for_nt      = [10, 20]
vector_keys_values = [(keys_for_nt[i], values_for_nt[i]) for i in eachindex(keys_for_nt)]

nt = NamedTuple(vector_keys_values)
```

```
julia> nt
(a = 10, b = 20)
```

DICTIONARY

```
dict = Dict(:a => 10, :b => 20)



nt = NamedTuple(vector_keys_values)
```

```
julia> nt
(a = 10, b = 20)
```

DESTRUCTURING TUPLES AND NAMED TUPLES

Previously, we demonstrated how to create tuples and named tuples from variables. Next, we show that the reverse operation is also possible, where **values are extracted from a tuple or named tuple and then assigned to separate variables**. This process is known as **destructuring**, enabling users to "unpack" the values of a collection into distinct variables.

Destructuring involves the assignment operator `=` with either a tuple or named tuple on the left-hand side. The choice between one or the other determines what objects can be used on the right-hand side. Specifically, tuples on the left-hand side are quite flexible, allowing values to be unpacked from a variety of collections. Named tuples on the left-hand side, instead, necessarily require a named tuple on the right-hand side. Next, we develop each case separately.

DESTRUCTURING COLLECTIONS THROUGH TUPLES

Given a collection `list` with two elements, destructuring via tuples allows us to unpack its values into the variables `x` and `y`. The syntax for this is `<tuple> = <collection>`, as in `x,y = list`. In the following, we illustrate the process by considering different objects as `list`.

VECTORS

```
list = [3,4]

x,y  = list
```

```
julia> x
3
julia> y
4
```

RANGES

```
list = 3:4

x,y  = list
```

```
julia> x
3
julia> y
4
```

TUPLES

```
list = (3,4)

x,y  = list
```

```
julia> x
3
julia> y
4
```

NAMED TUPLES

```
list = (a = 3, b = 4)

x,y  = list
```

```
julia> x
3
julia> y
4
```

In addition to unpacking all elements, destructuring can also be applied to only a subset of elements. The assignment is then carried out sequentially, following the collection's inherent order.

Importantly, this method doesn't allow skipping specific values. When a value must be disregarded, the conventional approach is to bind it to the special variable name `_`. This symbol serves purely as a placeholder, indicating that the value is unimportant and has no effect on execution.

For illustration, we'll use a vector as an example of `list`. Nonetheless, the same principle applies to any collection.

FIRST ELEMENT

```
list  = [3,4,5]

(x,)  = list
```

```
julia> x
3
```

FIRST ELEMENTS

```
list  = [3,4,5]

x,y   = list
```

```
julia> x
3
julia> y
4
```

LAST ELEMENT

```
list  = [3,4,5]

_,_,z = list        # _ or any symbol (_ just signals we don't care about that value)
```

```
julia> z
5
```

FIRST AND LAST ELEMENT

```
list  = [3,4,5]

x,_,z = list        # _ or any symbol (it just signals we don't care about that value)
```

```
julia> x
3
julia> y
5
```

DESTRUCTURING WITH NAMED TUPLES ON BOTH SIDES

Destructuring can also be applied with named tuples on the left-hand side of an assignment. In this form, values are extracted by referencing directly to their field names, rather than relying on their positional order. The main advantage of this approach is flexibility: variables can be assigned in any order, provided each name matches a field in the named tuple.

MULTIPLE VARIABLES

```
nt             = (; key1 = 10, key2 = 20, key3 = 30)

(; key3, key1) = nt            # keys in any order
```

```
julia> key1
10
julia> key3
30
```

A SINGLE VARIABLE

```
nt             = (; key1 = 10, key2 = 20, key3 = 30)

(; key2)       = nt            # only one key
```

```
julia> key2
20
```

> **Remark**
>
> When destructuring with a tuple on the left-hand side and a named tuple on the right-hand side, keep in mind that the assignment is carried out strictly by position. This means that variable names on the left don't influence the result. In other words, the keys of the named tuple are completely ignored during the process.

NAMED TUPLE

```
nt = (; key1 = 10, key2 = 20, key3 = 30)

(key2, key1)     = nt        # variables defined according to
POSITION
key2, key1     = nt     # alternative notation
```

```
julia> key2
10
julia> key1
20
```

TUPLE

```
nt = (; key1 = 10, key2 = 20, key3 = 30)

(; key2, key1) = nt     # variables defined according to KEY
 ; key2, key1  = nt     # alternative notation
```

```
julia> key2
20
julia> key1
10
```

The same caveat applies to assignments of single variables.

NAMED TUPLE

```
nt       = (; key1 = 10, key2 = 20)

(key2,)   = nt                # variable defined according to
POSITION
```

```
julia> key2
10
```

TUPLE

```
nt       = (; key1 = 10, key2 = 20)

(; key2) = nt            # variable defined according to KEY
```

```
julia> key2
20
```

APPLICATIONS OF DESTRUCTURING

Destructuring named tuples is particularly valuable in scientific modeling, where numerous parameters are referenced repeatedly throughout the code. By grouping all parameters into a single named tuple, we can pass them to a function as *one* consolidated argument. Functions written in this style typically begin by destructuring the named tuple, extracting exactly the parameters needed for the computations.

MULTIPLE ARGUMENTS

```
β = 3
δ = 4
ϵ = 5

# function 'foo' uses β and δ, but not ϵ
function foo(x, δ, β)
    x * δ + exp(β) / β


end

output = foo(1, δ, β)
```

```
julia> output
10.6952
```

NO DESTRUCTURING

```
parameters_list = (; β = 3, δ = 4, ϵ = 5)



# function 'foo' uses β and δ, but not ϵ
function foo(x, parameters_list)
    x * parameters_list.δ + exp(parameters_list.β) / parameters_list.β


end

output = foo(1, parameters_list)
```

```
julia> output
10.6952
```

DESTRUCTURING

```
parameters_list = (; β = 3, δ = 4, ϵ = 5)



# function 'foo' uses β and δ, but not ϵ
function foo(x, parameters_list)
    (; β, δ) = parameters_list

    x * δ + exp(β) / β
end

output = foo(1, parameters_list)
```

```
julia> output
10.6952
```

Destructuring also provides a convenient solution for retrieving multiple outputs from a function. When a function wraps its results in a tuple, we can then immediately unpack the tuple into separate variables. This often makes the subsequent code easier to read and reason about. In the example below, the function `foo` returns a tuple, which is then unpacked into variables `x`, `y`, and `z`.

TUPLE AS OUTPUT

```
function foo()
    out1 = 2
    out2 = 3
    out3 = 4

    out1, out2, out3
end

x, y, z = foo()
```

Another common use of destructuring arises when we only need a subset of a function's outputs. While both tuples and named tuples can be applied for this purpose, tuples offer greater flexibility, as they can be combined with various types of collections. In contrast, named tuples are restricted to returning another named tuple as the function's output, thus requiring knowing the field names in advance.

The following example illustrates this functionality by extracting the first and third output of `foo`, while ignoring the second.

TUPLE AS OUTPUT (TUPLE)

```
function foo()
    out1 = 2
    out2 = 3
    out3 = 4

    out1, out2, out3
end

x, _, z = foo()
```

NAMED TUPLE AS OUTPUT (NAMED TUPLES)

```
function foo()
    out1 = 2
    out2 = 3
    out3 = 4

    (; out1, out2, out3)
end

(; out1, out3) = foo()
```

FOOTNOTES

1. Not all collections map keys to values. For example, the type `Set`, which represents a group of unique unordered elements, doesn't have a key-value structure.

2. `Symbol` also enables the programmatic creation of variables. A typical use case arises in data analysis, where symbols are employed to generate new columns in tabular data structures.

3. The semicolon notation `;` may seem odd, but it actually comes from the syntax for keyword arguments in functions.

4. The package `OrderedCollections` addresses this, by offering a special dictionary called `OrderedDict`. It behaves similarly to regular dictionaries, including their syntax, but endows the dictionary with an order.

6c. Chaining Operations

Martin Alfaro
PhD in Economics

INTRODUCTION

This section introduces some strategies for managing computations that involve multiple intermediate steps. These approaches are designed to streamline the writing process. Moreover, they help preserve a clean namespace by avoiding the need to store variables for each intermediate result.

The first strategy relies on **let blocks**, which introduce a new variable scope and return the value of the final expression. They provide a compact way to group a sequence of operations, offering a function-like structure with less syntactic burden. Because each block introduces its own scope, all intermediate variables remain local, helping maintain a tidy namespace.

The second strategy leverages **pipes**, which chain a series of operations and return the final output. As the built-in pipe can become unwieldy beyond single-argument functions, we also present an alternative based on the `Pipe` package.

LET BLOCKS

Let blocks become particularly convenient when performing a series of operations but only the final result matters. To illustrate their utility, consider the task of computing the rounded logarithm of `a`'s absolute value. Formally, $\text{round}\left(\ln\left(|a|\right)\right)$.

In Julia, this operation can be directly written as `round(log(abs(a)))`, where `round(a)` returns the integer nearest to `a`. However, the nested parentheses make the expression hard to read, with the issue potentially exacerbated if the variables or functions had long names.

A straightforward way to improve its clarity is to break the whole operation into multiple steps: *i)* compute the absolute value of `a`, *ii)* compute the logarithm of the result, and *iii)* round the resulting output. While this can be implemented through three intermediate variables that store the output in each step, such an approach would clutter our namespace and potentially obscure the nested nature of the operations.

A more elegant solution is to introduce a **let-block**, which resembles functions in several respects. It introduces a new scope delimited by the `let` and `end` keywords, enabling multiple calculations to be performed locally. The result of the last calculation is then returned as the output. Like functions, let-blocks also allow arguments to be passed by incorporating them after the `let` keyword.

To highlight the benefits of let-blocks, the following examples add other approaches to computing `round(log(abs(a)))`.

VIA SINGLE LINE

```
a      = -2

output = round(log(abs(a)))
```

```
julia> output
1.0
```

VIA MULTIPLE LINES

```
a      = -2

temp1  = abs(a)
temp2  = log(temp1)
output = round(temp2)
```

```
julia> output
1.0

julia> temp1
2

julia> temp2
0.693147
```

VIA LET BLOCK

```
a      = -2

output = let b = a          # 'b' is a local variable having the value of 'a'
   temp1 = abs(b)
   temp2 = log(temp1)
   round(temp2)
end
```

```
julia> output
1.0

julia> temp1 #local to let-block
ERROR: UndefVarError: `temp1` not defined

julia> temp2 #local to let-block
ERROR: UndefVarError: `temp2` not defined
```

VIA LET BLOCK (EQUIVALENT)

```
a      = -2

output = let a = a          # the 'a' on the left of `=` defines a local variable
   temp1 = abs(a)
   temp2 = log(temp1)
   round(temp2)
end
```

```
julia> output
1.0
julia> temp1 #local to let-block
ERROR: UndefVarError: `temp1` not defined
julia> temp2 #local to let-block
ERROR: UndefVarError: `temp2` not defined
```

Let Blocks Can Mutate Variables

Let blocks follow the same rules as functions with respect to assignment and mutation. This means the values passed into a let block can be mutated, but passed global variables can't be reassigned.

VARIABLE MUTATION (POSSIBLE)

```
x = [2,2,2]

output = let x = x
   x[1] = 0
end
```

```
julia> x
3-element Vector{Int64}:
 0
 2
 2
```

VARIABLE REASSIGNMENT (NOT POSSIBLE)

```
x = [2,2,2]

output = let x = x
   x = 0
end
```

```
julia> x
3-element Vector{Int64}:
 2
 2
 2
```

> Given the possibility of unintended side effects from mutating global variables, you should exercise caution.

PIPES

For operations consisting of multiple intermediate step, pipes constitutes an alternative to let-blocks. Unlike let-blocks, they're specifically designed to chain operations together, with each step receiving the output of the previous one as its input. Each individual step in the chain is separated via the `|>` keyword.

Pipes are particularly well-suited for sequential applications of single-argument functions. To illustrate their use, let's revisit the example presented above.

OPERATION

```
a      = -2

output = round(log(abs(a)))
```

```
julia> output
1.0
```

OPERATION WITH PIPES

```
a      = -2

output = a |> abs |> log |> round
```

```
julia> output
1.0
```

> **Let Blocks and Pipes For Long Names**
>
> Both let blocks and pipes help create temporary aliases for variables with lengthy names. This lets users assign meaningful names to variables, while preserving code readability.

VIA SINGLE LINE

```
variable_with_a_long_name = 2

output          =          variable_with_a_long_name          -
log(variable_with_a_long_name) / abs(variable_with_a_long_name)
```

```
julia> output
1.65343
```

VIA MULTIPLE LINES

```
variable_with_a_long_name = 2

temp   = variable_with_a_long_name
output = temp - log(temp) / abs(temp)
```

```
julia> output
1.65343
```

VIA PIPE

```
variable_with_a_long_name = 2

output = variable_with_a_long_name  |>
         a -> a - log(a) / abs(a)
```

```
julia> output
1.65343
```

VIA LET BLOCK

```
variable_with_a_long_name = 2

output = let x = variable_with_a_long_name
    x - log(x) / abs(x)
end
```

```
julia> output
1.65343
```

BROADCASTING PIPES

Just like any other operator, pipes can be broadcast by prefixing them with a dot `.`. In this form, `.|>` indicates that the subsequent operation must be applied element-wise to the preceding output. For example, the expression `x .|> abs` is equivalent to `abs.(x)`.

We demonstrate this behavior below, where we transform each element of `x` with the logarithm of its absolute values, and then sum the results.

VIA SINGLE LINE

```
x      = [-1,2,3]

output = sum(log.(abs.(x)))
```

```
julia> output
1.79176
```

VIA MULTIPLE LINES

```
x      = [-1,2,3]

temp1  = abs.(x)
temp2  = log.(temp1)
output = sum(temp2)
```

```
julia> output
1.79176
```

VIA BROADCAST PIPES

```
x      = [-1,2,3]

output = x .|> abs .|> log |> sum
```

```
julia> output
1.79176
```

PIPES WITH MORE COMPLEX OPERATIONS

So far, our examples of pipes have followed a simple pattern, with each step consisting of a single-argument function. Unfortunately, this form precludes the application of pipes to multiple-argument functions or even operations. For example, it prevents the inclusion of expressions like `foo(x,y)` or `2 * x`.

To address this limitation, we can **combine pipes with anonymous functions**. This enables users to specify how the output of the previous step is integrated into the subsequent operation. In this way, the utility of pipes is significantly expanded, as demonstrated below.

VIA SINGLE LINE

```
a      = -2

output = round(2 * abs(a))
```

```
julia> output
4
```

VIA MULTIPLE LINES

```
a      = -2

temp1  = abs(a)
temp2  = 2 * temp1
output = round(temp2)
```

```
julia> output
4
```

VIA PIPES

```
a      = -2

output = a |> abs |> (x -> 2 * x) |> round

#equivalent and more readable
output = a              |>
         abs            |>
         x -> 2 * x     |>
         round
```

```
julia> output
4
```

PACKAGE PIPE

Combining pipes and anonymous functions can result in cumbersome code, defeating the very own purpose of using pipes in the first place.

The `Pipe` package provides a convenient solution, eliminating the need for anonymous functions. By prefixing the operation chain with the `@pipe` macro, you can reference the output of the previous step by the symbol `_`. Additionally, for single-argument operations that don't require anonymous functions, `@pipe` maintains the same syntax as built-in pipes.

To illustrate its convenience, below we reimplement the last code snippet.

VIA BUILT-IN PIPES

```
#
a      = -2

output = a |> abs |> (x -> 2 * x) |> round

#equivalent and more readable
output =       a            |>
               abs          |>
               x -> 2 * x   |>
               round
```

```
julia> output
4
```

VIA PACKAGE PIPE

```
using Pipe
a = -2

output = @pipe a |> abs |> 2 * _ |> round

#equivalent and more readable
output = @pipe a            |>
               abs          |>
               2 * _        |>
               round
```

```
julia> output
4
```

FUNCTION COMPOSITION (*OPTIONAL*)

An alternative approach to nesting functions is through the composition operator `∘`. This symbol can be inserted by tab completion through `\circ`, and its functionality is the same as in Mathematics. Specifically, given some functions `f` and `g`, `(f ∘ g)(x)` is equivalent to `f(g(x))`.

The operator `∘` can be considered as an alternative to piping, as `(f ∘ g)(x)` provides the same output as `x |> f |> g`. Moreover, `∘` is also available as a function, where `∘(f,g)(x)` is equivalent to `(f ∘ g)(x)`. The following examples demonstrate its use.

EXAMPLE WITH BUILT-IN FUNCTIONS

```
a        = -1



# all `output` are equivalent
output   = log(abs(a))
output   = a |> abs |> log
output   = (log ∘ abs)(a)
output   = ∘(log, abs)(a)
```

```
julia> output
0.0
```

EXAMPLE WITH USER-DEFINED FUNCTIONS

```
a        = 2
outer(a) = a + 2
inner(a) = a / 2

# all `output` are equivalent
output   = (a / 2) + 2
output   = outer(inner(a))
output   = a |> inner |> outer
output   = (outer ∘ inner)(a)
output   = ∘(outer, inner)(a)
```

```
julia> output
3.0
```

Importantly, the resulting function from function composition can be broadcast. To understand this notation more clearly, you should think of compositions as defining a new function: $h := f \circ g$. This entails that $h\left(x\right) := \left(f \circ g\right)\left(x\right)$, and therefore $h\left(x\right) := \left(f \circ g\right)\left(x\right) = f\left[g\left(x\right)\right]$. Given this, broadcasting `h` requires `h.(x)`, which is equivalent to `(f ∘ g).(x)` or `∘(f,g).(x)`.

EXAMPLE WITH BUILT-IN FUNCTIONS

```
x        = [1, 2, 3]



# all `output` are equivalent
output   = log.(abs.(x))
output   = x .|> abs .|> log
output   = (log ∘ abs).(x)
output   = ∘(log, abs).(x)
```

```
julia> output
3-element Vector{Float64}:
 0.0
 0.693147
 1.09861
```

EXAMPLE WITH USER-DEFINED FUNCTIONS

```
x        = [1, 2, 3]
outer(a) = a + 2
inner(a) = a / 2

# all `output` are equivalent
output   = (x ./ 2) .+ 2
output   = outer.(inner.(x))
output   = x .|> inner .|> outer
output   = (outer ∘ inner).(x)
output   = ∘(outer, inner).(x)
```

```
julia> output
3-element Vector{Float64}:
 2.5
 3.0
 3.5
```

We can also broadcast the composition operator `∘` itself, enabling the simultaneous application of multiple functions to the same object. For instance, the following implementation ensures that each function takes the absolute value of its argument.

BROADCASTING ∘

```
a            = -1

inners       = abs
outers       = [log, sqrt]
compositions = outers .∘ inners

# all `output` are equivalent
output       = [log(abs(a)), sqrt(abs(a))]
output       = [foo(a) for foo in compositions]
```

```
julia> compositions
2-element Vector{ComposedFunction{O, typeof(abs)} where O}:
 log ∘ abs
 sqrt ∘ abs
julia> output
2-element Vector{Float64}:
 0.0
 1.0
```

6d. Useful Functions for Vectors

Martin Alfaro
PhD in Economics

INTRODUCTION

This section introduces a set of core functions for manipulating vectors. We focus on operations that arise frequently in data processing and numerical computing, such as: sorting values, retrieving the indices that produce a sorted order, removing duplicates, counting occurrences, and computing rankings. The next section will show how these functions come together in a practical example.

SORTING VECTORS

The `sort` function arranges elements in ascending order, with the possibility of a descending order through the keyword argument `rev = true`. The function comes in two forms: `sort`, which returns a new sorted copy, and `sort!`, the in-place version that directly updates the vector.

SORT (ASCENDING)

```
x = [4, 5, 3, 2]

y = sort(x)
```

```
julia> y
4-element Vector{Int64}:
 2
 3
 4
 5
```

SORT (DESCENDING)

```
x = [4, 5, 3, 2]

y = sort(x, rev=true)
```

```
julia> y
4-element Vector{Int64}:
 5
 4
 3
 2
```

SORT!

```
x = [4, 5, 3, 2]

sort!(x)
```

```
julia> x
4-element Vector{Int64}:
 2
 3
 4
 5
```

Both `sort(x)` and `sort!(x)` have the option of defining the sorting order based on transformations of `x`. Specifically, given a function `foo`, the elements can be ordered by the values of `foo(x)`. Its implementation requires the keyword argument `by`.

SORT - ABSOLUTE

```
x      = [4, -5, 3]


y      = sort(x, by = abs)      # 'abs' computes the absolute value
```

```
julia> abs.(x)
3-element Vector{Int64}:
 4
 5
 3
julia> y
3-element Vector{Int64}:
  3
  4
 -5
```

SORT - QUADRATIC

```
x      = [4, -5, 3]

foo(a) = a^2
y      = sort(x, by = foo)      # same as sort(x, by = x -> x^2)
```

```
julia> foo.(x)
3-element Vector{Int64}:
 16
 25
  9
julia> y
3-element Vector{Int64}:
  3
  4
 -5
```

SORT - NEGATIVE

```
x      = [4, -5, 3]

foo(a) = -a
y      = sort(x, by = foo)      # same as sort(x, by = x -> -x)
```

```
julia> foo.(x)
3-element Vector{Int64}:
 -4
  5
 -3
julia> y
3-element Vector{Int64}:
  4
  3
 -5
```

RETRIEVING INDICES OF SORTED ELEMENTS

While `sort(x)` returns the ordered *values* of `x`, it's also useful to obtain the *indices* of the sorted elements. This capability is provided by the function `sortperm`, which returns the indices of `x` that would result in `sort(x)`. In other words, `x[sortperm(x)] == sort(x)` evaluates to `true`.[1]

EXAMPLE 1

```
x          = [3, 4, 5, 6]

sort_index = sortperm(x)
```

```
julia> sort_index
4-element Vector{Int64}:
 1
 2
 3
 4
```

EXAMPLE 2

```
x          = [1, 3, 4, 2]

sort_index = sortperm(x)
```

```
julia> sort_index
4-element Vector{Int64}:
 1
 4
 2
 3
```

In the first example, the elements are already in ascending order, so `sortperm` returns the trivial permutation `[1, 2, 3, 4]`. In contrast, the last example features an unordered vector `x = [1, 3, 4, 2]`. Thus, the resulting vector `[1, 4, 2, 3]` indicates that the smallest element appears at index 1, the second smallest at index 4, the third smallest at index 2, and the largest at index 3.

Like `sort`, `sortperm` also supports retrieving indices in descending order. This requires including the keyword argument `rev = true`.

EXAMPLE 1

```
x          = [9, 5, 3, 1]

sort_index = sortperm(x, rev=true)
```

```
julia> sort_index
4-element Vector{Int64}:
 1
 2
 3
 4
```

EXAMPLE 2

```
x          = [9, 3, 5, 1]

sort_index = sortperm(x, rev=true)
```

```
julia> sort_index
4-element Vector{Int64}:
 1
 3
 2
 4
```

Finally, `sortperm` also accepts the keyword argument `by` to define a custom transformation.

SORT - ABSOLUTE

```
x      = [4, -5, 3]


value  = sort(x, by = abs)      # 'abs' computes the absolute value
index  = sortperm(x, by = abs)
```

```
julia> abs.(x)
3-element Vector{Int64}:
 4
 5
 3
julia> value
3-element Vector{Int64}:
  3
  4
 -5
julia> index
3-element Vector{Int64}:
 3
 1
 2
```

SORT - QUADRATIC

```
x      = [4, -5, 3]

foo(a) = a^2
value  = sort(x, by = foo)      # same as sort(x, by = x -> x^2)
index  = sortperm(x, by = foo)
```

```
julia> foo.(x)
3-element Vector{Int64}:
 16
 25
  9
julia> value
3-element Vector{Int64}:
  3
  4
 -5
julia> index
3-element Vector{Int64}:
 3
 1
 2
```

SORT - NEGATIVE

```
x      = [4, -5, 3]

foo(a) = -a
value  = sort(x, by = foo)      # same as sort(x, by = x -> -x)
index  = sortperm(x, by = foo)
```

```
julia> foo.(x)
3-element Vector{Int64}:
 -4
  5
 -3
julia> value
3-element Vector{Int64}:
  4
  3
 -5
julia> index
3-element Vector{Int64}:
 1
 3
 2
```

AN EXAMPLE

One common application of `sortperm` is to reorder one variable based on the values of another. For example, suppose we want to assess the daily failures of a machine. Focusing on the first three days of the month, the following code snippet ranks these days by their corresponding failure counts.

DAYS SORTED BY LOWEST NUMBER OF FAILURES

```
days             = ["one", "two", "three"]
failures         = [8, 2, 4]

index            = sortperm(failures)
days_by_failures = days[index]        # days sorted by lowest failures
```

```
julia> index
3-element Vector{Int64}:
 2
 3
 1
julia> days_by_earnings
3-element Vector{String}:
 "two"
 "three"
 "one"
```

REMOVING DUPLICATES

The function `unique` removes duplicate entries from a vector, returning a new vector that contains each element exactly once. The function comes in two variants: `unique`, which produces a new copy, and `unique!`, which performs the operation in place and thus modifies the original vector.

UNIQUE

```
x = [2, 2, 3, 4]

y = unique(x)       # returns a new vector
```

```
julia> x
4-element Vector{Int64}:
 2
 2
 3
 4
julia> y
3-element Vector{Int64}:
 2
 3
 4
```

UNIQUE!

```
x = [2, 2, 3, 4]

unique!(x)          # mutates 'x'
```

```
julia> x
3-element Vector{Int64}:
 2
 3
 4
```

The `StatsBase` package provides a related function called `countmap`, which counts the occurrences of each element in a vector. It returns a dictionary in which the unique elements act as keys, and their corresponding values represent the number of times each element appears.

By default, the keys in the resulting dictionary are unsorted. If instead sorted keys are preferred, you must apply the `sort` function to the result. This will automatically convert an ordinary dictionary into an object with type `OrderedDict`.

UNSORTED COUNT

```
using StatsBase
x           = [6, 6, 0, 5]

y           = countmap(x)              # Dict with `element => occurrences`

elements    = collect(keys(y))
occurrences = collect(values(y))
```

```
julia> y
Dict{Int64, Int64} with 3 entries:
  0 => 1
  5 => 1
  6 => 2
julia> elements
3-element Vector{Int64}:
 0
 5
 6
julia> occurrences
3-element Vector{Int64}:
 1
 1
 2
```

SORTED COUNT

```
using StatsBase
x           = [6, 6, 0, 5]

y           = sort(countmap(x))        # OrderedDict with `element => occurrences`

elements    = collect(keys(y))
occurrences = collect(values(y))
```

```
julia> y
OrderedCollections.OrderedDict{Int64, Int64} with 3 entries:
  0 => 1
  5 => 1
  6 => 2
julia> elements
3-element Vector{Int64}:
 0
 5
 6
julia> occurrences
3-element Vector{Int64}:
 1
 1
 2
```

ROUNDING NUMBERS

Julia provides standard functions for approximating numerical values to a specified precision:

- `round` approximates a number to its nearest integer.
- `floor` returns the greatest integer less than or equal to the given number.
- `ceil` returns the smallest integer greater than or equal to the given number.

Below, we show that these functions are quite flexible, allowing users to specify the output type (e.g., `Int64` or `Float64`), the number of decimals places via the keyword argument `digits`, and the number of significant digits.

ROUND

```
x = 456.175

round(x)                        # 456.0

round(x, digits=1)              # 456.2
round(x, digits=2)              # 456.18

round(Int, x)                   # 456

round(x, sigdigits=1)           # 500.0
round(x, sigdigits=2)           # 460.0
```

FLOOR

```
x = 456.175

floor(x)                        # 456.0

floor(x, digits=1)              # 456.1
floor(x, digits=2)              # 456.17

floor(Int, x)                   # 456

floor(x, sigdigits=1)           # 400.0
floor(x, sigdigits=2)           # 450.0
```

CEIL

```
x = 456.175

ceil(x)                         # 457.0

ceil(x, digits=1)               # 456.2
ceil(x, digits=2)               # 456.18

ceil(Int, x)                    # 457

ceil(x, sigdigits=1)            # 500.0
ceil(x, sigdigits=2)            # 460.0
```

RANKINGS

Instead of sorting a vector, you may be interested in determining the rank position of each element. The `StatsBase` package offers two functions for this purpose: `competerank` and `ordinalrank`. Their main difference them lies in how they treat tied values: `competerank` assigns the same rank to all tied elements, while `ordinalrank` assigns consecutive ranks. In both cases, a rank of 1 corresponds to the smallest value. The keyword argument `rev = true` reverses this convention, assigning a rank of 1 to the largest value.

RANK (SAME RANK FOR TIES)

```
using StatsBase
x = [6, 6, 0, 5]

y = competerank(x)
```

```
julia> y
4-element Vector{Int64}:
 3
 3
 1
 2
```

DESCENDING RANK (SAME RANK FOR TIES)

```
using StatsBase
x = [6, 6, 0, 5]

y = competerank(x, rev=true)
```

```
julia> y
4-element Vector{Int64}:
 1
 1
 4
 3
```

RANK (UNIQUE POSITIONS)

```
using StatsBase
x = [6, 6, 0, 5]

y = ordinalrank(x)
```

```
julia> y
4-element Vector{Int64}:
 3
 4
 1
 2
```

DESCENDING RANK (UNIQUE POSITIONS)

```
using StatsBase
x = [6, 6, 0, 5]

y = ordinalrank(x, rev=true)
```

```
julia> y
4-element Vector{Int64}:
 1
 2
 4
 3
```

Do not confuse `ordinalrank` and `sortperm`

The function `ordinalrank` indicates the position of each value in the *sorted* vector. Instead, `sortperm` indicates the position of each value in the *unsorted* vector.

'ORDINALRANK'

```
using StatsBase
x = [3, 1, 2]

y = ordinalrank(x)
```

```
julia> y
3-element Vector{Int64}:
 3
 1
 2
```

'SORTPERM'

```
using StatsBase
x = [3, 1, 2]

y = sortperm(x)
```

```
julia> y
3-element Vector{Int64}:
 2
 3
 1
```

EXTREMA (MAXIMUM AND MINIMUM)

We conclude by presenting a method for identifying both the indices and the values of extrema within a collection. The following examples are based on the maximum, with similar functions available for the minimum.

VALUE

```
x = [6, 6, 0, 5]

y = maximum(x)
```

```
julia> y
6
```

INDEX

```
x = [6, 6, 0, 5]

y = argmax(x)
```

```
julia> y
1
```

VALUE AND INDEX

```
x = [6, 6, 0, 5]

y = findmax(x)
```

```
julia> y
(6, 1)
```

Julia additionally provides the function `max` and `min`, which respectively return the maximum and minimum of their *arguments*. These functions will become particularly useful in procedures built on binary operations that we'll cover later (e.g., reductions).

'MAX' FUNCTION

```
x = 3
y = 4

z = max(x,y)
```

```
julia> z
4
```

FOOTNOTES

[1] The name `sortperm` originates from "sorting permutation". Although the name might seem somewhat opaque, it arises because the operation returns the permutation of indices that would sort the original vector.

6e. Illustration - Johnny, the YouTuber

Martin Alfaro
PhD in Economics

INTRODUCTION

Through a practical example, this section will demonstrate the convenience of the following features:

1. Boolean indexing for working with subsets of the data
2. organizing code around functions
3. pipes to enhance code readability
4. use of views to modify subsets of the data

DESCRIBING THE SCENARIO

We'll explore the stats of Johnny's YouTube channel during a month. He has a median of 50,000 viewers per video, with a few viral videos exceeding 100,000 viewers. The information at our disposal is:

- `nr_videos`: 30 (one per day).
- `viewers`: viewers per video (in thousands).
- `payrates`: Dollars paid per video for 1,000 viewers. They range from $2 to $6. The fluctuation is consistent with YouTube's payment model, which depends on a video's feature (e.g., content, duration, retention).

The scenario is modeled by some mock data. The details of how data are generated are unimportant, but were added below for the sake of completeness. Ultimately, what matters is that the mock data creates the variables `viewers` and `payrates`.

MOCK DATA CREATION

```
using StatsBase, Distributions
using Random; Random.seed!(1234)

function audience(nr_videos; median_target)
    shape   = log(4,5)
    scale   = median_target / 2^(1/shape)

    viewers = rand(Pareto(shape,scale),  nr_videos)

    return viewers
end

nr_videos = 30

viewers  = audience(nr_videos, median_target = 50)      # in thousands of viewers
payrates = rand(2:6, nr_videos)                         # per thousands of viewers
```

```
julia> viewers # in thousands
30-element Vector{Float64}:
 38.8086
 70.8113
  ⋮
 72.3673
 30.2565
julia> payrates # per thousand viewers
30-element Vector{Int64}:
 5
 3
 ⋮
 2
 4
```

The variables `viewers` and `payrates` enable us to calculate the total payment per video.

VARIABLE "EARNINGS"

```
earnings = viewers .* payrates
```

```
julia> earnings
30-element Vector{Float64}:
 194.043
 212.434
   ⋮
 144.735
 121.026
```

SOME STATISTICS

We begin by examining the per-view payments made by YouTube. We first show that Johnny's payments range from \$2 to \$6. Moreover, using the `countmaps` function from the `StatsBase` package, we conclude that Johnny has eight videos reaching the maximum payment of \$6.

PAYMENT RANGE PER VIEWERS

```
range_payrates  = unique(payrates) |> sort
```

```
julia> range_payrates
5-element Vector{Int64}:
 2
 3
 4
 5
 6
```

COUNTS OF PAYMENTS PER VIEWERS

```
using StatsBase
occurrences_payrates = countmap(payrates) |> sort
```

```
julia> occurrences_payrates
OrderedDict{Int64, Int64} with 5 entries:
  2 => 5
  3 => 6
  4 => 8
  5 => 5
  6 => 6
```

We can also provide some insights into Johnny's most profitable videos. By applying the `sort` function, we can isolate his top 3 highest-earning videos. Moreover, we can apply the `sortperm` function to identify their indices, allowing us to extract the payment per view and total viewers associated with each video.

TOP 3 VIDEOS BY EARNINGS

```
top_earnings    = sort(earnings, rev=true)[1:3]
```

```
julia> top_earnings
3-element Vector{Float64}:
 7757.81
  693.813
  672.802
```

PAYMENTS PER VIEWERS (TOP 3 BY EARNINGS)

```
indices         = sortperm(earnings, rev=true)[1:3]

sorted_payrates = payrates[indices]
```

```
julia> sorted_payrates
3-element Vector{Int64}:
 6
 6
 6
```

VIEWERS PER VIDEO (TOP 3 BY EARNINGS)

```
indices         = sortperm(earnings, rev=true)[1:3]

sorted_viewers  = viewers[indices]
```

```
julia> sorted_viewers
3-element Vector{Float64}:
 1292.97
  115.636
  112.134
```

BOOLEAN VARIABLES

In the following, we demonstrate how to use Boolean indexing to extract and characterize subsets of data. Our focus will be on characterizing Johnny's viral videos, defined as those that have surpassed a threshold of 100k viewers. In particular, we'll determine the number of viewers and revenue generated by them.

To identify the viral videos, we'll create a `Bool` vector, where `true` identifies a viral video. This vector can then be employed as a logical index, allowing us to selectively extract data points from other variables. In the example below, we apply it to compute the total viewers and earnings attributable to the viral videos.

MOCK DATA CREATION

```
# characterization of viral videos
viral_threshold = 100
is_viral        = (viewers .≥ viral_threshold)

# stats
viral_nrvideos  = sum(is_viral)
viral_viewers   = sum(viewers[is_viral])
viral_revenue   = sum(earnings[is_viral])
```

```
julia> viral_nrvideos
4
julia> viral_viewers
1625.05
julia> viral_revenue
9750.3
```

Boolean indexing also enables subsetting data that satisfy multiple conditions. For instance, we can apply this technique to calculate the proportion of viral videos for which YouTube paid more than $3 per thousand viewers.

MORE COMPLEX CONDITION

```
# characterization
viral_threshold    = 100
payrates_above_avg = 3

is_viral           = (viewers .≥ viral_threshold)
is_viral_lucrative = (viewers .≥ viral_threshold) .&& (payrates .> payrates_above_avg)

# stat
proportion_viral_lucrative = sum(is_viral_lucrative) / sum(is_viral) * 100
```

```
julia> proportion_viral_lucrative
100.0
```

Rounding Outputs

You can express results with rounded numbers via the function `round`. By default, this returns the nearest integer expressed as a `Float64` number.

The function also offers additional specifications. For instance, the number of decimal places in the approximation can be controlled via the `digits` keyword argument. Furthermore, it's possible to represent the number as an integer using either `Int` or `Int64` as an argument. [1]

ROUND - FLOAT64

```
rounded_proportion = round(proportion_viral_lucrative)
```

```
julia> rounded_proportion
100.0
```

ROUND - TWO DECIMALS

```
rounded_proportion = round(proportion_viral_lucrative, digits=1)
```

```
julia> rounded_proportion
100.0
```

ROUND - INTEGER

```
rounded_proportion = round(Int64, proportion_viral_lucrative)
```

```
julia> rounded_proportion
100
```

FUNCTIONS TO REPRESENT TASKS

The approach employed so far allows for a quick exploration of Johnny's viral videos. However, it lacks the structure needed for a systematic analysis across different subsets of the data. To address this limitation, we can capture the characterization of videos through a function.

Recall that a well-designed function should embody a single clearly defined task. In our case, the goal is to subset data and extract key statistics, including the number of videos, viewers, and revenue generated. Furthermore, the function should remain independent of any specific application, so it can be reused to analyze different groups of videos without rewriting code each time.

The function below implements this task taking three arguments: the raw data (`viewers` and `payrates`) and a condition that defines the subset of data (`condition`). By keeping the condition generic, the function is flexible enough to target any subset of videos. The example also showcases the convenience of pipes to compute intermediate temporary steps.

TASK

```
#
function stats_subset(viewers, payrates, condition)
    nrvideos = sum(condition)
    audience = sum(viewers[condition])

    earnings = viewers .* payrates
    revenue  = sum(earnings[condition])

    return (; nrvideos, audience, revenue)
end
```

ADDING PIPE (BASE)

```
using Pipe
function stats_subset(viewers, payrates, condition)
    nrvideos = sum(condition)
    audience = sum(viewers[condition])


    revenue  = @pipe (viewers .* payrates) |> x -> sum(x[condition])

    return (; nrvideos, audience, revenue)
end
```

ADDING PIPE (PACKAGE PIPE)

```
using Pipe
function stats_subset(viewers, payrates, condition)
    nrvideos = sum(condition)
    audience = sum(viewers[condition])


    revenue  = @pipe (viewers .* payrates) |> sum(_[condition])

    return (; nrvideos, audience, revenue)
end
```

Below, we demonstrate the reusability of the function by characterizing various subsets of data.

VIRAL VIDEOS

```
viral_threshold  = 100
is_viral         = (viewers .≥ viral_threshold)
viral            = stats_subset(viewers, payrates, is_viral)
```

```
julia> viral
(nrvideos = 4, audience = 1625.05, revenue = 9750.3)
```

NON-VIRAL VIDEOS

```
viral_threshold  = 100
is_notviral      = .!(is_viral)      # '!' is negating a boolean value and we broadcast it
notviral         = stats_subset(viewers, payrates, is_notviral)
```

```
julia> notviral
(nrvideos = 26, audience = 1497.02, revenue = 5687.67)
```

SPECIFIC DAYS

```
days_to_consider = (1, 10, 25)      # subset of days to be characterized
is_day           = in.(eachindex(viewers), Ref(days_to_consider))
specific_days    = stats_subset(viewers, payrates, is_day)
```

```
julia> specific_days
(nrvideos = 3, audience = 182.939, revenue = 1030.33)
```

VARIABLE MUTATION

Suppose Johnny is exploring ways to increase viewership through targeted advertising. His projections suggest that ads will boost viewership per video by 20%. However, due to budget constraints, Johnny must choose between promoting either his non-viral or viral ones. To make an informed decision, Johnny decides to leverage the data at his disposal to crunch some rough estimates. In particular, he'll base his decision on the earnings he would've earned if he had run targeted ads.

The first step in this process involves creating a modified copy of `viewers`. This should now reflect the anticipated increase in viewership from running ads on the targeted videos (either viral or non-viral). With this updated audience data, Johnny can then apply the previously defined `stats_subset` function to estimate the potential earnings. By comparing the estimations for each group of targeted video, Johnny can determine which strategy offers the higher return on investment.

TARGET VIRAL

```
# 'temp' modifies 'new_viewers'
new_viewers     = copy(viewers)
temp            = @view new_viewers[new_viewers .≥ viral_threshold]
temp           .= 1.2 .* temp

allvideos       = trues(length(new_viewers))
targetViral     = stats_subset(new_viewers, payrates, allvideos)
```

```
julia> targetViral
(nrvideos = 30, audience = 3447.08, revenue = 17388.0)
```

TARGET NON-VIRAL

```
# 'temp' modifies 'new_viewers'
new_viewers     = copy(viewers)
temp            = @view new_viewers[new_viewers .< viral_threshold]
temp           .= 1.2 .* temp

allvideos       = trues(length(new_viewers))
targetNonViral  = stats_subset(new_viewers, payrates, allvideos)
```

```
julia> targetNonViral
(nrvideos = 30, audience = 3421.47, revenue = 16575.5)
```

Given the results in each tab, promoting viral videos appears to be the more profitable option.

Be Careful with Misusing 'view'

Updating `temp` requires an in-place operation to mutate the parent object. In our case, this was achieved via the broadcast operator `.=`. Below, we present some implementations that fail to produce the intended result.

RIGHT USE

```
new_viewers = copy(viewers)


temp  = @view new_viewers[new_viewers .≥ viral_threshold]
temp .= temp .* 1.2
```

WRONG USE 1

```
new_viewers = viewers      # it creates an alias, it's a view of
the original object!!!

# 'temp' modifies 'viewers' -> you lose the original info
temp  = @view new_viewers[new_viewers .≥ viral_threshold]
temp .= temp .* 1.2
```

WRONG USE 2

```
new_viewers = copy(viewers)

# wrong -> not using `temp .= temp .* 1.2`
temp  = @view new_viewers[new_viewers .≥ viral_threshold]
temp  = temp .* 1.2      # it creates a new variable 'temp', it
does not modify 'new_viewers'
```

Use of "Let Blocks" To Avoid Bugs

In the code above, "Target Viral" and "Target Non-Viral" reference variables with identical names. This creates the risk of accidentally referring to a variable from the wrong scenario.

A practical way to mitigate this risk is by employing "let blocks". Since each let block introduces its own scope, this helps maintain a clean namespace and prevents variable collisions.

TARGET VIRAL

```
targetViral     = let viewers = viewers, payrates = payrates,
threshold = viral_threshold
    new_viewers = copy(viewers)
    temp        = @view new_viewers[new_viewers .≥ threshold]
    temp       .= 1.2 .* temp

    allvideos  = trues(length(new_viewers))
    stats_subset(new_viewers, payrates, allvideos)
end
```

```
julia> targetViral
(nrvideos = 30, audience = 3837.09, revenue = 19728.1)
```

TARGET NON-VIRAL

```
targetNonViral  = let viewers = viewers, payrates = payrates,
threshold = viral_threshold
    new_viewers = copy(viewers)
    temp        = @view new_viewers[new_viewers .< threshold]
    temp       .= 1.2 .* temp

    allvideos  = trues(length(new_viewers))
    stats_subset(new_viewers, payrates, allvideos)
end
```

```
julia> targetNonViral
(nrvideos = 30, audience = 3746.48, revenue = 18525.6)
```

BROADCASTING OVER A LIST OF FUNCTIONS

A function like `stats_subset` is useful for computing a fixed set of summary statistics. However, since the choice of statistics is hard-coded into the function's definition, the output can't be changed without rewriting the code. This rigidity makes the function less reusable across different analytical contexts.

A more flexible approach consists of specifying which statistics to compute at the time of use. Julia makes this possible because functions are *first-class objects*, entailing that functions behave just like any other variable. This feature lets us define a list of statistical functions, eventually applying them element-wise to the variables we want to characterize.

Below, we apply this methodology to characterize the variable `viewers`.

DESCRIPTIVE STATISTICS

```
list_functions = [sum, median, mean, maximum, minimum]

stats_viewers  = [fun(viewers) for fun in list_functions]
```

```
julia> stats_viewers
5-element Vector{Float64}:
 3447.08
   64.8765
  114.903
 1551.56
   28.2954
```

The same methodology can also be employed for characterizing multiple variables at once. In fact, broadcasting makes this straightforward to implement. For instance, below we simultaneously characterize `viewers` and `earnings`.

DESCRIPTIVE STATISTICS

```
list_functions = [sum, median, mean, maximum, minimum]

stats_various  = [fun.([viewers, payrates]) for fun in list_functions]
```

```
julia> stats_various
5-element Vector{Vector{Float64}}:
 [3447.08, 121.0]
 [64.8765, 4.0]
 [114.903, 4.03333]
 [1551.56, 6.0]
 [28.2954, 2.0]
```

One major limitation of the current method is its inability to reflect each statistic's name. To address this, we can collect all statistics in a named tuple, enabling the access of each through its name. For instance, given a named tuple `stats_viewers`, it'll become possible to retrieve the average value of `viewers` by `stats_viewers.mean` or `stats_viewers[:mean]`.

To assign names to the statistics within the named tuple, we'll use the `Symbol` type. This translates strings into identifiers that can act as keys of a named tuple, enabling programmatic access to each statistic.

DESCRIPTIVE STATISTICS

```
vector_of_tuples = [(Symbol(fun), fun(viewers)) for fun in list_functions]
stats_viewers    = NamedTuple(vector_of_tuples)
```

```
julia> stats_viewers
(sum = 3447.08, median = 64.8765, mean = 114.903, maximum = 1551.56, minimum = 28.2954)
julia> stats_viewers.mean
114.903
julia> stats_viewers[:median]
64.8765
```

FOOTNOTES

[1] Recall that the type `Int` defaults to `Int64` on 64-bit systems and to `Int32` for 32-bit systems. Most modern computers fall into the former category, explaining why we usually employ `Int64`.

7a. Overview and Goals

Martin Alfaro
PhD in Economics

ON PART II OF THE BOOK

The first part of the book has laid the groundwork for working with Julia. This demanded introducing fundamental data types, such as numbers (integers and floating-point numbers), vectors, and tuples. Alongside, we've covered essential programming constructs like functions, conditionals, and for-loops. While these concepts may vary in syntax and usage across different programming languages, their underlying principles remain universal.

In the second part of the book, we'll shift our attention to one of Julia's most distinctive strengths: **high-performance computing**. Combined with its intuitive syntax and interactive design, this capability makes Julia an ideal choice for scientific applications.

The domain of high-performance computing is vast and complex. At the same time, each application has idiosyncratic features that may render certain optimizations more or less relevant. Because of this breadth, I've made deliberate choices about what to include and leave aside.

In particular, the book presents techniques that are either essential for achieving performance or broadly applicable across diverse applications. Specifically:

- **Type stability**
- **Reductions in memory allocations**
- **Parallelization**

The order in which these techniques are presented reflects their relative importance. Type stability in particular constitutes a necessary condition for achieving high performance, turning it into a prerequisite for any further optimization in Julia.

OVERVIEW OF THE CHAPTER

This chapter will provide theoretical background, necessary to understand the optimization techniques we'll eventually present. As such, none of the sections will treat a particular technique. Rather, we present necessary knowledge to identify the factors that influence performance. This foundation ensures readers can effectively apply specific methods later.

Section 7b deals with the very first question of **when to optimize**. It acknowledges that implementing certain optimizations may be time-consuming, reduce code readability, and complicate maintenance. In this context, it's important to weigh whether performance improvements are truly worthwhile before investing effort.

Once the decision to optimize has been made, the next step is determining which techniques to apply. Section 7c explains **how to benchmark code**, a critical process for guiding this choice. Note that benchmarking isn't simply a way to measure improvement. In fact, the techniques we present don't guarantee performance gains in every case. Parallelization is a prime example of this behavior: while it can accelerate computation, the overhead it introduces may ultimately slow execution.

The two sections that come after will deal with theoretical aspects related to optimization. The challenge there lay in striking the right balance between providing enough background to understand future techniques presented, while avoiding unnecessary specificity. My decisions in this respect were based on two considerations. First, I excluded subjects that, while important, could be bypassed without compromising the grasp of high performance. Second, the scope remained focused, to avoid diverting the reader's attention away from the central goal, which is learning optimization techniques.

In particular, Section 7d extends earlier explanations about **types**. Types matter for performance because computers ultimately implement different computations for different kinds of data. For instance, integer addition differs from floating-point addition at the machine level. The type system encodes these differences, allowing the compiler to generate the appropriate low-level operations. Note that Julia's type system is quite vast, and therefore many valuable concepts have necessarily been excluded. For example, the concept of user-defined structures (`structs`), which allow creation of custom objects, was left out.

Section 7e turns to the inner workings of **functions**. Unlike other languages, Julia generates specialized machine code only when operations are enclosed within functions. This means that wrapping code in functions isn't merely a stylistic choice, but a fundamental requirement for achieving high performance.

7b. When To Optimize Code?

Martin Alfaro
PhD in Economics

INTRODUCTION

Julia has been praised as solving the "two-language problem". This refers to the difficulty of finding a language that's fast, but still easy to read and write. Although it's true that Julia has some advantages relative to other languages, claims like this can be quite misleading for someone new to programming. It wrongly suggests that Julia is the only language you'll need to learn, regardless of your specific coding domain.

In reality, each programming language is designed with certain purposes in mind. Consequently, it's quite likely that you'll need to learn multiple programming languages, even if your focus is narrow. This is particularly true in data analysis, where a package implementing a specific task may only be available in one language. I, for one, tend to use Julia as my main language for data analysis, but complement it with libraries from R and Python when the task requires it. [1]

Getting the best performance in any language is also not immediate. It requires you to write code appropriately, with implementations that tend to be software-specific and involve several trade-offs. [2] Overall, the claim that "Julia is fast" should be replaced by "Julia *can* be fast." Considering this, the upcoming chapters aim to equip you with the essential tools to unlock Julia's performance capabilities.

WHEN SHOULD WE CARE ABOUT SPEED?

Achieving high performance often comes with trade-offs, and thus should never be the sole consideration when writing code. Optimizing performance frequently means rewriting parts of your script, which can reduce readability and make the code harder to maintain in the long run. Additionally, implementing these improvements requires significant time and effort, including tasks such as testing, identifying bottlenecks, and integrating third-party packages.

Considering this, you should assess your goals before embarking on any optimization efforts. Keep in mind that **most of YOUR time will be spent on writing, reading, and debugging code**. Reducing the computer's execution time by a millisecond may not be worth the trade-off if it demands investing hours. Moreover, even if speed is crucial for your project, you should prioritize which parts of the code to optimize. Typically, only a few operations impact runtime critically, with the rest having a negligible effect.

With these caveats in mind, the suggestions we'll present in the upcoming chapters serve a dual purpose. Firstly, they represent essential rules for speed. Not adhering to them would severely undermine performance, thereby negating any advantages of using Julia. Secondly, several tips we'll

consider have a minimal impact on code's readability, if any. In summary, the procedures to be presented will help you unlock Julia's speed, without sacrificing code readability or entailing excessive additional work.

FOOTNOTES

[1.] Julia has the capacity of calling programs from other software such as R or Python. R and Python also have this feature.

[2.] This explains the disparate results often seen in online benchmarks, where code can be written inefficiently in one language and highly optimized in another. Moreover, since languages tend to excel at certain tasks, it's possible to cherry-pick examples that make a particular language appear faster.

7c. Benchmarking Execution Time

Martin Alfaro
PhD in Economics

INTRODUCTION

This section introduces standard tools for benchmarking code performance. Our book reports results based on the `BenchmarkTools` package, which is currently the most mature and reliable option in the Julia ecosystem. That said, the newer `Chairmarks` package has demonstrated notable improvements in execution speed compared with `BenchmarkTools`. I recommend adopting `Chairmarks` once it's achieved sufficient stability and adoption within the community.

To set the stage, we'll start by addressing some key points for interpreting benchmark results. We'll also look at Julia's built-in `@time` macro, whose limitations explain why `BenchmarkTools` and `Chairmarks` should be used instead.

TIME METRICS

Julia uses the same time metrics described below, regardless of whether you use `BenchmarkTools` or `Chairmarks`. For quick reference, these metrics can be accessed at any point **in the left bar** under "**Notation & Hotkeys**".

Unit	Acronym	Measure in Seconds
Seconds	`s`	1
Milliseconds	`ms`	10^{-3}
Microseconds	`µs`	10^{-6}
Nanoseconds	`ns`	10^{-9}

Alongside execution times, each package also reports the amount of **memory allocated on the heap**, typically referred to simply as **allocations**. These allocations can play a major role in overall performance, and usually indicate suboptimal coding practices. As we'll explore in later sections, monitoring allocations tends to be crucial for achieving high performance.

"TIME TO FIRST PLOT"

The expression "Time to First Plot" refers to a specific performance characteristic inherent in Julia's operation. It describes the delay experienced when running a function for the first time during a session. This latency is a side effect of how the language translates code into action. Specifically, the first execution triggers the generation of low-level machine instructions to carry out the function's operations. This translation from human-readable code into machine-executable instructions is known as **compilation**. Unlike many other languages that compile before running, Julia relies on a just-in-time (JIT) compiler. This means the code compiles on-the-fly when the function is first executed.

While this approach enables high performance, it introduces latency during the initial run. Fortunately, recent developments have significantly reduced the practical impact of this delay. Thanks to improvements in the precompilation system since Julia 1.9, a vast majority of package functions are now compiled before you even start writing code. Consequently, the compilation penalty in any new session mainly affects functions defined by the user or novel combinations of argument types that precompilation doesn't cover. Once a function is compiled, its machine code is cached. This caching ensures that all subsequent calls to that specific function execute much faster than the first time.

The latency incurred varies significantly across functions. While it may be imperceptible for simple operations, it can still be noticeable when triggering a chain of user-defined methods or write complex operations. This explains the origin of the term "Time to First Plot", since originally drawing the first plot during a session was taking several seconds.

> ⚠️ **Warning!**
> Precompilation hasn't changed the JIT compilation model. Understanding that Julia compiles specialized machine code for your specific types remains crucial for writing performant Julia code.

@TIME

Julia comes with a built-in macro called `@time`, allowing you to get a quick sense of an operation's execution time. The results provided by this macro, nonetheless, suffer from several limitations that make it unsuitable for rigorous benchmarking.

First, a measurement based on just a single execution is often unreliable, as runtimes can fluctuate significantly due to background processes on your computer. Second, if that run represents a function's first call, the measurement will include compilation overhead. The extra time Julia spends generating machine code inflates the reported runtime, making it unrepresentative of subsequent calls.

Moreover, while running `@time` multiple times can address these issues, the macro still has a major flaw when used for rigorous benchmarking: it evaluates code in the global scope. We'll show in upcoming sections that non-constant global variables have a marked detrimental effect on performance. Consequently, the time reported doesn't accurately reflect how the function would perform in practice.

The following example illustrates the use of `@time`, highlighting the difference in execution time between the first and subsequent runs.

@TIME

```
x = 1:100

@time sum(x)         # first run                      -> it incorporates compilation time
@time sum(x)         # time without compilation time -> relevant for each subsequent run
```

```
 0.002747 seconds (3.56 k allocations: 157.859 KiB, 99.36% compilation time)
 0.000003 seconds (1 allocation: 16 bytes)
```

PACKAGE "BENCHMARKTOOLS"

A more reliable alternative for measuring execution time is provided by `BenchmarkTools`. This addresses the shortcomings of `@time` in several ways.

First, it reduces result variability by running operations multiple times and providing summary statistics. It also measures the execution time of functions without compilation latency, since the package discards the first run for the reported timing. Additionally, the package allows users to explicitly control variable scope: by prefixing function arguments with the `$` symbol, these are treated as local variables during a function call.

The package offers two macros, depending on the level of detail required: `@btime`, which only reports the minimum time, and `@benchmark`, which provides detailed statistics. Below, we demonstrate their use.

@BTIME

```
using BenchmarkTools

x = 1:100
@btime sum($x)        # provides minimum time only
```

```
  2.314 ns (0 allocations: 0 bytes)
```

@BENCHMARK

```
using BenchmarkTools

x = 1:100
@benchmark sum($x)    # provides more statistics than `@btime`
```

In upcoming sections, we'll exclusively benchmark functions. This means that you should always prefix the function arguments with `$`. **Omitting `$` will lead to inaccurate results**, including incorrect reports of memory allocations.

The following example demonstrates the consequence of excluding `$`, where the runtimes reported are higher than the actual runtime.

NO '$'

```
using BenchmarkTools
x = rand(100)

@btime sum(x)
```

```
  14.465 ns (1 allocation: 16 bytes)
```

WITH '$'

```
using BenchmarkTools
x = rand(100)

@btime sum($x)
```

```
  6.546 ns (0 allocations: 0 bytes)
```

PACKAGE "CHAIRMARKS"

A new alternative for benchmarking code is given by the `Chairmarks` package. Its notation closely resembles that of `BenchmarkTools`, with the macros `@b` and `@be` respectively providing a similar functionality to `@btime` and `@benchmark`. The main benefit of `Chairmarks` is its speed, as it can be orders of magnitude faster than `BenchmarkTools`.

As with `BenchmarkTools`, measuring the execution time of functions appropriately requires prepending function arguments with `$`.

@B

```
using Chairmarks
x = rand(100)

display(@b sum($x))        # provides minimum time only
```

```
6.550 ns
```

@BE

```
using Chairmarks
x = rand(100)

display(@be sum($x))       # analogous to `@benchmark` in BenchmarkTools
```

```
Benchmark: 3856 samples with 3661 evaluations
 min    6.679 ns
 median 6.815 ns
 mean   6.785 ns
 max    14.539 ns
```

REMARK ON RANDOM NUMBERS FOR BENCHMARKING

When comparing the performance of different methods, we must ensure that our measurements aren't skewed by variations in the input data. To accomplish this, each approach must be tested using *the exact same set of values*. In this way, differences in execution time can be attributed solely to the efficiency of the method itself, rather than to a change in the inputs.

This consistency can be achieved by using random number generators. They rely on a **random seed**, which is an arbitrary starting point that dictates the entire sequence of values to be produced. By setting the same seed before each test, we can generate identical deterministic sequences of random numbers across multiple runs. **Any arbitrary number can be used for the seed**. The only requirement is that the same number is employed, so that you replicate the exact same set of random numbers.

Below, we demonstrate random number generation by using the package `Random`. We choose the seed `1234` before executing each operation. Note, though, that any other number could be used as long as it's the same across runs.

SAME VALUES

```
using Random

Random.seed!(1234)      # 1234 is an arbitrary number, use any number you want
x = rand(100)

Random.seed!(1234)
y = rand(100)           # identical to `x`
```

DIFFERENT VALUES

```
using Random

Random.seed!(1234)      # 1234 is an arbitrary number, use any number you want
x = rand(100)


y = rand(100)           # different from `x`
```

For presentation purposes, code snippets in this book will omit the lines dedicated to setting the random seed. While adding these code lines is essential for ensuring reproducibility, their inclusion in every example would create unnecessary clutter. Below, we illustrate the code that will be displayed throughout the book, along with the actual code executed.

SNIPPET EXECUTED

```
using Random
Random.seed!(123)

x = rand(100)

y = sum(x)
```

SNIPPET SHOWN

```
# We omit the lines that seet the seed


x = rand(100)

y = sum(x)
```

BENCHMARKS IN PERSPECTIVE

When evaluating approaches for performing a task, execution times are often negligible, typically on the order of nanoseconds. Yet, this doesn't mean that the choice of method has no practical consequence: **the relevance of benchmarks becomes apparent when these operations are performed repeatedly**. This includes cases where the operation is called in a for-loop or embedded in iterative computations (e.g., solving systems of equations or optimizing functions). In these situations, small differences in timing are amplified, as they are replicated hundreds, thousands, or even millions of times.

AN EXAMPLE

To illustrate this matter, let's consider a specific example. Suppose we want to double each element of a vector `x`, and then calculate their sum. In the following, we'll compare two different approaches to accomplish this task. Both methods produce the same result.

The first method will be based on `sum(2 .* x)`, with `x` treated as a global variable. As we'll discuss in later sections, this approach is inefficient. A more performant alternative is given by `sum(a -> 2 * a, x)`, where `x` is passed as a function argument. The measured runtimes of each approach are as follows.

IMPLEMENTATION 1

```
x      = rand(100_000)

foo()  = sum(2 .* x)
```

```
 35.519 μs (5 allocations: 781.37 KiB)
```

IMPLEMENTATION 2

```
x      = rand(100_000)

foo(x) = sum(a -> 2 * a, x)
```

```
 6.393 μs (0 allocations: 0 bytes)
```

The results reveal that the second approach achieves a significant speedup, requiring less than 15% of the time consumed by the slower method. However, even the "slow" approach is remarkably fast when interpreted in absolute terms, completing in under 0.0001 seconds.

This pattern will be recurring in our benchmarks, where execution times are often negligible. In such cases, the relevance of our conclusions depends heavily on context. If the operation is performed only once, readability should guide the choice of implementation. Instead, if the operation is executed repeatedly, small performance differences accumulate and might become meaningful, making the faster approach a more suitable choice.

To make this point concrete, let's place these functions inside a for-loop that runs 100,000 times. Since our sole goal is to repeat the operation, the iteration variable itself plays no role. In such cases, it's a common practice to employ a **throwaway variable**: a placeholder that exists only to satisfy syntax requirements, without ever being referenced. This convention signals to other programmers that the variable's value can be safely ignored. In our example, `_` serves this purpose within the for-loop. It reflects that each iteration performs exactly the same operation, without depending on the for-loop index.

IMPLEMENTATION 1 WITHIN A FOR-LOOP

```
x      = rand(100_000)
foo()  = sum(2 .* x)

function replicate()
   for _ in 1:100_000
      foo()
   end
end
```

```
 5.697 s (500000 allocations: 74.52 GiB)
```

IMPLEMENTATION 2 WITHIN A FOR-LOOP

```
x      = rand(100_000)
foo(x) = sum(a -> 2 * a, x)

function replicate(x)
   for _ in 1:100_000
      foo(x)
   end
end
```

```
 677.130 ms (0 allocations: 0 bytes)
```

The example starkly reveals the consequences of calling these functions within a for-loop. The execution time of the slow version now jumps to more than 5 seconds, while the fast version finishes in under one second. Such a stark contrast underscores the importance of optimizing functions that are executed repeatedly: even seemingly minor improvements can accumulate into pronounced performance gains.

7d. Preliminaries on Types

Martin Alfaro
PhD in Economics

INTRODUCTION

High performance in Julia is intimately related to the notion of type stability. The definition of this concept is relatively straightforward: a function is type-stable when the types of its expressions can be inferred from the types of its arguments. When this property holds, Julia can specialize its computation method, resulting in fast code.

Despite its simplicity, type stability is subject to various nuances. In fact, a careful consideration of the property requires a solid foundation in two key areas: **Julia's type system and the inner workings of functions**. The current section equips you with the necessary knowledge to grasp the former, deferring the internals of functions to the next section. Moreover, **the explanations will focus on the case of scalars and vectors**, leaving more complex objects for subsequent sections.

Before you continue, I recommend reviewing the basics of types introduced here.

> ⚠️ **Warning!**
>
> **The subject is covered only to the necessary extent for understanding type stability**. Julia's type system is indeed quite vast, and a comprehensive exploration would warrant a dedicated chapter.

BASICS OF TYPES

Variables in Julia serve as mere labels for objects, with objects in turn holding values of specific types. The most common types for scalars are `Float64` and `Int64`, whose vector counterparts are `Vector{Float64}` and `Vector{Int64}`. Recall that `Vector` is an alias for a one-dimensional array, so that a type like `Vector{Float64}` is equivalent to `Array{Float,1}`.

> ❗ **`Int` As an Alternative to `Int64`**
>
> You'll notice that packages tend to use `Int` as the default type for integers. The type `Int` is an alias that adapts to your CPU's architecture. Since most modern computers are 64-bit systems, `Int` is equivalent to `Int64`. Instead, `Int` becomes `Int32` on 32-bit systems.

Julia's type system is organized in a hierarchical way. This feature permits the definition of subsets and supersets of types, which in the context of types are referred to as **subtypes** and **supertypes**. For instance, the type `Any` is a supertype that includes all possible types in Julia, thus occupying the highest position in the type hierarchy. Another example of supertype is `Number`, which encompasses all numeric types (`Float64`, `Float32`, `Int64`, etc.). [1]

Supertypes provide great flexibility for writing code. They enable the grouping of values under a common abstraction, making it possible to define operations generically. For instance, defining the `+` operator for the abstract type `Number` ensures its applicability to all numeric types, regardless of whether they are integers, floats, or their numerical precision.

A particular supertype known as `Union` will be instrumental for our examples. It allows variables to hold values of any type specified in its arguments. Its syntax is `Union{<type1>, <type2>, ...}`, so that a variable with type `Union{Int64, Float64}` could be either an `Int64` or `Float64`. Note that, by definition, union types are always supertypes of their constituent types.

ABSTRACT AND CONCRETE TYPES

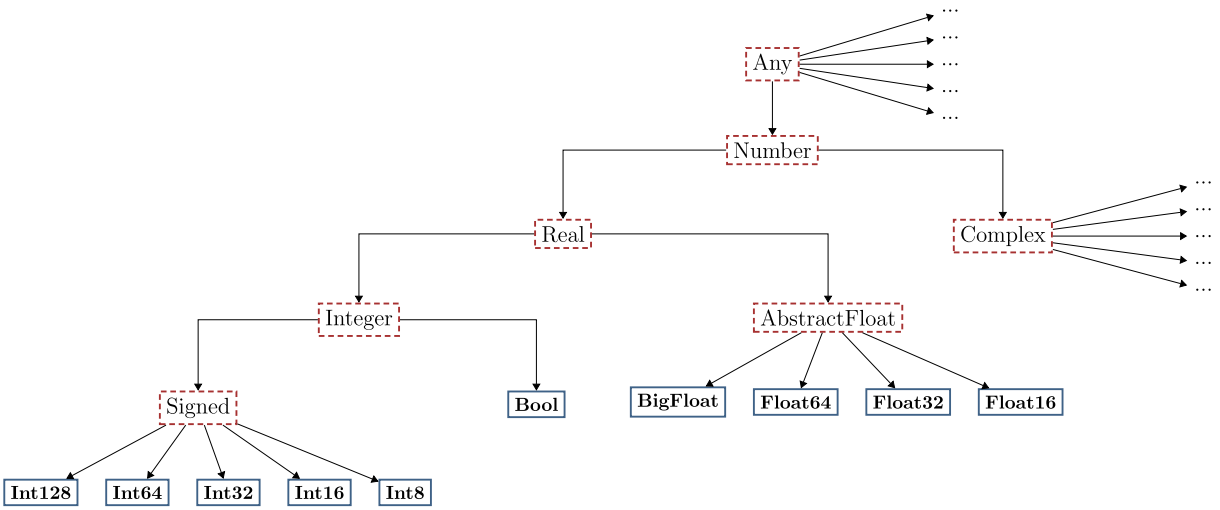
The hierarchical nature of types makes it possible to represent subtypes and supertypes through trees. Such a structure, in turn, gives rise to the notions of abstract and concrete types.

An **abstract type** acts as a parent category that groups related types, necessarily breaking down into subtypes. This means that it serves as a conceptual category, rather than a fully specified representation. The `Any` type in Julia is a prime example.

A **concrete type**, by contrast, is fully specified and has no subtypes. It represents an irreducible unit and therefore considered final, in the sense that it can't be further specialized within the hierarchy.

The diagram below illustrates the difference between abstract and concrete types for scalars. In particular, we present the hierarchy of the type `Number`. Note that the labels included in the diagram match the corresponding type name in Julia. [2]

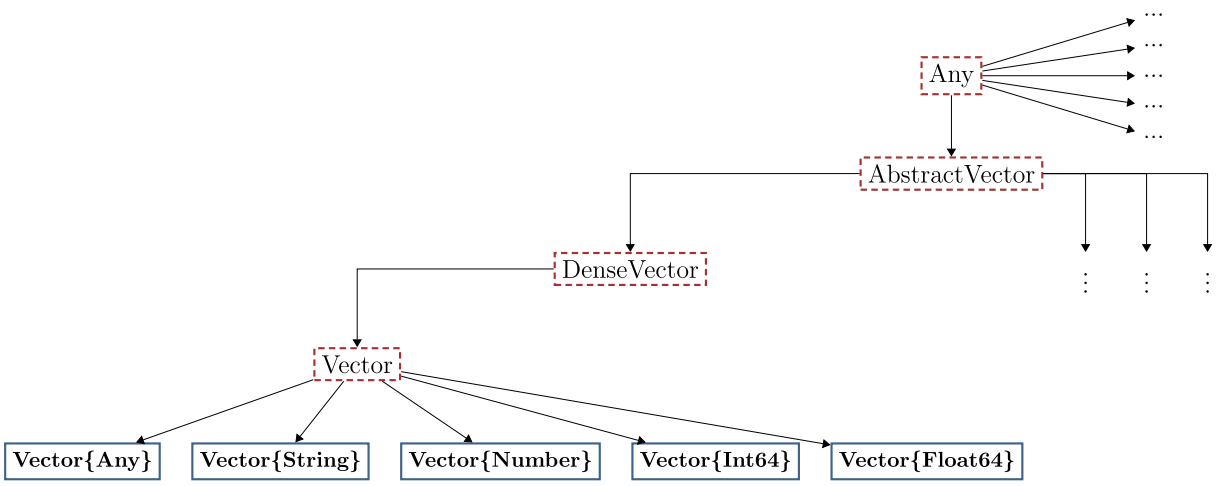
HIERARCHY OF TYPE NUMBER



Note: Dashed red borders indicate abstract types, while solid blue borders indicate concrete types.

The distinction between abstract and concrete types for scalars is relatively straightforward. Instead, their difference becomes more nuanced when considering vectors, as shown in the diagram below.

HIERARCHY OF TYPE VECTOR



Note: Dashed red borders indicate abstract types, while solid blue borders indicate concrete types.

The diagram reveals that `Vector{T}` **is a concrete type for each specific type** `T`. As a result, none of these concrete vector types have subtypes. This also means that a vector such as `Vector{Int64}` is not a subtype of `Vector{Any}`, even though `Int64` itself is a subtype of `Any` and `Any` is abstract.

The design is entirely consistent with the idea of vectors representing collections of homogeneous elements, where every element must share the same type.

ONLY CONCRETE TYPES CAN BE INSTANTIATED, ABSTRACT TYPES CAN'T

Instantiation refers to the act of creating a value of a specific type. A central principle of Julia's type system is that **only concrete types can be instantiated**. Abstract types, instead, describe sets of possible values, but never correspond to values themselves.

The distinction helps clarify the meaning of some widespread expressions used in Julia. For example, stating that a variable has type `Any` shouldn't be interpreted literally. Rather, it means the variable can hold values of any concrete type, considering that all concrete types are subtypes of `Any`.

The clarification is also important for what follows regarding type-annotation of variables. It implies that declaring a variable with an abstract type restricts the set of possible concrete types it can hold, even though the variable will ultimately adopt a concrete type.

RELEVANCE FOR TYPE STABILITY

At this point, you may be wondering how these concepts relate to type stability. The connection becomes clear when you consider how Julia performs computations.

High performance in Julia relies heavily on specializing the computation method. In the next section, we'll see that this specialization is unattainable in the global scope, as Julia treats global variables as potentially holding values of any type. In contrast, when code is written within a function, the execution process begins by determining the concrete types of each function argument. This information is then used to infer the concrete types of all the expressions in the function body.

When inference succeeds and all expressions have unambiguous concrete types, the function is considered **type stable**. This enables Julia to specialize its computation method and generate optimized machine code. If, instead, expressions could potentially take on multiple concrete types, performance is substantially degraded, as Julia must consider a separate implementation for each possible type.

For scalars and vectors, type stability essentially requires that expressions ultimately operate on **primitive types**. Examples of numeric primitive types include integers and floating-point numbers, such as `Int64`, `Float64`, and `Bool`. Thus, applying functions like `sum` to a `Vector{Int64}` or `Vector{Float64}` allows for full specialization, whereas applying them to a `Vector{Any}` prevents it.

> **String Objects**
>
> For text representation, the character type `Char` serves as a primitive type. Since a `String` is internally represented as a collection of `Char` elements, operations on `String` objects can also achieve type stability.

THE OPERATOR <: TO IDENTIFY SUPERTYPES

The remainder of this section is dedicated to operators and functions for handling types. Specifically, we'll introduce the operator `<:`, which checks whether one type is a subtype of another. Then, we'll examine strategies for constraining variables to specific types.

It's possible that you won't need to apply any of the techniques we present, as Julia automatically attempts to infer types when functions are called. Nonetheless, understanding these operators is essential for grasping upcoming material.

USE OF `<:`

The symbol `:<` tests whether a type `T` is a subtype of another type `S`. It can be used as an operator `T <: S` or as a function `<:(T,S)`. For example, `Int64 <: Number` and `<:(Int64, Number)` verify whether `Int64` is a subtype of `Number`, thus returning `true`. Below, we provide further examples.

ALL TRUE

```
# all the statements below are `true`
Float64 <: Any
Int64   <: Number
Int64   <: Int64
```

ALL FALSE

```
# all the statements below are `false`
Float64 <: Vector{Any}
Int64   <: Vector{Number}
Int64   <: Vector{Int64}
```

The fact that `Int64 <: Int64` evaluates to `true` illustrates a fundamental principle: **every type is a subtype of itself**. Moreover, in the case of concrete types, this is the only subtype.

THE KEYWORD `WHERE`

By combining `<:` with `Union`, you can also check whether a type belongs to a given set of types. For example, `Int64 <: Union{Int64, Float64}` assesses whether `Int64` equals `Int64` or `Float64`, thus returning `true`.

The approach can be made more widely applicable by using the `where` keyword with a type parameter `T`.[3] The syntax is `<type depending on T> where T <: <set of types>`. This entails that `T` covers multiple possible types.

CASE 1 - SCALARS

```
# all the statements below are `true`
Float64 <: Any
Int64   <: Union{Int64, Float64}
Int64   <: Union{T, String} where T <: Number       # `String` represents text
```

CASE 2 - VECTORS

```
# all the statements below are `true`
Vector{Float64} <: Vector{T} where T <: Any
Vector{Int64}   <: Vector{T} where T <: Union{Int64, Float64}
Vector{Number}  <: Vector{T} where T <: Any

# all the statements below are `false`
Vector{Float64} <: Vector{Any}
Vector{Int64}   <: Vector{Union{Int64, Float64}}
Vector{Number}  <: Vector{Any}
```

CASE 2 (EQUIVALENT)

```
# all the statements below are `true`
Vector{Float64} <: Vector{<:Any}
Vector{Int64}   <: Vector{<:Union{Int64, Float64}}
Vector{Number}  <: Vector{<:Any}

# all the statements below are `false`
Vector{Float64} <: Vector{Any}
Vector{Int64}   <: Vector{Union{Int64, Float64}}
Vector{Number}  <: Vector{Any}
```

Types relying on parameters like `T` are called **parametric types**. In the example above, these types allow us to distinguish between a concrete type like `Vector{Any}` and a set of concrete types `Vector{T} where T <: Any`, where the latter encompasses `Vector{Int64}`, `Vector{Float64}`, `Vector{String}`, etc.

> ⚠ **Warning!** - The Type `Any`
>
> When `<:` is omitted and we simply write `where T`, the statement is interpreted by Julia as `where T <: Any`. This is why the following equivalences hold.

EXAMPLE

```
# all the statements below are `true`
Float64       <: Any
Float64        <: T where T <: Any              # identical to
the line above

Vector{Int64} <: Vector{T} where T <: Any
```

ALTERNATIVE (EQUIVALENT)

```
# all the statements below are `true`
Float64       <: Any
Float64        <: T where T                        # identical to
the line above

Vector{Int64} <: Vector{T} where T
```

TYPE-ANNOTATING VARIABLES

In the following, we present methods for **type-annotating variables**. The technique can be used either to assert a variable's type **during an assignment** or to restrict the types of **function arguments**.

There are two approaches to type-annotating variables. The first one relies on the binary operator `::` and its syntax is `x::<type>`. The second approach leverages the Boolean binary operator `<:`, combined with `::` and the keyword `where`. Its syntax is `x::T where T <: <type>`, where `T` can be replaced with other symbol.

Next, we illustrate both methods, separately considering type-annotations for assignments and function arguments.

ASSIGNMENTS

Let's start illustrating the approaches based on scalar assignments. Each tab below declares an identical type for `x` and for `y`.

:: FOR ASSIGNMENT

```
x::Int64              = 2      # only reassignments to `Int64` are possible

y::Number              = 2      # only reassignments to `Float64`, `Float32`, `Int64`, etc
are possible
```

```
julia> x = 2.5
ERROR: InexactError: Int64(2.5)
julia> y = 2.5
2.5
julia> y = "hello"
ERROR: MethodError: Cannot convert an object of type String to an object of type Number
```

<: FOR ASSIGNMENT

```
x::T where T <: Int64  = 2      # only reassignments to `Int64` are possible

y::T where T <: Number = 2      # only reassignments to `Float64`, `Float32`, `Int64`, etc
are possible
```

```
julia> x = 2.5
ERROR: InexactError: Int64(2.5)
julia> y = 2.5
2.5
julia> y = "hello"
ERROR: MethodError: Cannot convert an object of type String to an object of type Number
```

> ⚠ **Warning!** - Modifying Types
>
> Once `x` has been assigned a type, that type can't be changed within the same Julia session. The only way to reset its type is to start a new session.

The fact that `x` holds the same type across all tabs follows because `T <: Float64` can only represent `Float64`. More specifically, `Float64` is a concrete type, which by definition has no subtypes other than itself. Considering this, scalar types are usually asserted using `::` rather than `<:`.

While this behavior holds for scalars, it doesn't apply to vectors. Specifically, using `::` in combination with `Vector{Number}` establishes that the object will have `Vector{Number}` as its concrete type. Instead, `Vector{T} where T <: Number` indicates that the elements of the vector will adopt a concrete subtype of `Number`.

VECTORS AND TYPE ANY

```
# `x` will always be `Vector{Any}`
x::Vector{Any}                = [1,2,3]

# `y` will always be `Vector{Number}`
y::Vector{Number}             = [1,2,3]
```

```
julia> typeof(x)
Vector{Any} (alias for Array{Any, 1})
julia> typeof(y)
Vector{Number} (alias for Array{Number, 1})
```

VECTORS AND TYPE NUMBER

```
# `x` is Vector{Int64}, but could eventually be `Vector{Float64}`, `Vector{String}`, etc
x::Vector{T} where T <:Any     = [1,2,3]

# `y` is now Vector{Int64}, but could eventually be `Vector{Float64}`, `Vector{Int32}`, etc
y::Vector{T} where T <: Number = [1,2,3]
```

```
julia> typeof(x)
Vector{Int64} (alias for Array{Int64, 1})
julia> typeof(y)
Vector{Int64} (alias for Array{Int64, 1})
```

The principles outlined apply even when a variable isn't explicitly type-annotated. The reason is that **an assignment without `::` implicitly assigns the type `Any` to the variable**. For example, the statements `x = 2` and `x::Any = 2` are equivalent.

The same occurs when omitting `<:` from the expression `where T`, which implicitly takes `T <: Any`. Thus, for instance, `x = 2` is equivalent to `x::T where T = 2` or `x::T where T <: Any = 2`. Considering this, all variables listed below have their types constrained in a similar manner.

TYPE 'ANY' WITH OPERATOR ::

```
# all are equivalent
a      = 2
b::Any = 2
```

TYPE 'ANY' WITH OPERATOR <:

```
# all are equivalent
a                   = 2
b::T where T        = 2
c::T where T <: Any = 2
```

Once we recognize that variables default to the type `Any`, it becomes clear why they can be reassigned with values of different types. For instance, given `a = 1`, executing `a = "hello"` afterward is valid, since `a` is implicitly type-annotated with `Any`.

> ⚠ **Warning!** - One-liner Statements Using `where`
>
> Be careful with one-liner statements using `where`, especially when `where T` is shorthand for `where T <: Any`. These concise statements can easily lead to confusion, as demonstrated below.

<: AND ASSIGNMENT

```
a::T where T                  = 2     # this is not `T = 2`, it's
`a = 2`

a::T where {T}                 = 2     # slightly less confusing
notation
a::T where {T <: Any}          = 2     # slightly less confusing
notation
```

<: AND FUNCTION

```
foo(x::T) where T             = 2     # this is not `T = 2`, it's
`foo(x) = 2`

foo(x::T) where {T}            = 2     # slightly less confusing
notation
foo(x::T) where {T <: Any}     = 2     # slightly less confusing
notation
```

FUNCTIONS

Function arguments can also be type-annotated. This is illustrated below, where functions are restricted to integer inputs exclusively.

OPERATOR ::

```
function foo1(x::Int64, y::Int64)
    x + y
end
```

```
julia> foo1(1, 2)
3
julia> foo1(1.5, 2)
ERROR: MethodError: no method matching foo1(::Float64, ::Int64)
```

OPERATOR <:

```
function foo2(x::Vector{T}, y::Vector{T}) where T <: Int64
    x .+ y
end
```

```
julia> foo2([1,2], [3,4])
2-element Vector{Int64}:
 4
 6
julia> foo2([1,2], [3.0, 4.0])
ERROR: MethodError: no method matching foo2(::Vector{Int64}, ::Vector{Float64})
```

Note that when both function arguments are annotated with the same type parameter `T`, they're constrained to share exactly the same type. Also notice that types like `Int64` preclude the use of `Float64`, even for numbers like `3.0`. Considering both facts, greater flexibility can be achieved by introducing separate type parameters, annotating them with a common abstract type like `Number`.

SAME TYPE

```
function foo2(x::T, y::T) where T <: Number
    x + y
end
```

```
julia> foo2(1.5, 2.0)
3.5
julia> foo2(1.5, 2)
ERROR: MethodError: no method matching foo2(::Float64, ::Int64)
```

DIFFERENT TYPES

```
function foo3(x::T, y::S) where {T <: Number, S <: Number}
    x + y
end
```

```
julia> foo3(1.5, 2.0)
3.5
julia> foo3(1.5, 2)
3.5
```

The greatest flexibility is achieved when we don't type-annotate function arguments at all, as they'll implicitly default to `Any`. This can be observed below, where all tabs define identical functions. Ultimately, type-annotating function arguments is only needed to prevent invalid usage (e.g., to ensure that `log` isn't applied to a negative value).

"ANY" (IMPLICIT)

```
function foo(x, y)
    x + y
end
```

"ANY" WITH ::

```
function foo(x::Any, y::Any)
    x + y
end
```

"ANY" WITH <:

```
function foo(x::T, y::S) where {T <: Any, S <: Any}
    x + y
end
```

EQUIVALENT TO <:

```
function foo(x::T, y::S) where {T, S}
    x + y
end
```

DEFINING VARIABLES WITH CERTAIN TYPE

To conclude this section, we present an approach for converting values into a specific type. The approach makes use of the so-called **constructors**, which are functions that create new instances of a concrete type. They're useful for transforming a variable `x` into another type.

Constructors are implemented via functions of the form `Type(x)`, where `Type` should be replaced with the name of the type (e.g., `Vector{Float64}`). Like any other function, constructors also support broadcasting.

SCALAR

```
x = 1

y = Float64(x)
z = Bool(x)
```

```
julia> y
1.0
julia> z
true
```

VECTOR

```
x = [1, 2, 3]

y = Vector{Any}(x)
```

```
julia> y
3-element Vector{Any}:
 1
 2
 3
```

BROADCASTING

```
x = [1, 2, 3]

y = Float64.(x)
```

```
julia> y
3-element Vector{Float64}:
 1.0
 2.0
 3.0
```

Remark

Parametric types can be used as constructors. Moreover, abstract types may still serve as constructors, despite that they can't be instantiated. In such cases, Julia will attempt to convert the object to a specific concrete type. Nonetheless, not all abstract types can be used for this purpose.

SCALARS

```
x = 1

y = Number(x)
```

```
julia> typeof(y)
Int64
```

VECTORS

```
x = [1, 2]

y = (Vector{T} where T)(x)
```

```
julia> typeof(y)
Vector{Int64}
```

UNFEASIBLE

```
x = 1

z = Any(x)
```

```
ERROR: MethodError: no constructors have been defined for Any
```

An alternative to transform `x`'s type into `T` is given by the function `convert(T,x)`. Note that this method only works when a valid conversion exists, such as when `Float64` can be translated into an equivalent `Int64` (e.g., `3.0`).

SCALAR

```
x = 1

y = convert(Float64, x)
z = convert(Bool, x)
```

```
julia> y
1.0
julia> z
true
```

VECTOR

```
x = [1, 2, 3]

y = convert(Vector{Any}, x)
```

```
julia> y
3-element Vector{Any}:
 1
 2
 3
```

BROADCASTING

```
x = [1, 2, 3]

y = convert.(Float64, x)
```

```
julia> y
3-element Vector{Float64}:
 1.0
 2.0
 3.0
```

FOOTNOTES

[1.] Types don't necessarily follow a subtype-supertype hierarchy. For example, `Float64` and `Vector{String}` exist independently, without a hierarchical relationship. This fact will become clearer when the concepts of abstract and concrete types are defined.

[2.] The `Signed` subtype of `Integers` allows for the representation of negative and positive integers. Julia also offers the type `Unsigned`, which only accepts positive integers and comprises subtypes such as `UInt64` and `UInt32`.

[3.] `T` can be replaced by any other letter

7e. Functions: Type Inference and Multiple Dispatch

Martin Alfaro
PhD in Economics

INTRODUCTION

Functions play a central role in Julia to achieving high performance. This role is by design: they have been engineered from the outset to generate efficient machine code. In practice, it means that wrapping code within functions isn't just good programming practice for organizing logic. It's actually necessary to attain fast execution.

However, to fully unlock the power of functions, we must first understand the underlying process of function calls. Essentially, when a function is called, Julia attempts to identify concrete types for its variables, eventually selecting a corresponding computation method. At the heart of the process are three interconnected mechanisms: **dispatch**, **compilation**, and **type inference**. This section will provide a detailed explanation of each.

VARIABLE SCOPE IN FUNCTIONS

To understand why functions are central to writing high-performance code, recall the variable scope of functions. **Local variables** consist of function arguments and variables created within the function body. They exist only during the function's execution and are inaccessible from outside. **Global variables**, on the other hand, refer to any variable defined outside the function and remain accessible throughout program execution.

When code runs inside a function and all its variables are local, the compiler can reason about types, optimize aggressively, and generate efficient machine code. Global variables disrupt this process, as their types and values can change at any time, forcing the compiler to insert dynamic lookups. This process prevents key optimizations, resulting in a substantial performance degradation.

The use of global variables isn't limited to code written in the global scope. It also arises when a function references or writes variables that either aren't passed as arguments or are defined in the function body. As a result, the performance drawbacks from global variables are present in all the following cases.

GLOBAL SCOPE

```
x = 2

y = 3 * x
```

```
julia> y
6
```

FUNCTION USING A GLOBAL VARIABLE

```
x = 2

f() = 3 * x
```

```
julia> f()
6
```

Recall that an assignment like `x = 2` is shorthand for `x::Any = 2`, reflecting that global variables default to `Any` if they aren't explicitly type-annotated. Furthermore, **only concrete types can be instantiated**, meaning that values can only adopt a concrete type. This is why `x::Any` shouldn't be interpreted as `x` having type `Any`, but rather that `x` can take on any concrete type that's a subtype of `Any`. Since `Any` sits at the top of Julia's type hierarchy, this simply means that `x`'s types are unrestricted.

The unrestricted nature of global variables is precisely what prevents Julia from specializing operations. In our example, `x` is a global variable, and so the specialization of `*` is precluded. Specifically, once a global variable is introduced, Julia must consider multiple possible methods for the computation of `*`, one for each possible concrete type of `x`. In practice, this leads to code generation with multiple branches, potentially involving type checks, conversions, and memory allocations. The consequence is degraded performance.

The underlying logic for Julia treating global variables as potentially embodying any type and value is that, even if a variable holds some value at a specific moment, the user may reassign it at any point in the program. Note that the performance issues wouldn't completely go away if we had type-annotated `x` with a concrete type such as `x::Int64 = 2`. The reason is that many optimizations depend not only on knowing the concrete type of a variable, but also on knowing its value and having a well-defined scope where that value exists. It's only when these assumptions are met that Julia can gain a comprehensive view of all the operations to be performed, creating opportunities for optimizations.

Functions in Julia are designed precisely to provide these guarantees. By enforcing local scope and predictable variable behavior, they enable method specialization and unlock the full range of performance optimizations. Next, we explore the specific steps that Julia takes to achieve this.

FUNCTIONS AND METHODS

A **function** is simply a name that can be associated with different implementations. Each implementation is known as a **method**. This specifies the function body for a particular combination of argument types and number of arguments. You can inspect the methods associated with a function `foo` by executing `methods(foo)`.

To illustrate, let's define several methods for the function `foo1`. Creating a method requires type-annotating the arguments of `foo1` during its definition, implemented via the `::` operator. In this way, we can set a distinct function body for each unique combination of argument types.

To keep matters simple, let's begin with a scenario where all the methods of `foo1` take the same number of arguments, thus differing only in their types.

METHODS

```
foo1(a,b)                   = a + b
foo1(a::String, b::String) = "This is $a and this is $b"
```

```
julia> methods(foo1)
2 methods for generic function "foo1" from Main
julia> foo1(1,2)
3
julia> foo1("some text", "more text")
"This is some text and this is more text"
```

Since `foo1(a,b)` is equivalent to `foo1(a::Any,b::Any)`, the first method sets the behavior of `foo1` for every possible pair of argument types. The second method `foo1(a::String, b::String)` overrides the function's behavior when both arguments are strings.

The existence of multiple methods explains why the two calls produce different outputs: executing `foo1(1, 2)` makes use of the first method of `foo1`, whereas `foo1("some text", "more text")` triggers the second method.

The example also reveals that **methods don't need to perform similar operations**. Although it's usually unwise to group unrelated behaviors under a single function name, this capability serves a different purpose. It allows developers to optimize algorithms for specific variable types, which is central to performance-oriented programming. Thus, by reusing the same function name across different implementations, the execution strategy is tailored to each type, while maintaining a consistent interface for users.

Also note that **methods don't need to have the same number of arguments**. For instance, it's possible to define the following methods for a function `foo2`.

METHODS WITH DIFFERENT NUMBERS OF ARGUMENTS

```
foo2(x)       = x
foo2(x, y)    = x + y
foo2(x, y, z) = x + y + z
```

```
julia> methods(foo2)
3 methods for generic function "foo2" from Main
julia> foo2(1)
1
julia> foo2(1, 2)
3
julia> foo2(1, 2, 3)
6
```

Defining methods with different number of arguments is particularly useful for extending a function's behavior. A prime example is given by the function `sum`. So far, we've only used its simplest form `sum(x)`, which adds all the elements of a collection `x`. However, `sum` also supports additional methods. One of them is `sum(<function>, x)`, where the elements of `x` are transformed via `<function>` before being summed.

METHODS FOR 'SUM'

```
x = [2, 3, 4]

y = sum(x)          # 2 + 3 + 4
z = sum(log, x)     # log(2) + log(3) + log(4)
```

FUNCTION CALLS

Building on our understanding of how functions and methods are defined, let's now analyze the process triggered when a function is called. All our explanations will be based on the following function `foo`:

EXAMPLE

```
foo(a, b) = 2 + a * b
```

```
julia> foo(1, 2)
4
julia> foo(3, 2)
8
julia> foo(3.0, 2)
8.0
```

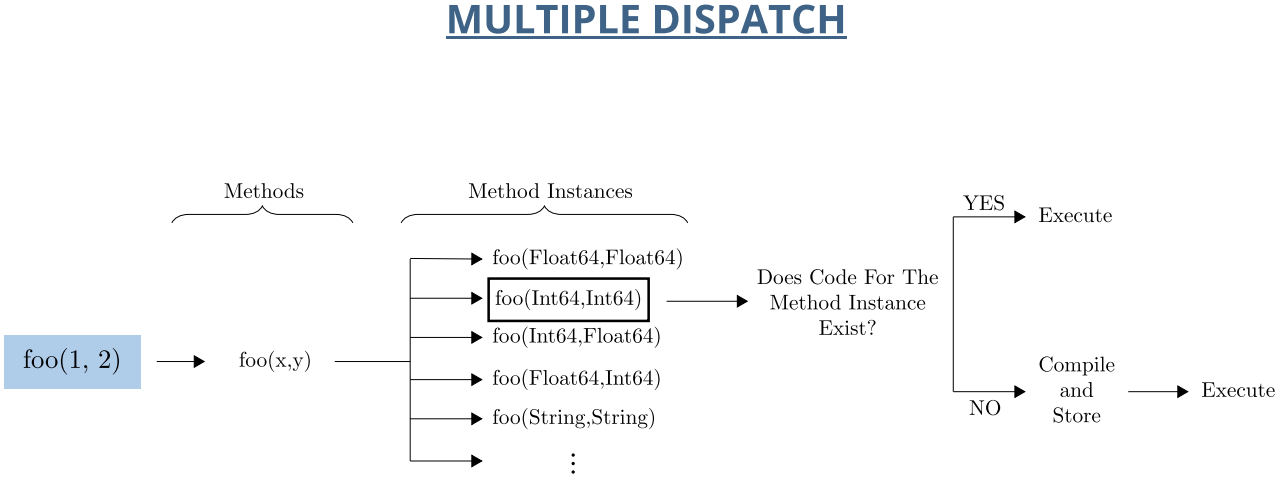
Defining a function like `foo(a, b)` is shorthand for creating a **method** with the signature `foo(a::Any, b::Any)`. This means the function body applies to every possible combination of argument types.

When we call `foo(1, 2)`, Julia evaluates the expression `2 + a * b` by following a well-defined sequence of steps.

The process begins with what's known as **multiple dispatch**, where Julia selects which method of the function to execute. Importantly, this decision is based solely on the concrete types of the function arguments, not their values. The information on types is then used to select a *method*, which defines the function body and hence the operations to be performed. This process involves searching through the available methods of `foo`, eventually choosing the most specific one that matches the concrete types of `a` and `b`. The function `foo` in our example has only the single method `foo(a,b) = 2 + a * b`. This applies to any argument types, including `a::Int64` and `b::Int64`, and so the corresponding function body is `2 + a * b`.

Once the method is chosen, Julia looks for a **method instance**, which is the compiled version of the method specialized for the signature `foo(a::Int64, b::Int64)`. If such an instance already exists, Julia reuses it immediately to compute `foo(1,2)`. If not, the compiler generates optimized machine code for that specific type combination, caches it for future calls, and then executes it. This on-demand specialization is one of the key reasons Julia can offer a performance that's close to low-level languages.

The following diagram depicts the process unfolded when `foo(1,2)` is executed.



The process outlined has implications for how the language works. In particular, the first time a function is called with a particular combination of concrete types, Julia incurs a time cost to generate specialized code for those types. This initial delay is often referred to as Time To First Plot, a phrase that highlights how noticeable the compilation overhead was in plotting libraries. Starting with Julia 1.9, the ability to precompile packages has improved substantially. As a result, the issue primarily affects user-defined functions and uncommon type combinations that weren't precompiled by packages. In these cases, the user will still experience compilation delays during the first execution.

Note, though, that once Julia compiles a method instance, it caches the resulting code. Consequently, any later call with the same argument types can reuse that cached instance immediately, avoiding the compilation step entirely. In practical terms, it implies that all subsequent function calls with the same concrete types are fast.

The behavior of `foo` illustrates this process clearly. After evaluating `foo(1, 2)`, Julia has already compiled a method instance for the specific signature `foo(a::Int64, b::Int64)`. Thus, the subsequent call `foo(3, 2)` is executed immediately using the cached method instance, without any need for recompilation. Instead, the execution of `foo(3.0, 2)` introduces a new combination of argument types `a::Float64` and `b::Int64`. Because no compiled method instance yet exists for this signature, Julia must generate one before executing the function.

TYPE INFERENCE

As we indicated, Julia generates the machine code to implement a function during the compilation process. **Most considerations for achieving high performance are tied to this step**.

Compilation in Julia happens on demand, right at the moment a function is first called with a new set of argument types. The strategy is known as **Just-In-Time Compilation (JIT)**. A key mechanism in it is **type inference**, whereby the compiler attempts to identify concrete types for *all* variables and expressions within the function body.

If the compiler succeeds in identifying concrete types, it can specialize instructions for each operation and yield fast code. For instance in the example above with the function `foo`, type inference involves determining concrete types for `2`, `a = 1`, and `b = 2`. Since all values have type `Int64`, the compiler can specialize the computation of `2 + a * b` for variables with type `Int64`. This is the essence behind **type stability**, which we'll cover extensively in the next chapter.

On the contrary, if the compiler is unable to identify concrete types for some expressions, it must create generic code to accommodate multiple combinations of types. This forces Julia to perform type checks and conversions during runtime, significantly degrading performance.

Below, we provide various remarks about type inference that are worth keeping in mind for next sections.

FUNCTIONS THEMSELVES DO NOT GUARANTEE THE IDENTIFICATION OF CONCRETE TYPES

Merely placing code inside a function doesn't guarantee that the compiler will identify concrete types for all operations. The following example illustrates this.

TYPE-UNSTABLE FUNCTION

```
x       = [1, 2, "hello"]    # Vector{Any}

foo3(x) = x[1] + x[2]        # type unstable
```

```
julia> foo3(x)
3
```

The issue in the example is that `x[1]` and `x[2]` come from an object with type `Vector{Any}`, and so the compiler assigns the type `Any` to both. This explains why the compiler can't specialize the computation of `+`. The example also highlights that compilation is exclusively based on types, not values. Thus, the generated code ignores that the actual values are `x[1] = 1` and `x[2] = 2`, which would otherwise reveal their type `Int64`.

GLOBAL VARIABLES INHERIT THEIR GLOBAL TYPE

Type inference is restricted to local variables. Instead, any global variable inherits its type from the global scope. In the following examples, `b` and `d` are global variables. Consequently, the compiler defines the type of `b` as `Any` and that of `d` as `Number`.

UNANNOTATED GLOBAL VARIABLE

```
a         = 2
b         = 1

foo4(a)   = a * b
```

```
julia> foo4(a)
2
```

TYPE-ANNOTATED GLOBAL VARIABLE

```
c         = 2
d::Number = 1

foo4(c)   = c * d
```

```
julia> foo4(c)
2
```

TYPE-ANNOTATING FUNCTION ARGUMENTS DOES NOT IMPROVE PERFORMANCE

The ability of the compiler to identify concrete types is crucial for achieving high performance. This might wrongly suggest that explicitly annotating function arguments is necessary for performance, or at least beneficial. Thanks to type inference, such annotations are actually redundant. In fact, adding them can be counterproductive: type-annotations unnecessarily restrict the types accepted by a function, thus limiting its flexibility and potential applications.

To see how this loss of flexibility plays out, compare the following scripts.

UNANNOTATED FUNCTION

```
foo5(a, b) = a * b
```

```
julia> foo5(0.5, 2.0)
1.0
julia> foo5(1, 2)
2
```

TYPE-ANNOTATED FUNCTION

```
foo6(a::Float64, b::Float64) = a * b
```

```
julia> foo6(0.5, 2.0)
1.0
julia> foo6(1, 2)
ERROR: MethodError: no method matching foo6(::Int64, ::Int64)
```

The function on the first tab only accepts arguments of type `Float64`, meaning that even integers are disallowed. By contrast, the function on the second tab also handles `Float64` inputs, but additionally allows for other types. This is due to the implicit type annotation `Any` on the function arguments.

> **Packages Commonly Type-Annotate Function Arguments**
>
> When inspecting code of packages, you may notice that function arguments are often type-annotated. The reason for this isn't related to performance, but rather to ensure that the function is used as intended, safeguarding against inadvertent type mismatches.
>
> For instance, suppose a function that computes theater revenue via `nr_tickets * price`. In Julia, the operator `*` not only implements the product of numbers, but also concatenates words when applied to expressions with type `String`. Without type-annotations, the function could potentially be misused, as exemplified in the first tab below. The second tab precludes this possibility by asserting types.
>
> **UNANNOTATED FUNCTION**
>
> ```
> revenue1(nr_tickets, price) = nr_tickets * price
> ```
>
> ```
> julia> revenue1(3, 2)
> 6
> julia> revenue1("this is ", "allowed")
> "this is allowed"
> ```

TYPE-ANNOTATED FUNCTION

```
revenue2(nr_tickets::Int64, price::Number) = nr_tickets * price
```

```
julia> revenue2(3, 2)
6
julia> revenue2("this is ", "not allowed")
ERROR: MethodError: no method matching revenue2(::String,
::String)
```

8a. Overview and Goals

Martin Alfaro
PhD in Economics

This chapter focuses on the role of **type stability** in achieving high performance. Its importance for Julia can't be overstated: **any attempt to generate fast code without ensuring type stability is destined to fail**. Because of this, type stability should be the starting point of the optimization process, before pursuing any further performance improvements.

The formal definition of type stability is provided in Section 8b. Its relevance becomes clear when we understand how computers execute operations. At the most basic level, the process for internal computation differs depending on the data types involved. Integer arithmetic, for instance, follows an entirely different execution path than floating-point arithmetic. Because computers must implement distinct operations for different types, performance hinges on identifying concrete types for all variables before code execution.

Julia addresses this requirement through type inference, where Julia analyzes the code and tries to deduce the concrete type of each variable. However, inference is only performed inside function bodies. Consequently, wrapping your code in a function is a necessary first step toward type stability. Importantly, though, this alone doesn't guarantee type stability. That's why we'll also explore additional conditions required.

The roadmap of the chapter is as follows:

- Section 8c considers how to achieve type stability when functions operate with two of the most common objects in Julia: scalars and vectors.
- Section 8d demonstrates how asserting variable types at the global scope ensures type stability. This technique is especially valuable for variables that act as constants throughout a program, letting us work with global variables without sacrificing performance.
- Section 8e provides a general technique to address type instability based on "barrier" functions. When a variable inside a function has ambiguous type information, the technique involves calling a second function (the "barrier" function) from within the original one. This triggers a new round of type inference, thereby restoring stability.
- Section 8f moves to the analysis of type stability when tuples are involved. Because tuples encode richer type information, their analysis requires more nuanced considerations.
- Section 8g deals with type stability when a function calls a higher-order function, which is a function that takes another function as an argument (e.g., the `map` function). Julia avoids specialization in this case, so that the higher-order function runs without considering the type of the variable being transformed. We explore several ways to retain type stability in this case.
- Section 8h addresses subtle scenarios where type instability can unexpectedly arise. These cases are referred to as "gotchas" and can surprise even experienced users.

8b. Defining Type Stability

Martin Alfaro
PhD in Economics

INTRODUCTION

One of the primary reasons Julia achieves high performance is its ability to generate specialized machine code for functions. Thus, instead of relying on generic instructions, Julia can tailor code to the concrete operations and types appearing in a function. This capability relies heavily on **type inference**, the process by which the compiler deduces the types of variables and intermediate results before execution. When the compiler can successfully predict the concrete types of all operations within a function body, the resulting code is described as **type stable**.

Type stability isn't just a mere implementation detail, but rather a practical requirement for writing efficient Julia programs. Without it, the compiler must fall back to generic code, leading to significant performance penalties.

This section formally defines type stability and reviews the tools employed for its verification. In the next section, we'll start to examine how type stability applies in specific scenarios.

AN INTUITION

When a function is called, Julia follows a well-defined sequence of steps to determine how that call should be executed. We already **described this process** and now briefly review it.

Consider a function `foo(x) = x + 2` and the execution of `foo(a)` for some variable `a`. We assume `a` has a specific value assigned and therefore a concrete type, although we omit explicitly specifying a value for `a`. This lets us highlight that the process unfolded depends on types, rather than values.

When evaluating `foo(a)`, Julia first determines the concrete type of `a`, which we'll denote as `T`. It then checks whether a compiled method instance of `foo` specialized for an argument of type `T` already exists. If such an instance exists, then `foo(a)` is executed immediately. Otherwise, Julia proceeds to compile one. This compilation step leverages type inference, wherein the compiler attempts to deduce concrete types for all terms within the function body. The resulting machine code is then stored, making it readily available for subsequent calls of `foo(b)` with `b` having type `T`.

TYPE STABILITY AND PERFORMANCE

The key to generating fast code lies in the information available to the compiler during the compilation stage. This information is primarily gathered through type inference, where the compiler identifies the specific type of each variable and expression involved. When the compiler can **accurately predict a single concrete type for the function's output**, the function call is said to be **type stable**.

While this constitutes the **formal definition of type stability**, a more stringent definition is usually applied in practice: the compiler must be able to **infer concrete types for *each* expression within the function**, rather than only for the final output. This definition aligns with the output provided by `@code_warntype`, which is the built-in macro to detect type instabilities. The use of this macro we'll be presented in the next subsection.

When type stability holds, the compiler can specialize the computational approach for each operation, resulting in fast execution. Essentially, type stability dictates that there's sufficient information to determine a straight execution path, thus avoiding unnecessary type checks and dispatches at runtime.

In contrast, type-unstable functions generate generic code, thus accommodating each possible combination of concrete types. This results in additional overhead during runtime, where Julia is forced to dynamically gather type information and perform extra calculations based on it. The consequence is a pronounced deterioration in performance.

> **Type Stability Characterizes Function *Calls***
>
> It's common to describe a function as "type stable". Strictly speaking, however, type stability isn't a property of the function. Rather, it's a property of how the function behaves when called with arguments of particular concrete types. The distinction is crucial in practice, since a function may exhibit type stability for certain input types, but not for others.

AN EXAMPLE

To identify type stability in practice, let's consider the following example.

TYPE STABLE

```
x      = [1, 2, 3]          # `x` has type `Vector{Int64}`

foo(x) = sum(x[1:2])        # type stable with this `x`
```

```
julia> @btime foo($x)
  23.485 ns (2 allocations: 80 bytes)
```

TYPE UNSTABLE

```
x      = [1, 2, "hello"]    # `x` has type `Vector{Any}`

foo(x) = sum(x[1:2])        # type UNSTABLE with this `x`
```

```
julia> @btime foo($x)
  47.666 ns (2 allocations: 80 bytes)
```

The two operations may look identical at first glance, since both ultimately evaluate `1 + 2`. However, the way Julia compiles and executes each version differs. In fact, the first approach is faster since the function is type stable.

In that first version, the expression `x[1] + x[2]` can be inferred to have type `Int64`. Because the compiler can determine that both `x[1]` and `x[2]` are `Int64`, it's able to generate efficient, specialized machine code for this concrete type. Importantly, this optimization isn't limited to the specific `x` used: any call of the form `sum(y)` where `y` is a `Vector{Int64}` benefits from the same specialization.

The second version, however, introduces type instability. Here, `x` has type `Vector{Any}`, which prevents the compiler from deducing a single concrete type for the expression `x[1] + x[2]`. Since the elements of a `Vector{Any}` may hold values of any subtype of `Any`, the compiler can't predict whether `x[1]` or `x[2]` will be an `Int64`, a `Float64`, a `Float32`, or other type. This results in slow code, and the penalty applies to every call of `sum(y)` where `y` is a `Vector{Any}`.

> **Remark**
>
> Julia's developers are continually refining the compiler, addressing and mitigating the effects of type instability. As a result, **many operations that were once type unstable are now type stable**. This means that type stability should be considered a dynamic property of the language, subject to change as the compiler evolves.

CHECKING FOR TYPE STABILITY

There are several mechanisms to determine whether a function call is type stable. One of them is based on the `@code_warntype` macro, which reports all the types inferred during a function call. To illustrate its use, consider a function that defines `y` as a transformation of `x` and then `y` is used to perform some operation.

TYPE UNSTABLE

```
function foo(x)
    y = (x < 0) ?  0  :  x

    return [y * i for i in 1:100]
end
```

```
julia> @code_warntype foo(1.0)
```

TYPE STABLE

```
function foo(x)
    y = (x < 0) ?  0  :  x

    return [y * i for i in 1:100]
end
```

```
julia> @code_warntype foo(1)
```

The output of `@code_warntype` can be difficult to interpret. Nonetheless, the inclusion of colors facilitates its understanding:

- If all lines are **blue**, the function is **type stable**. This means that Julia can identify a unique concrete type for each expression.
- If at least one line is **red**, the function is **type unstable**. It reflects that one or more expressions could potentially adopt multiple types.
- **Yellow** lines indicate type instabilities that the compiler can handle effectively, in the sense that they have a reduced impact on performance. As a rule of thumb, **you can safely ignore them**.

> ⚠ **Warning!**
> Throughout the book, we'll refer to **type instabilities** as those indicated by a red warning exclusively. Yellow warnings will be mostly ignored.

In the provided example, the compiler attempts to infer concrete types. This is done by identifying two pieces of information given `x`'s concrete type:

i) the type of `y`,

ii) the type of `y * i`, with `i` identified with type `Int64`. Implicitly, this also defines the type of `[y * i for i in 1:100]`.

The example clearly demonstrates that **the same function can be type stable or unstable depending on the types of its inputs**: `foo` is type stable when `x` has type `Int64`, but type unstable when `x` is `Float64`.

Specifically, in the scenario where `x = 1`, the compiler infers for *i)* that `y` can be equal to either `0` or `x`. Since both `0` and `1` are `Int64`, the compiler identifies a unique type for `y`, given by `Int64`. Regarding *ii)*, `y * i` also yields an `Int64`, as both `i` and `y` have type `Int64`. This determines that `[y * i for i in 1:100]` has type `Vector{Int64}`. Consequently, `foo(1)` is type stable, enabling Julia to invoke a tailored method for integers.

As for `x = 1.0`, the information for *i)* is that `y` could be either `0` or `1.0`. As a result, the compiler can't infer a unique type for `y`, which could be either `Int64` or `Float64`. The `@code_warntype` macro reflects this, identifying `y` as having type `Union{Float64, Int64}`. This ambiguity affects *ii)*,

forcing the compiler to consider approaches that handle both `Float64` and `Int64`, and hence preventing specialization. Overall, `foo(1.0)` is type unstable, which has a detrimental impact on performance.

> **Function Call Leverage Information on Types, Not Values**
>
> The conclusions regarding type stability wouldn't have changed if we had respectively considered `foo(-2)` or `foo(-2.0)` in each tab. Type stability depends on whether `x` has type `Int64` or `Float64`, regardless of its actual value. In other words, since the compiler analyzes type information exclusively, the actual numeric value plays no role in determining type stability.

YELLOW WARNINGS MAY TURN RED

Not all type instabilities carry the same performance cost, and Julia indicates their severity through yellow and red warnings. A yellow warning typically signals a mild issue: the compiler has detected a type instability, but it occurs in a small isolated computation that Julia can still optimize reasonably well. However, repeated execution of these operations may escalate into more serious performance issues, triggering a red warning. The following example demonstrates a scenario like this.

YELLOW WARNING

```
function foo(x)
    y = (x < 0) ?  0  :  x

    y * 2
end
```

```
julia> @code_warntype foo(1.0)
```

RED WARNING

```
function foo(x)
    y = (x < 0) ?  0  :  x

    [y * i for i in 1:100]
end
```

```
julia> @code_warntype foo(1.0)
```

YELLOW WARNING

```
function foo(x)
    y = (x < 0) ?  0  :  x

    for i in 1:100
      y = y + i
    end

    return y
end
```

```
julia> @code_warntype foo(1.0)
```

In the first case, the yellow warning appears because the expression `y * 2` may produce either a `Float64` or an `Int64`. Since this computation happens only once and involves types the compiler can handle efficiently, the impact on performance is minimal. In contrast, the second tab performs repeated evaluations of `y * i` without a concrete type for `y`, resulting in a red warning.

Note, though, that a yellow warning doesn't necessarily escalate into a red warning when incorporated into a for-loop. The third tab demonstrates a situation where the compiler can still manage the instability effectively, even under repeated execution. This highlights that type instabilities vary in severity, and not all of them pose a meaningful threat to performance.

For-Loops and Yellow Warnings

When running for-loops, Julia will always emit a yellow warning, even if the operation is type stable. The warning can safely be disregarded, as it simply reflects the inherent behavior of iterators: they return either the next element to iterate over, or `nothing` when the sequence is exhausted (a value with type `Nothing`).

YELLOW WARNING IN FOR-LOOPS

```
function foo()
    for i in 1:100
        i
    end
end
```

```
julia> @code_warntype foo()
```

8c. Type Stability with Scalars and Vectors

Martin Alfaro
PhD in Economics

INTRODUCTION

The previous section has defined type stability, along with approaches to checking whether the property holds. The formal definition of a type-stable function is that the function's output type can be inferred from its argument types. In practice, however, we often rely on a more stringent definition, which requires that the compiler can infer a single concrete type for each expression within the function body. This property guarantees that every operation is specialized, resulting in optimal performance. [1]

In this section, we start the analysis of type stability for specific objects. We cover in particular the case of scalars and vectors, providing practical guidance for achieving type stability with them.

TYPES OF SCALARS AND VECTORS

The notion of type stability applied to scalars is straightforward. It demands operations to be performed on variables with the same concrete type (e.g., `Float64`, `Int64`, `Bool`). Likewise, type stability for vectors requires that their *elements* have a concrete type.

The following table identifies some types for scalars and vectors satisfying this property.

Objects Whose Elements Have Concrete Types

Scalars	Vectors
`Int`	`Vector{Int}`
`Int64`	`Vector{Int64}`
`Float64`	`Vector{Float64}`
`Bool`	`BitVector`

Note: `Int` defaults to `Int64` or `Int32`, depending on the CPU architecture.

Next, we'll delve into type stability of scalars and vectors, considering each case separately.

TYPE STABILITY WITH SCALARS

To make the definition of type stability for scalars operational, let's revisit some concepts about types. Recall that only concrete types such as `Int64` or `Float64` can be instantiated. Moreover, every value that appears during execution is ultimately an instance of one of these concrete types. Instead, abstract types like `Any` or `Number` can't be instantiated, meaning they never hold values directly. Their role is organizational: they describe sets of possible concrete types and provide the structure that shapes the type hierarchy.

This distinction explains why a type annotation such as `x::Number` shouldn't be read as `x` having type `Number`. Rather, it constrains `x` to values whose concrete types are subtypes of `Number`. At runtime `x` must always have a concrete type. For instance, after evaluating `x::Number = 2`, the variable `x` contains the value 2, whose concrete type is `Int64`.

With this in mind, we can discuss how type instability arises. A common source of instability is mixing values of different types, such as combinations of `Int64` with `Float64`. However, mixing types doesn't automatically imply type instability due to type promotion and automatic conversions. We discuss these concepts next.

TYPE PROMOTION AND CONVERSION

Julia employs various mechanisms to handle cases that combine `Int64` and `Float64`. The first one is part of a concept known as **type promotion**, which converts dissimilar types to a common one whenever possible. The second one emerges when variables are type-annotated, in which case Julia engages in **type conversion**. By transforming values to the respective type declared, this feature also prevents the mix of types. Both mechanisms are illustrated below.

TYPE PROMOTION

```
foo(x,y)    = x * y

x1          = 2
y1          = 0.5

output        = foo(x1,y1)        # type stable: mixing `Int64` and `Float64` results in
`Float64`
```

```
julia> output
1.0
```

TYPE CONVERSION

```
foo(x,y)    = x * y

x2::Float64 = 2               # this is converted to `2.0`
y2          = 0.5

output      = foo(x2,y2)      # type stable: `x` and `y` are `Float64`, so output type is
predictable
```

```
julia> output
1.0
```

In the first tab, the output type depends on the types of the function arguments. However, in all cases the output type can be predicted, since mixing `Int64` and `Float64` results in `Float64` due to automatic type promotion.

As for the second tab, Julia transforms the value of `x2` to make it consistent with the type-annotation declared. Consequently, `x * y` is computed as the product of two values with type `Float64`.

TYPE INSTABILITY WITH SCALARS

While type promotion and conversion can handle several situations, they certainly don't cover all cases. One such scenario is when a scalar value depends on a conditional expression and each branch returns values of different types. In this situation, since the compiler reasons about types rather than values, it can't determine which branch is relevant for the function call. As a result, it'll generate code that accommodates both possibilities, as it happens in the following example.

INT64

```
function foo(x,y)
    a = (x > y) ?  x  :  y

    [a * i for i in 1:100_000]
end

foo(1, 2)           # type stable   -> `a * i` is always `Int64`
```

```
julia> @btime foo(1,2)
  17.949 μs (3 allocations: 781.320 KiB)
```

MIXING INT64 AND FLOAT64

```
function foo(x,y)
    a = (x > y) ?  x  :  y

    [a * i for i in 1:100_000]
end

foo(1, 2.5)         # type UNSTABLE -> `a * i` is either `Int64` or `Float64`
```

```
julia> @btime foo(1,2.5)
  38.658 µs (3 allocations: 781.320 KiB)
```

In the example, type instability is inevitable if `x` and `y` have different types. The reason is that type promotion only ensures that `a * i` will be converted to `Float64` if `a` is `Float64`. However, the compiler still needs to consider the possibility that `a` could be `Int64`, in which case `a * i` would be `Int64`.

Since both outcomes are possible from the compiler's perspective, the generated method instance has to support both. Only at runtime can Julia observe the actual types, narrow down the possibilities, and select the appropriate computation implementation.

TYPE STABILITY WITH VECTORS

Vectors in Julia are collections whose elements share the same element type. Given this definition, reasoning about type stability requires distinguishing between the type of the vector itself and the type of its elements.

Recall that, by construction, **every vector type is concrete**. For instance, `Vector{Any}` is a concrete type, even though its elements may belong to any subtype of `Any`. Consequently, this fact can't serve as a criterion for type stability, since otherwise all functions operating on vectors would automatically be type stable. In fact, a function operating on a vector like `Vector{Any}` would lead to type instability if not addressed properly.

Instead, what matters is that operations on vectors ultimately manipulate individual elements. Thus, **type stability is contingent on whether their element type is concrete**.[2]

TYPE INSTABILITY

Below, we compare type stability for vectors, depending on whether their element types are abstract or concrete. All examples rely on the function `sum`.

We begin with vectors whose element types are concrete.

VECTOR{INT64}

```
z1::Vector{Int64}   = [1, 2, 3]

sum(z1)             # type stable
```

VECTOR{FLOAT64}

```
z2::Vector{Float64} = [1, 2, 3]

sum(z2)             # type stable
```

BITVECTOR

```
z3::BitVector       = [true, false, true]

sum(z3)             # type stable
```

In contrast, the next examples use vectors whose elements have abstract types. They result in type instability.

VECTOR{NUMBER}

```
z4::Vector{Number} = [1, 2, 3]

sum(z4)                 # type UNSTABLE -> `sum` must consider all possible subtypes of
`Number`
```

VECTOR{ANY}

```
z5::Vector{Any}    = [1, 2, 3]

sum(z5)             # type UNSTABLE -> `sum` must consider all possible subtypes of `Any`
```

TYPE PROMOTION AND CONVERSION

By definition, vectors require all their elements to share the same type. This means that, when creating a vector, Julia will attempt to identify a type that can accommodate every element. This common type can be either abstract or concrete, depending on what the elements allow. For example, if you mix elements with disparate types, such as `String` and `Int64`, Julia will infer the vector's type as `Vector{Any}`. This will pose a problem for type stability.

In some cases, nonetheless, the elements can be converted to a shared concrete type. For example, this is the case when mixing `Float64` and `Int64`, in which case Julia will perform a conversion to `Float64` automatically. The following example illustrates this mechanism in a simple assignment.

AUTOMATIC CONVERSION

```
x = [1, 2, 2.5]      # automatic conversion to `Vector{Float64}`
```

```
julia> x
3-element Vector{Float64}:
 1.0
 2.0
 2.5
```

AUTOMATIC CONVERSION

```
y = [1, 2.0, 3.0]    # automatic conversion to `Vector{Float64}`
```

```
julia> y
3-element Vector{Float64}:
 1.0
 2.0
 3.0
```

Likewise, when assignments are declared with type-annotations and values differ in type, Julia will attempt to convert all values to the declared type.

ASSIGNMENT WITH CONVERSION

```
v1                = [1, 2.0, 3.0]     # automatic conversion to `Vector{Float64}`

w1::Vector{Int64} = v1                # conversion to `Vector{Int64}`
```

```
julia> w1
3-element Vector{Int64}:
 1
 2
 3
```

ASSIGNMENT WITH NO CONVERSION

```
v2                = [1, 2, 2.5]       # automatic conversion to `Vector{Float64}`

w2::Vector{Number} = v2               # `w2` is still `Vector{Number}`
```

```
julia> w2
3-element Vector{Number}:
 1.0
 2.0
 2.5
```

FOOTNOTES

[1.] Nevertheless, simply demanding that the output's type can be inferred from the input types already offers benefits. In particular, it ensures that type instability won't be propagated when the function is called in other operations.

[2.] Note that this condition isn't sufficient to guarantee type stability. Ultimately, whether a function is type stable still depends on how it's implemented. In practice, though, functions in well-designed packages are typically type stable if they receive vectors whose elements have a concrete type.

8d. Type Stability with Global Variables

Martin Alfaro
PhD in Economics

INTRODUCTION

Variables can be categorized as local or global, according to the code block in which they're defined. **Global variables** can be accessed and modified throughout the entire codebase, while **local variables** only exist within a specific scope. For this section, the scope of interest is a function, so local variables will exclusively refer to function arguments and variables defined within the function body.

The distinction is especially relevant for this chapter since **global variables are a frequent source of type instability**. This is because Julia doesn't assign concrete types to global variables, forcing the compiler to consider multiple potential types whenever these variables are used. Such a behavior prevents specialization, leading to reduced performance.

The current section explores two approaches to reduce or even eliminate the detrimental effect of global variables: **type-annotations** and **constants**. Defining global variables as constants is a natural choice when values are truly constants, such as in the case of $\pi = 3.14159$. More broadly, constants are appropriate whenever a value remains unchanged throughout the program. Compared to type annotations alone, constants offer better performance as the compiler gains knowledge of *both* the type and value, rather than just the type. This feature allows for further optimizations, effectively making **constants behave just like a literal value within a function**. [1]

> ⚠️ **Warning!** - You Should Always Wrap Code in a Function
>
> Even if you implement the fixes proposed for global variables, optimal performance still calls for wrapping tasks in functions. The reason is that **functions implement additional optimizations** that are unfeasible in the global scope.

WHEN ARE WE USING GLOBAL VARIABLES?

Let's begin by identifying operations that rely on global variables. To this end, we present two cases, each represented in a different tab. The first one considers the most direct use of global variables, where operations are performed directly in the global scope. The second tab illustrates a more nuanced case, where a function accesses and manipulates a global variable.

The third tab serves as a counterpoint, implementing the same operations but within a self-contained function. By definition, self-contained functions exclusively operate with locally defined variables.

GLOBAL SCOPE

```
# all operations are type UNSTABLE (they're defined in the global scope)
x = 2

y = 2 * x
z = log(y)
```

FUNCTION WITH A GLOBAL VARIABLE

```
x = 2

function foo()
    y = 2 * x
    z = log(y)

    return z
end

@code_warntype foo() # type UNSTABLE
```

SELF-CONTAINED FUNCTION

```
x = 2

function foo(x)
    y = 2 * x
    z = log(y)

    return z
end

@code_warntype foo(x)  # type stable
```

Self-contained functions offer advantages that extend beyond performance gains: they **promote clarity, predictability, testability, and reusability**. These benefits were briefly introduced in a previous section, where functions were framed as units that embody a specific task.

Specifically, you should get used to work with self-contained functions due to several reasons.

First, they're much easier to reason about, since all relevant information is local to the function. In particular, you don't need to track the state of variables scattered across the script. Instead, you can focus solely on the function's inputs and its internal logic.

Second, self-contained functions ensure that its behavior depends only on its own variables, not on the state of global variables. This makes its output more predictable and debugging more straightforward.

Finally, a self-contained function behaves like a small, reusable program with a clearly defined purpose. Once written, it can be applied to similar tasks without modification, reducing code duplication and improving the overall maintainability of the codebase.

ACHIEVING TYPE STABILITY WITH GLOBAL VARIABLES

The advantages of self-contained functions provide strong incentives to avoid global variables. Still, there are situations where globals are genuinely useful. A common example is when we work with true constants, defined as values that are fixed throughout the program.

With this in mind, the next section introduces two techniques. They let us work with global variables, while mitigating their performance costs.

CONSTANT GLOBAL VARIABLES

Declaring a global variable as a constant simply requires prefixing its name with the `const` keyword, as in `const x = 3`. This mechanism works for any type of value, including collections.

CONSTANT NUMBER

```
const a = 5
foo()   = 2 * a

@code_warntype foo()        # type stable
```

CONSTANT VECTOR

```
const b = [1, 2, 3]
foo()   = sum(b)

@code_warntype foo()        # type stable
```

> ⚠ **Warning!** - Avoid Reassignments of Global Variables
>
> **Global variables should be declared as constants only if their values will remain unchanged throughout the session**. Although redefining constants is feasible, doing so is highly discouraged. The possibility was only introduced to facilitate testing in interactive sessions, eliminating the need to restart Julia after each modification of a constant's value.
>
> Importantly, if a constant is reassigned, every function that depends on it must be redefined as well. Otherwise, those functions will continue to use the constant's original value. Because this requirement is easy to overlook, the most reliable practice is to rerun the entire script whenever a constant is modified.
>
> To illustrate the potential consequences of ignoring this guideline, let's compare the following code snippets that execute the function `foo`. Both define a constant value `x=1`, which is subsequently redefined as `x=2`. The first example runs the script without re-executing the

definition of `foo`, in which case the value returned by `foo` is still based on `x = 1`. In contrast, the second example reruns `foo`'s definition, thus ensuring that `foo` relies on the updated value of `x`.

WRONG USE

```
const x1 = 1
foo()    = x1
foo()              # it gives 1

x1       = 2

foo()              # it still gives 1
```

RIGHT USE

```
const x2 = 1
foo()    = x2
foo()              # it gives 1

x2       = 2
foo()    = x2
foo()              # it gives 2
```

TYPE-ANNOTATING A GLOBAL VARIABLE

The second approach to address type instability involves declaring *concrete* types for global variables. This is done by appending the operator `::` to the variable name, as in `x::Int64`. When working with vectors, ensure that the element type declared is concrete. Otherwise, the variable will remain type-unstable despite the annotation.

INT64

```
x3::Int64           = 5
foo()               = 2 * x3

@code_warntype foo()     # type stable
```

VECTOR{FLOAT64}

```
x4::Vector{Float64} = [1, 2, 3]
foo()               = sum(x4)

@code_warntype foo()     # type stable
```

VECTOR{NUMBER}

```
x5::Vector{Number}  = [1, 2, 3]
foo()               = sum(x5)

@code_warntype foo()     # type UNSTABLE
```

DIFFERENCES BETWEEN APPROACHES

The two approaches presented for handling global variables have different implications for both code behavior and performance. The key lies in that **type-annotations assert a variable's type, while constants fix both their types and values**. Next, we analyze the main differences between both approaches.

DIFFERENCES IN CODE

Unlike the case of constants, type-annotations allow you to reassign a global variable without unexpected consequences. This means you don't need to re-run the entire script when redefining a variable.

REDEFINING A GLOBAL VARIABLE

```
x6::Int64 = 5
foo()     = 2 * x6
foo()               # output is 10

x6        = 2
foo()     = 2 * x6
foo()               # output is 4
```

DIFFERENCES IN PERFORMANCE

Type-annotated global variables are more flexible than constants: they require only a declaration of types, without binding to a specific value. This flexibility, however, comes at a performance cost. The reason is that constants not only convey type information, but also act as a promise of immutability throughout the program. As a result, constants behave like literal values, embedded directly in the code. With this guarantee in place, the compiler can apply stronger optimizations, such as replacing certain expressions with their precomputed results.

The following code demonstrates this behavior. It performs an operation that can be precomputed if the value of the global variable is known at compile time. To make the performance difference more evident, we call this operation repeatedly inside a for-loop.

CONSTANT VARIABLE

```
const k1  = 2

function foo()
    for _ in 1:100_000
       2^k1
    end
end
```

```
julia> @btime foo()
  0.791 ns (0 allocations: 0 bytes)
```

TYPE-ANNOTATED VARIABLE

```
k2::Int64 = 2

function foo()
    for _ in 1:100_000
       2^k2
    end
end
```

```
julia> @btime foo()
  104.374 μs (0 allocations: 0 bytes)
```

Invariance of Operations

Even without declaring variables as constants, the compiler could still recognize the invariance of some operations across repeated calculations. In such cases, the compiler would compute the operation once and reuse the result whenever needed.

To illustrate this behavior, consider reexpressing each element of `x` as a proportion relative to the sum of all elements. A naive implementation would involve a for-loop with `sum(x)` inside the for-loop body, causing `sum(x)` to be recomputed on every iteration. By contrast, when shares are computed through `x ./ sum(x)`, the compiler is smart enough to recognize the invariance of `sum(x)` across iterations. Therefore, it proceeds to its pre-computation, eliminating redundant work.

FOR-LOOP

```
x           = rand(100_000)


function foo(x)
    y    = similar(x)

    for i in eachindex(x,y)
        y[i] = x[i] / sum(x)
    end

    return y
end
```

```
julia> @btime foo($x)
  830.068 ms (3 allocations: 781.312 KiB)
```

BROADCAST

```
x           = rand(100_000)


foo(x) = x ./ sum(x)
```

```
julia> @btime foo($x)
  49.166 µs (3 allocations: 781.312 KiB)
```

FOOTNOTES

[1] A literal value is one written directly in the code (e.g., `1`, `"hello"`, or `true`), rather than provided as a value of a variable.

8e. Barrier Functions

Martin Alfaro
PhD in Economics

INTRODUCTION

This section presents an approach to mitigating type instability through the use of barrier functions. A **barrier function** is a type-stable function that's called from within a type-unstable function, with the variables of uncertain type passed explicitly as arguments. Introducing barrier functions prompts the compiler to infer concrete types for those variables, effectively creating a "barrier" that prevents the propagation of type instability to subsequent operations.

A key benefit of this approach is that **barrier functions are agnostic to the underlying source of type instability**, making them widely applicable across scenarios.

> ⚠ **Warning!** - Barrier Functions Should Be Considered as a Second Option
>
> Barrier functions are preferred for situations where type instability is either difficult to fix or directly unavoidable. Keep in mind that the original function will remain type unstable, entailing different consequences depending on the instability nature. For this reason, it's best to aim for type-stable code from the outset whenever possible.

APPLYING BARRIER FUNCTIONS

To illustrate the technique, let's revisit a type-unstable function from a previous section. This defines a variable `y` based on `x`, subsequently performing an operation involving `y`.

TYPE INSTABILITY

```
function foo(x)
    y = (x < 0) ?  0  :  x

    [y * i for i in 1:100]
end

@code_warntype foo(1)       # type stable
@code_warntype foo(1.0)     # type UNSTABLE
```

In the example, `0` is an `Int64`, whereas `x` could be either an `Int64` or `Float64`. This leads to two possibilities:

- If `x` is an `Int64`, then `y` will also be an `Int64`, making `foo(1)` type stable.
- If `x` is a `Float64`, the compiler then can't determine whether `y` will be an `Int64` or a `Float64`, rendering `foo(1.0)` type unstable.

A barrier function can address the type instability of the second case. It requires embedding a type-stable function into `foo`, passing `y` as an argument. The function will then attempt to deduce `y`'s type, allowing the compiler to use this information for subsequent operations. The example below defines `operation` as a barrier function. [1]

BARRIER FUNCTION

```
operation(y) = [y * i for i in 1:100]

function foo(x)
    y = (x < 0) ?  0  :  x

    operation(y)
end

@code_warntype operation(1)    # barrier function is type stable
@code_warntype operation(1.0)  # barrier function is type stable

@code_warntype foo(1)          # type stable
@code_warntype foo(1.0)        # barrier-function solution
```

With the introduction of `operation`, the variable `y` in `foo(1.0)` can still be either an `Int64` or a `Float64`. Nevertheless, this ambiguity no longer matters, as `operation(y)` will determine the type of `y` before the array comprehension is executed. As a result, the expression `[y * i for i in 1:100]` will be computed with a method specialized for the specific type of `y`, ensuring type stability.

> ⚠ **Warning!**
>
> Barrier Functions should address the type instability *before* the type-unstable operation is executed. Otherwise, we're back to the original issue, where the compiler has to check `y`'s type at each iteration and select a method accordingly.
>
> For example, `foo` in the example below doesn't apply the technique correctly: `y` can be either `Float64` or `Int64`, but `operation(y,i)` only identifies the type inside the for-loop. Thus, the compiler is forced to check `y`'s type at each iteration, which is the original problem we intended to solve.

WRONG USE

```
operation(y,i) = y * i

function foo(x)
    y = (x < 0) ?  0  :  x

    [operation(y,i) for i in 1:100]
end

@code_warntype foo(1)          # type stable
@code_warntype foo(1.0)        # type UNSTABLE
```

REMARKS ON @CODE_WARNTYPE

Functions that introduce barrier functions are capable of addressing type instability. However, this effect isn't necessarily reflected when `@code_warntype` is executed. The reason is that barrier functions typically mitigate type instability, rather than completely eliminating it. When this is the case, nonetheless, the impact of the remaining type instability may be negligible. And even if the barrier function successfully eliminates the type instability, a red warning may still be triggered.

To illustrate this, let's start presenting a scenario where the barrier function completely eliminates the type instability, yet a red warning shows up.

TYPE UNSTABLE

```
x = ["a", 1]                    # variable with type 'Any'



function foo(x)
    y = x[2]

    [y * i for i in 1:100]
end
```

```
julia> @code_warntype foo(x)
```

BARRIER FUNCTION

```
x = ["a", 1]                    # variable with type 'Any'

operation(y) = [y * i for i in 1:100]

function foo(x)
    y = x[2]

    operation(y)
end
```

```
julia> @code_warntype foo(x)
```

In this example, `y` is defined from an object with type `Vector{Any}`. This leads to a red warning, as `x[2]` has type `Any` and therefore the compiler can't infer a concrete type for `y`. However, no operation is involved at that point, as we're only performing an assignment. Since the only operation performed uses a barrier function, the lack of type information is inconsequential. Overall, type instability is never impacting performance after introducing a barrier function.

In contrast, the example below demonstrates that a barrier function may only alleviate type instability, rather than eliminate it entirely. In this scenario, the operation `2 * x[2]` is type unstable, forcing the compiler to generate code for each possible concrete type of `x[2]`. Nonetheless, this operation has a negligible performance impact on `foo`, justifying why the barrier function only targets the more demanding operation.

TYPE UNSTABLE

```
x = ["a", 1]                    # variable with type 'Any'



function foo(x)
    y = 2 * x[2]

    [y * i for i in 1:100]
end
```

```
julia> @code_warntype foo(x)
```

BARRIER FUNCTION

```
x = ["a", 1]                    # variable with type 'Any'

operation(y) = [y * i for i in 1:100]

function foo(x)
    y = 2 * x[2]

    operation(y)
end
```

```
julia> @code_warntype foo(x)
```

TYPE STABLE

```
x = ["a", 1]                    # variable with type 'Any'

operation(y) = [y * i for i in 1:100]

function foo(z)
    y = 2 * z

    operation(y)
end
```

```
julia> @code_warntype foo(x[2])
```

The effectiveness of a barrier function ultimately hinges on how the function `foo` will be applied. In the given example, the barrier-function solution would be sufficient if `foo` is called only once. Instead, if `foo` is eventually called in a tight for-loop, the type instability of `2 * x[2]` would be incurred multiple times. In such cases, simultaneously addressing the type instability in `2 * x[2]` could entail substantial performance benefits.

FOOTNOTES

[1] In this particular example, there's an easier solution, where `0` is substituted with `zero(x)`. The function `zero(x)` has been designed to return the additive identity (i.e., the null element) of `x`'s type.

8f. Type Stability with Tuples

Martin Alfaro
PhD in Economics

INTRODUCTION

A function is type stable when, given its arguments' types, the compiler can deterministically infer a single concrete type for each expression involved. This definition, while universal, takes on different forms when applied to specific objects. So far, we've exclusively dealt with scalars and vectors, whose conditions for type stability are relatively straightforward.

In this section, we expand our analysis of type stability to other data structures. In particular, we consider **tuples**, whose coverage automatically encompasses **named tuples**. Guaranteeing type stability with tuples is more nuanced compared to vectors, as their type characterization demands more information. In fact, their exploration will challenge our understanding of type stability, demanding a clear grasp of their definition and subtleties.

> ⚠️ **Warning!** - Tuples Are Only Suitable For Small Collections
> Remember that tuples should be reserved for collections that **comprise a few elements**. Using them for large collections will result in significant performance degradation or directly trigger fatal errors.

COMPARING TUPLES AND VECTORS

Tuples and vectors are the most common forms of collections in Julia. While both seem to fulfill a similar purpose at the surface, they actually differ significantly in design philosophy and performance characteristics.

In particular, tuples are engineered for objects comprising a small fixed number of elements. When this is the case, tuples outperform vectors by avoiding the memory-allocation overhead incurred by vectors. This advantage will be **explained in more depth later**.

Another key distinction is that tuples possess a more intricate type system relative to vectors. To see this, let's compare the information needed to describe each type.

Vectors represent collections of elements sharing a *homogeneous* type and exhibiting a variable size. Thus, the information needed to describe the types of vectors is relatively minor: simply specifying the element type is sufficient. For instance, a type like `Vector{Float64}` establishes that *all* elements must have type `Float64`, without any restriction on the number of elements to be contained.

For their part, tuples are fixed-size collections that can accommodate *heterogeneous* types. This makes the characterization of its type more demanding, requiring both the number of elements and the type of *each* element. For instance, the variable `tup = ("hello", 1)` has type `Tuple{String, Int64}`, indicating that the first element has type `String` and the second one `Int64`. Furthermore, it implicitly sets the number of elements to two, as there's no possibility of appending or removing elements.

The fact that the element count is intrinsic to the type becomes evident when tuples contain `N` elements of the same type `T`. In such cases, Julia provides the convenient alias `Ntuple{N,T}`. This is just syntactic sugar for `Tuple{T, T,...,T}`, where `T` appears `N` times. [1]

In the following, we show that the choice between tuples and vectors may have different implications for type stability.

TUPLE SLICES WITH MIXED TYPES CAN STILL BE TYPE STABLE

As we indicated, a fundamental distinction between tuples and vectors in Julia lies in how they manage type information. While tuples explicitly define the type of each individual element, vectors require all elements to be of a uniform type.

Because vectors must maintain a consistent type, attempting to store mixed concrete types within a single vector compels Julia to determine a common type that accommodates them all. For example, if you create a vector containing both `Int64` and `Float64`, Julia will infer the type of the vector as `Vector{Float64}`, as it's the most general type encompassing both integers and floats.

However, this automatic widening can lead to significant performance penalties when dealing with highly diverse types. In extreme cases, Julia might resort to using the abstract type `Any`, resulting in a `Vector{Any}`. Working with such vectors is extremely undesirable from a performance standpoint.

This issue also affects vector slices, as they inherit the type information from their parent vector. Thus, if the parent vector has been widened to a more general type like `Vector{Any}`, operations performed on those slices will also be subject to that same type instability. This behavior contrasts sharply with **slices of tuples, where each element within the slice retains its concrete type.**

TUPLE

```
tup    = (1, 2, "hello")        # type is `Tuple{Int64, Int64, String}`

foo(x) = sum(x[1:2])

@code_warntype foo(tup)         # type stable (output is `Int64`)
```

VECTOR

```
vector = [1, 2, "hello"]        # type is `Vector{Any}`

foo(x) = sum(x[1:2])

@code_warntype foo(vector)      # type UNSTABLE
```

TUPLES CONTAIN MORE INFORMATION THAN VECTORS

Given the differences in type information, conversions between tuples and vectors can pose several challenges for type stability.

To see this, let's start with the simplest conversion, where a tuple is transformed into a vector. As our previous analysis suggests, type stability is preserved with this conversion only if all elements within the tuple share the same type or can be promoted to a common concrete type.

To illustrate, recall that each type automatically defines a constructor. This is a specialized function that transforms variables into a corresponding type. For instance, `Vector` can be used as a function that converts variables to the type `Vector`. When you invoke this constructor with a tuple argument, Julia must determine a single element type, capable of holding all items from that tuple. If the tuple contains heterogeneous types that can't be promoted (e.g., mixing strings and integers), the constructor fails to produce a concrete type during the conversion.

TYPE-HOMOGENEOUS TUPLES

```
tup = (1, 2, 3)                 # `Tuple{Int64, Int64, Int64}` or just `NTuple{3, Int64}`


function foo(tup)
    x = Vector(tup)             # 'x' has type `Vector{Int64}`
    sum(x)
end

@code_warntype foo(tup)         # type stable
```

TYPE PROMOTION

```
tup = (1, 2, 3.5)               # `Tuple{Int64, Int64, Float64}`


function foo(tup)
    x = Vector(tup)             # 'x' has type `Vector{Float64}`
    sum(x)
end

@code_warntype foo(tup)         # type stable
```

TYPE-HETEROGENEOUS TUPLES

```
tup = (1, 2, "hello")           # `Tuple{Int64, Int64, String}`


function foo(tup)
    x = Vector(tup)             # 'x' has type `Vector{Any}`
    sum(x)
end

@code_warntype foo(tup)         # type UNSTABLE
```

On the contrary, **constructing a tuple from a vector may cause type instability, even when the vector's type element is concrete**. The core issue is that vectors don't encode their length in their type. As a result, when a vector builds a tuple within a function, the compiler must consider every possible vector length as a distinct tuple type. This forces the generation of code that can handle an open-ended set of type possibilities, thus preventing specialization.

To pass the number of elements, the information must be hard-coded within the function. In other words, you must specify the information as a literal value, rather than as the output of a calculation. In fact, simply passing the count as a function argument won't address the problem, since functions only identify types, not values.

To illustrate, we begin with the case where the vector comprises elements holding an abstract type. In this case, we can't achieve type stability, regardless of how the tuple's length is handled.

TYPE-ELEMENT ABSTRACT

```
x   = [1, 2, "hello"]     # 'Vector{Any}' has no info on each individual type or number of
elements

function foo(x)
     tup    = Tuple(x)        # Tuple with type elements `Any` and undefined number of
arguments


    sum(tup)
end

@code_warntype foo(x)     # type UNSTABLE
```

TYPE-ELEMENT ABSTRACT

```
x   = [1, 2, "hello"]     # 'Vector{Any}' has no info on each individual type or number of
elements

function foo(x, limit)
     tup    = Tuple(x)        # Tuple with type elements `Any` and undefined number of
arguments
    slice = tup[1:limit]  # compiler won't identify number of elements through `limit`

    sum(slice)
end

@code_warntype foo(x,2)   # type UNSTABLE
```

TYPE-ELEMENT ABSTRACT

```
x    = [1, 2, "hello"]      # 'Vector{Any}' has no info on each individual type or number of
elements

function foo(x)
      tup    = Tuple(x)         # Tuple with type elements `Any` and undefined number of
arguments
     slice = tup[1:2]       # `Tuple{Any,Any}`, compiler smart enough to identify number of
elements

    sum(slice)
end

@code_warntype foo(x)     # type UNSTABLE (still operating with `Any`)
```

Instead, when the vector comprises elements holding a concrete type, we can achieve type stability if the number of elements is hard-coded in the function.

TYPE-ELEMENT CONCRETE

```
x    = [1 ,2 ,3]            # 'Vector{Int64}' identifies `Int64`, but no info on the number
of elements

function foo(x)
     tup    = Tuple(x)        # type `Tuple{Vararg{Int64}}` (`Vararg` is "variable number of
arguments")


    sum(tup)
end

@code_warntype foo(x)      # type UNSTABLE
```

TYPE-ELEMENT CONCRETE

```
x    = [1 ,2 ,3]            # 'Vector{Int64}' identifies `Int64`, but no info on the number
of elements

function foo(x,limit)
     tup    = Tuple(x)        # type `Tuple{Vararg{Int64}}` (`Vararg` is "variable number of
arguments")
    slice = tup[1:limit]   # compiler won't identify the number of elements through `limit`

    sum(tup)
end

@code_warntype foo(x,2)    # type UNSTABLE
```

TYPE-ELEMENT CONCRETE

```
x   = [1 ,2 ,3]             # 'Vector{Int64}' identifies `Int64`, but no info on the number
of elements

function foo(x)
    tup   = Tuple(x)        # type `Tuple{Vararg(Int64)}` (`Vararg` is "variable number of
arguments")
    slice = tup[1:2]       # compiler smart enough to identify number of elements

    sum(slice)
end

@code_warntype foo(x)      # type STABLE (the sum)
```

ADDRESSING VARIABLE ARGUMENTS

We indicated that, unless we identify the number of arguments, defining tuples from vectors will inevitably introduce type instability. The most straightforward and effective solution is to form the tuple prior to calling the function, and then pass it as a function argument. Given type inference, this ensures that the compiler identifies all the necessary information for handling tuples. This is demonstrated below.

TUPLE AS A FUNCTION ARGUMENT

```
x   = [1, 2, 3]
tup = Tuple(x)

foo(tup) = sum(tup[1:2])

@code_warntype foo(tup)         # type stable
```

The approach presented should be your first option when transforming vectors to tuples. Nonetheless, there may be scenarios where defining the tuple inside the function is unavoidable. In such cases, there are a few alternatives.

One solution is to define the tuple's length using a literal value. Below, we illustrate this approach. We also show that simply passing the vector's number of elements as a function argument doesn't solve the issue. The reason is that the compiler generates method instances based on information about types, not values. Consequently, a function argument like `length(x)` merely informs the compiler that the number of elements can be described as an object with type `Int64`, without providing any additional insight.

NOT A SOLUTION

```
x   = [1, 2, 3]

function foo(x)
    tup = NTuple{length(x), eltype(x)}(x)

    sum(tup)
end

@code_warntype foo(x)       # type UNSTABLE
```

INFLEXIBLE SOLUTION

```
x   = [1, 2, 3]

function foo(x)
    tup = NTuple{3, eltype(x)}(x)

    sum(tup)
end

@code_warntype foo(tup)     # type stable
```

The downside of this solution is that it defeats the purpose of having generic code: it sacrifices the generality of a function, restricting it to accept only tuples of a single predetermined size. To eliminate the type instability without constraining functionality, we need to introduce a more advanced solution. This is based on a technique known as **dispatch on value**. Because this approach is more complex to implement, *I recommend using it only when passing the tuple as a function argument is unfeasible*.

Next, we begin by laying out the general principles of dispatch on value. Only then we apply the technique to the specific case of tuples.

DISPATCHING ON VALUES

Dispatching on values enables passing information about specific values to the compiler. Since the compiler gathers information about types but not values, implementing this feature requires a workaround: introducing a generic type where the value itself becomes a type parameter. For example, in the case of tuples, this type parameter is simply the vector's number of elements.

The functionality is implemented via the built-in type `Val`, whose type parameter represents the value to be conveyed. Its use is best explained through an example. Suppose a function `foo` that behaves differently depending on a value `a`. The technique requires defining `foo` with a type-annotated argument `::Val{a}`. Technically, this acts as a **constructor**. Since the value of this argument is irrelevant, it's common to leave this argument unnamed and write it directly as `::Val{a}`. Then, when calling the function `foo`, we pass `Val(a)`, where `a` is the value we want the compiler to know.

To illustrate the use of `Val` beyond tuples, suppose that a variable `y` could be an `Int64` or `Float64` contingent upon a condition. Since the compiler can't predict the result of the condition, it can't know which branch will be executed. Consequently, any operation depending on `y` will be type unstable. The strategy is to lift the condition into the type domain using `Val{condition}`. In this way, we create separate methods for `Val{true}` and `Val{false}`, enabling the compiler to specialize each branch. The code snippet below demonstrates this approach in practice.

TYPE UNSTABLE

```
function foo(condition)
    y = condition ? 1 : 0.5      # either `Int64` or `Float64`

    [y * i for i in 1:100]
end

@code_warntype foo(true)         # type UNSTABLE
@code_warntype foo(false)        # type UNSTABLE
```

SOLUTION VIA "VAL"

```
function foo(::Val{condition}) where condition
    y = condition ? 1 : 0.5      # either `Int64` or `Float64`

    [y * i for i in 1:100]
end

@code_warntype foo(Val(true))    # type stable
@code_warntype foo(Val(false))   # type stable
```

> ⚠ **Warning!**
>
> The function argument `Val` must be defined with `{}`, as types define their parameters with `{}`. Instead, `Val` must be called with `()`, as with any other constructor (recall that constructors are ultimately functions).

DISPATCHING ON VALUE WITH TUPLES

Let's now revisit the conversion of vectors to tuples. As we previously discussed, type instability in those cases arises because vectors don't encode their size into their type, leaving the compiler without sufficient information to determine the tuple's type.

Dispatch on value provides a solution to this issue: by passing the vector's length as a type parameter, the function call becomes type stable.

TYPE UNSTABLE

```
x = [1, 2, 3]

function foo(x, N)
    tuple_x = NTuple{N, eltype(x)}(x)

    2 .+ tuple_x
end

@code_warntype foo(x, length(x))        # type UNSTABLE
```

SOLUTION VIA "VAL"

```
x = [1, 2, 3]

function foo(x, ::Val{N}) where N
    tuple_x = NTuple{N, eltype(x)}(x)

    2 .+ tuple_x
end

@code_warntype foo(x, Val(length(x)))   # type stable
```

FOOTNOTES

[1] Don't confuse `NTuple` with an abbreviation for the type `NamedTuple`. The "N" in the former case refers to the number of elements.

8g. Type Stability with Higher-Order Functions

Martin Alfaro
PhD in Economics

INTRODUCTION

Functions in Julia are **first-class objects**, a concept also referred to as **first-class citizens**. This means that functions can be handled just like any other variable: we can define vectors of functions, have functions whose outputs are other functions, and compose them in flexible ways. In particular, the property makes it possible to define **higher-order functions**, which are functions that take another function as an argument. We've already worked with several of them, often in the form of anonymous functions passed as function arguments. A familiar example is `map(<function>, <collection>)`, which applies `<function>` to every element of `<collection>`.

Our goal in this section is to understand when higher-order functions are type-stable. As we'll discover, these functions present some challenges in this regard.

> **Remark**
> Throughout the explanations, we'll often refer to the function passed as an argument as the *callback function*.

THE ISSUE

In Julia, **each function has its own unique concrete type**. Moreover, each of these concrete types is a subtype of an abstract type called `Function`, which serves as an umbrella for all possible functions defined in Julia. This design creates challenges to specialize the computation method of higher-order functions. In particular, if Julia were to generate a distinct specialized method for every possible callback function, the number of methods could grow explosively.

To address this issue, Julia adopts a conservative strategy, **often choosing not to specialize the methods of higher-order functions**. When specialization is skipped, the performance can drop sharply, sometimes resembling the overhead of executing code in global scope.

It's therefore important to recognize when specialization is inhibited and to monitor its consequences. If you notice that performance is severely impaired, there are still ways to enforce specialization. In this section, we'll explore these strategies.

AN EXAMPLE OF NO SPECIALIZATION

To understand when higher-order functions fail to specialize, consider a simple example: summing the transformed elements of a vector `x`. The only constraint we impose is that the transformation itself must remain generic, allowing us to supply different functions for the transformation.

Specifically, we define a function `foo` that broadcasts over `x` to transform its elements through some function `f`. To illustrate how `foo` works, we call it with the function `abs`, which provides absolute values.

NO SPECIALIZATION

```
x         = rand(100)

foo(f, x) = f.(x)
```

```
julia> @code_warntype foo(abs,x)
```

In the example, type instability arises because **specialization is avoided when a callback function isn't explicitly called inside the function body**. Specifically, the function `f` only enters `foo` as an argument of broadcasting, but there's no explicit line calling `f`. As a result, the compiler doesn't specialize `foo` for the specific function passed in.

Despite this, the macro `@code_warntype` **doesn't report any type-stability issues**. This happens because `@code_warntype` evaluates type stability *under the assumption that specialization was attempted*. In our case, that assumption simply doesn't hold, rendering `@code_warntype` of no use for diagnosing the problem.

To gather indirect evidence about the lack of specialization, we can compare the performance of the original `foo` function with a modified version that explicitly calls `f`.

NO SPECIALIZATION

```
x = rand(100)

function foo(f, x)

    f.(x)
end
```

```
julia> foo(abs, x)
100-element Vector{Float64}:
 0.9063
 0.443494
 ⋮
 0.121148
 0.20453
julia> @btime foo(abs, $x)
  817.886 ns (12 allocations: 1.250 KiB)
```

SPECIALIZATION

```
x = rand(100)

function foo(f, x)
    f(1)                # irrelevant computation to force specialization
    f.(x)
end
```

```
julia> foo(abs, x)
100-element Vector{Float64}:
 0.9063
 0.443494
 ⋮
 0.121148
 0.20453
julia> @btime foo(abs, $x)
  25.916 ns (2 allocations: 928 bytes)
```

The addition of `f(1)` forces the compiler to specialize the function, making the function call type stable. This is noticed through a significant decrease in execution time and memory allocations. Excessive allocations are often indicative of type instability.

FORCING SPECIALIZATION

> ⚠ **Warning!**
>
> Exercise caution when forcing specialization. Overly aggressive specialization can degrade performance severely, explaining why Julia's default approach is deliberately conservative. In particular, you should

> avoid specialization when your script repeatedly calls a higher-order function with many unique functions.[1]

Explicitly calling the callback function to force specialization isn't ideal, as it introduces an unnecessary computation. Fortunately, alternative solutions exist. One of them is to type-annotate `f`, thus providing Julia with a hint to specialize code for that function.

Another solution involves wrapping the function in a tuple, before passing it as an argument. This ensures the identification of the function's type, as tuples define a concrete type for each of their elements.

Below, we outline both approaches.

NO SPECIALIZATION

```
x     = rand(100)


function foo(f, x)
    f.(x)
end
```

```
julia> foo(abs, x)
100-element Vector{Float64}:
 0.9063
 0.443494
 ⋮
 0.121148
 0.20453
julia> @btime foo(abs, $x)
  807.353 ns (12 allocations: 1.250 KiB)
```

TYPE-ANNOTATED FUNCTION

```
x     = rand(100)


function foo(f::F, x) where F
    f.(x)
end
```

```
julia> foo(abs, x)
100-element Vector{Float64}:
 0.9063
 0.443494
 ⋮
 0.121148
 0.20453
julia> @btime foo(abs, $x)
  25.891 ns (2 allocations: 928 bytes)
```

FUNCTION IN TUPLE

```
x     = rand(100)
f_tup = (abs,)

function foo(f_tup, x)
    f_tup[1].(x)
end
```

```
julia> foo(f_tup, x)
100-element Vector{Float64}:
 0.9063
 0.443494
 ⋮
 0.121148
 0.20453
julia> @btime foo($f_tup, $x)
  26.825 ns (2 allocations: 928 bytes)
```

FOOTNOTES

[1] For discussions about the issue of excessive specialization, see here and here.

8h. Type-Stability Gotchas

Martin Alfaro
PhD in Economics

INTRODUCTION

This section presents subtle scenarios where type instabilities arise. Since the root cause of the type instability isn't immediately obvious, we refer to these cases as "gotchas", offering guidance on how to address them. To ensure self-containment, we revisit some examples previously discussed, but now providing recommendations for their mitigation.

GOTCHA 1: INTEGERS AND FLOATS

`Int64` and `Float64` are distinct types. Even though Julia promotes integers to floating-point numbers in many contexts, mixing them can still inadvertently introduce type instability.

To illustrate this, consider a function `foo` that takes a numeric variable `x` as its argument and performs two tasks: it first defines a variable `y` that replaces `x`'s negative values with zero, and then it executes an operation based on the resulting `y`.

In the following, we implement `foo` with an approach that suffers from type instability, and another one that addresses the issue.

UNSTABLE

```
function foo(x)
    y = (x < 0) ?  0  :  x

    return [y * i for i in 1:100]
end

@code_warntype foo(1)      # type stable
@code_warntype foo(1.0)    # type UNSTABLE
```

STABLE

```
function foo(x)
    y = (x < 0) ?  zero(x)  :  x

    return [y * i for i in 1:100]
end

@code_warntype foo(1)      # type stable
@code_warntype foo(1.0)    # type stable
```

The first implementation uses the literal `0`, whose type is `Int64`. If `x` is also `Int64`, no type instability arises. However, if `x` is `Float64`, the compiler treats `y` as potentially `Int64` or `Float64`, thus causing type instability. [1]

Note, though, that Julia can generally handle combinations of `Int64` and `Float64` quite effectively. Thus, this type instability wouldn't be a significant problem if the operation calls `y` only once. Indeed, `@code_warntype` in this case would simply issue a yellow warning, hinting at potential for optimization but not a severe performance bottleneck. However, `foo` in our example repeatedly performs an operation involving `y`, incurring the cost of type instability multiple times. As a result, `@code_warntype` issues a red warning, indicating a more serious performance issue.

The second tab proposes a **solution** for this scenario. It introduces a function that returns the zero element of `x`'s type, instead of `0`. In this way, `y` is created ensuring that types won't be mixed.

This approach to solving type instability can be extended to values different from zero, by use of the function `convert(typeof(x), <value>)` or `oftype(x, <value>)`. Both convert `<value>` to the same type as `x`. For instance, below we reimplement `foo` using the value `5` instead of `0`.

UNSTABLE

```
function foo(x)
    y = (x < 0) ?  5  :  x

    return [y * i for i in 1:100]
end

@code_warntype foo(1)      # type stable
@code_warntype foo(1.0)    # type UNSTABLE
```

STABLE

```
function foo(x)
    y = (x < 0) ?  convert(typeof(x), 5)  :  x

    return [y * i for i in 1:100]
end

@code_warntype foo(1)      # type stable
@code_warntype foo(1.0)    # type stable
```

STABLE

```
function foo(x)
    y = (x < 0) ?  oftype(x, 5)  :  x

    return [y * i for i in 1:100]
end

@code_warntype foo(1)      # type stable
@code_warntype foo(1.0)    # type stable
```

GOTCHA 2: COLLECTIONS OF COLLECTIONS

In data analysis, it's common to manipulate *collections of collections*, where one data structure nests others inside it. A well-known example in Julia is the `DataFrames` package, which organizes data into columns that represent multiple variables. In this case, each column is a collection itself, with the full set of columns acting as another collection. Since we haven't introduced this package, we'll consider a simpler analogous structure: a vector of vectors, whose type is `Vector{Vector}`.

The primary benefit of using `Vector{Vector}` is its inherent flexibility. Since it imposes no constraints on the element types stored in the inner vectors, each vector can hold whatever data you need: strings, integers, floating-point numbers, or even mixture of these. This makes it easy to represent heterogeneous datasets, without committing to a rigid schema.

That same flexibility, however, comes with a cost. The type system only knows that each element is some vector, with no information about the concrete element type inside each inner vector. Without that information, the compiler can't infer types when your code operates on these vectors, thus leading to type instability.

To see this more concretely, imagine a vector `data` whose elements are themselves vectors. Suppose we write a function `foo` that receives `data` and performs some computation on one of its inner vectors, `vec2`. As shown in the first tab, the compiler can determine that `vec2` is a vector, but it can't deduce the type of its elements. Consequently, calls to `foo` become type-unstable.

A simple and effective way to **address** this issue appears in the second tab. The solution is to introduce a barrier function that takes the inner vector `vec2` as its argument. The barrier function then rectifies the type instability by attempting to identify a concrete type for `vec2`.

UNSTABLE

```
vec1 = ["a", "b", "c"] ; vec2 = [1, 2, 3]
data = [vec1, vec2]

function foo(data)
    for i in eachindex(data[2])
        data[2][i] = 2 * i
    end
end

@code_warntype foo(data)            # type UNSTABLE
```

BARRIER FUNCTION

```
vec1 = ["a", "b", "c"] ; vec2 = [1, 2, 3]
data = [vec1, vec2]

foo(data) = operation!(data[2])

function operation!(x)
    for i in eachindex(x)
        x[i] = 2 * i
    end
end

@code_warntype foo(data)            # barrier-function solution
```

Note that the second tab defines the barrier function **in-place**. This means that the function directly updates the contents of the inner vector `vec2`, rather than creating a new copy. Because `vec2` is part of the larger structure data, the outer structure `data` is updated as well. This approach is typical in data-analysis workflows, where the objective is to transform an existing dataset, rather than allocate a fresh one every time a change is applied.

GOTCHA 3: BARRIER FUNCTIONS

Barrier functions are an effective technique to mitigate type instabilities. However, keep in mind that **the parent function may remain type unstable**. When this occurs and instability isn't resolved before executing a repeated operation, the associated performance penalty will be incurred multiple times.

To illustrate this point, let's revisit the last example involving a vector of vectors. Below, we present two incorrect approaches to using a barrier function, followed by a demonstration of its proper application.

BARRIER FUNCTION (WRONG)

```
vec1 = ["a", "b", "c"] ; vec2 = [1, 2, 3]
data = [vec1, vec2]

operation(i) = (2 * i)

function foo(data)
    for i in eachindex(data[2])
        data[2][i] = operation(i)
    end
end

@code_warntype foo(data)            # type UNSTABLE
```

BARRIER FUNCTION (WRONG)

```
vec1 = ["a", "b", "c"] ; vec2 = [1, 2, 3]
data = [vec1, vec2]

operation!(x,i) = (x[i] = 2 * i)

function foo(data)
    for i in eachindex(data[2])
        operation!(data[2], i)
    end
end

@code_warntype foo(data)            # type UNSTABLE
```

BARRIER FUNCTION (RIGHT)

```
vec1 = ["a", "b", "c"] ; vec2 = [1, 2, 3]
data = [vec1, vec2]

function operation!(x)
    for i in eachindex(x)
        x[i] = 2 * i
    end
end

foo(data) = operation!(data[2])

@code_warntype foo(data)            # barrier-function solution
```

GOTCHA 4: INFERENCE IS BY TYPE, NOT BY VALUE

Julia's compiler generates method instances solely on the basis of types, without taking actual values into account. To demonstrate this, consider the following example.

INFERENCE ISN'T BY VALUE

```
function foo(condition)
    y = condition ?  2.5  :  1

    return [y * i for i in 1:100]
end

@code_warntype foo(true)          # type UNSTABLE
@code_warntype foo(false)         # type UNSTABLE
```

At first glance, we might erroneously conclude that `foo(true)` is type stable: the value of `condition` is `true`, so that `y = 2.5` and therefore `y` will have type `Float64`. However, values don't participate in multiple dispatch, meaning that Julia's compiler ignores the value of `condition` when inferring `y`'s type. Ultimately, `y` is treated as potentially being either `Int64` or `Float64`, leading to type instability.

The issue in this case can be easily resolved by replacing `1` by `1.0`, thus ensuring that `y` is always `Float64`. More generally, we could employ similar techniques to the first "gotcha", where values are converted to a specific concrete type.

An alternative solution relies on dispatching by value, a technique we already explored and implemented for tuples. This technique makes it possible to communicate information about values directly to the compiler. It's based on the type `Val` in conjunction with the keyword `where` introduced here.

Specifically, for any function `foo` and value `a` that you seek the compiler to know, you need to include `::Val{a}` as an argument. In this way, `a` is interpreted as a type parameter, which can then be identified using the `where` keyword within the function definition. Finally, when calling `foo`, we need pass `Val(a)` as its input.

Applied to our example, type instability in `foo` is caused because the value of `condition` isn't known by the compiler. Dispatching by value provides a mechanism to explicitly convey this information and hence solve the type instability.

UNSTABLE

```
function foo(condition)
    y = condition ?  2.5  :  1

    return [y * i for i in 1:100]
end

@code_warntype foo(true)          # type UNSTABLE
@code_warntype foo(false)         # type UNSTABLE
```

STABLE (VAL APPROACH)

```
function foo(::Val{condition}) where condition
    y = condition ?  2.5  :  1

    return [y * i for i in 1:100]
end

@code_warntype foo(Val(true))    # type stable
@code_warntype foo(Val(false))   # type stable
```

GOTCHA 5: VARIABLES AS DEFAULT VALUES OF KEYWORD ARGUMENTS

Functions accept both positional and keyword arguments. The possibility of keyword arguments in particular allows the user to assign default values. If these default values are set through variables rather than literal values, a type instability will be introduced. The reason is that such variables will be treated as global variables.

NO DEFAULT VALUE

```
foo(; x) = x

β = 1
@code_warntype foo(x=β)          #type stable
```

LITERAL AS DEFAULT VALUE

```
foo(; x = 1) = x


@code_warntype foo()             #type stable
```

VARIABLE AS DEFAULT VALUE

```
foo(; x = β) = x

β = 1
@code_warntype foo()             #type UNSTABLE
```

In case you necessarily need to set a variable as a default value, there are still a few strategies you could follow to restore type stability.

One set of solutions leverages the techniques we introduced for global variables. hese include type-annotating the global variable (*Solution 1a*) or defining it as a constant (*Solution 1b*).

Another strategy involves defining a function that stores the default value. By doing so, you can take advantage of type inference, with the function attempting to infer a concrete type for the default value (*Solution 2*).

You can also solve the type instability by adopting a local approach, where type annotations are added to either the keyword argument (*Solution 3a*) or the default value itself (*Solution 3b*). Note that this isn't necessary when positional arguments are used as default values of keyword arguments (*Solution 4*).

All these scenarios are represented below.

SOLUTION 1A

```
foo(; x = β) = x

const β = 1
@code_warntype foo()            #type stable
```

SOLUTION 1B

```
foo(; x = β) = x

β::Int64 = 1
@code_warntype foo()            #type stable
```

SOLUTION 2

```
foo(; x = β()) = x

β() = 1
@code_warntype foo()            #type stable
```

SOLUTION 3A

```
foo(; x::Int64 = β) = x

β = 1
@code_warntype foo()            #type stable
```

SOLUTION 3B

```
foo(; x = β::Int64) = x

β = 1
@code_warntype foo()            #type stable
```

SOLUTION 4

```
foo(β; x = β) = x

β = 1
@code_warntype foo(β)           #type stable
```

GOTCHA 6: CLOSURES CAN EASILY INTRODUCE TYPE INSTABILITIES

Closures are a fundamental concept in programming. They refer to functions that capture and retain access to variables from the scope in which they were defined. In practical terms, a closure arises when **one function is defined inside another**, including the case where anonymous functions are used inside a function.

Although closures provide a convenient way to write modular and self-contained code, they can sometimes introduce type instabilities. While Julia has made progress in mitigating these issues, they have persisted for years and remain a source of potential inefficiency. For this reason, it's essential to understand not only the consequences of using closures carelessly, but also to learn strategies for addressing their performance challenges.

CLOSURES ARE COMMON IN CODING

There are several scenarios where closures emerge naturally. One such scenario is when a task requires multiple steps, but you prefer to keep a single self-contained unit of code. For instance, this approach is particularly useful if a function needs to perform multiple interdependent steps, such as data preparation (e.g., setting parameters or initializing variables) and subsequent computations based on that data. By nesting a function within another, you can keep related code organized and contained within the same logical block, promoting code readability and maintainability.

To illustrate how code implements a task with and without closures, we'll use generic code. This isn't intended to be executed, but rather to demonstrate the underlying structure.

CLOSURE

```
function task()
        # <here you define parameters and initialize variables>

    function output()
        # <here you do computations with the parameters and variables>
    end

    return output()
end

task()
```

NO CLOSURE

```
function task()
        # <here, you define parameters and initialize variables>

    return output(<variables>, <parameters>)
end

function output(<variables>, <parameters>)
        # <here, you do some computations with the variables and parameters>
end

task()
```

Although the approach using closures may seem more intuitive, it can easily introduce type instability. This occurs when one of these conditions hold:

- variables or arguments are redefined inside the function (e.g., when updating a variable)
- the order in which functions are defined is altered
- anonymous functions are introduced

Each of these cases is explored below, where we refer to the containing function as the *outer function* and the closure as the *inner function*.

WHEN THE ISSUE ARISES

Let's start examining three examples. They cover all the possible situations where closures could result in type instability.

The first examples reveal that the placement of the inner function could matter for type stability.

STABLE

```
function foo()
    x           = 1
    bar()       = x

    return bar()
end

@code_warntype foo()      # type stable
```

STABLE

```
function foo()
    bar(x)      = x
    x           = 1

    return bar(x)
end

@code_warntype foo()      # type stable
```

UNSTABLE

```
function foo()
    bar()        = x
    x            = 1

    return bar()
end

@code_warntype foo()      # type UNSTABLE
```

UNSTABLE

```
function foo()
    bar()::Int64 = x::Int64
    x::Int64     = 1

    return bar()
end

@code_warntype foo()      # type UNSTABLE
```

NO CLOSURE

```
function foo()
    x = 1

    return bar(x)
end

bar(x) = x

@code_warntype foo()      # type stable
```

The second example establishes that type instability arises when closures are combined with reassignments of variables or arguments. This issue even emerges when the reassignment involves the same object, including trivial expressions such as `x = x`. The example also reveals that type annotating the redefined variable or the closure doesn't resolve the problem.

STABLE

```
function foo()
    x            = 1
    x            = 1            # or 'x = x', or 'x = 2'

    return x
end

@code_warntype foo()            # type stable
```

STABLE

```
function foo()
    x            = 1
    x            = 1            # or 'x = x', or 'x = 2'
    bar(x)       = x

    return bar(x)
end

@code_warntype foo()            # type stable
```

UNSTABLE

```
function foo()
    x            = 1
    x            = 1            # or 'x = x', or 'x = 2'
    bar()        = x

    return bar()
end

@code_warntype foo()            # type UNSTABLE
```

UNSTABLE

```
function foo()
    x::Int64     = 1
    x            = 1
    bar()::Int64 = x::Int64

    return bar()
end

@code_warntype foo()            # type UNSTABLE
```

UNSTABLE

```
function foo()
    x::Int64     = 1
    bar()::Int64 = x::Int64
    x            = 1

    return bar()
end

@code_warntype foo()            # type UNSTABLE
```

UNSTABLE

```
function foo()
    bar()::Int64 = x::Int64
    x::Int64     = 1
    x            = 1

    return bar()
end

@code_warntype foo()            # type UNSTABLE
```

NO CLOSURE

```
function foo()
    x            = 1
    x            = 1            # or 'x = x', or 'x = 2'

    return bar(x)
end

bar(x) = x

@code_warntype foo()            # type stable
```

Finally, the last example deals with situations involving multiple closures. It highlights that the order in which you define them could matter for type stability. The third tab in particular demonstrates that passing closures as function arguments can sidestep the issue. However, such an approach is at odds with how code is generally written in Julia.

STABLE

```
function foo(x)
    closure1(x) = x
    closure2(x) = closure1(x)

    return closure2(x)
end

@code_warntype foo(1)            # type stable
```

UNSTABLE

```
function foo(x)
    closure2(x) = closure1(x)
    closure1(x) = x

    return closure2(x)
end

@code_warntype foo(1)            # type UNSTABLE
```

STABLE

```
function foo(x)
    closure2(x, closure1) = closure1(x)
    closure1(x)           = x

    return closure2(x, closure1)
end

@code_warntype foo(1)            # type stable
```

NO CLOSURE

```
function foo(x)
    closure2(x) = closure1(x)

    return closure2(x)
end

closure1(x) = x

@code_warntype foo(1)            # type stable
```

In the following, we'll examine specific scenarios where these patterns emerge. The examples reveal that the issue can occur more frequently than we might expect. For each scenario, we'll also provide a solution that enables the use of a closure approach. Nonetheless, if the function captures a performance critical part of your code, it's probably wise to avoid closures.

"BUT NO ONE WRITES CODE LIKE THAT"

i) Transforming Variables through Conditionals

UNSTABLE

```
x = [1,2]; β = 1

function foo(x, β)
    (β < 0) && (β = -β)          # transform 'β' to use its absolute value

    bar(x) = x * β

    return bar(x)
end

@code_warntype foo(x, β)         # type UNSTABLE
```

STABLE

```
x = [1,2]; β = 1

function foo(x, β)
    (β < 0) && (β = -β)         # transform 'β' to use its absolute value

    bar(x,β) = x * β

    return bar(x,β)
end

@code_warntype foo(x, β)        # type stable
```

STABLE

```
x = [1,2]; β = 1

function foo(x, β)
    δ = (β < 0) ? -β : β        # transform 'β' to use its absolute value

    bar(x) = x * δ

    return bar(x)
end

@code_warntype foo(x, β)        # type stable
```

STABLE

```
x = [1,2]; β = 1

function foo(x, β)
    β = abs(β)                    # 'δ = abs(β)' is preferable (you should avoid redefining
variables)

    bar(x) = x * δ

    return bar(x)
end

@code_warntype foo(x, β)        # type stable
```

Recall that the compiler doesn't dispatch by value, and so whether the condition holds is irrelevant. For instance, the type instability would still hold if we wrote `1 < 0` instead of `β < 0`. Moreover, the value used to redefine `β` is also unimportant, with the same conclusion holding if you write `β = β`.

ii) Anonymous Functions inside a Function

Using an anonymous function inside a function is another common form of closure. Considering this, type instability also arises in the example above if we replace the inner function `bar` for an anonymous function. To demonstrate this, we apply `filter` with an anonymous function that keeps

all the values in `x` that are greater than `β`.

UNSTABLE

```
x = [1,2]; β = 1

function foo(x, β)
    (β < 0) && (β = -β)          # transform 'β' to use its absolute value

    filter(x -> x > β, x)        # keep elements greater than 'β'
end

@code_warntype foo(x, β)         # type UNSTABLE
```

STABLE

```
x = [1,2]; β = 1

function foo(x, β)
    δ = (β < 0) ? -β : β         # define 'δ' as the absolute value of 'β'

    filter(x -> x > δ, x)        # keep elements greater than 'δ'
end

@code_warntype foo(x, β)         # type stable
```

STABLE

```
x = [1,2]; β = 1

function foo(x, β)
    β = abs(β)                     # 'δ = abs(β)' is preferable (you should avoid redefining
variables)

    filter(x -> x > β, x)        # keep elements greater than β
end

@code_warntype foo(x, β)         # type stable
```

iii) Variable Updates

UNSTABLE

```
function foo(x)
    β = 0                       # or 'β::Int64 = 0'
    for i in 1:10
        β = β + i               # equivalent to 'β += i'
    end

    bar() = x + β               # or 'bar(x) = x + β'

    return bar()
end

@code_warntype foo(1)           # type UNSTABLE
```

STABLE

```
function foo(x)
    β = 0
    for i in 1:10
        β = β + i
    end

    bar(x,β) = x + β

    return bar(x,β)
end

@code_warntype foo(1)           # type stable
```

NO DISPATCH BY VALUE

```
x = [1,2]; β = 1

function foo(x, β)
    (1 < 0) && (β = β)

    bar(x) = x * β

    return bar(x)
end

@code_warntype foo(x, β)         # type UNSTABLE
```

iv) The Order in Which you Define Functions Could Matter Inside a Function

To illustrate this claim, suppose you want to define a variable `x` that depends on a parameter `β`. However, `β` is measured in one unit (e.g., meters), while `x` requires `β` to be expressed in a different unit (e.g., centimeters). This implies that, before defining `x`, we must rescale `β` to the appropriate unit.

Depending on how we implement the operation, a type instability could emerge.

UNSTABLE

```
function foo(β)
    x(β)                  =  2 * rescale_parameter(β)
    rescale_parameter(β)  =  β / 10

    return x(β)
end

@code_warntype foo(1)      # type UNSTABLE
```

STABLE

```
function foo(β)
    rescale_parameter(β)  =  β / 10
    x(β)                  =  2 * rescale_parameter(β)

    return x(β)
end

@code_warntype foo(1)      # type stable
```

FOOTNOTES

[1] A similar problem would occur if we replaced `0` by `0.0` and `x` is an integer.

9a. Overview and Goals

Martin Alfaro
PhD in Economics

INTRODUCTION

In the previous chapter, we began our exploration of high performance in Julia by focusing on type stability. We now shift our attention to memory allocations, a critical aspect of performance optimization.

Memory allocations occur whenever a new object is created, involving the reservation of memory space to store its values. In particular, memory allocations on the heap, simply referred to as *memory allocations*, incur a notable cost due to the additional CPU instructions required for memory management.

From a practical standpoint, closely monitoring memory usage is essential when performance is critical. **Excessive allocation often signals inefficiency**: if two approaches exhibit large differences in memory allocation, their execution speeds are likely to differ significantly as well.

Despite this, the interplay between memory allocation and performance is more nuanced than it first appears. In fact, **reducing memory allocation is neither necessary nor sufficient for speeding up computations**. We'll present instances where the approach that allocates more memory is actually faster. This apparent paradox reflects a trade-off: while allocations introduce overhead, the resulting objects store their data in contiguous blocks of memory. Contiguity enables the CPU to access information more efficiently, sometimes offsetting the cost of allocation.

The roadmap of the chapter is as follows:

- We begin by explaining allocations and identifying allocation patterns. In particular, Section 9b defines memory allocations formally, distinguishing between allocations on the heap and the stack. With this definition, Section 9c lists which objects in Julia allocate and which don't. In particular, vectors are allocated on the heap, and so their repeated creation degrades performance. Because of this, the chapter is ultimately about using vectors appropriately. They're primarily necessary when working with large collections or objects that must be mutable or flexible in size.
- Section 9d starts the presentation of approaches to reducing memory allocations. It highlights the use of views for creating slices, instead of producing new copies.
- Section 9e introduces reductions, a technique that computes aggregate results (like summing a vector) by accumulating outputs into scalars (which don't allocate).
- Section 9f provides another technique to decrease allocations, based on lazy computations. They define the operations to be executed, deferring their computation until the results are required. This deferral allows Julia to fuse multiple steps together, eliminating temporary allocations. Section 9g extends this technique and focuses exclusively on its application to broadcasting.

- Section 9h discusses a technique that applies across programming languages, known as pre-allocation. This initializes vectors before execution begins, reusing them throughout computations. By favoring mutation over repeated object creation, pre-allocation minimizes heap allocations.
- Section 9i introduces a solution for operations involving small objects. It leverages tuples, which by design aren't allocated on the heap. Building on this idea, the package `StaticArrays` provides vectors that are implemented on top of tuples and therefore don't allocate.

9b. Stack vs Heap

Martin Alfaro
PhD in Economics

INTRODUCTION

Memory allocation occurs whenever a new object is created, requiring the system to set aside a portion of Random Access Memory (RAM) to store the object's data. For simple objects, the compiler may actually optimize away the allocation entirely and directly keep their data in CPU registers.

When RAM is required, the system manages it using two distinct logical models: the stack and the heap. These areas aren't physical locations, but rather management models that govern how memory is handled. Typically, the specific storage location is determined by the nature of the object. For instance, vectors in Julia are stored on the heap, whereas tuples can be allocated either on the stack or optimized into CPU registers.

The choice between heap and non-heap allocations has significant implications for performance. Allocating memory on the heap is comparatively an expensive operation. It requires a systematic search for an available block of memory, bookkeeping to track its status, and an eventual deallocation process to reclaim the space once it's no longer needed. [1] Stack, by contrast, is simpler and therefore faster.

The performance gap between stack and heap allocations can easily become a critical bottleneck when an operation is performed repeatedly. This disparity in performance explains a widespread convention in programming (including Julia), where **memory allocations exclusively refer to heap allocations**. In the following, we provide a brief overview of how each operates.

STACK ALLOCATIONS

The stack is typically reserved for storing data whose size and lifetime can be determined at compile time. This includes many primitive values (e.g., integers, floating-point numbers, and characters), as well as some fixed-size immutable aggregates like tuples. These objects have a known fixed layout, which allows the compiler to place them on the stack or even eliminate the allocation entirely by keeping their contents in CPU registers. Their fixed size and predictable lifetime make allocation and deallocation extremely efficient.

The primary downside of the stack is its limited capacity, which makes them suitable only for objects with a few elements. Indeed, attempting to allocate more memory than the stack can accommodate will result in a "stack overflow" error, causing program termination. And, even if an object fits on the stack, allocating too many elements will significantly degrade performance. [2]

HEAP ALLOCATIONS

Common objects stored on the heap include arrays—such as vectors and matrices—as well as strings. The heap is designed to support data structures whose size can't be determined at compile time, with the possibility of growing and shrinking dynamically. These objects can be arbitrarily large, up to the limits of available RAM, and often have lifetimes that extend beyond a single function call. As a result, they must be managed in a more flexible region of memory than the stack can provide.

While the heap offers greater flexibility, its more complex memory management comes at the cost of slower performance. [3] Due to its overhead, the upcoming sections of this book will discuss strategies for reducing heap allocations. These include computational techniques that translate vectorized operations into repeated scalar operations, as well as programming practices that favor mutating existing objects over the creation of new ones.

FOOTNOTES

[1.] This deallocation process is automated in Julia by what's known as a garbage collector.

[2.] There's no hard and fast rule about how many elements are "too many". Benchmarking is the only reliable way to determine the performance consequences for each particular case. As a rough guideline, objects with more than 100 elements will certainly suffer from poor performance, while those with fewer than 15 elements will benefit from stack allocation.

[3.] Heap memory is managed by what's known as the *garbage collector*, which automatically identifies and reclaims memory no longer in use. This process, while convenient, can be computationally expensive.

9c. Objects Allocating Memory

Martin Alfaro
PhD in Economics

INTRODUCTION

In the previous section, we outlined the basic ideas behind memory allocation, noting that objects may reside either on the stack (or even in CPU registers) or on the heap. We also adopted a common convention in programming language discussions: **memory allocations exclusively refer to those on the heap**. This convention isn't merely to economize on words. Rather, it highlights that heap allocations are the ones that meaningfully affect performance. They require more elaborate bookkeeping, can introduce latency, and often become a dominant source of overhead in performance-critical code.

Julia's own benchmarking tools reinforce this connection between performance and heap activity. Macros such as `@time` and `@btime` don't just measure execution time, but also report the number and size of heap allocations involved. This dual reporting encourages developers to think about performance not only in terms of speed, but also in terms of allocation patterns.

Given the importance of understanding when allocations occur, this section classifies objects according to whether they allocate on the heap or avoid allocation altogether. This distinction will guide our analysis of performance throughout the remainder of the chapter.

NUMBERS, TUPLES, NAMED TUPLES, AND RANGES DON'T ALLOCATE

We start by presenting objects that don't allocate memory. They include:

- Scalars (numbers)
- Tuples
- Named Tuples
- Ranges

As they don't allocate, neither does their creation, access, nor manipulation. This is demonstrated below.

NUMBERS

```
function foo()
    x = 1; y = 2; z = 3

    x + y + z
end
```

```
julia> @btime foo()
  1.822 ns (0 allocations: 0 bytes)
```

TUPLES

```
function foo()
    tup = (1,2,3)

    sum(tup[1:2]) + tup[3]
end
```

```
julia> @btime foo()
  1.822 ns (0 allocations: 0 bytes)
```

NAMED TUPLES

```
function foo()
    nt = (a=1, b=2, c=3)

    sum((nt.a, nt.b)) + nt.c
end
```

```
julia> @btime foo()
  1.822 ns (0 allocations: 0 bytes)
```

RANGES

```
function foo()
    rang = 1:3

    sum(rang[1:2]) + rang[3]
end
```

```
julia> @btime foo()
  1.821 ns (0 allocations: 0 bytes)
```

ARRAYS AND THEIR SLICES DO ALLOCATE MEMORY

Arrays are among the most common heap-allocated objects in Julia. A new allocation occurs not only when you explicitly construct an array and assign it to a variable, but also whenever an expression implicitly produces a fresh array as part of its computation. The examples below illustrate both

situations.

EXPLICIT VECTOR CREATION

```
foo() = [1,2,3]
```

```
julia> @btime foo()
  23.236 ns (2 allocations: 80 bytes)
```

IMPLICIT VECTOR CREATION

```
foo() = sum([1,2,3])
```

```
julia> @btime foo()
  16.569 ns (1 allocations: 48 bytes)
```

Slicing is another operation that results in memory allocation. By default, a slice produces a new array that copies the selected elements, instead of creating a lightweight view over the original data. This behavior ensures isolation between the slice and its source, but it also means that each slicing operation allocates fresh storage. The only exception occurs when a single element is accessed, in which case no allocations take place.

SLICE WITH RANGE AS INDEX

```
x      = [1,2,3]

foo(x) = x[1:2]                    # allocations only from 'x[1:2]' itself (ranges don't
allocate)
```

```
julia> @btime foo($x)
  25.229 ns (2 allocations: 80 bytes)
```

SLICE WITH VECTOR AS INDEX

```
x      = [1,2,3]

foo(x) = x[[1,2]]              # allocations from both '[1,2]' and 'x[[1,2]]' itself
```

```
julia> @btime foo($x)
  52.388 ns (4 allocations: 160 bytes)
```

SINGLE-ELEMENT SLICE (NO ALLOCATIONS)

```
x      = [1,2,3]

foo(x) = x[1] * x[2] + x[3]
```

```
julia> @btime foo($x)
  4.219 ns (0 allocations: 0 bytes)
```

Array comprehensions and broadcasting are two more constructs that result in fresh arrays. Notably, broadcasting also allocates memory for intermediate results computed on the fly, even when those values aren't explicitly returned. This behavior is demonstrated in the tab "Broadcasting 2" below.

ARRAY COMPREHENSION

```
foo()  = [a for a in 1:3]
```

```
julia> @btime foo()
  28.637 ns (2 allocations: 80 bytes)
```

BROADCASTING 1

```
x      = [1,2,3]
foo(x) = x .* x
```

```
julia> @btime foo($x)
  36.064 ns (2 allocations: 80 bytes)
```

BROADCASTING 2

```
x      = [1,2,3]
foo(x) = sum(x .* x)               # allocations from temporary vector 'x .* x'
```

```
julia> @btime foo($x)
  42.740 ns (2 allocations: 80 bytes)
```

9d. Slice Views

Martin Alfaro
PhD in Economics

INTRODUCTION

We previously defined a slice as a subvector derived from a parent vector `x`. Common examples include expressions such as `x[1:2]`, which extracts elements at positions 1 and 2, or `x[x .> 0]`, which selects those elements that are positive. By default, these operations create a copy of the data and therefore allocate memory, except when the slice comprises a single element.

In this section, we address the issue of memory allocations associated with slices. In particular, we highlight the role of **views**, which bypass the need for a copy by directly referencing the parent object. The strategy is effective when slices are indexed through ranges, although it's not for slices that employ Boolean indexing like `x[x .> 0]`, where memory allocation will still occur.

Interestingly, we'll show scenarios where **copying data could actually be faster than using views**, despite the additional memory allocation involved. This apparent paradox emerges because copied data is stored in a contiguous block of memory, which provides more efficient access patterns.

VIEWS OF SLICES

We begin by showing that views don't allocate memory *when a slice is indexed by a range*. This behavior can yield performance improvements over regular slices, which create a copy of the data by default.

SLICE AS A COPY

```
x      = [1, 2, 3]

foo(x) = sum(x[1:2])           # allocations from the slice 'x[1:2]'
```

```
julia> @btime foo($x)
  24.838 ns (2 allocations: 80 bytes)
```

SLICE AS A VIEW

```
x      = [1, 2, 3]

foo(x) = sum(@view(x[1:2]))    # it doesn't allocate
```

```
julia> @btime foo($x)
  2.924 ns (0 allocations: 0 bytes)
```

Keep in mind, though, that **views created through Boolean indexing neither reduce memory allocations nor are more performant**. This is why you shouldn't rely on views of these objects if your goal is to speed up computations. This fact is illustrated below.

BOOLEAN INDEX (COPY)

```
x      = rand(1_000)

foo(x) = sum(x[x .> 0.5])
```

```
julia> @btime foo($x)
  797.000 ns (6 allocations: 4.094 KiB)
```

BOOLEAN INDEX (VIEW)

```
x      = rand(1_000)

foo(x) = @views sum(x[x .> 0.5])
```

```
julia> @btime foo($x)
  1.070 μs (6 allocations: 4.094 KiB)
```

COPYING DATA MAY BE FASTER

Although views can reduce memory allocations, there are scenarios where copying data can result in faster performance. A detailed comparison of copies versus views will be provided in another section. Here, we simply remark on this possibility.

Essentially, the choice between copies and views reflects a fundamental trade-off between memory allocation and data access patterns. On one hand, newly created vectors store data in contiguous blocks of memory, enabling more efficient CPU access and allowing for certain optimizations. On the other hand, views avoid allocation, but may also require accessing data scattered across non-contiguous memory regions.

Below, we illustrate a scenario in which the overhead of creating a copy is outweighed by the benefits of contiguous memory access. This makes copying the more efficient choice.

COPY

```
x = rand(100_000)

function foo(x)
    y = x[1:2:length(x)]

    max.(y, 0.5)
end
```

```
julia> @btime foo($x)
  61.026 µs (6 allocations: 781.375 KiB)
```

VIEW

```
x = rand(100_000)

function foo(x)
    y = @view x[1:2:length(x)]

    max.(y, 0.5)
end
```

```
julia> @btime foo($x)
  364.969 µs (3 allocations: 390.688 KiB)
```

9e. Reductions

Martin Alfaro
PhD in Economics

INTRODUCTION

Reductions refer to **operations that take a collection as their input and return a single element as their output**. Such operations arise naturally in a wide range of contexts, with summary statistics as a typical example (e.g., averages, variances, or maxima of collections).

The technique involves iteratively applying an operation to pairs of elements, accumulating the result at each step until the final output is obtained. A classic example of reduction is the summation of all numeric elements in a vector. This arrives at the final result by applying the addition operator `+` to pairs of elements, iteratively updating the accumulated sum. The technique applied to this case is illustrated below.

OPERATION

```
x = rand(100)

foo(x) = sum(x)
```

```
julia> foo(x)
48.447
```

IMPLEMENTATION VIA REDUCTION

```
x = rand(100)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output = output + x[i]
    end

    return output
end
```

```
julia> foo(x)
48.447
```

IMPLEMENTATION VIA REDUCTION (EQUIVALENT)

```
x = rand(100)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo(x)
48.447
```

The last tab implements the reduction via an **update operator**. These operators are frequently used in reductions because they streamline notation, rewriting expressions like `x = x + a` in the more compact form `x += a`.

The main benefit of reductions is that, by operating on scalars, **they don't create memory allocations**. This is particularly convenient when a vector must be transformed prior to aggregating the result. For instance, if you need to compute `sum(log.(x))`, the procedure would avoid the allocations of the intermediate vector `log.(x)`.

IMPLEMENTING REDUCTIONS

Technically, reductions are implemented by iteratively applying a *binary operation* to pairs of values until a scalar result is produced. For the procedure to be valid, the binary operation must satisfy **two mathematical properties**:

- **Associativity**: the grouping of operations must not affect the outcome. For example, scalar addition is associative because $(a + b) + c = a + (b + c)$.
- **Existence of an identity element**: there must exist an element that, when combined with any other element through a binary operation, leaves that element unchanged. For example, the identity element of scalar addition is 0 because $a + 0 = a$.

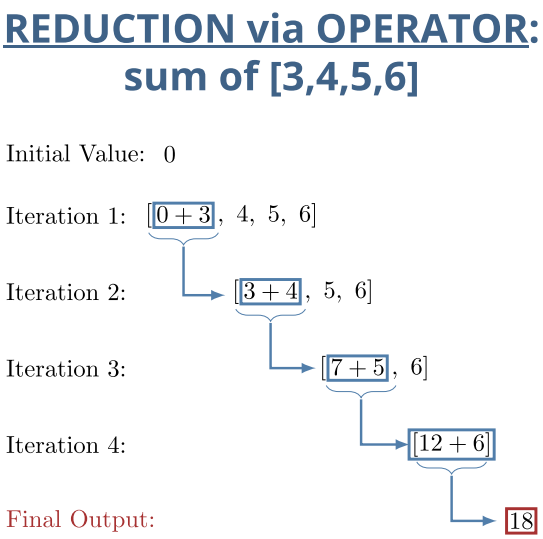
In practice, the identity element serves as the initial value of the accumulator variable. [1] Each operation has its own identity element, as shown in the table below.

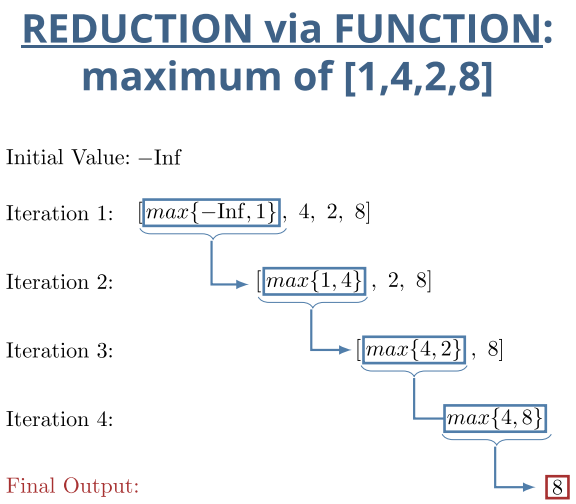
Operation	Identity Element
Sum	0
Product	1
Maximum	-Inf

Operation	Identity Element
Minimum	Inf

Remarkably, binary operations can be expressed either as **binary operators** or as two-argument functions. For instance, the symbol `+` can be employed in either form, making `output = output + x[i]` equivalent to `output = +(output, x[i])`. The possibility of using functions for reductions expands their scope. For instance, it enables the application of the `max` function to find the maximum value in a vector `x`, where `max(a,b)` returns the larger of the two scalars `a` and `b`.

The figures below visually illustrate how a reduction operates. They describe the process both when expressed with a binary operator and with a two-argument function.





To implement reductions manually, we typically employ for-loops. This iterates through the data and updates an accumulator step by step. To illustrate the formulation, we introduce the function `foo1`, which captures the desired outcome directly, and `foo2`, which reaches the same result through a reduction.

SUM

```
x       = rand(100)

foo1(x) = sum(x)

function foo2(x)
    output = 0.0

    for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo1(x)
48.447
julia> foo2(x)
48.447
```

PRODUCT

```
x       = rand(100)

foo1(x) = prod(x)

function foo2(x)
    output = 1.0

    for i in eachindex(x)
        output *= x[i]
    end

    return output
end
```

```
julia> foo1(x)
1.80982e-43
julia> foo2(x)
1.80982e-43
```

MAXIMUM

```
x       = rand(100)

foo1(x) = maximum(x)

function foo2(x)
    output = -Inf

    for i in eachindex(x)
        output = max(output, x[i])
    end

    return output
end
```

```
julia> foo1(x)
0.991405
julia> foo2(x)
0.991405
```

MINIMUM

```
x       = rand(100)

foo1(x) = minimum(x)

function foo2(x)
    output = Inf

    for i in eachindex(x)
        output = min(output, x[i])
    end

    return output
end
```

```
julia> foo1(x)
0.0303789
julia> foo2(x)
0.0303789
```

AVOIDING MEMORY ALLOCATIONS THROUGH REDUCTIONS

One of the key advantages of reductions is that they avoid unnecessary memory allocation, especially for intermediate results.

To illustrate, consider the operation `sum(log.(x))` for a vector `x`. Its computation involves two steps: transforming `x` by applying `log` element-wise, and summing the transformed elements. Since broadcasting computes its results **eagerly**, it implies the internal creation of a new vector to store the

values of `log.(x)`. Consequently, this step results in memory allocations.

In many cases like this, however, only the scalar output matters, with the intermediate result being of no interest. In this context, computational strategies that obtain the final output while bypassing the allocation of intermediate results like `log.(x)` are preferred.

Reductions make this possible by defining a scalar `output`, which is iteratively updated by summing the transformed values of `x`. In our example, this means that each element of `x` is transformed by the logarithm, and the result is then immediately added to the accumulator. In this way, the storage of the intermediate vector `log.(x)` is entirely avoided.

SUM

```
x       = rand(100)

foo1(x) = sum(log.(x))

function foo2(x)
    output = 0.0

    for i in eachindex(x)
        output += log(x[i])
    end

    return output
end
```

```
julia> @btime foo1($x)
  719.923 ns (2 allocations: 928 bytes)
julia> @btime foo2($x)
  706.610 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x       = rand(100)

foo1(x) = prod(log.(x))

function foo2(x)
    output = 1.0

    for i in eachindex(x)
        output *= log(x[i])
    end

    return output
end
```

```
julia> @btime foo1($x)
  753.135 ns (2 allocations: 928 bytes)
julia> @btime foo2($x)
  727.667 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x       = rand(100)

foo1(x) = maximum(log.(x))

function foo2(x)
    output = -Inf

    for i in eachindex(x)
        output = max(output, log(x[i]))
    end

    return output
end
```

```
julia> @btime foo1($x)
  1.218 μs (2 allocations: 928 bytes)
julia> @btime foo2($x)
  948.161 ns (0 allocations: 0 bytes)
```

MINIMUM

```
x       = rand(100)

foo1(x) = minimum(log.(x))

function foo2(x)
    output = Inf

    for i in eachindex(x)
        output = min(output, log(x[i]))
    end

    return output
end
```

```
julia> @btime foo1($x)
  1.295 μs (2 allocations: 928 bytes)
julia> @btime foo2($x)
  981.767 ns (0 allocations: 0 bytes)
```

REDUCTIONS VIA BUILT-IN FUNCTIONS

So far, we've manually implemented reductions with intermediate transformations through explicit for-loops. While this approach makes the underlying mechanics transparent, it also introduces considerable verbosity.

To avoid this inconvenience, Julia provides several streamlined alternatives for expressing reductions. One such alternative is through specialized methods of common reduction functions, including `sum`, `prod`, `maximum`, and `minimum`. These methods accept a transforming function as their first argument, with the collection to be reduced as a second argument. The general syntax is `foo(<transforming function>, x)`, where `foo` is the reduction function and `x` is the vector to be transformed and reduced.

The following examples illustrate this approach by applying a logarithmic transformation prior to the reduction.

SUM

```
x      = rand(100)

foo(x) = sum(log, x)        #same output as sum(log.(x))
```

```
julia> @btime foo($x)
  799.583 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x      = rand(100)

foo(x) = prod(log, x)       #same output as prod(log.(x))
```

```
julia> @btime foo($x)
  768.722 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x      = rand(100)

foo(x) = maximum(log, x)    #same output as maximum(log.(x))
```

```
julia> @btime foo($x)
  1.363 μs (0 allocations: 0 bytes)
```

MINIMUM

```
x      = rand(100)

foo(x) = minimum(log, x)    #same output as minimum(log.(x))
```

```
julia> @btime foo($x)
  1.375 μs (0 allocations: 0 bytes)
```

The specialized function methods are commonly applied using anonymous functions, as shown below.

SUM

```
x      = rand(100)

foo(x) = sum(a -> 2 * a, x)       #same output as sum(2 .* x)
```

```
julia> @btime foo($x)
  17.333 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x      = rand(100)

foo(x) = prod(a -> 2 * a, x)      #same output as prod(2 .* x)
```

```
julia> @btime foo($x)
  20.968 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x      = rand(100)

foo(x) = maximum(a -> 2 * a, x)   #same output as maximum(2 .* x)
```

```
julia> @btime foo($x)
  406.685 ns (0 allocations: 0 bytes)
```

MINIMUM

```
x      = rand(100)

foo(x) = minimum(a -> 2 * a, x)   #same output as minimum(2 .* x)
```

```
julia> @btime foo($x)
  415.786 ns (0 allocations: 0 bytes)
```

These specialized methods also accept transforming functions with multiple arguments. In this case, the arguments must be paired using `zip`, with indices corresponding to each argument within the transforming function. This is illustrated below, with the transforming operation `x .* y`.

SUM

```
x      = rand(100)
y      = rand(100)

foo(x,y) = sum(a -> a[1] * a[2], zip(x,y))        #same output as sum(x .* y)
```

```
julia> @btime foo($x)
  78.454 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x        = rand(100)
y        = rand(100)

foo(x,y) = prod(a -> a[1] * a[2], zip(x,y))        #same output as prod(x .* y)
```

```
julia> @btime foo($x)
  94.980 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x        = rand(100)
y        = rand(100)

foo(x,y) = maximum(a -> a[1] * a[2], zip(x,y))     #same output as maximum(x .* y)
```

```
julia> @btime foo($x)
  385.753 ns (0 allocations: 0 bytes)
```

MINIMUM

```
x        = rand(100)
y        = rand(100)

foo(x,y) = minimum(a -> a[1] * a[2], zip(x,y))     #same output as minimum(x .* y)
```

```
julia> @btime foo($x)
  380.757 ns (0 allocations: 0 bytes)
```

THE "REDUCE" AND "MAPREDUCE" FUNCTIONS

Beyond the specific cases discussed, reductions are applicable whenever a binary operation meets the necessary conditions for their implementation. To accommodate this generality, Julia provides the functions `reduce` and `mapreduce`. Both functions are convenient especially for packages that implement specialized versions of reductions. By simply defining these two functions, a package is capable of covering all possible reductions.

The `reduce` function applies a binary operation directly to the elements of a collection, combining them into a single result. By contrast, `mapreduce` transforms each element before applying the reduction.

It's worth remarking that reductions for sums, products, maximum, and minimum should still be implemented via their dedicated functions. This is because the specialized methods in `sum`, `prod`, `maximum`, and `minimum` have been carefully optimized for their respective tasks, typically outperforming `reduce` and `mapreduce`.

FUNCTION "REDUCE"

The function `reduce` uses the syntax `reduce(<function>,x)`, where `<function>` is a two-argument function.

SUM

```
x      = rand(100)

foo(x) = reduce(+, x)           #same output as sum(x)
```

```
julia> @btime foo($x)
  15.738 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x      = rand(100)

foo(x) = reduce(*, x)           #same output as prod(x)
```

```
julia> @btime foo($x)
  16.330 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x      = rand(100)

foo(x) = reduce(max, x)         #same output as maximum(x)
```

```
julia> @btime foo($x)
  379.038 ns (0 allocations: 0 bytes)
```

MINIMUM

```
x      = rand(100)

foo(x) = reduce(min, x)         #same output as minimum(x)
```

```
julia> @btime foo($x)
  361.919 ns (0 allocations: 0 bytes)
```

FUNCTION "MAPREDUCE"

The `mapreduce` function integrates `map` and `reduce` into a unified operation, explaining its name. Thus, it applies a transformation via the function `map` before doing the reduction. Its syntax is `mapreduce(<transformation>,<reduction>,x)`. To illustrate its use, we consider a `log` transformation.

SUM

```
x      = rand(100)

foo(x) = mapreduce(log, +, x)       #same output as sum(log.(x))
```

```
julia> @btime foo($x)
  700.262 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x      = rand(100)

foo(x) = mapreduce(log, *, x)       #same output as prod(log.(x))
```

```
julia> @btime foo($x)
  732.325 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x      = rand(100)

foo(x) = mapreduce(log, max, x)     #same output as maximum(log.(x))
```

```
julia> @btime foo($x)
  1.244 μs (0 allocations: 0 bytes)
```

MINIMUM

```
x      = rand(100)

foo(x) = mapreduce(log, min, x)     #same output as minimum(log.(x))
```

```
julia> @btime foo($x)
  1.256 μs (0 allocations: 0 bytes)
```

`mapreduce` can also be used with anonymous functions and functions with multiple arguments. Below, we illustrate these possibilities.

SUM

```
x        = rand(100)
y        = rand(100)

foo(x,y) = mapreduce(a -> a[1] * a[2], +, zip(x,y))       #same output as sum(x .* y)
```

```
julia> @btime foo($x)
  75.167 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x        = rand(100)
y        = rand(100)

foo(x,y) = mapreduce(a -> a[1] * a[2], *, zip(x,y))       #same output as prod(x .* y)
```

```
julia> @btime foo($x)
  89.958 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x        = rand(100)
y        = rand(100)

foo(x,y) = mapreduce(a -> a[1] * a[2], max, zip(x,y))     #same output as maximum(x .* y)
```

```
julia> @btime foo($x)
  370.026 ns (0 allocations: 0 bytes)
```

MINIMUM

```
x        = rand(100)
y        = rand(100)

foo(x,y) = mapreduce(a -> a[1] * a[2], min, zip(x,y))     #same output as minimum(x .* y)
```

```
julia> @btime foo($x)
  365.562 ns (0 allocations: 0 bytes)
```

REDUCE OR MAPREDUCE?

`mapreduce(<transformation>,<operator>,x)` yields the same result as `reduce(<operator>, map(<transformation>,x))`. Despite this, `mapreduce` should be employed if the vector input must be transformed beforehand. The reason is that `mapreduce` avoids the internal memory allocations of the transformed vector, while `map` doesn't. This aspect is demonstrated below, where `sum(2 .* x)` is computed through a reduction.

MAPREDUCE

```
x      = rand(100)

foo(x) = mapreduce(a -> 2 * a, +, x)
```

```
julia> @btime foo($x)
  15.887 ns (0 allocations: 0 bytes)
```

REDUCE

```
x      = rand(100)

foo(x) = reduce(+, map(a -> 2 * a, x))
```

```
julia> @btime foo($x)
  82.306 ns (2 allocations: 928 bytes)
```

FOOTNOTES

[1] Strictly speaking, an identity element isn't always required. In many cases, no natural identity exists, yet the first element of the collection can serve as the initial value. For simplicity, however, we assume the existence of an identity element as a requirement.

9f. Lazy Operations

Martin Alfaro
PhD in Economics

INTRODUCTION

Computational approaches can be broadly classified as either "lazy" or "eager". **Eager operations** are executed immediately upon definition, providing instant access to the results. For most programming tasks, this is the default behavior.

Lazy operations take a different approach. Instead of executing code immediately, they define what should be computed, deferring the actual execution until the results are actually needed. This mechanism execution offers significant advantages for operations involving heavy intermediate computations. By enabling operation fusion, intermediate calculations can be computed on the fly and fed directly into the final calculation, thereby **sidestepping unnecessary memory allocations**.

This section provides basic implementations of lazy computations. The first approach presented is based on the so-called **generators**, which are the lazy analogue of array comprehensions. After this, we'll introduce several functions from the built-in package **Iterators**. This provides lazy variants of various functions, including `map` and `filter`.

GENERATORS

Array comprehensions offer an expressive way to construct vectors, employing a syntax that mirrors for-loops. The elements defined by them are computed and stored right away, making array comprehensions an eager operation. **Generators** simply represent **the lazy counterpart of array comprehensions**, deferring the creation of elements until they're actually needed.

In terms of syntax, generators are identical to array comprehensions, with the sole difference that they're enclosed in parentheses `( )` instead of square brackets `[ ]`. Just like array comprehensions, generators also retain the ability to add conditions and simultaneously iterate over multiple collections.

ARRAY COMPREHENSIONS

```
x = [a for a in 1:10]

y = [a for a in 1:10 if a > 5]
```

```
julia> x
10-element Vector{Int64}:
  1
  2
  ⋮
  9
 10
julia> y
5-element Vector{Int64}:
  6
  7
  8
  9
 10
```

GENERATORS

```
x = (a for a in 1:10)

y = (a for a in 1:10 if a > 5)
```

```
julia> x
Base.Generator{UnitRange{Int64}, typeof(identity)}(identity, 1:10)
julia> y
Base.Generator{Base.Iterators.Filter{var"#42#43", UnitRange{Int64}}, typeof(identity)}
(identity, Base.Iterators.Filter{var"#42#43", UnitRange{Int64}}(var"#42#43"(), 1:10))
```

The examples show that array comprehensions compute and give immediate access to the elements of the vector. In contrast, generators formally define an object with type `Base.Generator`, where operations are described but no output is materialized.

This characteristic makes generators particularly useful for applying **reductions** to transformed values of a collection. By producing values on-demand and fusing them with the reduction function, generators avoid the materialization of temporary vectors, thus reducing memory allocations.

To illustrate the performance benefits this entails, let's compute the sum of all elements in a vector `y`. In particular, `y` is obtained by doubling each element of a vector `x`. One way to compute this operation is by first creating the vector `y` and then summing all its elements. Alternatively, we can describe the transformation through a generator, which bypasses the storage of the intermediate output `y` and instead feeds the transformation directly into the `sum` function. This allows the compiler to perform the addition as a cumulative operation on scalars, thereby reducing memory usage.

ARRAY COMPREHENSION

```
x = rand(100)

function foo(x)
    y = [a * 2 for a in x]       # it does allocate

    sum(y)
end
```

```
julia> @btime foo($x)
  58.431 ns (2 allocations: 928 bytes)
```

GENERATOR

```
x = rand(100)

function foo(x)
    y = (a * 2 for a in x)      # it does NOT allocate

    sum(y)
end
```

```
julia> @btime foo($x)
  28.663 ns (0 allocations: 0 bytes)
```

GENERATOR (EQUIVALENT)

```
x = rand(100)


foo(x) = sum(a * 2 for a in x)  # it does NOT allocate
```

```
julia> @btime foo($x)
  28.520 ns (0 allocations: 0 bytes)
```

The last tab shows that generators can be incorporated directly as a function argument, resulting in a compact syntax. Remarkably, this syntax is applicable to *any* function that accepts a collection as its input.

ITERATORS

Iterators are formally defined as lazy objects that create sequential values on demand, rather than storing them all in memory upfront. Throughout the book, we've already encountered numerous scenarios involving iterators. A typical example of an iterator is a range, such as `1:length(x)`, which defines a sequence of numbers to be generated on the fly. Their lazy evaluation explains why the

function `collect` is needed when we want to materialize the entire sequence into a vector. Without `collect`, iterators merely describe the numbers to be created, without actually creating and storing them in memory.

Beyond simple ranges, we've also covered other types of iterators that offer more specialized functionality. They include `eachindex` for accessing array indices, `enumerate` for pairing elements with their positions, and `zip` for combining multiple sequences.

The lazy nature of iterators makes them particularly efficient in for-loops: by generating each value as the for-loop progresses, we eliminate unnecessary memory allocations that would arise from materializing the list being iterated over.

RANGE

```
x = 1:10
```

```
julia> x
1:10
```

RANGE + COLLECT

```
x = collect(1:10)
```

```
julia> x
10-element Vector{Int64}:
  1
  2
  ⋮
  9
 10
```

The built-in package `Iterators`, which is automatically "imported" in every Julia session, provides multiple functions for generating lazy sequences. Additionally, it offers lazy counterparts of various functions such as `filter` and `map`, which can be accessed as `Iterators.filter` and `Iterators.map`. [1]

The following example demonstrates the use of these functions to avoid memory allocations of intermediate computations.

FILTER

```
x = collect(1:100)

function foo(x)
    y = filter(a -> a > 50, x)              # it does allocate

    sum(y)
end
```

```
julia> @btime foo($x)
  72.889 ns (3 allocations: 1.344 KiB)
```

FILTER (ITERATORS)

```
x = collect(1:100)

function foo(x)
    y = Iterators.filter(a -> a > 50, x)     # it does NOT allocate

    sum(y)
end
```

```
julia> @btime foo($x)
  50.891 ns (0 allocations: 0 bytes)
```

MAP

```
x = rand(100)

function foo(x)
    y = map(a -> a * 2, x)                   # it does allocate

    sum(y)
end
```

```
julia> @btime foo($x)
  68.000 ns (2 allocations: 928 bytes)
```

MAP (ITERATORS)

```
x = rand(100)

function foo(x)
    y = Iterators.map(a -> a * 2, x)         # it does NOT allocate

    sum(y)
end
```

```
julia> @btime foo($x)
  25.518 ns (0 allocations: 0 bytes)
```

FOOTNOTES

[1] The `IterTools` package further extends the functionality of `Iterators`, offering even more tools for working with lazy sequences.

9g. Lazy Broadcasting and Loop Fusion

Martin Alfaro
PhD in Economics

INTRODUCTION

This section continues the analysis of lazy operations as a means of reducing memory allocations. The focus now shifts to broadcasting operations, which strike a balance between code readability and performance.

A key feature of broadcasting in Julia is its default eager behavior. This means that broadcast operations compute and materialize outputs immediately upon execution. As a consequence, broadcasting objects like vectors inevitably leads to memory allocation. Such behavior becomes especially relevant in scenarios with intermediate broadcast operations, whose allocations are potentially avoidable.

To address this performance cost, we'll present two strategies. The first one highlights the notion of **loop fusion**. This enables the combination of multiple broadcasting operations into a single more efficient one. By executing the combined operations in one pass over the data, loop fusion significantly minimizes memory allocations, although not necessarily completely eliminating them.

After this, we'll explore the `LazyArrays` package, which evaluates broadcasting operations **lazily**. The main advantage of this approach is for reductions requiring intermediate calculations. In those cases, a lazy technique can completely bypass memory allocations by avoiding the materialization of temporary arrays.

HOW DOES BROADCASTING WORK INTERNALLY?

Broadcasting operations are essentially syntactic sugar for for-loops, sparing developers from writing them explicitly. This makes it possible to write code that's concise and expressive, without sacrificing performance.

Despite this, you'll often notice performance differences in practice. These discrepancies stem primarily from compiler optimizations, rather than inherent differences between broadcasting and for-loops. The reason is that an operation supporting a broadcast form is automatically revealing additional information about its underlying structure. Consequently, the compiler is allowed to apply more aggressive optimizations. In contrast, for-loops are conceived as a general-purpose construct, precluding the compiler from automatically making such assumptions.

It's worth noting, though, that carefully optimized for-loops always match or exceed the performance of broadcasting. The following code snippets demonstrate this point. The first tab outlines the operation being performed, while the second tab provides a rough internal translation of broadcasting into a for-loop.

The third tab demonstrates the equivalence more directly, using a for-loop that resembles the broadcast code more closely. Its implementation adds the `@inbounds` macro, which broadcasting always applies automatically. The definition of `@inbounds` will be studied in a later section. Its inclusion here is only to illustrate the internal implementation of broadcasting.

BROADCASTING

```
x      = rand(100)

foo(x) = 2 .* x
```

```
julia> @btime foo($x)
  134.005 ns (2 allocations: 928 bytes)
```

ROUGH INTERNAL IMPLEMENTATION

```
x      = rand(100)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = 2 * x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  103.603 ns (2 allocations: 928 bytes)
```

ACTUAL INTERNAL IMPLEMENTATION

```
x      = rand(100)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = 2 * x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  70.333 ns (2 allocations: 928 bytes)
```

> ⚠ **Warning!** - About `@inbounds`
>
> In the example provided, `@inbounds` **was solely added to demonstrate the internal implementation of broadcasting, not as a general recommended practice**. In fact, `@inbounds` **can cause serious issues** when used incorrectly.
>
> To understand the role of this macro, recall that Julia performs bounds checks in for-loops by default. This means Julia verifies the existence of elements accessed during every iteration. Bounds checking prevents out-of-range access, so that a vector `x` with 3 elements isn't accessed at index `x[4]`. Instead, placing `@inbounds` at the beginning of a for-loop instructs Julia to disable these checks.
>
> Removing bounds checking can improve performance, but it comes at the cost of safety: an out-of-range access may result in program termination or trigger more severe issues. Out-of-bounds access can't occur when operations support broadcasting, since the broadcast mechanism guarantees a valid index range. Consequently, Julia safely omits these checks by automatically applying `@inbounds`.

REMARK (OPTIONAL) Other Optimization Differences Between Broadcasting and For-Loops

CONSEQUENCES OF HOW BROADCASTING IS INTERNALLY COMPUTED

Once we understand how broadcasting is computed internally, we can also anticipate its consequences for memory allocations. First, broadcasting involves the creation of a collection like `output` to store the result. Therefore, the operation will necessarily allocate when broadcasting vectors, since `output` will inherit the type of its inputs.

Importantly, **memory allocations in broadcasting arise even when the result isn't explicitly stored**. For example, evaluating `sum(2 .* x)` involves storing internally the intermediate output `2 .* x`.

Second, Julia's broadcasting is eager by default. Continuing with the same example, this means that `2 .* x` in `sum(2 .* x)` is computed immediately. As we'll see, adopting a lazy strategy can be advantageous in such cases. By deferring the computation of `2 .* x` until `sum(2 .* x)` is executed, we let Julia know that the broadcast operation will be used as part of a reduction. Thus, the compiler can optimize the internal computation, by generating a for-loop that reduces a transformed version of `x`. This avoids materializing the intermediate result altogether, thereby eliminating the temporary allocation for `2 .* x`.

LOOP FUSION

When working with long broadcast operations, splitting them into smaller intermediate steps can significantly improve readability and reduce the likelihood of errors. However, this approach comes at the cost of separately allocating each broadcasting operation.

To mitigate unnecessary memory allocations, it's essential to preserve **loop fusion**: a compiler optimization that merges multiple element-wise operations into a single loop over the data. Thus, with loop fusion enabled, the compiler can perform all operations in a single pass over the data. This not only eliminates the creation of multiple intermediate vectors, but also provides the compiler with a holistic view of the operations, thus unlocking further optimizations.

Loop fusion is applied automatically when all operations are expressed as a single broadcast operation. Yet, as indicated before, writing a single lengthy monolithic expression is inconvenient for complex computations. To overcome this limitation, we can rely on the lazy design of function definitions. Below, we show how this approach can break down an operation into partial calculations, while still preserving loop fusion.

OPERATION SPLIT

```
x        = rand(100)

function foo(x)
    a      = x .* 2
    b      = x .* 3

    output = a .+ b
end
```

```
julia> @btime foo($x)
  293.200 ns (6 allocations: 2.719 KiB)
```

LOOP FUSION

```
x        = rand(100)

foo(x)   = x .* 2 .+ x .* 3     # or @. x * 2 + x * 3
```

```
julia> @btime foo($x)
  114.974 ns (2 allocations: 928 bytes)
```

OPERATION SPLIT + LOOP FUSION

```
x        = rand(100)

term1(a) = a * 2
term2(a) = a * 3

foo(a)   = term1(a) + term2(a)
```

```
julia> @btime foo.($x)
  127.305 ns (2 allocations: 928 bytes)
```

Even with a single simple operation, certain coding patterns can inadvertently break loop fusion. When this occurs, Julia will internally fall back to evaluating sub-expressions in separate for-loops, which are eventually combined to present the final output. In the following, we present two of these cases. The key practical takeaway is that you should broadcast via the macro `@.` when possible, rather than manually adding dots to each operator and function within an expression.

MIXING BROADCASTING WITH VECTOR OPERATIONS BREAKS LOOP FUSION

To understand the issue, there are two facts to consider. The first one is that some vector operations produce an equivalent result to their broadcast counterparts. For instance, adding two vectors with `+` yields the same result as summing them element-wise with `.+`.

BROADCASTING

```
x        = [1, 2, 3]
y        = [4, 5, 6]

foo(x,y) = x .+ y
```

```
julia> foo(x,y)
3-element Vector{Int64}:
 5
 7
 9
```

VECTOR OPERATION

```
x        = [1, 2, 3]
y        = [4, 5, 6]

foo(x,y) = x + y
```

```
julia> foo(x,y)
3-element Vector{Int64}:
 5
 7
 9
```

Another example is with product operations.

BROADCASTING

```
x        = [1, 2, 3]
β        = 2

foo(x,β) = x .* β
```

```
julia> foo(x,β)
3-element Vector{Int64}:
 2
 4
 6
```

VECTOR OPERATION

```
x        = [1, 2, 3]
β        = 2

foo(x,β) = x * β
```

```
julia> foo(x,β)
3-element Vector{Int64}:
 2
 4
 6
```

The second fact to consider is that vector operations don't participate in loop fusion. Thus, even if all these operations were combined into a single expression, Julia will evaluate each sub-expression separately, allocating temporary vectors at every step.

VECTOR OPERATION

```
x = rand(100)

foo(x)  = x * 2 + x * 3
```

```
julia> @btime foo($x)
  394.873 ns (6 allocations: 2.719 KiB)
```

INTERNAL IMPLEMENTATION

```
x = rand(100)

function foo(x)
    term1  = x * 2
    term2  = x * 3

    output = term1 + term2
end
```

```
julia> @btime foo($x)
  312.227 ns (6 allocations: 2.719 KiB)
```

Putting both facts together, mixing broadcasting with vector operations in the same expression may yield the correct result, but only achieve loop fusion partially. This means Julia will only fuse contiguous broadcast segments, internally partitioning the computation into separate for-loops when a vector operation is encountered. Each of these for-loops will produce a temporary intermediate vector.

PARTIAL FUSION

```
x      = rand(100)

foo(x) = x * 2 .+ x .* 3
```

```
julia> @btime foo($x)
  244.920 ns (4 allocations: 1.812 KiB)
```

INTERNAL IMPLEMENTATION

```
x      = rand(100)

function foo(x)
    term1  = x * 2

    output = term1 .+ x .*3
end
```

```
julia> @btime foo($x)
  230.167 ns (4 allocations: 1.812 KiB)
```

This behavior leads to a clear and actionable guideline: for full loop fusion, every operator and function in the expression must be explicitly broadcast. Yet manually adding dots throughout a large expression is both tedious and error-prone: one missing dot is enough to reintroduce unnecessary allocations. There are two coding practices that mitigate this risk.

One option is to broadcast via the macro `@.`, as shown below in the tab "Equivalent 1". By design, this adds a dot to *all* operators and functions within an expression, guaranteeing loop fusion. An alternative is to express the operation through a *scalar* function, which we then broadcast. This is presented below in the tab "Equivalent 2".

LOOP FUSION

```
x      = rand(100)

foo(x) = x .* 2 .+ x .* 3
```

```
julia> @btime foo($x)
  117.949 ns (2 allocations: 928 bytes)
```

EQUIVALENT 1

```
x      = rand(100)

foo(x) = @. x * 2 + x * 3
```

```
julia> @btime foo($x)
  122.426 ns (2 allocations: 928 bytes)
```

EQUIVALENT 2

```
x       = rand(100)

foo(a) = a * 2 + a * 3
```

```
julia> @btime foo.($x)
  124.282 ns (2 allocations: 928 bytes)
```

LAZY BROADCASTING

Broadcasting results in memory allocations since the technique is eager by default. This property means that its output becomes readily available when a broadcast expression is evaluated. Considering this, another way to achieve loop fusion is by evaluating all broadcast sub-expressions lazily, until the final output is computed. The approach is similar in spirit to the use of helper functions to accomplish the same goal, but without cluttering the namespace with new functions.

The functionality is provided by the macro `@~` from the `LazyArrays` package. By prepending this macro to the broadcasting operation, its computation is deferred until its output is required.

EAGER BROADCASTING

```
x          = rand(100)

function foo(x)
    term1  = x .* 2
    term2  = x .* 3

    output = term1 .+ term2
end
```

```
julia> @btime foo($x)
  394.852 ns (6 allocations: 2.719 KiB)
```

LAZY BROADCASTING @~

```
x          = rand(100)

function foo(x)
    term1  = @~ x .* 2
    term2  = @~ x .* 3

    output = term1 .+ term2
end
```

```
julia> @btime foo($x)
  120.214 ns (2 allocations: 928 bytes)
```

SPLITTING OPERATIONS WITH FUNCTIONS

```
x          = rand(100)

term1(a)   = a * 2
term2(a)   = a * 3

foo(a)     = term1(a) + term2(a)
```

```
julia> @btime foo($x)
  134.263 ns (2 allocations: 928 bytes)
```

Beyond this resemblance with an approach based on functions, lazy broadcasting additionally addresses certain scenarios that functions can't handle. To understand this, note that the operations explored thus far resulted in a *vector* output. In those cases, memory allocations could at best be reduced to a single unavoidable allocation, necessary for storing the final output.

When the final output is instead given by a scalar, as occurs with reductions, lazy broadcasting is capable of entirely eliminating memory allocations. The reason is that lazy broadcasting fuses with reduction operations, letting the compiler apply a **non-allocating procedure**. This is illustrated in the example below.

EAGER BROADCASTING

```
# eager broadcasting (default)
x      = rand(100)

foo(x) = sum(2 .* x)
```

```
julia> @btime foo($x)
  120.489 ns (2 allocations: 928 bytes)
```

LAZY BROADCASTING

```
using LazyArrays
x      = rand(100)

foo(x) = sum(@~ 2 .* x)
```

```
julia> @btime foo($x)
  25.216 ns (0 allocations: 0 bytes)
```

> **Lazy Broadcasting May Be Faster Than Other Lazy Alternatives**
>
> An additional advantage of `@~` is that it implements extra optimizations. This explains why `@~` tends to be faster than alternatives like a lazy map, even though neither allocates memory. This performance benefit can be noticed in the following comparison.

LAZY BROADCASTING @~

```
x        = rand(100)

term1(a) = a * 2
term2(a) = a * 3
temp(a)  = term1(a) + term2(a)

foo(x)   = sum(@~ temp.(x))
```

```
julia> @btime foo($x)
  39.288 ns (0 allocations: 0 bytes)
```

LAZY MAP

```
x        = rand(100)

term1(a) = a * 2
term2(a) = a * 3
temp(a)  = term1(a) + term2(a)

foo(x)   = sum(Iterators.map(temp, x))
```

```
julia> @btime foo($x)
  88.509 ns (0 allocations: 0 bytes)
```

9h. Mutations and Pre-Allocations

Martin Alfaro
PhD in Economics

INTRODUCTION

Certain operations executed with for-loops may trigger the creation of new vectors during each iteration. This repeated memory allocation may be unnecessary in many cases. Particularly, when the vectors only store temporary intermediate results that are discarded after the current iteration. A similar issue arises when the same operations must be executed with different inputs, as in simulations or Monte Carlo experiments. If only the final output is required, generating new intermediate vectors for every run results in avoidable allocations and wasted resources.

In these situations, performance can be improved through **pre-allocation**. This technique consists of initializing a vector before execution begins, which is then reused throughout each computation. By allocating memory upfront and updating the vector in place, the program avoids the repeated overhead of memory allocation.

The technique isn't exclusive to Julia. Rather, it represents an optimization strategy applicable across programming languages. Its effectiveness ultimately stems from **favoring mutations over the creation of new objects**, thus minimizing allocations on the heap.

Our presentation will include a review of methods for initializing vectors. This is a prerequisite for implementing a pre-allocation strategy. We then present some scenarios where pre-allocation proves advantageous, with special emphasis on its advantages within nested for-loops.

THE ADVANTAGES OF MUTATING VECTORS

The benefits of pre-allocation stem from the broader advantages of **in-place operations**. This strategy encourages reusing existing vectors rather than creating new objects. Because this principle is valuable beyond pre-allocation, we begin by illustrating the performance gains that come from reusing the same memory address.

To make this concrete, consider the operation of sorting a vector `x`. In Julia, many built-in functions have an **in-place function** counterpart. They're easily identified by inclusion of the symbol `!` at the end of the function name. For example, `sort(x)` allocates a new sorted vector, while `sort!(x)` rearranges the elements of `x` in place. When preserving the original ordering isn't necessary, creating a separate copy provides no benefit. In those cases, the mutating version `sort!(x)` is preferable, as it avoids unnecessary allocations by reusing the memory associated with `x`. The gains from this approach are presented below.

SORT

```
x = rand(1_000_000)
```

```
julia> @btime sort($x)
  43.514 ms (9 allocations: 15.267 MiB)
```

SORT!

```
x = rand(1_000_000)
```

```
julia> @btime sort!($x)
  1.750 ms (0 allocations: 0 bytes)
```

As we can see, the differences in runtime are substantial. If the same operation were executed repeatedly inside a for-loop, the performance gaps would become even more pronounced.

Pre-allocation builds on the same underlying idea, but applies to situations where a vector like `x` doesn't yet exist. Considering this, we begin with a review of how to initialize vectors efficiently.

INITIALIZING VECTORS

> **Remark**
>
> The review of methods for vector initialization will be relatively brief and centered on performance considerations. For a more detailed coverage, see the section about vector initialization.

Vector initialization refers to the process of creating a vector that subsequently will be filled with values. The process typically involves two steps: reserving space in memory and populating that space with some initial values. An efficient approach to initializing a vector involves performing only the first step, keeping whatever content is in the memory address. These values held in memory are referred to as `undef` (undefined). Although Julia will display them as numerical values, they're essentially arbitrary and meaningless.

There are two methods for initializing a vector with `undef` values. The first one is through a constructor, requiring the specification of the length and element types. The second one is based on the function `similar`, which creates a vector with the same type and dimension as another existing vector. This approach is particularly useful when the output sought matches the structure of an input.

Below, we compare the performance of several approaches to initializing vectors. In particular, we establish that working with `undef` values is faster than populating vectors with specific values. To starkly show the differences in execution time, we repeat the process of vector creation 100,000 times.

UNDEF

```
x           = collect(1:100)
repetitions = 100_000                       # repetitions in a for-loop

function foo(x, repetitions)
    for _ in 1:repetitions
        Vector{Int64}(undef, length(x))
    end
end
```

```
julia> @btime foo($x, $repetitions)
  3.455 ms (100000 allocations: 85.449 MiB)
```

SIMILAR (UNDEF)

```
x           = collect(1:100)
repetitions = 100_000                       # repetitions in a for-loop

function foo(x, repetitions)
    for _ in 1:repetitions
        similar(x)
    end
end
```

```
julia> @btime foo($x, $repetitions)
  2.977 ms (100000 allocations: 85.449 MiB)
```

ZEROS

```
x           = collect(1:100)
repetitions = 100_000                       # repetitions in a for-loop

function foo(x, repetitions)
    for _ in 1:repetitions
        zeros(Int64, length(x))
    end
end
```

```
julia> @btime foo($x, $repetitions)
  10.797 ms (100000 allocations: 85.449 MiB)
```

ONES

```
x           = collect(1:100)
repetitions = 100_000                       # repetitions in a for-loop

function foo(x, repetitions)
    for _ in 1:repetitions
        ones(Int64, length(x))
    end
end
```

```
julia> @btime foo($x, $repetitions)
  11.003 ms (100000 allocations: 85.449 MiB)
```

FILL

```
x           = collect(1:100)
repetitions = 100_000                       # repetitions in a for-loop

function foo(x, repetitions)
    for _ in 1:repetitions
        fill(2, length(x))                  # vector filled with integer 2
    end
end
```

```
julia> @btime foo($x, $repetitions)
  10.170 ms (100000 allocations: 85.449 MiB)
```

> **Remark**
>
> Recall that `_` isn't a keyword, but rather a convention widely adopted by programmers to denote **dummy variables**. These are variables included solely to meet syntactical requirements, but are never actually used or referenced within the code. In our example, the inclusion of `_` is because for-loops must always include a variable to iterate over. While any other symbol could be used, `_` signals programmers that its value can be safely ignored.

INITIALIZING VECTORS IN FUNCTIONS

We can initialize a vector by passing it to the function as a keyword argument. This strategy even enables the use of `similar(x)`, with `x` being a previous function argument. Considering this, the following implementations are equivalent.

IMPLEMENTATION 1

```
function foo(x)
   output = similar(x)
   # <some calculations using 'output'>
end
```

IMPLEMENTATION 2

```
function foo(x; output = similar(x))

   # <some calculations using 'output'>
end
```

When multiple variables of the same type need to be initialized, array comprehensions offer a concise solution. The downside is that they first create a vector that holds all the initialized vectors, which is unnecesary. Building on the same construct, a more efficient alternative is based on **generators**. They're the lazy counterpart of array comprehensions, reducing memory allocations by directly assigning each vector to its corresponding variable.

Below we illustrate these strategies. Although the example is based on `similar(x)`, the same principle applies to constructors of the form `Vector{Float64}(undef, length(x))`.

ARRAY COMPREHENSION

```
x = [1,2,3]

function foo(x)
    a,b,c = [similar(x) for _ in 1:3]
    # <some calculations using a,b,c>
end
```

```
julia> @btime foo($x)
  101.973 ns (8 allocations: 320 bytes)
```

GENERATOR

```
x = [1,2,3]

function foo(x)
    a,b,c = (similar(x) for _ in 1:3)
    # <some calculations using a,b,c>
end
```

```
julia> @btime foo($x)
  40.751 ns (3 allocations: 144 bytes)
```

DESCRIBING THE PRE-ALLOCATION TECHNIQUE

To describe how pre-allocation works, we'll consider a scenario where the strategy proves to be advantageous: **nested for-loops**. In these constructions, the output of a for-loop serves as an intermediate input for another for-loop. To illustrate them, we'll provide examples with both explicit for-loops and with operations computed internally via for-loops (e.g., broadcasting).

Let's first describe the inner for-loop. Suppose we're assessing a worker's performance over a 30-day period, with daily scores recorded on a scale from 0 to 1. Defining successful performance as any score above 0.5, the following code snippets generate vectors indicating the days on which the goal was achieved.

BROADCASTING

```
nr_days             = 30
score               = rand(nr_days)

performance(score) = score .> 0.5
```

```
julia> @btime performance($score)
  49.875 ns (3 allocations: 96 bytes)
```

FOR-LOOP (EQUIVALENT)

```
nr_days             = 30
score               = rand(nr_days)

function performance(score)
    target = similar(score)

    for i in eachindex(score)
        target[i] = score[i] > 0.5
    end

    return target
end
```

```
julia> @btime performance($score)
  45.300 ns (2 allocations: 304 bytes)
```

FOR-LOOP (EQUIVALENT)

```
nr_days             = 30
score               = rand(nr_days)

function performance(score; target=similar(score))


    for i in eachindex(score)
        target[i] = score[i] > 0.5
    end

    return target
end
```

```
julia> @btime performance($score)
  44.777 ns (2 allocations: 304 bytes)
```

Consider now that the output of this operation (the vector of days in which the target was met) represents an intermediate step in another for-loop. For instance, suppose we have performance scores for multiple workers and wish to summarize each worker's data using a summary statistic. Specifically, we'll use the ratio of the standard deviation to the mean, which expresses variability in units of the average.

In practice, calculating this statistic requires both the mean and the standard deviation. Below, we show that computing each statistic via reductions is inefficient. The reason is that, while reductions avoid memory allocations, they require computing which days the target was met twice. Instead, it's more efficient to compute and store this vector beforehand, and then reuse it as the input for both the mean and the standard deviation.

The result highlights a more general principle: when the same intermediate input is required for multiple outputs, the cost of allocating memory for the intermediate vector tends to be lower than its repeated computation. In the example, the inefficient approach is implemented via lazy broadcasting, which internally relies on reduction techniques. The other versions employ explicit or implicit for-loops, storing the intermediate input in a vector.

REDUCTIONS (INEFFICIENT)

```
nr_days             = 30
scores              = [rand(nr_days), rand(nr_days), rand(nr_days)]  # 3 workers



function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)

      stats[col] = std(@~ scores[col] .> 0.5) / mean(@~ scores[col] .> 0.5)
    end

    return stats
end
```

```
julia> @btime repeated_call($scores)
  685.195 ns (2 allocations: 80 bytes)
```

IMPLICIT NESTED FOR-LOOP

```
nr_days            = 30
scores             = [rand(nr_days), rand(nr_days), rand(nr_days)]   # 3 workers

performance(score) = score .> 0.5

function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
      target     = performance(scores[col])
      stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call($scores)
  495.696 ns (11 allocations: 368 bytes)
```

EXPLICIT NESTED FOR-LOOP

```
nr_days            = 30
scores             = [rand(nr_days), rand(nr_days), rand(nr_days)]   # 3 workers

function performance(score)
    target = similar(score)

    for i in eachindex(score)
        target[i] = score[i] > 0.5
    end

    return target
end

function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
      target     = performance(scores[col])
      stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call($scores)
  362.846 ns (8 allocations: 992 bytes)
```

EXPLICIT NESTED FOR-LOOP

```
nr_days            = 30
scores             = [rand(nr_days), rand(nr_days), rand(nr_days)]    # 3 workers

function performance(score; target = similar(score))


    for i in eachindex(score)
        target[i] = score[i] > 0.5
    end

    return target
end

function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
      target     = performance(scores[col])
      stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call($scores)
  376.145 ns (8 allocations: 992 bytes)
```

Since storing the intermediate vector is more performant, an efficient approach will necessarily involve memory allocations. Yet, there's still room for improvement, as the methods described above allocate more than required. In particular, during each iteration, a new vector is created for every worker to store the days the target was met. Such allocations are unnecessary, since this intermediate vector doesn't need to be preserved for future use. Indeed, the examples store this vector in a local variable, implying that it's dropped once the iteration concludes.

Such cases are strong candidates for pre-allocating intermediate results. By defining a single vector `target`, we can reuse the same object across iterations to compute the days when the target was met for each worker. This strategy incurs the cost of creating the vector only once, after which it's reused for every worker.

The implementation requires initializing `target` before the outer for-loop begins. Then, during each iteration, we mutate `target` using an in-place function, which we'll call `performance!`. This function can be defined to update contents either with a standard for-loop or by applying the broadcasting operator `.=`.

NO PRE-ALLOCATION

```
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]


performance(score) = score .> 0.5

function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        target     = performance(scores[col])
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call($scores)
  524.400 ns (11 allocations: 368 bytes)
```

PRE-ALLOCATION (VIA BROADCASTING)

```
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]
target  = similar(scores[1])

performance!(target, score) = (@. target = score > 0.5)

function repeated_call!(target, scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        performance!(target, scores[col])
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call!($target, $scores)
  358.638 ns (2 allocations: 80 bytes)
```

PRE-ALLOCATION (VIA FOR-LOOP)

```
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]
target  = similar(scores[1])

function performance!(target, score)
    for i in eachindex(score)
        target[i] = score[i] > 0.5
    end
end

function repeated_call!(target, scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        performance!(target, scores[col])
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call!($target, $scores)
  339.744 ns (2 allocations: 80 bytes)
```

> ⚠ **Warning!** - Use of @. to update values
>
> When your goal is to update the values of a vector, recall that `@.` has to be *placed at the beginning* of the statement.
>
> **NEW VARIABLE (NOT AN UPDATE)**
>
> ```
> # the following are equivalent and define a new variable
>   output  = @. 2  * x
>   output  =    2 .* x
> ```
>
> **AN UPDATE**
>
> ```
> # the following are equivalent and update 'output'
> @. output  = 2  * x
>   output .= 2 .* x
> ```

Compared to a for-loop, the method using `.=` provides a simpler syntax. For this reason, when the function `performance!` is simple enough as in our example, it's common to skip its definition and directly insert the operation into the inner for-loop. The approach also enables implementing the update via a built-in in-place function. For instance, `map!` can be used with this purpose.

NO PRE-ALLOCATION

```
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]


function repeated_call(scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        target     = @. score > 0.5
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call($scores)
  822.471 ns (23 allocations: 608 bytes)
```

PRE-ALLOCATION (VIA BROADCASTING)

```
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]
target  = similar(scores[1])

function repeated_call!(target, scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        @. target  = scores[col] > 0.5
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call!($target, $scores)
  336.372 ns (2 allocations: 80 bytes)
```

PRE-ALLOCATION (VIA MAP!)

```
nr_days = 30
scores  = [rand(nr_days), rand(nr_days), rand(nr_days)]
target  = similar(scores[1])

function repeated_call!(target, scores)
    stats = Vector{Float64}(undef, length(scores))

    for col in eachindex(scores)
        map!(a -> a > 0.5, target, scores[col])
        stats[col] = std(target) / mean(target)
    end

    return stats
end
```

```
julia> @btime repeated_call!($target, $scores)
  329.814 ns (2 allocations: 80 bytes)
```

9i. Static Vectors for Small Collections

Martin Alfaro
PhD in Economics

INTRODUCTION

Due to the memory-allocation overhead it involves, repeated vector creation can quickly become a performance bottleneck. The issue has far-reaching implications, since vector creation doesn't just occur when a vector is explicitly defined. It also happens internally in various scenarios, such as when referencing a slice like `x[1:2]` in `sum(x[1:2])` or when computing intermediate results on the fly like `x .* y` in `sum(x .* y)`.

The current section introduces one strategy to address this issue, while retaining the convenience of vectors for expressing collections. It leverages the so-called **static vectors**, provided by the `StaticArrays` package. Unlike built-in vectors, which are allocated on the heap, static vectors are stack-allocated.

Given the small capacity of the stack, **static vectors are only suitable for collections comprising a few elements**. As a rule of thumb, **you should use static vectors for collections with up to 50 elements**. Exceeding this threshold can lead to increased overhead, potentially offsetting any performance benefits or even resulting in a fatal error. [1]

Static vectors are built on top of tuples. Despite this, static vectors offer additional benefits beyond what tuples offer. Firstly, they maintain their performance benefits even at sizes where tuples typically begin to degrade. Secondly, their manipulation closely resembles that of regular vectors, making them more convenient to work with. Third, the `StaticArrays` package extends support to other array types (including matrices) and offer mutable variants. These mutable versions make static vectors more flexible than tuples, which are only available in an immutable form. It's worth indicating though that, while the mutable version provides performance benefits relative to regular vectors, the immutable option still offers the best performance.

> ⚠️ **Warning!**
>
> **The entire section assumes all collections are small**. Thus, all the benefits highlighted are contingent upon this assumption.
>
> We also suppose that the `StaticArrays` package has been loaded by executing `using StaticArrays`. This is why we omit this command from each code snippet.

PERFORMANCE GAINS OF TUPLES FOR SMALL COLLECTIONS

Since static vectors are built on top of tuples, we begin by illustrating the advantages of tuples over vectors. In particular, we emphasize that these advantages only materialize when the collections are small.

Specifically, recall that tuples are immutable and have a fixed size. This allows the compiler to aggressively optimize operations on them. However, as the number of elements grows, performance degrades sharply. In fact, once a tuple exceeds a certain threshold, the computation may fail to complete, causing the program to stall. The underlying reason is that, unlike heap-allocated objects like vectors, tuples live on the stack, which has very limited space.

SMALL COLLECTION

```
_size  = 10
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x .* x)
```

```
julia> foo(vector)
  34.806 ns (2 allocations: 144 bytes)
julia> foo(tuple)
  1.742 ns (0 allocations: 0 bytes)
```

MEDIUM COLLECTION

```
_size  = 200
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x .* x)
```

```
julia> foo(vector)
  109.528 ns (2 allocations: 1.625 KiB)
julia> foo(tuple)
  120.992 ns (0 allocations: 0 bytes)
```

BIG COLLECTION

```
_size  = 2_000
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x .* x)
```

```
julia> foo(vector)
  1.217 µs (3 allocations: 15.688 KiB)
julia> foo(tuple)
  2.306 µs (0 allocations: 0 bytes)
```

BIG COLLECTION - PROGRAM STALLED

```
_size  = 10_000
vector = rand(_size)
tuple  = Tuple(vector)

foo(x) = sum(x.* x)
```

```
julia> foo(vector)
  8.197 µs (3 allocations: 78.188 KiB)
julia> foo(tuple)
# program keeps working indefinitely
```

Large collections can also trigger a different kind of failure: a stack overflow. This typically occurs when the program exhausts the available stack space. Unlike the gradual slowdown caused by performance degradation, a stack overflow produces an immediate and fatal error. In such cases, Julia aborts execution and reports the problem directly, rather than allowing the computation to hang indefinitely.

STACK OVERFLOW

```
function foo(n)
    if n == 0
        ()
    else
        (n, foo(n-1)...)
    end
end
```

```
julia> foo(50_000)
ERROR: StackOverflowError
```

STATIC VECTORS

The package `StaticArrays` includes several variants of static arrays. They preserve the advantages of tuples, while extending their functionality.

Our primary focus here is on the type `SVector`, whose objects we'll simply refer to as SVectors. Like tuples, **SVectors are immutable**, meaning their elements can't be added, removed, or modified.

There are two approaches to creating an SVector, each serving a distinct purpose. The first one creates an SVector directly from literal values, while the second converts an existing regular vector into an SVector. The following examples illustrate both approaches in practice.

VIA LITERAL VALUES

```
# all 'sx' define the same static vector '[3,4,5]'


sx = SVector(3,4,5)
sx = SVector{3, Int64}(3,4,5)
sx = SA[3,4,5]
sx = @SVector [i for i in 3:5]
```

```
julia> sx
3-element SVector{3, Int64} with indices SOneTo(3):
 3
 4
 5
```

VIA EXISTING VECTOR

```
# all 'sx' define a static vector with same elements as 'x'
x = collect(1:10)

sx = SVector(x...)
sx = SVector{length(x), eltype(x)}(x)
sx = SA[x...]
sx = @SVector [a for a in x]
```

```
julia> sx
10-element SVector{10, Int64} with indices SOneTo(10):
  1
  2
  3
  ⋮
  9
 10
```

Of these approaches to creating SVectors, we'll primarily rely on the function `SVector`, occasionally employing `SA` for indexing purposes. [2] Note that the use of **splat operator** `...` is necessary when converting an existing vector into an SVector. This operator expands a collection into a sequence of arguments. For instance, if `x` is a vector or tuple with 3 elements, `foo(x...)` is equivalent to `foo(x[1], x[2], x[3])`.

Regarding slices of SVectors, the **method used for indexing determines whether the resulting slice is a regular vector or an SVector**: a slice remains an SVector when indices are provided as SVectors, whereas the slice becomes a regular vector when indices are ranges or regular vectors. The

sole exception to this rule is when the slice references the whole object (i.e., `sx[:]`), in which case an SVector is returned.

SLICES AS SVECTORS

```
x      = collect(3:10)
sx     = SVector(x...)

# both define static vectors
slice1 = sx[:]
slice2 = sx[SA[1,2]]        # or slice2 = sx[SVector(1,2)]
```

```
julia> slice1
8-element SVector{8, Int64} with indices SOneTo(8):
  3
  4
  ⋮
  9
 10
julia> slice2
2-element SVector{2, Int64} with indices SOneTo(2):
 3
 4
```

WRONG APPROACH

```
x      = collect(3:10)
sx     = SVector(x...)

# both define and ordinary vector
slice1 = sx[1:2]
slice2 = sx[[1,2]]
```

```
julia> slice
2-element Vector{Int64}:
 3
 4
```

SVECTORS DON'T ALLOCATE MEMORY AND ARE FASTER

Since SVectors are internally implemented on top of tuples, they don't allocate memory.

COPY

```
x   = rand(10)


function foo(x)
    a = x[1:2]              #   allocation (copy of slice)
    b = [3,4]               #   allocation (vector creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($x)
  42.844 ns (3 allocations: 128 bytes)
```

VIEW

```
x   = rand(10)


@views function foo(x)
    a = x[1:2]              # 0 allocation (view of slice)
    b = [3,4]               #   allocation (vector creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($x)
  17.411 ns (1 allocations: 48 bytes)
```

TUPLE

```
x   = rand(10)
tup = Tuple(x)

function foo(x)
    a = x[1:2]              # 0 allocation (slice of tuple)
    b = (3,4)               # 0 allocation (tuple creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($tup)
  1.909 ns (0 allocations: 0 bytes)
```

STATIC VECTOR

```
x   = rand(10)
sx  = SA[x...]

function foo(x)
    a = x[SA[1,2]]          # 0 allocation (slice of static array)
    b = SA[3,4]             # 0 allocation (static array creation)

    sum(a) * sum(b)         # 0 allocation (scalars don't allocate)
end
```

```
julia> @btime foo($sx)
  1.821 ns (0 allocations: 0 bytes)
```

The fact that SVectors don't allocate is especially relevant in operations that generate temporary vectors. For instance, this is the case when a reduction requires an intermediate transformation of the vector being reduced.

VECTOR

```
x      = rand(10)


foo(x) = sum(2 .* x)
```

```
julia> @btime foo($x)
  36.695 ns (2 allocations: 144 bytes)
```

SVECTOR

```
x      = rand(10)
sx     = SVector(x...)

foo(x) = sum(2 .* x)
```

```
julia> @btime foo($sx)
  1.822 ns (0 allocations: 0 bytes)
```

The performance benefits of SVectors extend beyond the absence of memory allocation. This is why, even if operations on regular vectors don't allocate memory, SVectors still offer significant speedups. The point is demonstrated below through the function `sum(f, <vector>)`, which adds all elements of `<vector>` after they're transformed via `f`. In the example, the implementation with SVectors yields faster execution times, even when that of regular vectors already avoids memory allocations.

VECTOR

```
x      = rand(10)


foo(x) = sum(a -> 10 + 2a +  3a^2, x)
```

```
julia> @btime foo($x)
  9.420 ns (0 allocations: 0 bytes)
```

SVECTOR

```
x      = rand(10)
sx     = SVector(x...)

foo(x) = sum(a -> 10 + 2a +  3a^2, x)
```

```
julia> @btime foo($sx)
  1.822 ns (0 allocations: 0 bytes)
```

SVECTOR TYPE AND ITS MUTABLE VARIANT

SVectors are immutable, and so their elements can't be added, removed, or modified. While this property is essential for enabling certain compiler optimizations, it also reduces their scope of application. To address this limitation, the package `StaticArrays` additionally provides a mutable variant given by the `MVector` type. The creation of MVectors and their slices follow the same syntax as SVectors, but with the function `SVector` replaced with `MVector`. This is illustrated below.

IMMUTABLE STATIC VECTOR (SVECTOR)

```
x     = [1,2,3]
sx    = SVector(x...)

sx[1] = 0
```

```
ERROR: setindex!(::SVector{3, Int64}, value, ::Int) is not defined
```

MUTABLE STATIC VECTOR (MVECTOR)

```
x     = [1,2,3]
mx    = MVector(x...)

mx[1] = 0
```

```
julia> mx
3-element MVector{3, Int64} with indices SOneTo(3):
 0
 2
 3
```

Because MVectors are mutable, they're well-suited for scenarios where a vector must be initialized and then filled iteratively via a for-loop. In fact, executing `similar(sx)` automatically returns an MVector when `sx` is an SVector.

'SIMILAR' FOR SVECTORS

```
sx = SVector(1,2,3)

mx = similar(sx)        # it defines an MVector with undef elements
```

```
julia> mx
3-element MVector{3, Int64} with indices SOneTo(3):
 129635981728016
 129635981728000
               2
```

TYPE STABILITY: SIZE IS PART OF THE STATIC VECTOR'S TYPE

Formally, SVectors are defined as objects with type `SVector{N,T}`, where `N` specifies the number of elements and `T` denotes the element's type. For instance, `SVector(4,5,6)` has type `SVector{3,Int64}`, since it comprises 3 elements with type `Int64`. The example illustrates that **the number of elements is part of the `SVector` type**. This behavior, inherited from tuples and shared with MVectors, can readily introduce type instabilities if not handled carefully.

To preserve type stability when working with SVectors, we must then employ strategies similar to those employed for tuples. In practice, this means either passing SVectors directly as function arguments, or dispatching on the number of elements using the `Val` technique.

TYPE UNSTABLE

```
x  = rand(50)


function foo(x)

    output = MVector{length(x), eltype(x)}(undef)

    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end

@code_warntype foo(x)                    # type unstable
```

SVECTOR AS FUNCTION ARGUMENT (TYPE STABLE)

```
x  = rand(50)
sx = SVector(x...)

function foo(x)

    output = MVector{length(x), eltype(x)}(undef)

    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end

@code_warntype foo(sx)                    # type stable
```

VAL (TYPE STABLE)

```
x  = rand(50)


function foo(x, ::Val{N}) where N
    sx     = SVector{N, eltype(x)}(x)
    output = MVector{N, eltype(x)}(undef)

    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end

@code_warntype foo(x, Val(length(x)))     # type stable
```

PERFORMANCE COMPARISON

While MVectors offer performance benefits over regular vectors, bear in mind that they're never more performant than SVectors. For this reason, the general guideline is to prefer an SVector if the collection won't be mutated, reserving MVectors for scenarios that require in-place modifications.

The examples below highlight these principles. They show that SVectors and MVectors either deliver a similar performance or SVectors are clearly superior. Furthermore, both SVectors and MVectors consistently outperform regular vectors for small collections.

SIMILAR PERFORMANCE OF SVECTOR AND MVECTOR

```
x      = rand(10)
sx     = SVector(x...)
mx     = MVector(x...)

foo(x) = sum(a -> 10 + 2a +  3a^2, x)
```

```
julia> @btime foo($x)
  9.426 ns (0 allocations: 0 bytes)
julia> @btime foo($sx)
  1.821 ns (0 allocations: 0 bytes)
julia> @btime foo($mx)
  6.614 ns (0 allocations: 0 bytes)
```

SVECTOR BETTER

```
x      = rand(10)
sx     = SVector(x...)
mx     = MVector(x...)

foo(x) = 10 + 2x +  3x^2
```

```
julia> @btime foo.($x)
  35.112 ns (2 allocations: 144 bytes)
julia> @btime foo.($sx)
  1.909 ns (0 allocations: 0 bytes)
julia> @btime foo.($mx)
  14.586 ns (1 allocations: 96 bytes)
```

STATIC VECTORS VS PRE-ALLOCATIONS

Considering the advantages of static vectors over regular vectors, let's compare their performance with other strategies for reducing memory allocations. In particular, we'll examine how they stack up against **pre-allocating memory** for intermediate outputs. Our examples demonstrate that static vectors can efficiently store intermediate results, making pre-allocation techniques unnecessary. Moreover, storing the final output in an MVector can lead to performance gains over using a regular vector.

For the illustration, consider a reduction via for-loop that requires an intermediate result during each iteration `i`. This consists of counting the number of elements in `x` that are greater than `x[i]`. Formally, `x .> x[i]`. To make the comparison stark, we'll isolate the computation of this intermediate step.

Note that every implementation below requires pre-allocating the vector `output`, explaining why each incurs at least one memory allocation. Any additional allocation depends on whether the temporary vector `temp` is also pre-allocated.

NO PRE-ALLOCATION

```
x  = rand(50)


function foo(x; output = similar(x))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($x)
  2.788 µs (152 allocations: 5.156 KiB)
```

PRE-ALLOCATION

```
x  = rand(50)


function foo(x; output = similar(x), temp = similar(x))
    for i in eachindex(x)
        @. temp      = x > x[i]
           output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($x)
  1.562 µs (4 allocations: 960 bytes)
```

SVECTOR

```
x  = rand(50)
sx = SVector(x...)

function foo(x; output = Vector{Float64}(undef, length(x)))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($sx)
  344.444 ns (2 allocations: 480 bytes)
```

SVECTOR AND MVECTOR ('SIMILAR')

```
x  = rand(50)
sx = SVector(x...)

function foo(x; output = similar(x))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($sx)
  309.357 ns (1 allocations: 448 bytes)
```

SVECTOR AND MVECTOR

```
x  = rand(50)
sx = SVector(x...)

function foo(x; output = MVector{length(x),eltype(x)}(undef))
    for i in eachindex(x)
        temp      = x .> x[i]
        output[i] = sum(temp)
    end

    return output
end
```

```
julia> @btime foo($sx)
  308.000 ns (1 allocations: 448 bytes)
```

The "No Preallocation" tab serves as a reference point for comparison with the other methods. As for the "Pre-allocation" tab, it reuses a regular vector to compute `temp`. In contrast, the "SVector" tab converts `x` to an SVector `sx`, without pre-allocating `temp`. This approach is more performant due to the benefits provided by SVectors.

As for the last two tabs, they continue defining `x` as an SVector, but additionally treat `output` as an MVector. One does this by using `similar(sx)` to initialize `output`, whereas the other tab explicitly specifies an MVector. The strategy delivers additional performance gains.

FOOTNOTES

[1.] The recommended number of elements provided is actually lower than the documentation's suggested (100 elements). The reason for this discrepancy is that, as you approach the upper limit, the performance benefits of static vectors compared to regular vectors decrease significantly. As a result, the time spent benchmarking with collections of 100 elements will likely offset any potential advantage.

[2.] The approach based on `@SVector` comes with certain caveats. For instance, it doesn't support defining SVectors based fromlocal variables. In particular, it precludes the use of `eachindex(x)` within an array comprehension, unless `x` is a global variable.

10a. Overview and Goals

Martin Alfaro
PhD in Economics

The previous chapters established the foundations of performance optimization, focusing on strategies that apply broadly: type stability and memory allocation. This chapter transitions to **specialized optimization techniques**, by concentrating on **parallelization strategies**. The current chapter will be about parallelization within a single core, referred to as SIMD (Single Instruction, Multiple Data) or vectorization. The next chapter will extend the discussion to parallelization across multiple cores of a machine, known as multithreading.

While these two chapters will center on parallelization, there are some general lessons to carry forward. First, **advanced optimizations almost always involve trade-offs**. Once type stability is ensured and memory allocations are reduced, further speed gains almost always require a compromise. They may involve *reduced precision* (accepting less accurate results for faster computation), *diminished safety* (bypassing safeguards that prevent errors), or *limited generality* (writing code that is highly specific to one problem). This is precisely why such techniques aren't applied by default in Julia, which prioritizes correctness and safety above all.

The second lesson is **the use of macros to implement optimizations**. When the demand for performance justifies these trade-offs, macros provide an elegant way to implement them. By prefixing a code block with `@` and the macro name, developers can execute an operation using an optimized strategy. This makes advanced optimization highly accessible, letting users apply powerful techniques without requiring implementation knowledge.

The roadmap of the chapter is as follows:

Section 10b explains how to use macros. Their application goes beyond parallelization. Formally, macros take code blocks as input and transform them into new code blocks. In this way, macros are capable of implementing complex algorithms, but also of simplifying repetitive expressions and automating tasks.

In Section 10c, we start to explore vectorization by explaining conceptually what SIMD does. Basically, SIMD allows a single core to perform the same computation simultaneously across multiple data elements.

Importantly, not all operations are suitable for SIMD, whose implementation requires specific code structure. The subsequent sections establish the conditions necessary for efficient implementation:

- Section 10d: iterations must be independent. In other words, the computation in one iteration can't depend on another. The only exception is reductions, whose dependencies are handled automatically by SIMD.

- Section 10e: SIMD requires contiguous memory and unit strides (sequential access). Although non-contiguous layouts can sometimes be adapted, the cost of transforming the data may outweigh the benefits. A practical implication is that copies of slices, which guarantee contiguous blocks of memory, can outperform views, which avoid allocations but may involve non-contiguous access.
- Section 10f: SIMD requires branchless code. Importantly, branches aren't limited to explicit conditional statements, such as those involving `if`. Other operations, such as type instabilities and index checks, can introduce hidden branches through their internal computation.

Finally, Section 10g considers the package `LoopVectorization` and its macro `@turbo`. This implements more aggressive SIMD optimizations than Julia's default approach. However, such aggressive optimization can result in correctness issues if misapplied.

10b. Macros as a Means for Optimizations

Martin Alfaro
PhD in Economics

INTRODUCTION

Customized approaches often have an edge over general-purpose built-in solutions, as they can tackle the unique challenges of a given scenario. However, the complexity of specialized techniques often deters their adoption among practitioners, who may lack the necessary expertise to implement them.

Macros offer a practical solution to bridge this gap, making specialized computational approaches more accessible to users. They're particularly well-suited for this purpose, due to their ability to take entire code blocks as inputs and transform them into an optimized execution strategy. In this way, practitioners benefit from specialized algorithms, without having to write the implementation themselves.

In the upcoming sections, the role of macros in boosting performance will be central. By leveraging them, we'll be able to effectively separate the benefits provided by an algorithm from its actual implementation. This decoupling will let us shift our focus from the nitty-gritty details of how to write algorithms to the more practical question of when to apply them. The current section in particular will concentrate on the procedure for applying macros.

USES OF MACROS

Macros can be easily identified in code by the `@` symbol preceding their name. They resemble functions, in that they take input and produce output. Their primary difference lies in what they operate on and what they can return.

Specifically, *functions operate on values*: they take evaluated expressions as arguments and return a final computed value. In contrast, *macros operate on code*: they take expressions as input and return a new transformed expression, which is then compiled and executed in place of the original macro call. This unique feature enables macros to handle tasks that functions can't. Two notable applications of macro are worth mentioning.

First, macros can be used **to simplify code**. By automating repetitive tasks and eliminating boilerplate, macros can enhance code readability and maintainability. For instance, suppose a function requires multiple slices of `x` to be converted into views. Without macros, this would involve repeatedly writing `view(x, <indices>)` for each slice, resulting in verbose and error-prone code. Instead, prepending the function definition with `@views` will automatically handle all the slice conversions for us. This is demonstrated below.

NO MACRO

```
x = rand(1_000)

function foo(x)
    x1 = view(x, x .> 0.7)
    x2 = view(x, x .< 0.5)
    x3 = view(x, 1:500)
    x4 = view(x, 501:1_000)

    x1, x2, x3, x4
end
```

MACRO @VIEWS

```
x = rand(1_000)

@views function foo(x)
    x1 = x[x .> 0.7]
    x2 = x[x .< 0.5]
    x3 = x[1:500]
    x4 = x[501:1_000]

    x1, x2, x3, x4
end
```

A second application of macros is **to modify how operations are computed**, which is the focus of the current section. This lets developers bundle sophisticated optimization techniques, making advanced solutions widely accessible. As a result, users unfamiliar with a method's underlying complexity can focus on choosing the most suitable computational approach, rather than grappling with the implementation details.

While macros are powerful tools, they're not without their limitations. Their black-box nature means that **misuse of macros can lead to unexpected results or compromise computational safety**. That's why it's crucial to identify the right scenarios for each macro. Although this requires some upfront work, it's considerably less demanding than implementing the functionality from scratch.

MACROS APPLIED IN FOR-LOOPS

One distinctive feature of Julia is its ability to execute for-loops with exceptional speed. In fact, carefully optimized for-loops can reach peak performance within the language. This efficiency stems from the versatility of for-loops, which lets users fine-tune them for their specific needs. As a result, it's no surprise that one prominent application of macros is to customize how for-loops are computed.

To illustrate this usage, let's consider the `@inbounds` macro. To understand what this macro accomplishes, we must first understand how for-loops behave in Julia. By default, the language implements **bounds checking**: when an element `x[i]` is accessed during the *i*-th iteration, Julia

verifies that *i* falls within the valid range of indices for `x`. This built-in mechanism safeguards against errors and security issues caused by out-of-bounds access.

While bounds checking prevents bugs, it comes at a performance cost: the additional checks not only introduce computational overhead, but also limit the compiler's ability to implement certain optimizations. In situations where iterations are guaranteed to stay within an array's bounds, these safety checks become redundant. Consequently, we can safely boost performance by disabling bounds checking with the `@inbounds` macro.

> **Trade-Offs Entailed by @inbounds**
>
> The `@inbounds` macro perfectly illustrates both the power and risks associated with macro usage. When applied judiciously, it can yield substantial performance gains, especially when multiple slices are involved.
>
> Despite this, disabling bounds checking simultaneously renders code unsafe: it increases the risk of crashes and silent errors, additionally creating security vulnerabilities. In this context, `@inbounds` shifts the responsibility of applying the macro onto the user, who must be absolutely certain that the iteration range lies within the arrays' bounds.

@INBOUNDS AS AN EXAMPLE

A macro that modifies the entire structure of the for-loop requires its inclusion at the beginning of the for-loop. For instance, to disable bounds checking for every index accessed within a for-loop, we need to prepend the for-loop with `@inbounds`.

@INBOUNDS PREPENDED TO FOR-LOOP

```
x = rand(1_000)

function foo(x)
    output = 0.0

    @inbounds for i in eachindex(x)
                  a      = log(x[i])
                  b      = exp(x[i])
                  output += a / b
    end

    return output
end
```

```
julia> @btime foo($x)
  5.825 µs (0 allocations: 0 bytes)
```

> **Alternative Application of @inbounds**

We can alternatively apply `@inbounds` individually to any specific line within the for-loop. This possibility is specific to macros like `@inbounds`, which aren't exclusive to for-loops. Rather, it can be applied to any expression that access elements of a collection.

@INBOUNDS INDIVIDUALLY ADDED

```
x = rand(1_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        @inbounds a      = log(x[i])
        @inbounds b      = exp(x[i])
                  output += a / b
    end

    return output
end
```

```
julia> @btime foo($x)
  5.940 µs (0 allocations: 0 bytes)
```

The performance advantages of `@inbounds` come not only from eliminating bounds checking itself, but also from giving the compiler more freedom to implement additional optimizations. This is because bounds checking is essentially a conditional statement, where the iteration is executed only if all indices are within range. As we'll see in the next sections, conditional statements limit the compiler's ability to apply the so-called SIMD instructions, which are a form of parallelism within a single core.

MACROS COULD BE APPLIED AUTOMATICALLY OR BE DISREGARDED BY THE COMPILER

The influence of a macro on code execution isn't always predictable. Because the compiler has the final say on the optimization strategy, the macro might actually have no impact at all. This would occur if the compiler is already applying the optimization suggested, or if it determines that the macro's recommendation is unhelpful and simply ignores it. In practice, you'll notice that a macro is inconsequential when execution time remains essentially unchanged after its introduction. [1] Scenarios like this can arise with `@inbounds`, as we show below.

REDUNDANT MACROS

The compiler could determine on its own that a for-loop is safe and therefore disable bounds check. In those cases, `@inbounds` becomes redundant. This behavior typically occurs in simple cases, such as when iterating over a single collection `x` using `eachindex(x)`.

NO MACRO

```
x = rand(1_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.206 µs (0 allocations: 0 bytes)
```

ADDING @INBOUNDS

```
x = rand(1_000)

function foo(x)
    output = 0.0

    @inbounds for i in eachindex(x)
        output += log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.266 µs (0 allocations: 0 bytes)
```

DISREGARDED MACRO

A macro can also serve as a lightweight hint that the compiler is free to disregard. A user might introduce a macro to indicate that the conditions for a particular optimization are met, giving the compiler permission to consider more aggressive strategies. The compiler then analyzes the surrounding code, evaluates whether the suggested approach is actually advantageous, and implements it only when it expects a performance gain. If its analysis suggests that the optimization would be counterproductive, the compiler simply discards the hint. In this role, macros guide the compiler toward potential optimizations, without imposing mandatory directives.

A prime example of a macro that plays this advisory role is `@simd`. This macro suggests the application of SIMD instructions, an optimizaiton that will be explored in upcoming sections. When `@simd` is added to a for-loop, nonetheless, the compiler retains full autonomy in deciding whether to implement it.

In the example below, the compiler concludes that SIMD would likely degrade performance, thus ignoring its application. The decision is reflected by the execution time, which remain the same with and without the `@simd` annotation.[2]

WITHOUT @SIMD

```
x = rand(2_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = if (200_000 > i > 100_000)
                        x[i] * 1.1
                    else
                        x[i] * 1.2
                    end
    end

    return output
end
```

```
julia> @btime foo($x)
  1.128 ms (3 allocations: 15.259 MiB)
```

WITH @SIMD

```
x = rand(2_000_000)

function foo(x)
    output = similar(x)

    @simd for i in eachindex(x)
        output[i] = if (200_000 > i > 100_000)
                        x[i] * 1.1
                    else
                        x[i] * 1.2
                    end
    end

    return output
end
```

```
julia> @btime foo($x)
  1.415 ms (3 allocations: 15.259 MiB)
```

FOOTNOTES

[1.] One can verify that macros are indeed ignored by examining the generated machine code. Because inspecting machine code is beyond the scope of this book, we'll instead use the runtime behavior as an indicator.

[2.] It's possible to confirm that the generated machine code is identical by inspecting the compiler's implemented code.

10c. Introduction to SIMD

Martin Alfaro
PhD in Economics

INTRODUCTION

Single Instruction, Multiple Data (SIMD) is an optimization technique widely embraced in modern CPU architectures. At its core, SIMD allows a single CPU instruction to process multiple data points concurrently, rather than sequentially processing them one by one. This parallel approach can yield substantial performance gains, especially for workloads involving simple identical calculations repeated across multiple data elements. [1]

SIMD is a form of parallelism within a single CPU core. By operating on vectors rather than individual elements, SIMD instructions can execute the same operation on multiple data points simultaneously. This is why the process of applying SIMD is often referred to as **vectorization**.

To illustrate, consider a computation involving four separate addition operations. Without SIMD, the computer would need to execute four distinct instructions, one for each addition. Instead, SIMD makes it possible to bundle the four additions into a single instruction, allowing the CPU to process them all at once. In an ideal scenario, the time required to complete four additions with SIMD would be the same as completing one addition without it.

Throughout the different sections, we'll cover two approaches for implementing SIMD instructions.

- Julia's native capabilities.
- The package `LoopVectorization`.

This section will exclusively concentrate on the built-in tools for applying SIMD. In particular, we'll explore the conditions that trigger automatic vectorization. The next section we'll introduce the `@simd` macro, which lets the user manually suggest SIMD in for-loops. The discussion of `LoopVectorization` will be saved for later sections. This package implements more aggressive optimizations, but can also introduce bugs if misused.

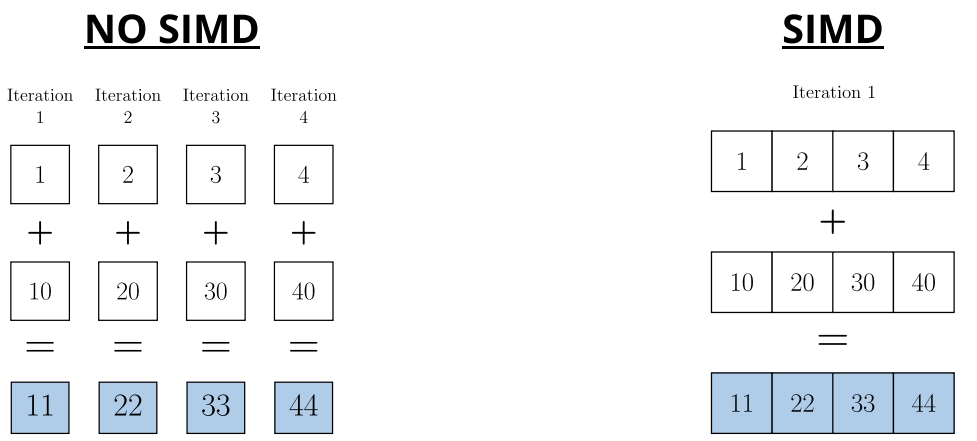
WHAT IS SIMD?

SIMD is a form of data-level parallelism within a single processor core, where the same operation is applied to multiple data elements at once. It's particularly effective for basic arithmetic operations, such as addition and multiplication. Given the nature of these operations, it's unsurprising that one of SIMD's primary applications is in linear algebra, where operations like vector addition or matrix multiplication involve performing identical arithmetic steps across many elements.

At the heart of SIMD lies the process of vectorization, where data is split into sub-vectors that can be processed as single units. To facilitate this operation, modern processors usually include dedicated SIMD registers to support this execution model. In particular, contemporary processors typically feature 256-bit registers for vectorized operations, which are wide enough to hold four `Float64` or `Int64` values at once.

To illustrate the workings of SIMD, consider adding two vectors, `x = [1, 2, 3, 4]` and `y = [10, 20, 30, 40]`. In traditional scalar execution, the processor would perform the operation `x + y` in four separate steps, one for each pair of numbers: `1+10`, `2+20`, `3+30`, `4+40`. Each addition requires its own instruction, so that the final output `[11, 22, 33, 44]` would take four sequential steps.

SIMD changes this dynamic entirely. Instead of issuing four independent additions, the processor loads the four elements of `x` and the four elements of `y` into a 256-bit SIMD register. Once the data is packed, a single vector-addition instruction triggers all four additions simultaneously. Internally, the hardware views the register as a collection of lanes, with each lane holding one element of the vector. The instruction applies the same operation to every lane in parallel, collapsing what would've been four scalar instructions into one. This ability to collapse multiple scalar operations into one instruction is what gives SIMD its substantial performance advantage in data-parallel workloads.



For larger vectors that exceed the width of a single register, the process remains fundamentally the same. The only difference is that processor first partitions the input into consecutive sub-vectors that fit the register size. Then, it performs the vectorized operation on each sub-vector, continuing until all elements have been processed.

BROADCASTING AND FOR-LOOPS

There are **two categories of operations that can potentially benefit from SIMD instructions: for-loops and broadcasting**. In fact, we could've explicitly mentioned only the former, since broadcasting is essentially syntactic sugar for for-loops.

In its built-in form, the decision to apply SIMD instructions is nonetheless made entirely by the compiler. Its decision relies on a set of heuristics to determine when vectorization will pay off.

For broadcasting in particular, the compiler implements SIMD automatically, without any user intervention. Instead, the automatic application of SIMD in for-loops is only restricted to a few simple cases, requiring the user to explicitly indicate that SIMD should be considered.

This is why the upcoming sections will identify conditions under which SIMD instructions should be suggested to the compiler. If these conditions aren't met, SIMD will substantially reduce its effectiveness or directly become infeasible. For such cases, we'll also provide guidance on how to structure computations so that SIMD remains efficient, despite not being well-suited for its application.

SIMD IN BROADCASTING

As we indicated, SIMD is automatically applied in broadcasting. To devote our full attention in subsequent sections to for-loops, we conclude this section by demonstrating this behavior in broadcasting.

The following example compares the same computation implemented via a for-loop and via broadcasting. While broadcasting automatically takes advantage of SIMD, the for-loop version doesn't.

FOR-LOOP (NO SIMD)

```
x      = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = 2 / x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  788.201 μs (3 allocations: 7.629 MiB)
```

BROADCASTING (SIMD)

```
x      = rand(1_000_000)

foo(x) = 2 ./ x
```

```
julia> @btime foo($x)
  411.572 μs (3 allocations: 7.629 MiB)
```

FOOTNOTES

[1.] SIMD isn't exclusive to CPUs. GPUs also take advantage of it. The architecture of GPUs is a natural fit for SIMD, as they were designed for parallel processing of certain simple identical operations.

10d. SIMD: Independence of Iterations

Martin Alfaro
PhD in Economics

INTRODUCTION

Broadcasting in Julia automatically leverages SIMD instructions. In contrast, whether and when for-loops will be vectorized is more subtle. Furthermore, the heuristics of the compiler, while powerful, aren't without flaws. Indeed, it's entirely possible that SIMD is implemented when it actually degrades performance, or not applied when it would've been advantageous. To address these possibilities, Julia provides the `@simd` macro to suggest the application of SIMD, giving developers a more granular control over the optimization process.

Using SIMD effectively requires understanding the conditions under which vectorization is both valid and worthwhile. Failing to meet these criteria can render SIMD infeasible or necessitate code adaptations that will slow down computations.

The ideal conditions for leveraging SIMD instructions are:

- **Independence of Iterations**: The operations in each iteration should not depend on the output of previous iterations. The only exception is **reductions**, which are specifically handled to ensure their feasibility.
- **Unit Strides**: the elements of each collection must be accessed sequentially.
- **No Conditional Statements**: the for-loop body should consist solely of straight-line code, avoiding branches or other conditional logic.

In this and upcoming sections, we'll elaborate on each of these items. Additionally, we'll provide guidance on how to address scenarios not conforming to them. This section in particular exclusively focuses on the independence of iterations.

> ⚠ **Warning!** - Determining Whether SIMD Has Been Implemented
>
> Assessing whether SIMD instructions have been implemented requires inspecting the compiled code. Due to the complexity of this approach, we'll instead rely on execution times as a practical indicator.

REMARKS ABOUT @SIMD IN FOR-LOOPS

Recall from the previous section that macros don't work as absolute commands. Rather, they function as optimization hints that the compiler may adopt, refine, or discard depending on the context.

In many cases, a macro simply suggests an optimization the compiler was already planning to apply, or it recommends a transformation that the compiler ultimately rejects as unsafe or inefficient. The compiler always retains the final authority over which optimizations are actually performed. In this context, adding `@simd` to a for-loop is far from a guarantee that SIMD will actually be implemented.

The challenge runs deeper still, since it's notoriously difficult to predict whether SIMD instructions are beneficial. Different CPU architectures provide different SIMD instruction sets and capabilities, and these variations can dramatically shift the performance gains you'd expect. As a result, the effectiveness of SIMD tend to vary significantly across hardware platforms. [1]

Despite this complexity, structuring operations in certain ways can improve the likelihood of implementing SIMD beneficially. By identifying these conditions, we'll be able to write code that's more amenable to SIMD optimization. It's worth remarking, though, that **the recommendations we'll present should be interpreted as general principles, rather than absolute rules**. Given ow intricate SIMD behavior can be, benchmarking remains the most reliable tool to corroborate any performance gains.

> **Safety of SIMD**
>
> Technically speaking, SIMD is a form of parallelization. We'll see in subsequent sections that parallelization can compromise code safety and cause serious errors when misused. The `@simd` macro doesn't carry these risks, since it's been designed to apply only when it's safe to do so. In particular, the compiler will disregard SIMD if the conditions for its safe application aren't met.

INDEPENDENCE OF ITERATIONS

A condition to apply SIMD effectively in a for-loop is that its iterations are independent. This means that none of them should depend on the results of previous iterations or affect the results of subsequent ones. When this independence holds, each iteration can be executed in parallel seamlessly.

A typical scenario where iterations are independent is when transforming each element of a vector `x` using some function `foo`. In the following, we illustrate this case via a polynomial transformation of `x`. The implementation will be done via a for-loop, considering variants with and without SIMD. We'll also compare these approaches with broadcasting, which applies SIMD automatically.

Importantly, applying `@simd` in for-loops requires adding the `@inbounds` macro. Otherwise, the compiler may decide that vectorization isn't beneficial. The reason for this will be explored in a subsequent section. Basically, bounds checking introduces conditional branches, which can obstruct or entirely prevent SIMD vectorization.

FOR-LOOP (NO SIMD)

```
x      = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = x[i] / 2 + x[i]^2 / 3
    end

    return output
end
```

```
julia> @btime foo($x)
  1.688 ms (3 allocations: 7.629 MiB)
```

FOR-LOOP (WITH SIMD)

```
x      = rand(1_000_000)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = x[i] / 2 + x[i]^2 / 3
    end

    return output
end
```

```
julia> @btime foo($x)
  873.697 μs (3 allocations: 7.629 MiB)
```

BROADCASTING (AUTOMATIC SIMD)

```
x      = rand(1_000_000)

foo(x) = @. x / 2 + x^2 / 3
```

```
julia> @btime foo($x)
  3.658 ms (3 allocations: 7.629 MiB)
```

A SPECIAL CASE OF DEPENDENCE: REDUCTIONS

While SIMD generally requires for-loop iterations to be independent, **reductions** represent a notable exception to this rule. Even though each iteration of a reduction depends on the partial result produced by the previous one, SIMD instructions have been designed so that they can be applied safely to reductions.

In Julia, the application of SIMD in reductions differs depending on the data type. For integer reductions, SIMD optimization is implemented automatically. Instead, floating-point reductions require explicit intervention by annotating the for-loop with the `@simd` macro.

To illustrate this behavior, let's first consider the case of integer reductions. Because SIMD is applied automatically, you'll find that execution times remain essentially unchanged regardless of whether you explicitly include the `@simd` macro. The following example demonstrates this in practice.

INT64 (AUTOMATIC SIMD)

```
x = rand(1:10, 10_000_000)   # random integers between 1 and 10

function foo(x)
    output = 0

    for a in x
        output += a
    end

    return output
end
```

```
julia> @btime foo($x)
  3.845 ms (0 allocations: 0 bytes)
```

INT64 (ADDING @SIMD)

```
x = rand(1:10, 10_000_000)   # random integers between 1 and 10

function foo(x)
    output = 0

    @simd for a in x
        output += a
    end

    return output
end
```

```
julia> @btime foo($x)
  4.508 ms (0 allocations: 0 bytes)
```

The behavior contrasts with a sum reduction consisting of floating-point operations, as shown below.

FLOAT64 (NO AUTOMATIC SIMD)

```
x = rand(10_000_000)

function foo(x)
    output = 0.0

    for a in x
        output += a
    end

    return output
end
```

```
julia> @btime foo($x)
  9.660 ms (0 allocations: 0 bytes)
```

FLOAT64 (ADDING @SIMD)

```
x = rand(10_000_000)

function foo(x)
    output = 0.0

    @simd for a in x
        output += a
    end

    return output
end
```

```
julia> @btime foo($x)
  4.356 ms (0 allocations: 0 bytes)
```

Why Are Floating Points Treated Differently?

Addition of floating-point numbers doesn't obey associativity, unlike what occurs with integers: due to the inherent imprecision of floating-point arithmetic, `(x+y)+z` may differ from `x+(y+z)`. This loss of associativity illustrates a broader point: floating-point numbers are finite-precision approximations, and therefore they don't always satisfy the mathematical properties that hold for real numbers.

The following code snippets illustrate this point.

OPTION 1

```
x = 0.1 + (0.2 + 0.3)
```

```
julia> x
0.6
```

OPTION 2

```
x = (0.1 + 0.2) + 0.3
```

```
julia> x
0.6000000000000001
```

By instructing the compiler to disregard the non-associativity of floating-point arithmetic, SIMD instructions can improve performance by reordering terms. However, this approach assumes that the operations don't rely on a specific order of execution.

FOOTNOTES

[1] For instance, x86 architectures (Intel and AMD processors) support SSE (Streaming SIMD Extensions) and AVX (Advanced Vector Extensions). Each of them comprises several variants that offer different vector widths and operations.

10e. SIMD: Contiguous Access and Unit Strides

Martin Alfaro
PhD in Economics

INTRODUCTION

This section examines performance differences between copies and views in the context of SIMD instructions. While views may initially appear advantageous due to their ability to **avoid memory allocations**, this section demonstrates that copies possess a memory layout better aligned with SIMD requirements.

To understand why this occurs, let's first revisit how SIMD improves performance in the first place. SIMD accelerates computation by applying the same operation simultaneously to multiple data elements. This is made possible by specialized vector registers in the CPU, which can hold several values at once (e.g., 4 floating-point numbers or 4 integers). With data packed into these registers, operations like additions or multiplications can complete with a single instruction instead of many.

For SIMD to fully leverage this vector-based processing, **data must adhere to specific rules regarding memory organization**. Specifically, there are two fundamental conditions: contiguous memory layout and unit-stride access.

- **Contiguous Memory Layout**: data elements must reside in adjacent memory addresses, one after another, with no gaps between them. Freshly allocated arrays automatically meet this requirement, enabling entire segments to be loaded directly into vector registers. In contrast, array views don't guarantee contiguity. By referencing the original data structure, views could result in highly irregular memory access patterns depending on the elements selected.

- **Unit Strides**: strides refer to the step size between consecutive memory accesses. Unit strides mean in particular that elements are accessed sequentially. For example, iterating through a freshly allocated vector `x` using `eachindex(x)` ensures unit stride: each access moves to the next adjacent address in memory. This contrasts with non-unit strides, such as accessing every other element with `1:2:length(x)`, or unpredictable accesses patterns resulting from an index list like `[1, 5, 3, 4]`.

Because SIMD performs best when both conditions hold, choosing between views and copies involves trade-offs. On one hand, using views reduces memory allocations, but at the expense of making SIMD less effective if views violate contiguity or unit stride. On the other hand, copies create contiguous arrays amenable to SIMD, but at the cost of increased memory usage. This section examines how these trade-offs play out in practice.

REVIEW OF INDEXING APPROACHES

The performance trade-offs between copies and views are highly dependent on the slicing method employed. For this reason, we begin with a brief overview of common slicing techniques: vector indexing, ranges, and Boolean indexing. We illustrate each below.

VECTOR INDEXING

```
x        = [10, 20, 30]

indices  = [3, 2, 1]
elements = x[indices]
```

```
julia> indices
3-element Vector{Int64}:
 3
 2
 1
julia> elements
3-element Vector{Int64}:
 30
 20
 10
```

RANGES

```
x        = [2, 3, 4, 5, 6]

indices_1 = 1:length(x)         # unit strides, equivalent to 1:1:length(x)
indices_2 = 1:2:length(x)       # strides two
```

```
julia> collect(indices_1)
5-element Vector{Int64}:
 1
 2
 3
 4
 5
julia> collect(indices_2)
3-element Vector{Int64}:
 1
 3
 5
```

BOOLEAN INDEXING

```
x          = [20, 10, 30]

indices    = x .> 15
elements   = x[indices]
```

```
julia> indices
3-element BitVector:
 1
 0
 1
julia> elements
2-element Vector{Int64}:
 20
 30
```

Vector Indexing selects elements by explicitly listing their indices. This approach offers the greatest flexibility in choosing arbitrary elements. However, it can also result in memory access patterns that are highly non-contiguous.

Ranges are a structured form of indexing that selects elements according to a given stride. Their general syntax is `<first index>:<stride>:<last index>`, where omitting `<stride>` defaults to a step size of 1. Ranges with unit stride maintain contiguous access patterns, while larger strides produce predictable but non-contiguous access patterns.

Finally, **Boolean indexing** uses a Boolean vector with `true` entries indicating the elements to retain. This technique is commonly used when broadcasting conditions to filter data. Because the resulting pattern of selected elements often lacks regularity, Boolean indexing typically leads to irregular and fragmented memory access.

BENEFITS OF SEQUENTIAL ACCESS

In a previous section, we highlighted the benefits of using views over copies when working with slices: by maintaining references to the original data, views avoid memory allocation. Yet, we also remarked that copies could outperform views in some scenarios. We're now in a position to explain in more depth why this occurs.

Creating a copy of some data structure involves allocating the information in a new contiguous block of memory. This layout ensures that elements sit next to each other, thus offering two key advantages: faster element retrieval and a more effective use of SIMD instructions. Views can't guarantee this property. If the referenced elements are scattered throughout memory, views will necessarily introduce irregular access patterns.

An analogy may help clarify this distinction. Imagine retrieving books from a library. If every book you need is neatly arranged on a single shelf, collecting them is straightforward: you move once, grab each, and proceed. This mirrors contiguous memory access. Conversely, if the books are dispersed

across different floors and sections, each retrieval demands additional time and effort. This is akin to non-contiguous access.

The analogy extends further by introducing a cart that lets you carry several books at once. The cart plays the role of vector registers, which enable SIMD instructions to process multiple adjacent data elements in a single operation.

ILLUSTRATING EACH BENEFIT IN ISOLATION

The following examples illustrate the potential advantages of copies compared to views. They're all based on the computation of a reduction operation.

To isolate the benefits from copying, we construct the slices outside the benchmarked function. In this way, performance differences aren't influenced by the memory allocations associated with copying.

To begin with, let's consider access to elements that follow a regular pattern but don't exhibit unit stride.

VIEW (NO SIMD)

```
x = rand(5_000_000)
y = @view x[1:2:length(x)]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  1.540 ms (0 allocations: 0 bytes)
```

VIEW (SIMD)

```
x = rand(5_000_000)
y = @view x[1:2:length(x)]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  1.831 ms (0 allocations: 0 bytes)
```

COPY (NO SIMD)

```
x = rand(5_000_000)
y = x[1:2:length(x)]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  1.094 ms (0 allocations: 0 bytes)
```

COPY (SIMD)

```
x = rand(5_000_000)
y = x[1:2:length(x)]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  644.124 μs (0 allocations: 0 bytes)
```

The example brings several points into focus. First, when comparing the two view-based variants, non-unit strides prevent the compiler from applying SIMD efficiently. As a result, the extra work required to rearrange elements for vectorization ends up slowing down the computation.

Second, when comparing the view and copy implementations without SIMD, the copy performs better. Since neither version benefits from SIMD, the performance gain stems entirely from contiguous memory access.

Finally, when the computation is based on a copy and applies SIMD instructions, the implementation combines the benefits of contiguous memory access and vectorization. This yields the fastest execution among all four variants.

When the access of elements doesn't follow any predictable pattern, the benefits of copying become even more pronounced. To illustrate, let's access elements of some vector `x` via the `sortperm` function. This returns the indices that would sort the vector. For instance, given `x = [5, 4, 6]`,

executing `sort(x)` returns `[4, 5, 6]`, while `sortperm(x)` provides `[2, 1, 3]`. Assuming that `x` hasn't been ordered previously, views that access elements according to the indices of `sortperm(x)` will result in unpredictable memory access pattern.

VIEW (NO SIMD)

```
x       = rand(5_000_000)

indices = sortperm(x)
y       = @view x[indices]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  29.691 ms (0 allocations: 0 bytes)
```

VIEW (SIMD)

```
x       = rand(5_000_000)

indices = sortperm(x)
y       = @view x[indices]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  31.315 ms (0 allocations: 0 bytes)
```

COPY (NO SIMD)

```
x       = rand(5_000_000)

indices = sortperm(x)
y       = x[indices]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  2.276 ms (0 allocations: 0 bytes)
```

COPY (SIMD)

```
x       = rand(5_000_000)

indices = sortperm(x)
y       = x[indices]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  1.634 ms (0 allocations: 0 bytes)
```

A similar conclusion can be obtained when slices are created with Boolean indexing.

VIEW (NO SIMD)

```
x       = rand(5_000_000)

indices = x .> 0.5
y       = @view x[indices]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  2.465 ms (0 allocations: 0 bytes)
```

VIEW (SIMD)

```
x       = rand(5_000_000)

indices = x .> 0.5
y       = @view x[indices]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  2.326 ms (0 allocations: 0 bytes)
```

COPY (NO SIMD)

```
x       = rand(5_000_000)

indices = x .> 0.5
y       = x[indices]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  1.095 ms (0 allocations: 0 bytes)
```

COPY (SIMD)

```
x       = rand(5_000_000)

indices = x .> 0.5
y       = x[indices]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  622.719 µs (0 allocations: 0 bytes)
```

COPIES VS VIEWS: TOTAL EFFECTS

The previous examples defined slices outside the benchmarked functions, thereby avoiding the memory allocations associated with copying. This was done intentionally to isolate the benefits of sequential memory access and SIMD.

In practice, however, the choice between copies and views must ultimately account for the overhead of memory allocations. Once this effect is incorporated, the trade-off becomes apparent, and whether copies or views yield better performance is highly case-specific. In the end, benchmarking remains the only reliable way to determine which approach delivers superior performance in a given context.

In what follows, we present cases that account for all these effects simultaneously. We begin with an example where views prove more performant. This scenario is characterized by significant memory allocation costs. Instead, there are only marginal benefits from SIMD, due to the simplicity of the operation being executed.

VIEW

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = @view x[indices]
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  33.449 ms (0 allocations: 0 bytes)
```

COPY

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = x[indices]
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  49.233 ms (2 allocations: 38.147 MiB)
```

Instead, the following operation is more computationally intensive and benefits substantially from SIMD instructions. Thus, a copy outperforms a view.

VIEW

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = @view x[indices]
    output = 0.0

    @simd for a in y
        output += a^(3/2)
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  407.670 ms (0 allocations: 0 bytes)
```

COPY

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = x[indices]
    output = 0.0

    @simd for a in y
        output += a^(3/2)
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  159.115 ms (2 allocations: 38.147 MiB)
```

CHOOSING WITHOUT BENCHMARKING

Certain patterns allow us to predict the faster approach, without the need of benchmarking the approach.

One scenario where views always outperform copies is when referencing sequential elements. These scenarios call for the use of view, as they provide the same benefits as copies, but without incurring memory allocations.

VIEW (NO SIMD)

```
x       = rand(1_000_000)
indices = 1:length(x)

function foo(x, indices)
    y      = @view x[indices]
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  481.124 µs (0 allocations: 0 bytes)
```

VIEW (WITH SIMD)

```
x       = rand(1_000_000)
indices = 1:length(x)

function foo(x, indices)
    y      = @view x[indices]
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  121.409 µs (0 allocations: 0 bytes)
```

COPY (WITH SIMD)

```
x       = rand(1_000_000)
indices = 1:length(x)

function foo(x, indices)
    y      = x[indices]
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  677.714 µs (2 allocations: 7.629 MiB)
```

In contrast, a common scenario where copies tend to outpace views is when performing multiple operations on the same slice. In this case, the cost of additional memory allocations is usually offset by the speed gains from contiguous memory access and SIMD instructions.

VIEW (NO SIMD)

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = @view x[indices]
    output1, output2, output3 = (0.0 for _ in 1:3)

    for a in y
        output1 += a^(3/2)
        output2 += a / 3
        output3 += a * 2.5
    end

    return output1, output2, output3
end
```

```
julia> @btime foo($x, $indices)
  404.078 ms (0 allocations: 0 bytes)
```

VIEW (WITH SIMD)

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = @view x[indices]
    output1, output2, output3 = (0.0 for _ in 1:3)

    @simd for a in y
        output1 += a^(3/2)
        output2 += a / 3
        output3 += a * 2.5
    end

    return output1, output2, output3
end
```

```
julia> @btime foo($x, $indices)
  422.823 ms (0 allocations: 0 bytes)
```

COPY (WITH SIMD)

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = x[indices]
    output1, output2, output3 = (0.0 for _ in 1:3)

    @simd for a in y
        output1 += a^(3/2)
        output2 += a / 3
        output3 += a * 2.5
    end

    return output1, output2, output3
end
```

```
julia> @btime foo($x, $indices)
  155.528 ms (2 allocations: 38.147 MiB)
```

10f. SIMD: Branchless Code

Martin Alfaro
PhD in Economics

INTRODUCTION

SIMD accelerates workloads by applying a single instruction to many elements at once. Its effectiveness critically hinges on all elements following the same control flow. Yet, this pattern can be disrupted when conditionals are introduced, either explicitly in the source code or implicitly by the compiler. While the compiler attempts to mitigate the issue by transforming code into SIMD-friendly forms, these adaptations tend to introduce additional work that reduce the net benefit of SIMD.

The current section explores strategies for efficiently applying SIMD in the presence of conditional behavior. In particular, we'll show how to restructure computations to make control-flow branches disappear, allowing the program to preserve a uniform instruction stream.

We'll first examine scenarios in which the compiler introduces branches as a byproduct of its internal computation techniques. In these situations, users may be unaware that conditional control flow has been generated. For this reason, it's essential to recognize when such hidden branches occur.

After this, we consider explicit conditional statements written directly by the user. These conditions are intrinsic to program logic, ensuring that certain computations occur only when specific criteria are met. Because they can't simply be eliminated, we revisit common approaches to expressing conditions and analyze their internal implementation. The aim is to compare their relative strengths and limitations, indicating which ones interact more favorably with SIMD execution.

Finally, we show that conditional statements can often be reformulated as algebraic operations. This transformation removes branching logic and restores SIMD's ability to operate on a uniform instruction stream. In fact, compilers may implement these transformations automatically when it's likely beneficial.

HIDDEN BRANCHING

Certain coding patterns result in **hidden branching**. These are conditional execution paths that the compiler inserts internally to handle the computation, rather than being explicitly written by the programmer.

TYPE INSTABILITY

Type instability is one source of hidden branching. When a function can return values of different types, Julia must generate separate execution paths to handle each possibility. These extra branches remain invisible in the source code, yet they still break the uniform instruction flow that SIMD depends

on. The solution is simply to restore type stability, for which we can apply all the techniques explored in Chapter 8 of this book.

It's worth remarking, though, that type stability should always be a priority, regardless of any SIMD consideration: any attempt to achieve high performance is nearly impossible without it. Considering this, we'll proceed throughout the section assuming that type stability has been guaranteed.

BOUNDS CHECKING

Bounds checking in for-loops represents another source of hidden branching. This safety mechanism, enabled by default, verifies that indices stay within valid limits before executing each iteration.

The verification creates a subtle form of conditional execution, preventing the compiler from treating the loop as a uniform sequence of operations. Consequently, disabling these checks is **a prerequisite for the application of SIMD**. This is implemented by prepending the for-loop with `@inbounds`.

If bound checks remain in place, inserting `@simd` alone will rarely lead the compiler to emit SIMD instructions. In fact, merely adding `@inbounds` may be sufficient to enable SIMD on its own. In those cases, you'll notice that using `@inbounds @simd` rather than `@inbounds` won't yield any additional speedup. Both of these points are demonstrated below. [1]

NO MACROS

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  2.392 ms (3 allocations: 7.629 MiB)
```

@SIMD

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @simd for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  2.384 ms (3 allocations: 7.629 MiB)
```

@INBOUNDS

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @inbounds for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  1.226 ms (3 allocations: 7.629 MiB)
```

@INBOUNDS + @SIMD

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  5.382 ms (3 allocations: 7.629 MiB)
```

Overall, when signaling to the compiler that SIMD is appropriate, for-loops should be annotated with `@inbounds @simd` rather than `@simd` alone.

Broadcasting and For-Loops

Broadcasting in Julia automatically disables bounds checking and applies SIMD instructions. This often makes broadcast versions appear faster than a straightforward for-loop. However, **broadcasting is just compact notation for implementing for-loops**. As a result, a for-loop annotated with `@inbounds @simd` usually achieves a similar performance to its broadcast variant, as demonstrated below.

BROADCASTING

```
x      = rand(1_000_000)
foo(x) = 2 ./ x
```

```
julia> @btime foo($x)
  1.190 ms (3 allocations: 7.629 MiB)
```

FOR-LOOP (NO MACROS)

```
x      = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  2.384 ms (3 allocations: 7.629 MiB)
```

FOR-LOOP (OPTIMIZED WITH MACROS)

```
x      = rand(1_000_000)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  1.191 ms (3 allocations: 7.629 MiB)
```

APPROACHES TO CONDITIONAL STATEMENTS

Certain workflows require conditional statements, making them an integral part of a program's logical flow. Because they can't be avoided, it's important to consider the most effective way to express them.

We've already examined several ways to introduce **conditional statements**. In the context of SIMD, a relevant distinction among these approaches is whether they evaluate conditional statements **eagerly or lazily**.

To illustrate, let's consider computing `1 + 1` only when certain condition `c` is met. A lazy strategy checks `c` first, performing the addition only if the condition is true. By contrast, an eager approach performs `1 + 1` immediately, regardless of whether `c` is satisfied. If the condition eventually evaluates to `false`, the result of `1 + 1` is simply discarded.

When a conditional is evaluated just once, a lazy approach is more performant because it avoids unnecessary work. In contrast, when that conditional appears inside a for-loop and is evaluated repeatedly, SIMD can apply the same operation across many elements simultaneously. In these cases, it's often advantageous to compute all branches upfront, selecting the relevant results only afterward. This eager strategy works particularly well when the branches involve inexpensive computations.

In Julia, whether a conditional expression is evaluated lazily or eagerly depends on how the code is written. Next, we'll examine this aspect in more detail.

IFELSE VS IF

The `ifelse` function in Julia follows an eager evaluation strategy, computing all possible outcomes before selecting the one that matches the condition. In contrast, the `if` construct favors lazy computations, where only the branch corresponding to the true condition is executed.

The following example demonstrates the computational difference between `if` and `ifelse`. It uses a reduction operation in which the elements to be summed are contingent on a condition. Because `ifelse` requires an argument for both the true and false cases, we return zero when the condition isn't met.

IF

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        if x[i] > 0.5
            output += x[i]/2
        end
    end

    return output
end
```

```
julia> @btime foo($x)
  1.312 ms (0 allocations: 0 bytes)
```

IF + SIMD

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.5
            output += x[i]/2
        end
    end

    return output
end
```

```
julia> @btime foo($x)
  1.321 ms (0 allocations: 0 bytes)
```

IFELSE

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += ifelse(x[i] > 0.5, x[i]/2, 0)
    end

    return output
end
```

```
julia> @btime foo($x)
  1.221 ms (0 allocations: 0 bytes)
```

IFELSE + SIMD

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(x[i] > 0.5, x[i]/2, 0)
    end

    return output
end
```

```
julia> @btime foo($x)
  349.996 µs (0 allocations: 0 bytes)
```

The example shows that `ifelse` doesn't automatically trigger SIMD execution, even though it uses eager evaluation. This is precisely why the code required the `@inbounds @simd` macros.

It's also worth noting that applying SIMD instructions with `ifelse` doesn't necessarily improve performance. The following example demonstrates this point.

IFELSE

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    for i in eachindex(x)
        output[i] = ifelse(x[i] > 0.5, x[i]/2, 0)
    end
end
```

```
julia> foo!($output,$x)
  56.122 ms (0 allocations: 0 bytes)
```

IFELSE + SIMD

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(x[i] > 0.5, x[i]/2, 0)
    end
end
```

```
julia> foo!($output,$x)
  50.983 ms (0 allocations: 0 bytes)
```

TERNARY OPERATOR

An alternative approach to expressing conditions is given by the ternary operation, whic follows the syntax `<condition> ? <action if true> : <action if false>`. Unlike the previous methods, this form doesn't enforce an eager or a lazy approach. Instead, it relies on compiler heuristics to determine which strategy will likely be faster for the specific application.

To illustrate this behavior, we revisit the same reduction example used earlier, now applying `@inbounds @simd` directly within each construct. In the following, the ternary operator chooses an eager strategy, thus evaluating both expressions before selecting the relevant branch.

IF

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.5
            output += x[i]/2
        end
    end

    return output
end
```

```
julia> foo!($x)
  1.318 ms (0 allocations: 0 bytes)
```

IFELSE

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(x[i]>0.5, x[i]/2, 0)
    end

    return output
end
```

```
julia> foo!($x)
  386.313 μs (0 allocations: 0 bytes)
```

TERNARY

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += x[i]>0.5 ? x[i]/2 : 0
    end

    return output
end
```

```
julia> foo!($x)
  392.661 µs (0 allocations: 0 bytes)
```

On the contrary, the ternary operator opts for a lazy approach in the following example.

IF

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.99
            output += log(x[i])
        end
    end

    return output
end
```

```
julia> foo!($x)
  1.267 ms (0 allocations: 0 bytes)
```

IFELSE

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(x[i] > 0.99, log(x[i]), 0)
    end

    return output
end
```

```
julia> foo!($x)
  11.496 ms (0 allocations: 0 bytes)
```

TERNARY

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += x[i]>0.99 ? log(x[i]) : 0
    end

    return output
end
```

```
julia> foo!($x)
  1.358 ms (0 allocations: 0 bytes)
```

It's worth noting that the method chosen by **the ternary operator isn't foolproof**, possibly choosing a less performant approach. This is illustrated below.

IF

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.5
            output[i] = log(x[i])
        end
    end
end
```

```
julia> foo!($output,$x)
  79.400 ms (0 allocations: 0 bytes)
```

IFELSE

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(x[i] > 0.5, log(x[i]), 0)
    end
end
```

```
julia> foo!($output,$x)
  54.072 ms (0 allocations: 0 bytes)
```

TERNARY

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        output[i] = x[i]>0.5 ? log(x[i]) : 0
    end
end
```

```
julia> foo!($output,$x)
  80.596 ms (0 allocations: 0 bytes)
```

WHEN IS EACH APPROACH BETTER?

As a rule of thumb, conditional statements with **computationally-demanding operations are more likely to benefit from a lazy implementation**. In contrast, **branches comprising simple computations tend to benefit from an eager approach**. These are precisely the heuristics that typically drive the decision of the ternary operator.

To make this concrete, consider a case where only one branch performs a computation, and that computation is straightforward. Under these conditions, an eager evaluation combined with SIMD typically outperforms a lazy alternative. This is precisely the strategy adopted by the ternary operator.

IF

```
x              = rand(1_000_000)
condition(a)   = a > 0.5
computation(a) = a * 2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if condition(x[i])
            output += computation(x[i])
        end
    end

    return output
end
```

```
julia> foo!($x)
  1.296 ms (0 allocations: 0 bytes)
```

IFELSE

```
x             = rand(1_000_000)
condition(a)   = a > 0.5
computation(a) = a * 2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(condition(x[i]), computation(x[i]), 0)
    end

    return output
end
```

```
julia> foo!($x)
  412.153 μs (0 allocations: 0 bytes)
```

TERNARY

```
x             = rand(1_000_000)
condition(a)   = a > 0.5
computation(a) = a * 2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += condition(x[i]) ? computation(x[i]) : 0
    end

    return output
end
```

```
julia> foo!($x)
  312.681 μs (0 allocations: 0 bytes)
```

The situation reverses when the conditional branch involves a computationally intensive operation. Here, a lazy strategy avoids performing expensive work unnecessarily, leading to better performance. Below, we illustrate a scenario like this, where the ternary operator chooses a lazy evaluation.

IF

```
x              = rand(2_000_000)
condition(a)   = a > 0.5
computation(a) = exp(a)/3 - log(a)/2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if condition(x[i])
            output += computation(x[i])
        end
    end

    return output
end
```

```
julia> foo!($x)
  39.390 ms (0 allocations: 0 bytes)
```

IFELSE

```
x              = rand(2_000_000)
condition(a)   = a > 0.5
computation(a) = exp(a)/3 - log(a)/2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(condition(x[i]), computation(x[i]), 0)
    end

    return output
end
```

```
julia> foo!($x)
  40.469 ms (0 allocations: 0 bytes)
```

TERNARY

```
x            = rand(2_000_000)
condition(a)   = a > 0.5
computation(a) = exp(a)/3 - log(a)/2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += condition(x[i]) ? computation(x[i]) : 0
    end

    return output
end
```

```
julia> foo!($x)
  40.580 ms (0 allocations: 0 bytes)
```

VECTORS WITH CONDITIONS

In many situations, you may already have a vector of Boolean conditions defined. This can happen because the condition vector is part of the dataset itself, or because the same conditions will be reused throughout the code, making it worthwhile to compute and store them in advance.

Julia represents such vectors using either `Vector{Bool}` or `BitVector`. The latter is the default output for broadcast expressions like `x .> 0`. While `BitVector` is memory-efficient and can be faster in some contexts, it can also inhibit SIMD optimizations. When this occurs, converting the `BitVector` to a `Vector{Bool}` can improve performance. The example below illustrates this effect: even after accounting for the cost of converting the type, the version using `Vector{Bool}` achieves a shorter execution time.

BITVECTOR

```
x         = rand(1_000_000)
bitvector = x .> 0.5

function foo(x,bitvector)
    output     = similar(x)


    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(bitvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x,$bitvector)
  13.156 ms (3 allocations: 7.629 MiB)
```

VECTOR{BOOL}

```
x         = rand(1_000_000)
bitvector = x .> 0.5

function foo(x,bitvector)
    output     = similar(x)
    boolvector = Vector(bitvector)

    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(boolvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x,$bitvector)
  2.588 ms (5 allocations: 8.583 MiB)
```

No Vector Holding Conditions

When the vector of conditions isn't already available, the conclusions outlined above no longer apply. In those cases, the most efficient strategy to leverage SIMD is to avoid building a condition vector altogether. Instead, you should apply `ifelse` directly to scalar values inside the for-loop. This approach eliminates unnecessary memory allocations, while still allowing the compiler to generate SIMD instructions effectively. The following example demonstrates this point.

BITVECTOR

```
x = rand(1_000_000)

function foo(x)
    output     = similar(x)
    bitvector  = x .> 0.5


    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(bitvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x)
  9.641 ms (7 allocations: 7.749 MiB)
```

VECTOR{BOOL}

```
x = rand(1_000_000)

function foo(x)
    output     = similar(x)
    boolvector = Vector{Bool}(undef,length(x))
        boolvector .= x .> 0.5

    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(boolvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x)
  2.314 ms (5 allocations: 8.583 MiB)
```

NO VECTOR

```
x = rand(1_000_000)

function foo(x)
    output     = similar(x)



    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(x[i]>0.5, x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x)
  1.436 ms (3 allocations: 7.629 MiB)
```

ALGEBRAIC OPERATIONS AS COMPOUND CONDITIONS

Branches can often be eliminated by rewriting logical conditions in terms of simple algebraic identities. By doing so, we're eliminating the hurdle that impedes a direct application of SIMD.

To see how this works, let's first formalize Boolean logic in mathematical terms. Given a set $\{b_i\}_{i=1}^{n}$ where $b_i \in \{0, 1\}$:

- All conditions are satisfied when

$$\prod_{i=1}^{n} c_i = 1$$

- At least one condition is satisfied when

$$1 - \prod_{i=1}^{n} (1 - c_i) = 1$$

These identities translate directly into Boolean expressions. Specifically, given two Boolean scalars `c1` and `c2`:

- `c1 && c2` can be expressed as `Bool(c1 * c2)`,
- `c1 || c2` can be expressed as `Bool(1 - !c1 * !c2)`.

Once this strategy is adopted, control flow can be rewritten in a manner that's friendlier to vectorization. The following example illustrates this approach by replacing branching logic in a for-loop.

&&

```
x = rand(1_000_000)
y = rand(1_000_000)

function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3) && (y[i] < 0.6) && (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  6.708 ms (0 allocations: 0 bytes)
```

&& (ALGEBRAIC)

```
x = rand(1_000_000)
y = rand(1_000_000)

function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3)  * (y[i] < 0.6)  * (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  2.924 ms (0 allocations: 0 bytes)
```

||

```
x = rand(1_000_000)
y = rand(1_000_000)

function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3) || (y[i] < 0.6) || (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  9.961 ms (0 allocations: 0 bytes)
```

|| (ALGEBRAIC)

```
x = rand(1_000_000)
y = rand(1_000_000)

function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if Bool(1 - !(x[i] > 0.3)  * !(y[i] < 0.6)  * !(x[i] > y[i]))
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  2.806 ms (0 allocations: 0 bytes)
```

The following example does the same, but for the case of broadcasting.

&&

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse((x>0.3) && (y<0.6) && (x>y), x, y)
```

```
julia> foo($x,$y)
  15.562 ms (3 allocations: 7.629 MiB)
```

ALGEBRAIC &&

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse((x>0.3) *  (y<0.6) *  (x>y), x, y)
```

```
julia> foo($x,$y)
  1.748 ms (3 allocations: 7.629 MiB)
```

||

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse((x>0.3) || (y<0.6) || (x>y), x, y)
```

```
julia> foo($x,$y)
  11.352 ms (3 allocations: 7.629 MiB)
```

ALGEBRAIC ||

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse(Bool(1 - !(x>0.3) * !(y<0.6) * !(x>y)), x, y)
```

```
julia> foo($x,$y)
  1.459 ms (3 allocations: 7.629 MiB)
```

Notice that such algebraic transformations may be performed automatically by the compiler. Whether this occurs depends on how the code is written. For example, revisiting the case of for-loops, the compiler performs the transformation automatically when expressing the condition through a separate function.

&&

```
x              = rand(1_000_000)
y              = rand(1_000_000)


function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3) && (y[i] < 0.6) && (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  6.732 ms (0 allocations: 0 bytes)
```

&& (ALGEBRAIC)

```
x               = rand(1_000_000)
y               = rand(1_000_000)


function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3)  * (y[i] < 0.6)  * (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  2.727 ms (0 allocations: 0 bytes)
```

AUTOMATIC TRANSFORMATION

```
x               = rand(1_000_000)
y               = rand(1_000_000)
condition(a,b) = (a > 0.3)  * (b < 0.6)  * (a > b)

function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if condition(x[i],y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  2.791 ms (0 allocations: 0 bytes)
```

FOOTNOTES

[1] Recall that the compiler may automatically disable bounds checking in simple scenarios. For instance, this is the case when only `x` is indexed and `eachindex(x)` is employed as the iteration range. Instead, explicit use of `@inbounds` becomes necessary when both `x` and another vector indexed, as in the examples we introduce below.

10g. SIMD Packages

Martin Alfaro
PhD in Economics

INTRODUCTION

Up to this point, we've leaned on the `@simd` macro to encourage vectorization in for-loops. This approach, nonetheless, exhibits several limitations.

The first one is control. The `@simd` macro acts as a suggestion, rather than a strict command: it hints to the compiler that SIMD optimizations might improve performance, but the implementation decision is ultimately up to the compiler's discretion. Second, `@simd` is designed to be conservative, prioritizing code safety over speed. In practice, this means it won't implement many transformations required to unlock SIMD's full capabilities.

To overcome these shortcomings, we'll introduce the `@turbo` macro from the `LoopVectorization` package. Unlike `@simd`, this macro can be used in both for-loops and broadcast operations.

Rather than merely suggesting vectorization, `@turbo` rewrites both for-loops and broadcast expressions into a form that's explicitly structured for SIMD execution. Furthermore, it applies more aggressive optimizations. In particular, it integrates with the `SLEEFPirates` package. This enables SIMD-accelerated implementations of common mathematical functions, including logarithms, exponentials, powers, and trigonometric operations.

The resulting additional performance comes with an important shift in responsibility: `@turbo` assumes that the code meets the conditions required for these transformations, placing the burden of safety and correctness on the user.

CAVEATS ABOUT IMPROPER USE OF @TURBO

In contrast to `@simd`, applying `@turbo` demands deliberate care, as an incorrect application can silently produce wrong results. The risk stems from the additional assumptions that `@turbo` makes to enable more aggressive optimizations. In particular, `@turbo` operates under two key assumptions:

- **No out-of-bound access**: `@turbo` removes bounds checks entirely. Thus, all indices should be valid.
- **Iteration order invariance**: `@turbo` supposes that the outcome doesn't depend on the order in which iterations execute. Nonetheless, reductions are allowed.

The second assumption becomes especially important, since it may conflict with for-loops that have dependent iterations. The example below illustrates this issue: because each iteration consumes the result produced by the previous one, applying `@turbo` breaks the required iteration-order invariance and therefore produces incorrect results.

NO MACRO

```
x = [0.1, 0.2, 0.3]

function foo!(x)
    for i in 2:length(x)
        x[i] = x[i-1] + x[i]
    end
end
```

```
julia> foo!(x)
julia> x
3-element Vector{Float64}:
 0.1
 0.3
 0.6
```

@SIMD

```
x = [0.1, 0.2, 0.3]

function foo!(x)
    @inbounds @simd for i in 2:length(x)
        x[i] = x[i-1] + x[i]
    end
end
```

```
julia> foo!(x)
julia> x
3-element Vector{Float64}:
 0.1
 0.3
 0.6
```

@TURBO

```
x = [0.1, 0.2, 0.3]

function foo!(x)
    @turbo for i in 2:length(x)
        x[i] = x[i-1] + x[i]
    end
end
```

```
julia> foo!(x)
julia> x
3-element Vector{Float64}:
 0.1
 0.3
 0.5
```

Even when a for-loop has no explicit loop-carried dependency, floating-point computations remain sensitive to reordering. In particular, because **floating-point addition isn't associative**, reordering a sequence of additions can change the final result. Note that integer arithmetic, by contrast, is unaffected by such reordering.

Considering that `@turbo` isn't suitable for all operations, we next present cases where the macro can be safely applied.

SAFE APPLICATIONS OF @TURBO

There are two categories of operations that represent safe uses of `@turbo`: **embarrassingly parallel problems** and **reductions**.

In the first category, **each iteration is completely independent**, making execution order irrelevant for the final outcome. The example below illustrates this scenario, by performing an independent transformation on each element of a vector.

DEFAULT

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.721 ms (3 allocations: 7.629 MiB)
```

@SIMD

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.940 ms (3 allocations: 7.629 MiB)
```

@TURBO (FOR-LOOP)

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  279.068 µs (3 allocations: 7.629 MiB)
```

@TURBO (BROADCASTING)

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

foo(x)         = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  272.868 µs (3 allocations: 7.629 MiB)
```

The second safe application is **reductions**. While reductions introduce dependencies across iterations, the package only requires that iterations can be arbitrarily reordered. This is satisfied by reduction operations.

DEFAULT

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.424 ms (0 allocations: 0 bytes)
```

@SIMD

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.758 ms (0 allocations: 0 bytes)
```

@TURBO

```
x              = rand(1_000_000)
calculation(a) = a * 0.1 + a^2 * 0.2 - a^3 * 0.3 - a^4 * 0.4

function foo(x)
    output = 0.0

    @turbo for i in eachindex(x)
        output += calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  169.195 µs (0 allocations: 0 bytes)
```

ACCELERATED VERSIONS OF CERTAIN FUNCTIONS

Another advantage of `LoopVectorization` comes from its integration with the library *SLEEF* (an anocrym for "SIMD Library for Evaluating Elementary Functions"). This feature accelerates the evaluation of several mathematical functions, including the exponential, logarithmic, power, and trigonometric functions. SLEEF is exposed in `LoopVectorization` via the `SLEEFPirates` package,

Below, we illustrate the use of `@turbo` across these different functions. For a complete list of supported functions, consult the SLEEFPirates documentation. All examples are based on an element-wise transformation of `x` via the function `calculation`, which will take different definitions depending on the function illustrated.

LOGARITHM

DEFAULT

```
x              = rand(1_000_000)
calculation(a) = log(a)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.511 ms (3 allocations: 7.629 MiB)
```

@SIMD

```
x              = rand(1_000_000)
calculation(a) = log(a)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.526 ms (3 allocations: 7.629 MiB)
```

@TURBO (FOR-LOOP)

```
x              = rand(1_000_000)
calculation(a) = log(a)

function foo(x)
    output = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  1.556 ms (3 allocations: 7.629 MiB)
```

@TURBO (BROADCASTING)

```
x              = rand(1_000_000)
calculation(a) = log(a)

foo(x)         = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  1.615 ms (3 allocations: 7.629 MiB)
```

EXPONENTIAL FUNCTION

DEFAULT

```
x              = rand(1_000_000)
calculation(a) = exp(a)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  2.577 ms (3 allocations: 7.629 MiB)
```

@SIMD

```
x              = rand(1_000_000)
calculation(a) = exp(a)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  2.719 ms (3 allocations: 7.629 MiB)
```

@TURBO (FOR-LOOP)

```
x              = rand(1_000_000)
calculation(a) = exp(a)

function foo(x)
    output = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  581.658 µs (3 allocations: 7.629 MiB)
```

@TURBO (BROADCASTING)

```
x              = rand(1_000_000)
calculation(a) = exp(a)

foo(x)         = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  552.402 µs (3 allocations: 7.629 MiB)
```

POWER FUNCTIONS

This includes any operation comprising terms x^y.

DEFAULT

```
x              = rand(1_000_000)
calculation(a) = a^4

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.695 ms (3 allocations: 7.629 MiB)
```

@SIMD

```
x              = rand(1_000_000)
calculation(a) = a^4

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.751 ms (3 allocations: 7.629 MiB)
```

@TURBO (FOR-LOOP)

```
x              = rand(1_000_000)
calculation(a) = a^4

function foo(x)
    output = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  272.640 µs (3 allocations: 7.629 MiB)
```

@TURBO (BROADCASTING)

```
x              = rand(1_000_000)
calculation(a) = a^4

foo(x)         = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  297.336 µs (3 allocations: 7.629 MiB)
```

The implementation for power functions includes calls to other functions, such as the one used for computing square roots.

DEFAULT

```
x              = rand(1_000_000)
calculation(a) = sqrt(a)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  1.209 ms (3 allocations: 7.629 MiB)
```

@SIMD

```
x              = rand(1_000_000)
calculation(a) = sqrt(a)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  1.230 ms (3 allocations: 7.629 MiB)
```

@TURBO (FOR-LOOP)

```
x              = rand(1_000_000)
calculation(a) = sqrt(a)

function foo(x)
    output = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  601.689 μs (3 allocations: 7.629 MiB)
```

@TURBO (BROADCASTING)

```
x              = rand(1_000_000)
calculation(a) = sqrt(a)

foo(x)         = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  601.851 μs (3 allocations: 7.629 MiB)
```

TRIGONOMETRIC FUNCTIONS

Among others, `@turbo` can handle the functions `sin`, `cos`, and `tan`. Below, we demonstrate its use with `sin`.

DEFAULT

```
x              = rand(1_000_000)
calculation(a) = sin(a)

function foo(x)
    output     = similar(x)

    for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  4.127 ms (3 allocations: 7.629 MiB)
```

@SIMD

```
x              = rand(1_000_000)
calculation(a) = sin(a)

function foo(x)
    output     = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  4.165 ms (3 allocations: 7.629 MiB)
```

@TURBO (FOR-LOOP)

```
x              = rand(1_000_000)
calculation(a) = sin(a)

function foo(x)
    output     = similar(x)

    @turbo for i in eachindex(x)
        output[i] = calculation(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  1.337 ms (3 allocations: 7.629 MiB)
```

@TURBO (BROADCASTING)

```
x              = rand(1_000_000)
calculation(a) = sin(a)

foo(x)         = @turbo calculation.(x)
```

```
julia> @btime foo($x)
  1.307 ms (3 allocations: 7.629 MiB)
```

11a. Overview and Goals

Martin Alfaro
PhD in Economics

The preceding chapter explored *single-core* parallelization through SIMD, which allows one processor core to perform the same computation simultaneously across multiple data elements. This chapter shifts focus to *multi-core* parallelization, where programs can execute simultaneously on several processor cores.

By default, most programming languages follow a sequential model: a single path of execution that advances one operation at a time. This linear approach simplifies reasoning about program behavior, as each operation completes before the next begins. However, hardware these days is commonly equipped with multiple processor cores. Consequently, a sequential execution does all the work on one core, while the others sit idle. This leaves substantial computational power untapped.

Multithreading addresses this limitation by simultaneously running different segments of a program across multiple cores. While this capability opens up significant opportunities for performance improvement, it also introduces new challenges that developers need to navigate carefully.

Compared to SIMD, **multithreading introduces even greater complexity**. Because multiple threads share the same memory space, execution can fail when operations are dependent. Thus, simple operations that work flawlessly in single-threaded programs may yield incorrect results in a multithreaded setting. In this context, writing multithreaded code requires a shift in how users think about execution. All this makes multithreaded code inherently more difficult to design, test, and debug than its single-threaded counterpart.

Despite these challenges, multithreading is essential for harnessing the full potential of modern processors. In particular, it's valuable for computationally intensive applications or when the same operation must be applied to multiple independent objects. The key is to recognize when multithreading is appropriate and how to avoid its pitfalls.

The roadmap of the chapter is as follows:

- Section 11b introduces the foundational concepts needed to reason about multithreading, including tasks and threads.
- Section 11c introduces task-based parallelism, implemented through the `@spawn` macro. This macro generates tasks that execute concurrently, with each task scheduled to run as soon as a thread becomes available. The section also emphasizes the overhead involved in managing tasks, implying that multithreading is only effective for large workloads.
- Section 11d examines which operations can be parallelized safely. Dependent operations may yield incorrect results when executed in parallel, turning common operations like reductions unsafe under naive parallelization. In contrast, "embarrassingly parallel problems", whose iterations are completely independent, can be parallelized seamlessly.

- The following sections transition to practical implementation. Section 11e compares two approaches to parallelizing for-loops: the `@threads` macro and `@spawn`. Building on this, Section 11f addresses how to parallelize for-loops effectively, focusing on embarrassingly parallel problems and reductions.
- Acknowledging the complexity of parallelizing code, Section 11g explores packages that simplify multithreading. They free developers from low-level implementation details. Notable examples are the `OhMyThreads` package and `@tturbo` macro from `LoopVectorization`. The latter combines SIMD and multithreading for maximum performance.

11b. Introduction to Multithreading

Martin Alfaro
PhD in Economics

INTRODUCTION

A correct implementation of multithreading requires a basic understanding of how computers execute instructions. In particular, it's essential to understand the procedure in light of data dependencies, where one operation relies on the output of a previous one. If these dependencies aren't properly managed, multithreading can lead to incorrect results and introduce several forms of unsafe code.

This section introduces the preliminary concepts needed to reason about these issues, laying the foundation for the more practical discussions that follow. **The emphasis here is on explanation rather than implementation**. Accordingly, many of the macros and functions presented won't be reused elsewhere in this book.

> ⚠️ **Warning!** - Loaded Package
>
> All the scripts below assume that ou've executed the line `using Base.Threads`. This allows direct access to the macro `@spawn`.

NATURE OF COMPUTATIONS

Operations can be broadly classified based on their data dependency. A **dependent operation** is one whose outcome is influenced by the result of another operation. In such cases, the order of execution is critical, because changing the sequence can alter the final outcome. In contrast, an **independent operation** always produces the same result, regardless of the order in which it's executed relative to other operations.

The following code gives rise to a dependent or independent operation, depending on which values `job_B` sums.

DEPENDENT OPERATIONS

```
job_A()  = 1 + 1
job_B(A) = 2 + A

function foo()
    A = job_A()
    B = job_B(A)

    return A,B
end
```

INDEPENDENT OPERATIONS

```
job_A()  = 1 + 1
job_B()  = 2 + 2

function foo()
    A = job_A()
    B = job_B()

    return A,B
end
```

Furthermore, regardless of their dependency status, operations can be computed either sequentially or concurrently. A **sequential** procedure involves executing operations one after the other, ensuring each completes before the next one begins. Conversely, **concurrency** allows multiple operations to be processed simultaneously, opening up opportunities for parallel execution.

Like most programming languages, **Julia defaults to a sequential execution**. This is a deliberate design choice that prioritizes result correctness, as **concurrent execution of dependent operations can yield incorrect results if mishandled**. The issue arises because concurrency introduces non-determinism in the execution order, which can lead to timing inconsistencies when reading and writing data. A sequential approach precludes this possibility by guaranteeing a predictable order of execution.

Despite its advantages regarding safety, a sequential approach can be quite inefficient for independent tasks: by restricting computations to one at a time, computational resources may go underutilized. In contrast, **a simultaneous approach allows for operations to be calculated in parallel, thereby fully utilizing all the available computational resources**. This can lead to significant reductions in computation time.

Because most programming languages default to sequential execution, certain nuances of concurrent programming can be difficult to grasp (e.g., concurrency doesn't necessarily imply simultaneity). Such misunderstandings can lead to flawed program design. To address this, we next revisit the topic in light of the fundamental concepts of tasks and threads.

TASKS AND THREADS

When computing an operation, Julia internally breaks it down into a set of instructions that need to be processed. These instructions are organized into units called **tasks**, and each task must be assigned to a **thread** to be executed. Because a single task can only run on one thread at a time, **the number of threads available on your machine directly determines how many tasks can be computed simultaneously**. Importantly, every Julia session starts with a predefined pool of available threads. By default, **a fresh installation of Julia configures each session to run on a single thread**, regardless of your machine's hardware capabilities. This means that, unless you explicitly request more threads, your program won't take advantage of the multiple cores of your processor.

To explain concurrency and distinguish it from simultaneity, it helps start with a strictly single-threaded setting. Consider two workers A and B, each responsible for a different task. To establish a parallelism that helps build intuition, think of them as employees working for a company. B's job consists of performing the same operation continuously for a certain period of time. In the code, this is represented by summing `1+1` repeatedly for one second. For its part, A's job consists of waiting for some delivery, which will arrive after a certain period of time. In the code, this job is represented by performing no computations for two seconds, simulated by calling the function `sleep(2)`.

The following code snippet defines functions to capture each worker's job.

A'S JOB

```
function job_A(time_working)
    sleep(time_working)        # do nothing (waiting for some delivery in the example)

    println("A completed his task")
end
```

B'S JOB

```
function job_B(time_working)
    start_time = time()

    while time() - start_time < time_working
        1 + 1                  # compute `1+1` repeatedly during `time_working` seconds
    end

    println("B completed his task")
end
```

Due to the lazy nature of function definitions, these code snippets merely create a blueprint for a set of operations. No computation is performed. It's only when we call these functions by adding lines like `job_A(2)` and `job_B(1)` that the operations are sent for execution.

To make the underlying execution model transparent, let's adopt a lower-level approach. In particular, we define the function calls `job_A(2)` and `job_B(1)` as tasks. As shown below, tasks aren't merely abstractions to organize our discussion, but actual constructs in Julia's codebase.

DEFINING TASKS

```
A = @task job_A(2)      # A's task takes 2 seconds
B = @task job_B(1)      # B's task takes 1 second
```

Once a task is defined, the first step toward execution is **scheduling**. This places the task into the queue of operations, waiting to be executed by the computer's processor. As soon as a thread becomes available, the computer begins its computation.

Importantly, multiple tasks can be *processed* concurrently, without implying that they'll be *computed* simultaneously. Indeed, this is the case in a single-thread session. The distinction can be understood through an analogy with juggling: a juggler manages multiple balls at the same time, but only holds one ball at any given moment. Similarly, multiple tasks can be processed simultaneously, even when only one is being executed on the CPU.

The implication of this statement is that true parallelism isn't feasible in single-threaded sessions. Nonetheless, concurrency can still improve efficiency through **task switching**, which is enabled by a **task-yielding** mechanism: when a task becomes idle (e.g., when waiting for input or data), it can voluntarily relinquish control of the thread, allowing other tasks to utilize the thread's time. By fostering a cooperative approach, concurrency ensures plenty of computer resource utilization at any given time.

In the following, we describe this mechanism in more detail.

SEQUENTIAL AND CONCURRENT COMPUTATIONS

While code in Julia executes sequentially by default, **tasks are designed to run concurrently**. This means that if we want to enforce a sequential execution of tasks, we must instruct Julia to run them one at a time. The way to do this is by introducing a "wait" instruction immediately after scheduling a task. By waiting, we ensure that the task finishes before proceeding with the rest of the program.

The code snippet below demonstrates this mechanism by introducing the functions `schedule` and `wait`.

JULIA'S DEFAULT PROCEDURE (SEQUENTIAL)

```
A = job_A(2)            # A's task takes 2 seconds
B = job_B(1)            # B's task takes 1 second
```



SEQUENTIAL

```
A = @task job_A(2)      # A's task takes 2 seconds
B = @task job_B(1)      # B's task takes 1 second

schedule(A) |> wait
schedule(B) |> wait
```



CONCURRENT

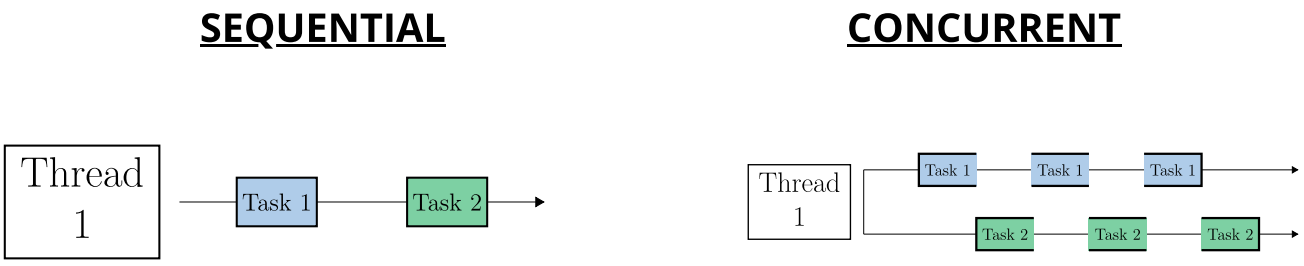
```
A = @task job_A(2)      # A's task takes 2 seconds
B = @task job_B(1)      # B's task takes 1 second

(schedule(A), schedule(B)) .|> wait
```

Note that `wait` was added even in the concurrent case and after both tasks were scheduled. The purpose is to pause the main program flow, until each scheduled task has completed. In this way, we prevent subsequent code from executing prematurely.

The example reveals the benefits of task switching under concurrency: although only one task can run at any moment, task A can yield control of the thread to task B when it becomes idle. In the code, the idle state is simulated by the function `sleep`, during which the computer performs no operations. Once task A becomes idle, its state is saved, allowing it to eventually resume execution from where it left off. In the meantime, task B can use that thread's processing time, explaining why task B finishes first.

By taking turns efficiently and sharing the single available thread, tasks make the most of the CPU's processing power. This contrasts with a sequential approach, where task A must finish before moving to the next task. The difference is reflected in the execution times, resulting in 2 seconds for the concurrent approach and 3 seconds for the sequential one.



Examples of idle states emerge naturally in real programs. A common case is when a program waits for user input, such as a keystroke or mouse click. It can also arise when browsing the internet, where the CPU may idle while waiting for a server to send data. Task switching is so ubiquitous in certain

contexts that we often take it for granted. For instance, you never wonder whether the computer can be used while a document prints in the background.

Notice, though, that concurrency with a single thread offers no benefits if both tasks require active computations. In that scenario, the CPU would be fully utilized, leaving no chance for task switching. As a result, the sequential and concurrent approaches would become equivalent. In our example, this possibility occurs when both task A and task B compute `1+1` repeatedly, leading to an identical execution time of 3 seconds in either approach.

DEFINING TASK - SAME FOR A AND B

```
function job(name_worker, time_working)
    start_time = time()

    while time() - start_time < time_working
        1 + 1                  # compute `1+1` repeatedly during `time_working` seconds
    end

    println("$name_worker completed his task")
end
```

SEQUENTIAL

```
function schedule_of_tasks()
    A = @task job("A", 2)      # A's task takes 2 seconds
    B = @task job("B", 1)      # B's task takes 1 second

    schedule(A) |> wait
    schedule(B) |> wait
end
```



CONCURRENT

```
function schedule_of_tasks()
    A = @task job("A", 2)      # A's task takes 2 seconds
    B = @task job("B", 1)      # B's task takes 1 second

    (schedule(A), schedule(B)) .|> wait
end
```



MULTITHREADING

The procedure outlined established that, **when a task is scheduled, the system looks for an available thread to execute it**. In our discussion so far, we have assumed that the session was running on a single thread, which means tasks are processed one at a time. **When a session is started with multiple threads, however, true parallel execution becomes possible**: several tasks can now be running at the same time, each on its own thread. This setup is what we call **multithreading**.

To build intuition, let's revisit the case where both workers A and B perform meaningful computations. The only change introduced is that Julia's session now starts with more than one available thread. For the concurrent version, we also specify that the tasks are "non-sticky". This means that a task isn't bound to the thread on which it was first scheduled, and can instead run on any available thread. This flexibility allows the system to allocate tasks to whichever thread is free, leading to more efficient use of the available hardware.

Once there's more than one thread available, concurrency implies simultaneity: each task can run on a different thread at the same time. This is why task B finishes first in the following implementation.

SEQUENTIAL

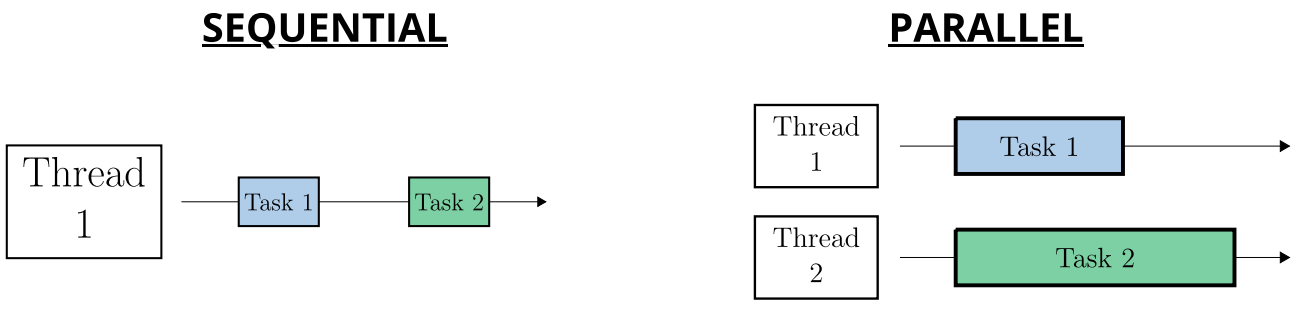
```
function schedule_of_tasks()
    A = @task job("A", 2)                    # A's task takes 2 seconds
    B = @task job("B", 1)                    # B's task takes 1 second

    schedule(A) |> wait
    schedule(B) |> wait
end
```

PARALLEL

```
function schedule_of_tasks()
    A = @task job("A", 2) ; A.sticky = false      # A's task takes 2 seconds
    B = @task job("B", 1) ; B.sticky = false      # B's task takes 1 second

    (schedule(A), schedule(B)) .|> wait
end
```



To see the benefits of multithreading in action, let's compare Julia's default implementation (sequential) with a multithreaded one. For the latter, we'll use the macro `@spawn`, which will be covered in the next section. This provides a simple way to create and schedule a non-sticky task.

SEQUENTIAL (JULIA'S DEFAULT PROCEDURE)

```
function schedule_of_tasks()
    A = job("A", 2)             # A's task takes 2 seconds
    B = job("B", 1)             # B's task takes 1 second
end
```



MULTITHREADED

```
function schedule_of_tasks()
    A = @spawn job("A", 2)      # A's task takes 2 seconds
    B = @spawn job("B", 1)      # B's task takes 1 second

    (A,B) .|> wait
end
```



THE IMPORTANCE OF WAITING FOR THE RESULTS

Before concluding this section, it's worth stressing that you must always instruct your program to wait for operations to complete, before proceeding. This holds true even for concurrent computations. **Failing to wait may produce incorrect results, even in a single-threaded environment**.

The issue can be observed when mutating a vector in a single-threaded session. Suppose that we introduce a one-second delay for each element update. If we don't wait for the mutation to finish, any subsequent operation will be based on the vector's value at the moment it's accessed. This value doesn't necessarily reflect its final mutated state, but merely its value at the moment of calling it.

For instance, suppose mutating the vector `x = [0,0,0]` into `x = [1,1,1]` and then summing all its value. Under Julia's default sequential execution, the mutation is guaranteed to complete before the program moves on to the sum.

MUTATION - JULIA'S DEFAULT APPROACH

```
# Description of job
function job!(x)
    for i in 1:3
        sleep(1)      # do nothing for 1 second
        x[i] = 1      # mutate x[i]

        println("`x` at this moment is $x")
    end
end

# Execution of job
function foo()
    x = [0, 0, 0]

    job!(x)           # slowly mutate `x`

    return sum(x)
end

output = foo()
println("the value stored in `output` is $(output)")
```



Let's now consider the same implementation, but through tasks. The task that performs the mutation is defined as follows.

DEFINE TASK

```
function job!(x)
    @task begin
        for i in 1:3
            sleep(1)    # do nothing for 1 second
            x[i] = 1    # mutate x[i]

            println("`x` at this moment is $x")
        end
    end
end
```

The following snippets show the consequences of waiting versus not waiting for the mutation task to complete.

NOT WAITING

```
function foo()
    x = [0, 0, 0]

    job!(x) |> schedule             # define job, start execution, don't wait for job to be
done

    return sum(x)
end

output = foo()
println("the value stored in `output` is $(output)")
```



WAITING

```
function foo()
    x = [0, 0, 0]

     job!(x) |> schedule |> wait       # define job, start execution, only continue when
finished

    return sum(x)
end

output = foo()
println("the value stored in `output` is $(output)")
```



Without a `wait` call, the main program schedules the mutation task, immediately proceeding to the next line. Since the task contains a `sleep` instruction, the mutation hasn't yet begun when `x` is accessed for being summed. This results in the use of `x`'s original value `[0,0,0]`, determining that `sum(x)` provides `0` as output.

11c. Task-Based Parallelism: @spawn

Martin Alfaro
PhD in Economics

INTRODUCTION

The previous section introduced the basics of multithreading, where we highlighted that operations can be computed either sequentially (Julia's default) or concurrently. The latter enables the processing of multiple operations simultaneously, with each of them running as soon as a thread becomes available. In particular, when Julia's session is initialized with more than one thread, computations can be executed in parallel on different CPU cores.

The current section will focus on Julia's native multithreading mechanisms. The topic will span several sections. Our primary goal here is to demonstrate how to write multithreaded code, rather than exploring how and when to apply the technique. We've deliberately structured our explanation in this way to smooth subsequent discussions.

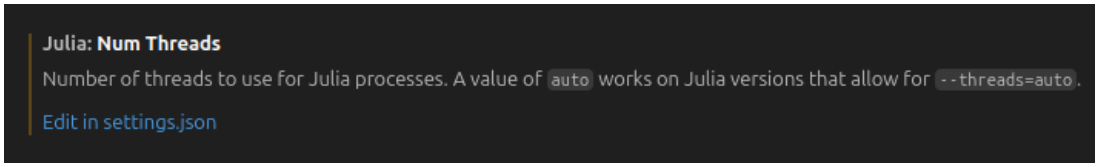
> ⚠ **Warning!**
>
> While multithreading can offer significant performance advantages, it's not applicable in all scenarios. In particular, **multithreading demands extreme caution in handling dependencies between operations**, as mismanagement can lead to silent catastrophic bugs.
>
> This is why the current section only aims at grasping a basic understanding of parallelism techniques. An entire section will be devoted to thread-unsafe operations.

ENABLING MULTITHREADING

Each Julia session is initialized with a given pool of threads available. Since each thread can run a set of instructions independently, the total number of threads determines how many instructions can be processed simultaneously.

By default, Julia runs in single-threaded mode. Consequently, you must configure it to enable multithreading. Importantly, **once this configuration is set, the change is permanent**: every new Julia session will start with the number of threads you specified.

In VSCode or VSCodium, multithreading is enabled by navigating to *File* > *Preferences* > *Settings*. Then, you must search for the keyword *threads*, which will display the following option:



After selecting *Edit in settings.json*, add the line `"julia.NumThreads": "auto"` in the file that opens. The specification `auto` automatically detects the number of threads supported by your system, typically matching the logical or physical cores of your CPU.

Notice the effects will only take place after a new session is started. To check whether this is the case, run the command `Threads.nthreads()`. This displays the number of threads available in the running session. Any number greater than one will indicate multithreading is active.

Once a multithreaded session is started, there are several packages for performing parallel computations. The current section will focus on the capabilities built directly into Julia, which are provided by the `Threads` package. This package is automatically "imported" in every Julia session, so you can call its functions and macros by prefixing them with `Threads`. Alternatively, you can load the package with `using`, which lets you use its macros and functions without qualification.

PACKAGE THREADS AUTOMATICALLY IMPORTED

```
# package `Threads` is automatically imported when you start a Julia session

Threads.nthreads()
```

```
24
```

USING THREADS

```
using Base.Threads      # or `using .Threads`

nthreads()
```

```
24
```

> ⚠ **Warning!** - Loaded Package
>
> All the scripts below assume that you've executed `using Base.Threads`. This allows direct access to macros like `@spawn`.

TASK-BASED PARALLELISM: @SPAWN

The first approach to parallel execution we introduce is the macro `@spawn`. By prefixing an expression with `@spawn`, Julia creates a (non-sticky) task that's immediately scheduled for execution. If a thread is available, the task begins running right away.

Unlike other parallel programming approaches we'll examine later, `@spawn` requires explicit instructions to wait for task completion. The method for doing so depends on the nature of the task output.

The `fetch` function is required when tasks return a result. It serves a dual purpose: it waits for a task to complete and returns the task's output. Since parallel programs involve multiple spawned tasks, `fetch` should be broadcast over a collection containing all tasks.

The following example demonstrates `fetch` with two spawned tasks.

BASELINE (SINGLE-THREADED)

```
x = 2

function foo(x)
    a      = x * -2
    b      = x *  2

    a,b
end
```

```
julia> foo(x)
(-4, 4)
```

FETCH

```
x = 2

function foo(x)
    task_a = @spawn x * -2
    task_b = @spawn x *  2

    a,b    = fetch.((task_a, task_b))
end
```

```
julia> foo(x)
(-4, 4)
```

Note that `fetch` takes tasks as input. This is why it's important to distinguish between `task_a` and `a`:

- `task_a` denotes the task creating the vector `a` (i.e., the task itself),
- `a` refers to the vector created (i.e., the task's output).

Alternatively, for tasks that perform actions without returning a value, we can use either the function `wait` or the macro `@sync`. The function `wait` works analogously to `fetch`, except that it doesn't return any value. For its part, the macro `@sync` requires enclosing all the operations to be synchronized between the keywords `begin` and `end`.

To illustrate, consider a mutating function. Such functions are suitable examples for both cases, since they don't return a value.

BASELINE (SINGLE-THREADED)

```
x = rand(10)
y = rand(10)

function foo!(x,y)
    @. x = -x
    @. y = -y
end
```

WAIT

```
x = rand(10)
y = rand(10)

function foo!(x,y)
    task_a = @spawn (@. x = -x)
    task_b = @spawn (@. y = -y)

    wait.((task_a, task_b))
end
```

@SYNC

```
x = rand(10)
y = rand(10)

function foo!(x,y)
    @sync begin
        @spawn (@. x = -x)
        @spawn (@. y = -y)
    end
end
```

MULTITHREADING OVERHEAD

To see `@spawn` in action, let's calculate both the sum and the maximum of a vector `x`. Their computation will be implemented following a sequential and a simultaneous approach. These implementations will reveal that the total runtime of the sequential procedure is essentially the sum of the individual runtimes. In contrast, thanks to parallelism, the runtime under multithreading is roughly equivalent to the longer of the two computations.

JULIA'S DEFAULT (SEQUENTIAL)

```
x = rand(10_000_000)

function non_threaded(x)
    a           = maximum(x)
    b           = sum(x)

    all_outputs = (a,b)
end
```

```
julia> @btime maximum($x)
  7.494 ms (0 allocations: 0 bytes)
julia> @btime sum($x)
  3.041 ms (0 allocations: 0 bytes)
julia> @btime non_threaded($x)
  10.666 ms (0 allocations: 0 bytes)
```

MULTITHREADING

```
x = rand(10_000_000)

function multithreaded(x)
    task_a      = @spawn maximum(x)
    task_b      = @spawn sum(x)

    all_tasks   = (task_a, task_b)
    all_outputs = fetch.(all_tasks)
end
```

```
julia> @btime maximum($x)
  7.494 ms (0 allocations: 0 bytes)
julia> @btime sum($x)
  3.041 ms (0 allocations: 0 bytes)
julia> @btime multithreaded($x)
  7.631 ms (13 allocations: 1.031 KiB)
```

Although the multithreaded runtime is close to the maximum of the individual runtimes, the equivalence isn't exact. The reason is that **multithreading introduces overhead** due to the creation, scheduling, and synchronization of tasks. A practical consequence of this behavior is that **multithreading can actually be counterproductive for small workloads**, where the overhead can outweigh any performance gains.

To make this trade-off concrete, let's compare the execution times of a sequential and multithreaded approach for different sizes of `x`. In the examples, multithreading is only advantageous for sizes greater than 100,000, with the single-threaded approach dominating otherwise. The high threshold illustrates just how significant the overhead can be relative to the actual work performed.

SINGLE-THREADED

```
x_small  = rand(    1_000)
x_medium = rand(  100_000)
x_big    = rand(1_000_000)

function foo(x)
    a           = maximum(x)
    b           = sum(x)

    all_outputs = (a,b)
end
```

```
julia> @btime foo($x_small)
  878.688 ns (0 allocations: 0 bytes)
julia> @btime foo($x_medium)
  59.726 μs (0 allocations: 0 bytes)
julia> @btime foo($x_big)
  634.935 μs (0 allocations: 0 bytes)
```

MULTITHREADED

```
x_small  = rand(    1_000)
x_medium = rand(  100_000)
x_big    = rand(1_000_000)

function foo(x)
    task_a      = @spawn maximum(x)
    task_b      = @spawn sum(x)

    all_tasks   = (task_a, task_b)
    all_outputs = fetch.(all_tasks)
end
```

```
julia> @btime foo($x_small)
  2.644 μs (13 allocations: 1.031 KiB)
julia> @btime foo($x_medium)
  63.325 μs (13 allocations: 1.031 KiB)
julia> @btime foo($x_big)
  593.296 μs (13 allocations: 1.031 KiB)
```

11d. Thread-Safe Operations

Martin Alfaro
PhD in Economics

INTRODUCTION

Multithreading allows multiple threads to run simultaneously within a single process, enabling parallel execution of operations on the same machine. Unlike other forms of parallelization such as multiprocessing, where each process has its own memory space, **threads in multithreading share a common memory space**.

This shared-memory environment makes parallelization easy, but it also introduces some complexity while writing code: since all threads can access and modify the same data simultaneously, **parallel execution can lead to unintended side effects if not managed carefully**. In particular, the interaction between threads may cause corrupted results or even program crashes.

To reason about these issues, it's useful to distinguish between thread-safe and thread-unsafe operations. An operation is considered **thread-safe** if it can be executed in parallel without causing inconsistencies, crashes, or corrupted results. Conversely, **thread-unsafe** operations require explicit synchronization or algorithmic restructuring to prevent corruption.

The current section will identify key features that render operations unsafe under multithreading. In particular, we'll see that common operations like reductions aren't thread-safe, leading to incorrect results if multithreading is applied naively. We'll also explore the concept of embarrassingly parallel programs, which are a prime example of thread-safe operations. These programs can be divided into independent units of work, requiring no communication or synchronization. These types of programs can be parallelized directly, without the need to restructure their computation.

> ⚠️ **Warning!** - Loaded Package
>
> All the scripts below assume you've executed `using Base.Threads`. This allows direct access to macros like `@threads` and `@spawn`.
>
> Furthermore, we assume your Julia session is running with more than one thread available. Refer to these instructions to enable multithreading.

THREAD-UNSAFE OPERATIONS

In a multithreaded environment, an operation is deemed unsafe if executing it concurrently can lead to incorrect behavior, data corruption, or even program crashes. These issues typically arise when tasks exhibit some degree of dependency, either in terms of operations or shared resources. The examples below illustrate how these dependencies can introduce serious issues when multithreading is applied without proper safeguards.

WRITING ON A SHARED VARIABLE

One of the simplest examples of a thread-unsafe operation is writing to a shared variable. To illustrate, consider a scalar variable `output` initialized to zero. This is then updated in a for-loop with two iterations, where the *i*-th iteration sets `output` to the value `i`.

To starkly show the challenges of concurrent execution, we'll deliberately introduce a decreasing delay before each update of `output`. This is implemented with `sleep(1/i)`, causing the first iteration to pause for 1 second and the second iteration for half a second. Although this delay is artificially induced via `sleep`, it mimics the kind of timing gaps that real computations can introduce. Such gaps can cause `output` to be updated in an unexpected order.

SEQUENTIAL

```
function foo()
    output = 0

    for i in 1:2
        sleep(1/i)
        output = i
    end

    return output
end
```

```
julia> foo()
2
```

PARALLEL

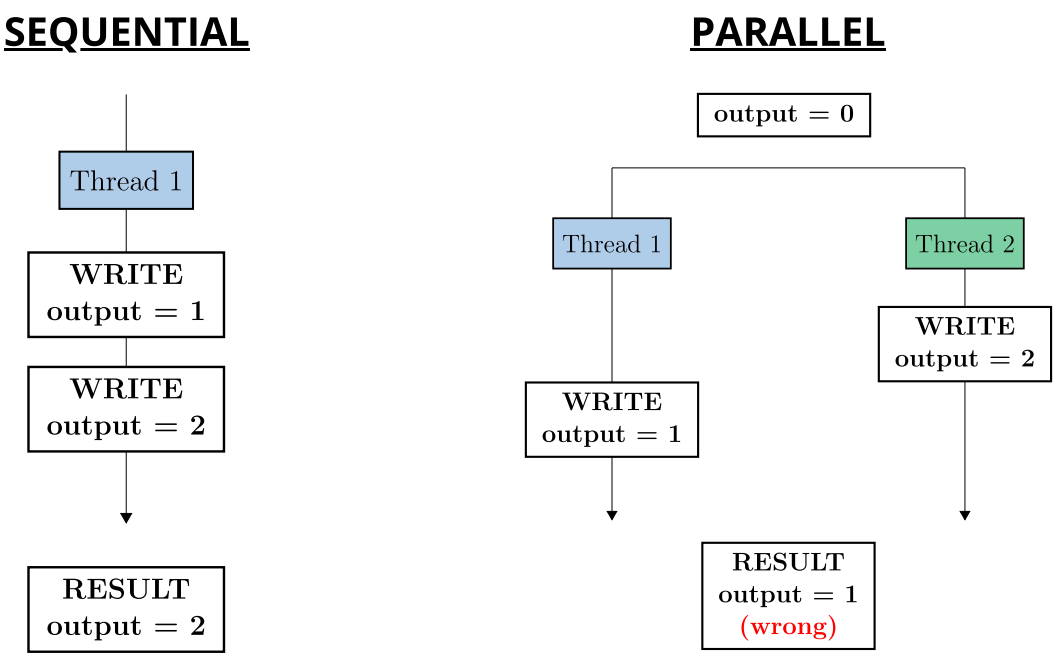
```
function foo()
    output = 0

    @threads for i in 1:2
        sleep(1/i)
        output = i
    end

    return output
end
```

```
julia> foo()
1
```

The delay is inconsequential for a sequential procedure, where `output` takes on the values 0, 1, and 2 as the program progresses. However, when executed concurrently, the first iteration completes after the second iteration has finished. As a result, the sequence of values for `output` is 0, 2, and 1.



While the problem may seem apparent in this simple example, it can manifest in more complex and subtle ways in real-world applications. The core issue is that the order of execution isn't guaranteed in a multithreaded environment. Thus, when multiple threads modify the same shared variable, the final value depends on which thread executes last.

In fact, the issue can be exacerbated when each iteration additionally involves reading a shared variable. Next, we consider such a scenario.

READING AND WRITING A SHARED VARIABLE

Reading and writing shared data doesn't necessarily yield incorrect results. For instance, a parallel for-loop could safely mutate a vector: even if multiple threads are simultaneously modifying the same shared object, each thread would be operating on distinct elements of the vector. Since no two threads would interfere with one another, the updates would remain independent.

Problems arise, however, **when the correctness of reading and writing shared data depends on the order in which threads execute**. This situation is known as a **race condition**. The term reflects that the final output may change from one run to the next, depending on which thread finishes and updates the data last.

To demonstrate the issue, let's modify our previous example by introducing a variable `temp`, whose value is updated in each iteration. This variable will be shared across threads and used to mutate the *i*-th entry of the vector `output`. By introducing a delay before writing each entry of `output`, a race condition is introduced, where all threads end up using the last value of `temp` (in this case, 2).

SEQUENTIAL

```
function foo()
    output = Vector{Int}(undef,2)
    temp   = 0

    for i in 1:2
        temp      = i; sleep(i)
        output[i] = temp
    end

    return output
end
```

```
julia> foo()
2-element Vector{Int64}:
 1
 2
```

PARALLEL (SHARED VARIABLE)

```
function foo()
    output = Vector{Int}(undef,2)
    temp   = 0

    @threads for i in 1:2
        temp      = i; sleep(i)
        output[i] = temp
    end

    return output
end
```

```
julia> foo()
2-element Vector{Int64}:
 2
 2
```

PARALLEL (NO SHARED VARIABLE)

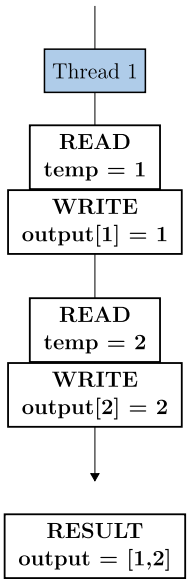
```
function foo()
    output = Vector{Int}(undef,2)


    @threads for i in 1:2
        temp      = i; sleep(i)
        output[i] = temp
    end

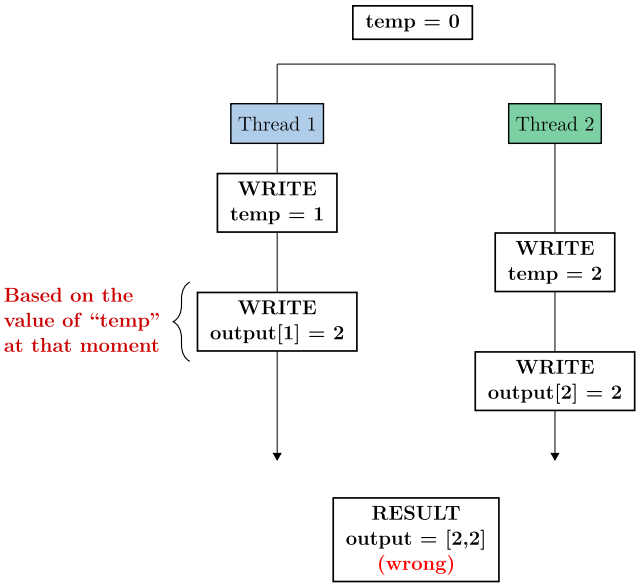
    return output
end
```

```
julia> foo()
2-element Vector{Int64}:
 1
 2
```

SEQUENTIAL



PARALLEL



In this specific scenario, the issue can be easily circumvented: instead of defining `temp` outside the for-loop, we simply define it as variable local to the for-loop. In this way, each thread would work with its own private copy of `temp`, thereby eliminating the data race entirely.

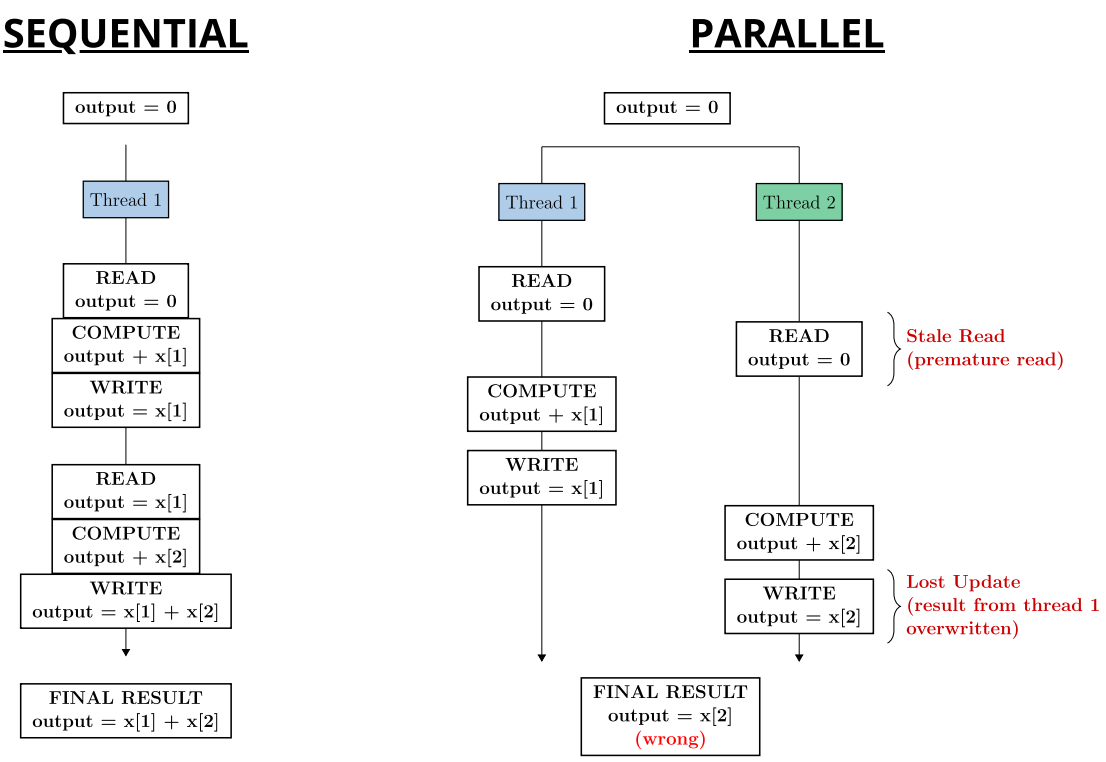
Beyond any solution proposed, the example highlights the subtleties of parallelizing operations. Even simple patterns can conceal dependencies that lead to unsafe behavior when executed concurrently. To illustrate this further, we now turn to a more common scenario where data races occur: reductions.

RACE CONDITIONS WITH REDUCTIONS

Reductions are a prime example of thread-unsafe operations. To illustrate why this is so, consider summing a collection in parallel. At first glance, this seems straightforward: each thread computes a partial contribution and adds it to a shared accumulator. However, the variable holding each partial sum is shared across all threads: during each iteration, every thread attempts to read its current value, add its contribution, and write the result back.

When multiple threads perform these steps concurrently, their actions can overlap in unpredictable ways. One thread's update may overwrite another's, meaning that some contributions are lost rather than combined. As a result, the final sum is nondeterministic and often incorrect, varying from run to run.

Below, we illustrate this issue when we sum the first two elements of a vector `x`.



When performing reductions in parallel without addressing the underlying race condition, the outcome becomes unpredictable. More specifically, the final sum tends to be lower than the correct value, because some updates are lost when threads overwrite one another's contributions.

SEQUENTIAL

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo(x)
500658.01158503356
```

MULTITHREADED 1

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @threads for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo(x)
21436.48668413443
```

MULTITHREADED 2

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @threads for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo(x)
21590.997961948713
```

MULTITHREADED 3

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @threads for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo(x)
21461.273851717895
```

The key insight from this example isn't that reductions are incompatible with multithreading. Rather, it's that the strategy to apply multithreading needs to be adapted accordingly. The upcoming sections will present these strategies. Before that, we conclude this section by looking at the opposite end of the spectrum: problems that naturally lend themselves to parallel execution.

EMBARRASSINGLY PARALLEL PROGRAMS

In the context of thread-safe programming, the simplest case occurs when tasks are completely independent of one another. These cases are known as **embarrassingly parallel problems**. In these problems, the computation can be decomposed into many independent tasks, each of which can be executed in parallel without communication, synchronization, or shared state. Because no task influences any other, the program has full freedom to schedule and execute them in any order, making parallelization highly efficient.

In the context of for-loops, a simple way to parallelize embarrassingly parallel problems is through the macro `@threads` from the package `Threads`. This is a form of thread-based parallelism, where the distribution of work is based on the number of threads available. In particular, `@threads` attempts to balance the workload by dividing the iterations as evenly as possible. Unlike `@spawn`, `@threads` automatically schedules the tasks and waits for their completion before execution proceeds. In the next section, we'll present a detailed comparison between `@threads` and `@spawn`. For now, we simply demonstrate the simplicity of `@threads` through the following example.

NOT THREADED

```
x_small  = rand(    1_000)
x_medium = rand(  100_000)
x_big    = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x_small)
  3.319 µs (3 allocations: 7.883 KiB)
julia> @btime foo($x_medium)
  332.609 µs (3 allocations: 781.320 KiB)
julia> @btime foo($x_big)
  3.396 ms (3 allocations: 7.629 MiB)
```

MULTITHREADED

```
x_small  = rand(    1_000)
x_medium = rand(  100_000)
x_big    = rand(1_000_000)

function foo(x)
    output = similar(x)

    @threads for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x_small)
  9.092 µs (125 allocations: 20.508 KiB)
julia> @btime foo($x_medium)
  42.710 µs (125 allocations: 793.945 KiB)
julia> @btime foo($x_big)
  336.284 µs (125 allocations: 7.642 MiB)
```

11e. Parallel For-Loops

Martin Alfaro
PhD in Economics

INTRODUCTION

Commonly, the most computationally intensive parts of a program are implemented using for-loops. To accelerate these operations, we can leverage parallel processing to distribute iterations across multiple processor threads. The central challenge, however, lies in how best to divide this work, as the optimal strategy heavily depends on the nature of the workload.

This section introduces Julia's `@threads` macro as a straightforward approach to parallelizing for-loops. This operates by splitting iterations evenly among all available threads. To shed light on the underlying mechanisms of `@threads`, we'll contrast it with the task-based parallelism offered by `@spawn`.

> ⚠ **Warning!** - Use of @spawn
>
> To clearly illustrate the differences between `@threads` and `@spawn`, **our examples will use `@spawn` in a simple but inefficient manner**. Specifically, a separate task will be created for every single iteration. While this suffices for our comparative analysis with `@threads`, it's not representative of a typical `@spawn` usage.
>
> In the next section, we'll revisit `@spawn` to show how efficiently define tasks. You'll see that `@spawn` is flexible enough to implement sophisticated parallel strategies, including the one employed by `@threads`.

> ⚠ **Warning!** - Loaded Package
>
> All the scripts below assume that you've executed `using Base.Threads`. This allows direct access to macros like `@threads` and `@spawn`.
>
> Furthermore, we assume your Julia session is running with more than one thread available. Refer to these instructions to enable multithreading.

SOME PRELIMINARIES

Parallelism techniques target code that performs multiple operations, making it a natural fit for for-loops. Using the macro `@spawn` introduced in the previous sections, we can parallelize for-loops through a task-based parallelism. For now, we'll consider a simple (inefficient) case where each iteration defines a separate task. The code implementing this technique is shown below.

@SPAWN WITH FOR-LOOP

```
@sync begin
    for i in 1:4
        @spawn println("Iteration $i is computed on Thread $(threadid())")
    end
end
```

```
Iteration 1 is computed on Thread 1
Iteration 2 is computed on Thread 2
Iteration 4 is computed on Thread 2
Iteration 3 is computed on Thread 2
```

MANUAL IMPLEMENTATION (EQUIVALENT)

```
@sync begin
    @spawn println("Iteration 1 is computed on Thread $(threadid())")
    @spawn println("Iteration 2 is computed on Thread $(threadid())")
    @spawn println("Iteration 3 is computed on Thread $(threadid())")
    @spawn println("Iteration 4 is computed on Thread $(threadid())")
end
```

```
Iteration 1 is computed on Thread 1
Iteration 2 is computed on Thread 2
Iteration 3 is computed on Thread 1
Iteration 4 is computed on Thread 2
```

When there are only a few iterations involved, creating one task per iteration can be a straightforward and effective way to parallelize the code. However, as the number of iterations increases, the approach becomes less efficient due to the overhead of task creation. To mitigate this issue, we need to consider alternative ways of parallelizing for-loops.

One such alternative is to create tasks that encompass multiple iterations, rather than just one per task. The techniques to do this will be explored in upcoming sections. They offer more granular control, but at the expense of adding substantial complexity to the code. In light of this complexity, Julia provides the `@threads` macro from the package `Threads`. The goal is to reduce the overhead of task creation, while keeping the parallelization simple. Specifically, `@threads` divides the set of iterations evenly among threads, thereby restricting the creation of tasks to the number of threads available.

The following example demonstrates the implementation of `@threads`, highlighting its difference from the approach with `@spawn`. The scenario considered is based on 4 iterations and 2 worker threads. Additionally, we track the thread on which each iteration is executed by using the `threadid()` function. This identifies the specific thread that is computing the operation.

JULIA'S DEFAULT (SEQUENTIAL)

```
for i in 1:4
    println("Iteration $i is computed on Thread $(threadid())")
end
```

```
Iteration 1 is computed on Thread 1
Iteration 2 is computed on Thread 1
Iteration 3 is computed on Thread 1
Iteration 4 is computed on Thread 1
```

@THREADS

```
@threads for i in 1:4
    println("Iteration $i is computed on Thread $(threadid())")
end
```

```
Iteration 1 is computed on Thread 1
Iteration 2 is computed on Thread 1
Iteration 3 is computed on Thread 2
Iteration 4 is computed on Thread 2
```

@SPAWN

```
@sync begin
    for i in 1:4
        @spawn println("Iteration $i is computed on Thread $(threadid())")
    end
end
```

```
Iteration 2 is computed on Thread 2
Iteration 1 is computed on Thread 1
Iteration 4 is computed on Thread 2
Iteration 3 is computed on Thread 2
```

One key distinction between `@threads` and `@spawn` lies in the strategy for thread allocation. Thread assignments with `@threads` are predetermined: before the for-loop begins, the macro pre-allocates threads and distributes iterations evenly. Thus, each thread is assigned a fixed number of iterations upfront, creating a predictable workload distribution. In the example, the feature is observed in the allocation of two iterations per thread.

In contrast, `@spawn` creates a separate task for each iteration, dynamically scheduling them as soon as a thread becomes available. This method allows for more flexible thread utilization, with task assignments adapting in real-time to the current system load and available thread capacity. In the case of the previous example, a single thread ended up computing three out of the four iterations.

@SPAWN VS @THREADS

The macros `@threads` and `@spawn` embody two distinct approaches to work distribution, thus catering to different types of scenarios. By comparing the creation of one task per iteration relative to `@threads`, we can highlight the inherent trade-offs involved in parallelizing code.

`@threads` employs a coarse-grained approach, making it well-suited for workloads with similar computational requirements. By reducing the overhead associated with task creation, this approach excels in scenarios where tasks have comparable execution times. However, it's less effective in handling workloads with unbalanced execution times, where some iterations are computationally intensive while others are relatively lightweight.

In contrast, `@spawn` adopts a fine-grained approach, treating each iteration as a separate task that can be scheduled independently. This allows for more flexible work distribution, with tasks dynamically allocated to available threads as soon as they become available. As a result, `@spawn` is particularly well-suited for scenarios with varying computational efforts, where iteration completion times can differ significantly. While this approach has a bigger overhead due to the creation of numerous smaller tasks, it simultaneously enables more efficient resource utilization. This is because no thread remains idle while tasks await computation.

In the following, we demonstrate the efficiency of the approaches under each scenario. With this goal, consider a situation where the *i*-th iteration computes `job(i;time_working)`. This function represents potential calculations performed during `time_working` seconds. It's formally defined as follows.

ITERATION I'S OPERATION

```
function job(i; time_working)
    println("Iteration $i is on Thread $(threadid())")

    start_time = time()

    while time() - start_time < time_working
        1 + 1                   # compute `1+1` repeatedly during `time_working` seconds
    end
end
```

Note that `job` additionally identifies the thread on which it's running, also displaying it on the REPL.

Based on a for-loop with four iterations and a session with two worker threads, we next consider two scenarios. They differ by the computational workload of the iterations.

SCENARIO 1: UNBALANCED WORKLOAD

The first scenario represents a situation with unbalanced work, where some iterations require more computational effort. The feature is captured by assuming that the *i*-th iteration has a duration of `i` seconds.

We start by presenting the coding implementing each approach, and then provide explanations for each.

JULIA'S DEFAULT (SEQUENTIAL)

```
nr_iterations = 4

function foo(nr_iterations)
    for i in 1:nr_iterations
      job(i; time_working = i)
    end
end
```

```
Iteration 1 is on Thread 1
Iteration 2 is on Thread 1
Iteration 3 is on Thread 1
Iteration 4 is on Thread 1
  10.000 s (40 allocations: 1.562 KiB)
```

@THREADS

```
nr_iterations = 4

function foo(nr_iterations)
    @threads for i in 1:nr_iterations
        job(i; time_working = i)
    end
end
```

```
Iteration 1 is on Thread 1
Iteration 3 is on Thread 2
Iteration 2 is on Thread 1
Iteration 4 is on Thread 2
  7.000 s (51 allocations: 2.625 KiB)
```

@SPAWN

```
nr_iterations = 4

function foo(nr_iterations)
    @sync begin
        for i in 1:nr_iterations
            @spawn job(i; time_working = i)
        end
    end
end
```

```
Iteration 1 is on Thread 1
Iteration 2 is on Thread 2
Iteration 3 is on Thread 1
Iteration 4 is on Thread 2
  6.000 s (69 allocations: 3.922 KiB)
```

Given the execution times for each iteration, a sequential approach would take 10 seconds. As for parallel implementations, `@threads` ensures that there are as many tasks created as number of threads. In the example, this means that there are two tasks created, with the first task computing iterations 1 and 2, and the second task computing iterations 3 and 4. As a result, the overall execution time is reduced to 7 seconds.

In contrast, `@spawn` creates a separate task for each iteration, which increases the overhead of task creation. Although the overhead is negligible in this example, it can be appreciated in the increased memory allocation. Despite this disadvantage, the approach allows each iteration to be executed as soon as a thread becomes available. Given the varying execution times between iterations, this dynamic allocation becomes advantageous, enabling iterations 3 and 4 to run in parallel.

The example demonstrates this, where iterations 1 and 2 are now executed on different threads. Since the first iteration only requires one second, the thread becomes available to compute the third iteration immediately. The final distribution of tasks on threads is such that iterations 1 and 3 are executed on one thread, while iterations 2 and 4 are executed on the other thread. This results in a total execution time of 6 seconds.

SCENARIO 2: BALANCED WORKLOAD

Consider now a scenario where the execution of `job` requires exactly the same time regardless of the iteration considered. To make the overhead more apparent, we'll use a larger number of iterations. In this context, `@threads` ensures parallelization with a reduced overhead, explaining why it's faster than the approach relying on `@spawn`.

JULIA'S DEFAULT (SEQUENTIAL)

```
nr_iterations = 1_000

function foo(nr_iterations)
    fixed_time = 1 / 1_000_000

    for i in 1:nr_iterations
        job(i; time_working = fixed_time)
    end
end
```

```
julia> @btime foo($nr_iterations$)
  1.714 ms (0 allocations: 0 bytes)
```

@THREADS

```
nr_iterations = 1_000

function foo(nr_iterations)
    fixed_time = 1 / 1_000_000

    @threads for i in 1:nr_iterations
        job(i; time_working = fixed_time)
    end
end
```

```
julia> @btime foo($nr_iterations$)
  80.212 µs (122 allocations: 12.625 KiB)
```

@SPAWN

```
nr_iterations = 1_000

function foo(nr_iterations)
    fixed_time = 1 / 1_000_000

    @sync begin
        for i in 1:nr_iterations
            @spawn job(i; time_working = fixed_time)
        end
    end
end
```

```
julia> @btime foo($nr_iterations$)
  606.384 µs (5015 allocations: 522.125 KiB)
```

11f. Parallelization in Practice

Martin Alfaro
PhD in Economics

INTRODUCTION

So far, we've explored two macros for parallelization: `@spawn` and `@threads`. The macro `@spawn` provides granular control over the parallelization process, letting users explicitly define the tasks to be executed concurrently. In contrast, `@threads` offers a simpler approach for parallelizing for-loops, where iterations are automatically partitioned into tasks according to the number of available threads. Additionally, we've pointed out that, due to inherent dependencies between computations, not all workloads are equally amenable to parallelization. In particular, a naive approach to parallelization can lead to severe issues.

Essentially, our discussions have largely focused on the *syntax* and *work distribution* of parallelization approaches. Yet, we have to address how to apply multithreading in real scenarios. Furthermore, given the possibility of dependencies between computations, *how* to parallelize is only part of the challenge: knowing *when* to parallelize is equally important.

This section and the next one aim to bridge this gap, providing practical guidance on implementing multithreading. We begin by highlighting the advantages of coarse-grained parallelization over fine-grained parallelization. By dividing the workload into a small number of large tasks, coarse-grained parallelization reduces the scheduling overhead from managing numerous lightweight tasks.

After this, we revisit the parallelization of for-loops, this time using `@spawn`. In particular, leveraging the additional control that `@spawn` provides over task creation, we'll demonstrate how to apply multithreading in the presence of a ubiquitous type of dependency: reductions.

We conclude by showing a performance issue arising with multithreading, known as false sharing. While this doesn't affect the correctness the result, it can significantly slow down computations if not addressed.

> ⚠️ **Warning!** - Loaded Package
>
> All the scripts below assume you've executed `using Base.Threads`. This allows direct access to macros like `@threads` and `@spawn`.
>
> Furthermore, we assume your Julia session is running with more than one thread available. Refer to these instructions to enable multithreading.

BETTER TO PARALLELIZE AT THE TOP

Given the overhead involved in multithreading, there's an inherent trade-off between creating new tasks and fully utilizing machine resources. This is why we must always analyze whether parallelization is worthwhile in the first place. For instance, when it comes to operations over collections, multithreading is only justified if the collections have enough elements to offset the associated overhead. Otherwise, single-threaded approaches will consistently outperform parallelized ones.

In case multithreading is deemed beneficial, we immediately face another decision: at what level code should be parallelized. Next, we'll demonstrate that **parallelism at the highest possible level is preferable, compared to multithreading individual operations**. By adopting this strategy, we minimize the overhead of task creation.

Note that the level of parallelization is always constrained by the degree of dependency between operations. Hence, our qualification of highest *possible* level. For instance, in problems requiring strictly serial computation, the best we can achieve is parallelization within each individual step.

To illustrate, let's consider a for-loop where each iteration needs to sequentially compute three operations.

SINGLE THREAD

```
step1(a) = a ^ 2
step2(a) = sqrt(a)
step3(a) = log(a + 1)

function all_steps(a)
   y      = step1(a)
   z      = step2(y)
   output = step3(z)

   return output
end

function foo(x)
   output = similar(x)

   for i in eachindex(output)
      output[i] = all_steps(x[i])
   end

   return output
end

x_small  = rand(  1_000)
x_large  = rand(100_000)
```

```
julia> @btime foo($x_small)
  5.206 µs (3 allocations: 7.883 KiB)
julia> @btime foo($x_large)
  537.663 µs (3 allocations: 781.320 KiB)
```

PARALLELIZATION AT THE HIGHEST LEVEL POSSIBLE

```
step1(a) = a ^ 2
step2(a) = sqrt(a)
step3(a) = log(a + 1)

function all_steps(a)
   y      = step1(a)
   z      = step2(y)
   output = step3(z)

   return output
end

function foo(x)
   output = similar(x)

   @threads for i in eachindex(output)
      output[i] = all_steps(x[i])
   end

   return output
end

x_small  = rand(  1_000)
x_large  = rand(100_000)
```

```
julia> @btime foo($x_small)
  13.667 µs (125 allocations: 20.508 KiB)
julia> @btime foo($x_large)
  71.050 µs (125 allocations: 793.945 KiB)
```

EACH OPERATION PARALLELIZED

```
step1(a) = a ^ 2
step2(a) = sqrt(a)
step3(a) = log(a + 1)

function parallel_step(f, x)
   output = similar(x)

   @threads for i in eachindex(output)
      output[i] = f(x[i])
   end

   return output
end

function foo(x)
   y      = parallel_step(step1, x)
   z      = parallel_step(step2, y)
   output = parallel_step(step3, z)

   return output
end

x_small  = rand(  1_000)
x_large  = rand(100_000)
```

```
julia> @btime foo($x_small)
  35.841 µs (375 allocations: 61.523 KiB)
julia> @btime foo($x_big)
  104.260 µs (375 allocations: 2.326 MiB)
```

The examples make two points. First, parallelization is advantageous only with large collections. Otherwise, the question of whether to parallelize shouldn't even arise. Second, once multithreading proves to be advantageous, grouping all operations into a single task is faster than parallelizing each operation individually.

IMPLICATIONS

The strategy of parallelizing code at the highest possible level has significant implications for program design. In particular, when the program will be applied repeatedly to multiple independent objects. It suggests a practical guideline: start with an implementation for a single object, without introducing parallelism. After thoroughly optimizing the single-case code, integrate parallel execution at the top level. The approach not only improves performance, but also simplifies the development by making debugging and testing more straightforward.

A common application of this strategy arises in scientific simulations, where independent executions of the same model are required. In those cases, the most effective approach is to maintain a single-threaded implementation of the model, eventually launching multiple instances in parallel. This design ensures that each run remains efficient at the single-thread level, while taking advantage of full resource utilization.

THE IMPORTANCE OF WORK DISTRIBUTION

Multithreading performance is heavily influenced by how evenly the computational workload is distributed across iterations. The `@threads` macro spawns a task for every iteration, making it highly effective when each iteration requires roughly equal processing time. Scenarios with uneven computational effort are more challenging. In such cases, some threads may finish early and remain idle, while others continue processing heavier tasks. This imbalance undermines parallel efficiency, substantially diminishing the performance gains of multithreading.

To address this issue, we need greater control over how work is distributed among threads. This calls for the use of `@spawn`, deploying different strategies regarding work distribution.

One strategy is to make each iteration a separate task. However, such approach is extremely inefficient if there's a large number of iterations: creating far more tasks than there are threads introduces substantial and unnecessary overhead. The following example illustrates this problem.

@THREADS

```
x = rand(10_000_000)

function foo(x)
    output = similar(x)

    @threads for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  5.877 ms (125 allocations: 76.309 MiB)
```

@SPAWN

```
x = rand(10_000_000)

function foo(x)
    output = similar(x)

    @sync for i in eachindex(x)
        @spawn output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  11.095 s (60005888 allocations: 5.277 GiB)
```

An alternative strategy, which lets users fine-tune the workload of each task, is to partition iterations into smaller subsets that can be processed in parallel. Before detailing the implementation, we'll first explore how to partition a collection along with its indices through the `ChunkSplitters` package.

PARTITIONING COLLECTIONS

The package `ChunkSplitters` provides two functions for lazy partitioning: `chunks` and `index_chunks`. These functions support `n` and `size` as keyword arguments, depending on the type of partition desired. Specifically, `n` sets the number of subsets to create, with each subset sized to distribute elements evenly. In contrast, `size` specifies the number of elements to be contained in each subset. Since an even distribution across all subsets can't be guaranteed in all cases, the package adjusts the number of elements in one of the subsets if necessary.

Below, we apply these functions to a variable `x` that contains the 26 letters of the alphabet. Note that the outputs provided require the use of `collect`, since `chunks` and `index_chunks` are lazy.

PARTITION BY NUMBER OF CHUNKS

```
x             = string.('a':'z')            # all letters from "a" to "z"

nr_chunks     = 5

chunk_indices = index_chunks(x, n = nr_chunks)
chunk_values  = chunks(x, n = nr_chunks)
```

```
julia> collect(chunk_indices)
5-element Vector{UnitRange{Int64}}:
 1:6
 7:11
 12:16
 17:21
 22:26
julia> collect(chunk_values)
5-element Vector{SubArray{String, 1, Vector{String}, Tuple{UnitRange{Int64}}, true}}:
 ["a", "b", "c", "d", "e", "f"]
 ["g", "h", "i", "j", "k"]
 ["l", "m", "n", "o", "p"]
 ["q", "r", "s", "t", "u"]
 ["v", "w", "x", "y", "z"]
```

PARTITION BY SIZE OF CHUNKS

```
x             = string.('a':'z')            # all letters from "a" to "z"

chunk_length  = 10

chunk_indices = index_chunks(x, size = chunk_length)
chunk_values  = chunks(x, size = chunk_length)
```

```
julia> collect(chunk_indices)
3-element Vector{UnitRange{Int64}}:
 1:10
 11:20
 21:26
julia> collect(chunk_values)
3-element Vector{SubArray{String, 1, Vector{String}, Tuple{UnitRange{Int64}}, true}}:
 ["a", "b", "c", "d", "e", "f", "g", "h", "i", "j"]
 ["k", "l", "m", "n", "o", "p", "q", "r", "s", "t"]
 ["u", "v", "w", "x", "y", "z"]
```

For multithreading, a relevant partition is a number of chunks proportional to the number of worker threads. The example below implements this, generating both chunk indices and chunk values. Since this partition will eventually be used with for-loops, we also show how to use `enumerate` to pair each chunk with the values or subindices of its corresponding subset.

PARTITION BY NUMBER OF THREADS

```
x             = string.('a':'z')            # all letters from "a" to "z"

nr_chunks     = nthreads()

chunk_indices = index_chunks(x, n = nr_chunks)
chunk_values  = chunks(x, n = nr_chunks)
```

```
julia> collect(chunk_indices)
24-element Vector{UnitRange{Int64}}:
 1:2
 3:4
 ⋮
 25:25
 26:26
julia> collect(chunk_values)
24-element Vector{SubArray{String, 1, Vector{String}, Tuple{UnitRange{Int64}}, true}}:
 ["a", "b"]
 ["c", "d"]
 ⋮
 ["y"]
 ["z"]
```

PARTITION BY NUMBER OF THREADS - ENUMERATE

```
x             = string.('a':'z')            # all letters from "a" to "z"

nr_chunks     = nthreads()

chunk_indices = index_chunks(x, n = nr_chunks)
chunk_values  = chunks(x, n = nr_chunks)

chunk_iter1   = enumerate(chunk_indices)    # pairs (i_chunk, chunk_index)
chunk_iter2   = enumerate(chunk_values)     # pairs (i_chunk, chunk_value)
```

```
julia> collect(chunk_iter1)
24-element Vector{Tuple{Int64, UnitRange{Int64}}}:
 (1, 1:2)
 (2, 3:4)
 ⋮
 (23, 25:25)
 (24, 26:26)

julia> collect(chunk_iter2)
24-element Vector{Tuple{Int64, SubArray{String, 1, Vector{String},
Tuple{UnitRange{Int64}}, true}}}:
 (1, ["a", "b"])
 (2, ["c", "d"])
 ⋮
 (23, ["y"])
 (24, ["z"])
```

WORK DISTRIBUTION: DEFINING TASKS THROUGH CHUNKS

Leveraging the `ChunkSplitters` package together with `@spawn`, we can control how a for-loop is parallelized. Instead of relying on the fixed scheduling strategy of `@threads`, the package explicitly divides the iteration space into user-defined chunks, with each chunk mapped to an independent task executed concurrently.

Below, we illustrate possible strategies to define chunks. To begin with, we replicate the exact behavior of `@threads` via `@spawn`. Specifically, both approaches follow the same execution pattern when the number of chunks matches the number of worker threads.

@THREADS

```
x = rand(10_000_000)

function foo(x)
    output = similar(x)

    @threads for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  5.877 ms (125 allocations: 76.309 MiB)
```

@SPAWN

```
x = rand(10_000_000)

function foo(x, nr_chunks)
    chunk_ranges = index_chunks(x, n=nr_chunks)
    output       = similar(x)

    @sync for chunk in chunk_ranges
        @spawn (@views @. output[chunk] = log(x[chunk]))
    end

    return output
end
```

```
julia> @btime foo($x, nthreads())
  5.829 ms (157 allocations: 76.311 MiB)
```

@SPAWN (EQUIVALENT)

```
x = rand(10_000_000)

function foo(x, nr_chunks)
    chunk_ranges = index_chunks(x, n=nr_chunks)
    output       = similar(x)
    task_indices = Vector{Task}(undef, nr_chunks)

    for (i, chunk) in enumerate(chunk_ranges)
        task_indices[i] = @spawn (@views @. output[chunk] = log(x[chunk]))
    end

    return wait.(task_indices)
end
```

```
julia> @btime foo($x, nthreads())
  5.841 ms (151 allocations: 76.310 MiB)
```

However, the flexibility of `@spawn` means we're not limited to the partitioning scheme used by `@threads`. A widely used strategy is to create more chunks than threads to improve load balancing. This is especially effective on systems where cores don't offer uniform performance. By using smaller chunks, faster cores can pick up additional work as soon as they finish their current tasks, preventing idle time and therefore fully utilizing the available hardware.

The next example demonstrates this approach by choosing numbers of chunks proportional to the number of worker threads.

@SPAWN

```
x = rand(10_000_000)

function foo(x, nr_chunks)
    chunk_ranges = index_chunks(x, n=nr_chunks)
    output       = similar(x)

    @sync for chunk in chunk_ranges
        @spawn (@views @. output[chunk] = log(x[chunk]))
    end

    return output
end
```

```
julia> @btime foo($x, 1 * nthreads())
  7.058 ms (157 allocations: 76.311 MiB)
julia> @btime foo($x, 2 * nthreads())
  5.492 ms (302 allocations: 76.325 MiB)
julia> @btime foo($x, 4 * nthreads())
  4.982 ms (590 allocations: 76.352 MiB)
```

@SPAWN

```
x = rand(10_000_000)

function compute!(output, x, chunk)
     @turbo for j in chunk
        output[j] = log(x[j])
     end
end

function foo(x, nr_chunks)
    chunk_ranges = index_chunks(x, n=nr_chunks)
    output       = similar(x)

    @sync for chunk in chunk_ranges
        @spawn compute!(output, x, chunk)
    end

    return output
end
```

```
julia> @btime foo($x, 1 * nthreads())
  4.379 ms (133 allocations: 76.310 MiB)
julia> @btime foo($x, 2 * nthreads())
  4.529 ms (254 allocations: 76.323 MiB)
julia> @btime foo($x, 4 * nthreads())
  4.080 ms (494 allocations: 76.347 MiB)
```

HANDLING DEPENDENCIES

So far, our discussion of parallelization has focused on embarrassingly parallel for-loops, where each iteration is completely independent of the others. In these situations, every iteration can run in isolation, which makes parallelization conceptually simple and very effective.

Things become more subtle once dependencies enter the picture. When operations rely on the results of earlier computations, attempts to parallelizing without first addressing those dependencies can lead to wasted work, poor performance, or even incorrect results.

There's no universal recipe for handling dependencies, because the right approach depends entirely on the structure of the program. In practice, you need to reformulate the computation so that the units of work you intend to parallelize are independent. Only after this restructuring can parallelization proceed safely. If such a reformulation isn't possible, then parallelization simply isn't viable for that part of the computation.

It's also important to recognize that once dependencies are present, some fraction of the work will inevitably remain sequential. In extreme cases, the computation may be inherently serial, leaving no subset of independent tasks to parallelize at all.

REDUCTIONS AS A PARTICULAR TYPE OF DEPENDENCE

Reductions are a common technique that introduces dependencies between iterations. To take advantage of parallel execution despite this dependency, the computation must be reorganized. The standard approach is to partition the data into chunks, perform a partial reduction on each chunk in parallel, and then combine those partial results in a final reduction step. This restructuring removes the original loop-carried dependency, since each partial reduction operates on a disjoint subset of the data and therefore doesn't interfere with the others.

To illustrate, let's compute the sum of elements of a vector `x`. The implementation relies on the `ChunkSplitters` package to divide the data into independent segments.

JULIA'S DEFAULT (SEQUENTIAL)

```
x = rand(10_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += x[i]
    end

    output
end
```

```
julia> @btime foo($x)
  5.203 ms (0 allocations: 0 bytes)
```

@THREADS

```
x = rand(10_000_000)

function foo(x)
    chunk_ranges    = index_chunks(x, n=nthreads())
    partial_outputs = Vector{Float64}(undef, length(chunk_ranges))

    @threads for (i,chunk) in enumerate(chunk_ranges)
        partial_outputs[i] = sum(@view(x[chunk]))
    end

    return sum(partial_outputs)
end
```

```
julia> @btime foo($x)
  1.268 ms (124 allocations: 13.250 KiB)
```

@SPAWN

```
x = rand(10_000_000)

function foo(x)
    chunk_ranges    = index_chunks(x, n=nthreads())
    partial_outputs = Vector{Float64}(undef, length(chunk_ranges))

    @sync for (i, chunk) in enumerate(chunk_ranges)
        @spawn partial_outputs[i] = sum(@view(x[chunk]))
    end

    return sum(partial_outputs)
end
```

```
julia> @btime foo($x)
  1.286 ms (156 allocations: 13.781 KiB)
```

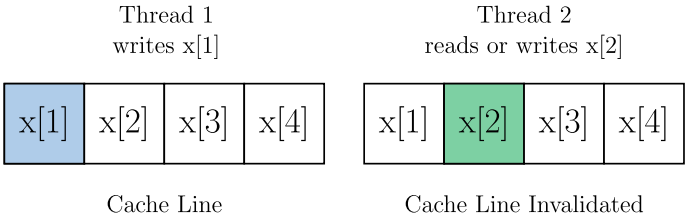
FALSE SHARING

Addressing dependencies between operations is essential when parallelizing. Without doing so, a program can easily produce incorrect results. Yet, even after eliminating these dependencies and ensuring correctness, performance may still degrade due to underlying hardware-level effects.

One common bottleneck is **cache contention**, where multiple processor cores compete for shared cache resources. A manifestation of this issue is what's known as **false sharing**, where multiple cores simultaneously access data stored in the same cache line. To understand why this behavior degrades performance, it helps first review how CPU caches work.

Processors rely on caches to hold copies of frequently accessed data. These caches are much smaller but significantly faster than the RAM, and are organized into fixed-size blocks called cache lines (typically 64 bytes). When the processor needs a piece of data, it first checks whether it's already present in the cache. If not, the data must be fetched from RAM and placed into a cache line, a process that's considerably slower.

Likewise, when multiple cores access data within the same cache line, the transfer of information is governed by a cache coherency protocol. Its goal is to ensure consistency across cores. However, the protocol can create inefficiencies: even if one core accesses data that remains unmodified, any modification to another value within the same cache line may cause the entire line to be invalidated. As a result, all cores are forced to reload the block, despite the absence of a logical need to do so. This phenomenon, known as **false sharing**, leads to unnecessary cache invalidations and refetches. The outcome is a notable degradation in program performance, particularly in workloads where threads frequently update their variables.



Below, we focus on the emergence of false sharing in reduction operations.

FALSE SHARING IN REDUCTIONS: AN ILLUSTRATION AND SOLUTIONS

Consider the task of summing the elements of a vector after applying a logarithmic transformation to each entry. To illustrate how implementation choices affect performance, we'll look at two versions of this computation. The first one is a straightforward sequential routine that processes the vector from start to finish. It serves as a clear baseline: simple, predictable, and free of concurrency concerns.

The second implementation parallelizes the work. It assigns different segments of the vector to different threads, with each responsible for accumulating its own partial sum. At first glance, this design seems efficient, since the partial sums are logically independent. However, it results in false sharing.

Specifically, each thread stores its partial result in a distinct element of the `partial_outputs` array. Even though these elements represent independent data, they're typically placed in contiguous memory locations. When several of these locations fall within the same cache line, updates from one thread cause that entire line to be invalidated on other cores. Those cores must then reload the line before continuing, creating unnecessary coherence traffic. This repeated invalidation and reloading can significantly slow down the computation, despite the algorithm's apparent parallel structure.

SEQUENTIAL

```
x = rand(10_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += log(x[i])
    end

    output
end
```

```
julia> @btime foo($x)
  37.046 ms (0 allocations: 0 bytes)
```

FALSE SHARING

```
x = rand(10_000_000)

function foo(x)
    chunk_ranges    = index_chunks(x, n=nthreads())
    partial_outputs = Vector{Float64}(undef, length(chunk_ranges))

    @threads for (i,chunk) in enumerate(chunk_ranges)
        for j in chunk
            partial_outputs[i] += log(x[j])
        end
    end

    return sum(partial_outputs)
end
```

```
julia> @btime foo($x)
  17.434 ms (124 allocations: 13.250 KiB)
```

Several techniques can mitigate this problem. All of them prevent multiple threads from writing to memory locations sharing a cache line.

One common strategy is **vector padding**, where extra spacing is inserted between the elements of `partial_outputs`. This strategy ensures that each thread's accumulator is placed on a distinct cache line, so that concurrent writes no longer interfere with one another at the cache level. We implement this below by storing the partial outputs in a vector with sufficient separation between rows. In particular, a separation of 8 entries.

PADDING

```
x = rand(10_000_000)

function foo(x)
    chunk_ranges    = index_chunks(x, n=nthreads())
    nr_strides      = 8
    partial_outputs = Vector{Float64}(undef, length(chunk_ranges) * nr_strides)

    @threads for (i, chunk) in enumerate(chunk_ranges)
        for j in chunk
            partial_outputs[(i-1)*nr_strides + 1] += log(x[j])
        end
    end

    return sum(@view(partial_outputs[1:nr_strides:end]))
end
```

```
julia> @btime foo($x)
  6.243 ms (124 allocations: 14.625 KiB)
```

Although padding is intuitive, it's not especially practical. The amount of spacing required depends on both the element type and the machine's cache-line size, which varies across architectures. Rather than tuning these details manually, it's usually better to rely on approaches that avoid false sharing in a more robust and portable way.

The first method introduces a thread-local variable `temp` to store partial results. In this way, each thread updates only its own local variable, finally writing to the shared array exactly once at the end. Below, we implement this solution via `@threads` and `@spawn`.

An alternative solution is to compute each partial reduction inside a separate function. Because the function's local variables are thread-private, the accumulation proceeds without any shared writes until the final store.

LOCAL VARIABLE (@THREADS)

```
x = rand(10_000_000)

function foo(x)
    chunk_ranges    = index_chunks(x, n=nthreads())
    partial_outputs = Vector{Float64}(undef, length(chunk_ranges))

    @threads for (i,chunk) in enumerate(chunk_ranges)
        temp = 0.0
        for j in chunk
            temp += log(x[j])
        end
        partial_outputs[i] = temp
    end

    return sum(partial_outputs)
end
```

```
julia> @btime foo($x)
  4.820 ms (124 allocations: 13.250 KiB)
```

LOCAL VARIABLE (@SPAWN)

```
x = rand(10_000_000)

function foo(x)
    chunk_ranges    = index_chunks(x, n=nthreads())
    partial_outputs = Vector{Float64}(undef, length(chunk_ranges))

    @sync for (i,chunk) in enumerate(chunk_ranges)
        @spawn begin
            temp = 0.0
            for j in chunk
                temp += log(x[j])
            end
            partial_outputs[i] = temp
        end
    end

    return sum(partial_outputs)
end
```

```
julia> @btime foo($x)
  4.385 ms (156 allocations: 13.781 KiB)
```

FUNCTION

```
x = rand(10_000_000)

function compute(x, chunk)
    temp = 0.0

    for j in chunk
        temp += log(x[j])
    end

    return temp
end

function foo(x)
    chunk_ranges    = index_chunks(x, n=nthreads())
    partial_outputs = Vector{Float64}(undef, length(chunk_ranges))

    @threads for (i,chunk) in enumerate(chunk_ranges)
        partial_outputs[i] = compute(x, chunk)
    end

    return sum(partial_outputs)
end
```

```
julia> @btime foo($x)
  4.851 ms (124 allocations: 13.250 KiB)
```

11g. Multithreading Packages

Martin Alfaro
PhD in Economics

INTRODUCTION

Parallelizing code may seem straightforward at first glance. However, once we start delving into its subtleties, it's rapidly revealed that an effective implementation can be a daunting task. Naive implementations can lead to various issues, including performance problems like suboptimal load balancing or **false sharing**, and more severe concerns such as **data races**. Furthermore, even if we master the necessary skills for a correct implementation, manual parallelization often impairs the readability and maintainability of code.

To assist users in overcoming these obstacles, several packages for parallelization have emerged. These tools aim to simplify the implementation of multithreading, allowing users to leverage its benefits without grappling with low-level intricacies. In this section, we'll present a few of these packages. In particular, the focus will be on those that facilitate the application of multithreading to **embarrassingly parallel problems** and **reductions**.

The first package we explore is `OhMyThreads`. This offers a collection of high-level functions and macros that help developers parallelize operations with minimal effort. For instance, it eliminates the need to manually partition tasks and also tackles performance issues like false sharing. We'll then examine the `Polyester` package. Thanks to its reduced overhead, this package excels at parallelizing workloads involving small objects. Finally, we revisit the `LoopVectorization` package to introduce the `@tturbo` macro, which combines the benefits of SIMD instructions with multithreading.

> ⚠️ **Warning!** - Loaded Package
>
> All the scripts below assume you've executed `using Base.Threads`. This allows direct access to the macros `@threads` and `@spawn`.
>
> Furthermore, we assume your Julia session is running with more than one thread. Refer to **these instructions** to enable multithreading.

PACKAGE "OHMYTHREADS"

`OhMyThreads` is designed as a user-friendly package for seamlessly applying multithreading. Consistent with its minimalist approach, it introduces only a handful of essential functionalities that could easily belong to Julia's `Base`. Despite its simplicity, the package covers a wide range of operations, including reductions.

The package's primary goal is to make parallelization widely accessible, including users without deep expertise in multithreading. Indeed, its functions and macros automatically address common pitfalls, such as **data races** and performance issues like **false sharing**.

To achieve broad applicability, `OhMyThreads` extends familiar `Base` operations into the multithreaded domain through a set of higher-order functions. Each parallelized variant follows a simple naming convention: the original function name prefixed with `t`. For example, the parallel counterpart to `map` becomes `tmap` in the package. At the same time, the package remains flexible by integrating with `ChunkSplitters`, allowing users to fine-tune how work is distributed across tasks. This customization is controlled through two keyword arguments: `nchunks` (or equivalently `ntasks`) to specify the number of subsets in the partition, and `chunksize` to define the number of elements in each partition.

> ⚠ **Warning!**
>
> All the scripts below assume you've executed `using OhMyThreads` to load the package.

PARALLEL MAPPING

`OhMyThreads` provides `tmap` as a multithreaded analogue of Julia's `map` function. The typical and most efficient way to call it is `tmap(foo, T, x)`, where `foo` is the function applied to each element of the collection `x`, and `T` denotes the element type of the resulting array. For instance, if the computation produces a `Vector{Float64}`, then `T` should be `Float64`.

Although you can omit the type parameter and write `tmap(foo, x)`, doing so introduces a performance penalty. This slowdown arises from a subtle source: the type instability of the underlying `Task` objects used to schedule work across threads. Because the compiler can't reliably infer the output type in this case, it's forced into less efficient code paths. To avoid this and recover full performance, you should explicitly specify the output element type. In practice, rather than hard-coding `T`, it's often clearer and safer to use `eltype(x)`, which ensures that the output array mirrors the element type of the input collection.

The package also offers an in-place variant `tmap!`, whose syntax is `tmap!(foo, output, x)`. Since the destination array is provided explicitly, the function already knows the output type, so there's no need to supply `T` to prevent performance issues.

The examples that follow illustrate how to use both `tmap` and `tmap!`. For context, we also include the corresponding results from `map` and `map!`, which serve as single-threaded baselines.

MULTITHREADED MAP

```
x                       = rand(1_000_000)


foo(x)                  =  map(log, x)
foo_parallel1(x)        = tmap(log, x)
foo_parallel2(x)        = tmap(log, eltype(x), x)
```

```
julia> @btime foo($x)
  3.407 ms (3 allocations: 7.629 MiB)
julia> @btime foo_parallel1($x)
  2.548 ms (251 allocations: 27.197 MiB)
julia> @btime foo_parallel2($x)
  336.252 μs (169 allocations: 7.643 MiB)
```

MULTITHREADED MAP!

```
x                       = rand(1_000_000)
output                  = similar(x)

foo!(output,x)          =  map!(log, output, x)
foo_parallel!(output,x) = tmap!(log, output, x)
```

```
julia> @btime foo($output, $x)
  3.394 ms (0 allocations: 0 bytes)
julia> @btime foo_parallel!($output, $x)
  333.281 μs (163 allocations: 13.516 KiB)
```

`tmap` also allows us to control the work distribution among tasks through the keyword arguments `nchunks` and `chunksize`. These options are internally implemented via the package `ChunkSplitters`.

`tmap` also gives you fine-grained control over how work is divided among tasks through the keyword arguments `nchunks` and `chunksize`. These options rely internally on the `ChunkSplitters` package. Specifically, `nchunks` controls the number of subsets in the partition, while `chunksize` sets the number of elements per task. Note that `nchunks` and `chunksize` are mutually exclusive options, so that only one of them can be used at a time.

To illustrate the use `nchunks`, we'll set its value equal to `nthreads()`. By setting a number of chunks equal to the number of worker threads, we're adopting an even distribution among tasks, similar to how `@threads` operates. To replicate the same behavior with `chunksize`, we'll make use of the floor division operator `÷`. This is a binary operator that rounds a division down to the nearest integer towards zero. [1]

NUMBER OF CHUNKS

```
x      = rand(1_000_000)

foo(x) = tmap(log, eltype(x), x; nchunks = nthreads())
```

```
julia> @btime foo($x)
  341.808 μs (169 allocations: 7.643 MiB)
```

SIZE OF CHUNKS

```
x      = rand(1_000_000)

foo(x) = tmap(log, eltype(x), x; chunksize = length(x) ÷ nthreads())
```

```
julia> @btime foo($x)
  336.906 μs (177 allocations: 7.643 MiB)
```

Do-Block Syntax

When passing anonymous functions into `tmap`, the do-block syntax comes in handy for keeping code readable. It enables the creation of multi-line functions, without the need to introduce a new function before applying `tmap`.

ANONYMOUS FUNCTION

```
x = rand(1_000_000)

function foo(x)

    output = tmap(a -> 2 * log(a), x)



    return output
end
```

DO-BLOCK SYNTAX

```
x = rand(1_000_000)

function foo(x)

    output = tmap(x) do a
                 2 * log(a)
             end

    return output
end
```

ARRAY COMPREHENSIONS

`OhMyThreads` also provides a parallel implementation of **array comprehensions**. Unlike the standard syntax of array comprehensions, the version from `OhMyThreads` combines a **generator** with a multithreaded variant of `collect` named `tcollect`.

As with `tmap`, specifying the output's element type is optional, abut necessary to avoid performance losses. The recommended syntax is therefore `tcollect(T, <generator>)`, where `T` denotes the element type of the output. A common practice is to use `eltype(x)` as `T`, with `x` being the variable iterated over in the generator. This ensures that the output type matches the input collection.

MULTITHREADED ARRAY COMPREHENSION

```
x                = rand(1_000_000)
output           = similar(x)

foo(x)           = [log(a) for a in x]
foo_parallel1(x) = tcollect(log(a) for a in x)
foo_parallel2(x) = tcollect(eltype(x), log(a) for a in x)
```

```
julia> @btime foo($x)
  3.552 ms (3 allocations: 7.629 MiB)
julia> @btime foo_parallel1($x)
  1.549 ms (251 allocations: 27.197 MiB)
julia> @btime foo_parallel2($x)
  335.952 µs (169 allocations: 7.643 MiB)
```

REDUCTIONS AND MAP-REDUCTIONS

`OhMyThreads` additionally provides multithreaded counterparts to `reduce` and `mapreduce`, respectively referred to as `treduce` and `tmapreduce`. These functions automatically handle the inherent race conditions arising in reductions. They also address performance issues such as **false sharing**. Unlike `tmap`, these functions are capable of achieving optimal performance without the need to explicitly specify an output type.

MULTITHREADED MAP

```
x              = rand(1_000_000)

foo(x)         =  reduce(+, x)
foo_parallel(x) = treduce(+, x)
```

```
julia> @btime foo($x)
  97.204 µs (0 allocations: 0 bytes)
julia> @btime foo_parallel($x)
  22.221 µs (158 allocations: 13.891 KiB)
```

MULTITHREADED ARRAY COMPREHENSION

```
x               = rand(1_000_000)

foo(x)          =  mapreduce(log, +, x)
foo_parallel(x) = tmapreduce(log, +, x)
```

```
julia> @btime foo($x)
  3.464 ms (0 allocations: 0 bytes)
julia> @btime foo_parallel($x)
  361.760 μs (158 allocations: 13.891 KiB)
```

FOREACH AS A FASTER OPTION FOR MAPPINGS

`OhMyThreads` also offers a multithreaded version of `foreach` called `tforeach`. Since we haven't covered the single-threaded version `foreach`, we begin by presenting it. The function follows a syntax identical to `map`, and is usually implemented using a do-block syntax.

FOR-LOOP

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.521 ms (3 allocations: 7.629 MiB)
```

FOREACH (ANONYMOUS FUNCTION)

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    foreach(i -> output[i] = log(x[i]), eachindex(x))



    return output
end
```

```
julia> @btime foo($x)
  3.374 ms (3 allocations: 7.629 MiB)
```

FOREACH (DO-BLOCK SYNTAX)

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    foreach(eachindex(x)) do i
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.374 ms (3 allocations: 7.629 MiB)
```

Despite the similarities between `tforeach` and `tmap`, the former is more performant. Furthermore, it doesn't incur a performance penalty when the output type isn't specified.

FOR-LOOP (SINGLE-THREADED)

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.442 ms (3 allocations: 7.629 MiB)
```

TMAP

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    tmap(eachindex(x)) do i
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  1.955 ms (260 allocations: 34.827 MiB)
```

TMAP (WITH OUTPUT'S TYPE)

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    tmap(eltype(x), eachindex(x)) do i
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  575.555 μs (179 allocations: 15.273 MiB)
```

TFOREACH

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    tforeach(eachindex(x)) do i
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  339.212 μs (164 allocations: 7.643 MiB)
```

Similar to `tmap`, the `tforeach` function offers the keyword arguments `nchunks` and `chunksize`, which control the workload distribution among worker threads. To illustrate, we use a work distribution analogous to the one used above for `tmap`.

NUMBER OF CHUNKS

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    tforeach(eachindex(x); nchunks = nthreads()) do i
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  339.700 μs (164 allocations: 7.643 MiB)
```

SIZE OF CHUNKS

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    tforeach(eachindex(x); chunksize = length(x) ÷ nthreads()) do i
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  341.067 μs (172 allocations: 7.643 MiB)
```

PARALLEL FOR-LOOPS

`OhMyThreads` also offers the possibility of directly parallelizing for-loops. This is done by prefixing for-loops with the `@tasks` keyword. The parallelization process can be refined by combining `@tasks` with the `@set` macro. This macro must appear as the first statement inside the for-loop body, giving you control over how work is partitioned. In particular, we can choose how many tasks to spawn using the `nchunks` option (or equivalently `ntasks`). Alternatively, we can specify the size of each chunk through the option `chunksize`, letting the number of tasks to be spawned be implicitly determined.

@TASKS

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @tasks for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  339.714 μs (164 allocations: 7.643 MiB)
```

WITH NUMBER OF CHUNKS

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @tasks for i in eachindex(x)
        @set nchunks = nthreads()                    # number of tasks spawned equal to the
number of threads
            output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  339.809 μs (164 allocations: 7.643 MiB)
```

WITH SIZE OF CHUNKS

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @tasks for i in eachindex(x)
        @set chunksize = length(x) ÷ nthreads()      # size that evenly distributes work
            output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  339.564 μs (172 allocations: 7.643 MiB)
```

The `@set` macro also enables parallel reductions. This is achieved by the `reducer` option, which requires specifying the binary operator used for the reduction (e.g., `reducer = +` or `reducer = max`). Importantly, reductions in this package don't follow the usual pattern of updating an accumulator with operators like `+=`. Instead, `@tasks` assumes that the last expression of the for-loop is the value to be reduced.

SINGLE-THREAD REDUCTION

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.417 ms (0 allocations: 0 bytes)
```

PARALLEL REDUCTION

```
x = rand(1_000_000)

function foo(x)
    output = @tasks for i in eachindex(x)
                @set reducer = +
                    log(x[i])
            end


    return output
end
```

```
julia> @btime foo($x)
  350.873 µs (162 allocations: 13.609 KiB)
```

POLYESTER: PARALLELIZATION FOR SMALL PROGRAMS

> ⚠ **Warning!**
>
> All the scripts below assume you've executed `using Polyester` to load the package

One key limitation of multithreading is the overhead introduced by the creation and scheduling of tasks. For smaller computational workloads, this overhead can outweigh any potential performance gain, rendering parallelization detrimental. As a result, multithreading is typically reserved for objects large enough to justify the cost.

The `Polyester` package addresses this limitation by providing a low-overhead implementation of for-loops. Its approach makes it possible to parallelize operations on objects that, otherwise, would be deemed too small to benefit from multithreading. To use `Polyester`, we simply prefix the for-loop with the `@batch` macro.

To illustrate its benefits, let's consider a for-loop with 500 iterations, a relatively low number for applying multithreading. The first tab below shows that an approach based on `@threads` is slower than its single-threaded variant. In contrast, `Polyester` achieves a comparable performance to the single-threaded variant, even with such a small workload.

FOR-LOOP (SINGLE-THREADED)

```
x = rand(500)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  1.658 μs (3 allocations: 4.008 KiB)
```

@THREADS

```
x = rand(500)

function foo(x)
    output = similar(x)

    @threads for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  9.819 μs (125 allocations: 16.633 KiB)
```

@BATCH

```
x = rand(500)

function foo(x)
    output = similar(x)

    @batch for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  1.134 μs (3 allocations: 4.008 KiB)
```

For larger workloads, it's worth noting that `Polyester` may not consistently outperform or underperform other multithreading approaches. Ultimately, performance will depend on the specifics of the computation and data involved. In such cases, you must benchmark the different implementations.

REDUCTIONS

`Polyester` also supports reduction operations. These can be implemented by prepending the for-loop with the expression `@batch reduce=(<tuple containing operation and variable reduced>)`. The implementation has been designed to avoid common pitfalls of reductions, such as data races and false sharing. This ensures both correctness and performance.

SINGLE THREADED

```
x = rand(250)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  766.042 ns (0 allocations: 0 bytes)
```

@BATCH

```
x = rand(250)

function foo(x)
    output = 0.0

    @batch reduction=( (+, output) ) for i in eachindex(x)
        output += log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  582.652 ns (0 allocations: 0 bytes)
```

It's possible to incorporate more than one reduction operation per iteration, as demonstrated below.

SINGLE THREADED

```
x = rand(250)

function foo(x)
    output1 = 1.0
    output2 = 0.0

    for i in eachindex(x)
        output1 *= log(x[i])
        output2 += exp(x[i])
    end

    return output1, output2
end
```

```
julia> @btime foo($x)
  1.374 µs (0 allocations: 0 bytes)
```

@BATCH

```
x = rand(250)

function foo(x)
    output1 = 1.0
    output2 = 0.0

    @batch reduction=( (*, output1), (+, output2) ) for i in eachindex(x)
        output1 *= log(x[i])
        output2 += exp(x[i])
    end

    return output1, output2
end
```

```
julia> @btime foo($x)
  697.073 ns (0 allocations: 0 bytes)
```

@BATCH (EQUIVALENT)

```
x = rand(250)

function foo(x)
    output1 = 1.0
    output2 = 0.0

    @batch reduction=( (*, output1), (+, output2) ) for i in eachindex(x)
        output1  = output1 * log(x[i])
        output2  = output2 + exp(x[i])
    end

    return output1, output2
end
```

```
julia> @btime foo($x)
  700.439 ns (0 allocations: 0 bytes)
```

LOCAL VARIABLES

Unlike `@threads`, the macros from `Polyester` treat variables as local per iteration.

SINGLE THREADED

```
function foo()
    out  = zeros(Int, 2)
    temp = 0

    for i in 1:2
        temp   = i; sleep(i)
        out[i] = temp
    end

    return out
end
```

```
julia> foo()
2-element Vector{Int64}:
 1
 2
```

@THREADS (OK)

```
function foo()
    out  = zeros(Int, 2)


    @threads for i in 1:2
        temp   = i; sleep(i)
        out[i] = temp
    end

    return out
end
```

```
julia> foo()
2-element Vector{Int64}:
 1
 2
```

@THREADS (DATA RACE)

```
function foo()
    out  = zeros(Int, 2)
    temp = 0

    @threads for i in 1:2
        temp   = i; sleep(i)
        out[i] = temp
    end

    return out
end
```

```
julia> foo()
2-element Vector{Int64}:
 1
 1
```

@BATCH (OK)

```
function foo()
    out  = zeros(Int, 2)
    temp = 0

    @batch for i in 1:2
        temp   = i; sleep(i)
        out[i] = temp
    end

    return out
end
```

```
julia> foo()
2-element Vector{Int64}:
 1
 2
```

SIMD + MULTITHREADING

> ⚠ **Warning!**
>
> All the scripts below assume you've executed `using LoopVectorization` to load the package

We've already covered the package `LoopVectorization` in the context of SIMD instructions. We now revisit this package to demonstrate its ability to combine SIMD with multithreading. The parallelization is achieved through its integration with `Polyester`.

The primary way to simultaneously implement SIMD and multithreading is via the `@tturbo` macro. This provides a parallelized version of `@turbo` in for-loops. Unlike the `@threads` macro, where the application of SIMD optimizations is left to the compiler's discretion, `@tturbo` enforces its application.

To illustrate the benefits of `@tturbo`, let's consider an example where SIMD isn't applied automatically by `@threads`, even when the operation is well-suited for this purpose.

SINGLE THREADED

```
x = BitVector(rand(Bool, 100_000))
y = rand(100_000)

function foo(x,y)
    output = similar(y)

    for i in eachindex(x)
        output[i] = ifelse(x[i], log(y[i]), y[i] * 2)
    end

    output
end
```

```
julia> foo($x,$y)
  608.400 μs (3 allocations: 781.320 KiB)
```

@THREADS

```
x = BitVector(rand(Bool, 100_000))
y = rand(100_000)

function foo(x,y)
    output = similar(y)

    @threads for i in eachindex(x)
        output[i] = ifelse(x[i], log(y[i]), y[i] * 2)
    end

    output
end
```

```
julia> foo($x,$y)
  82.093 μs (125 allocations: 793.945 KiB)
```

@TTURBO

```
x = BitVector(rand(Bool, 100_000))
y = rand(100_000)

function foo(x,y)
    output = similar(y)

    @tturbo for i in eachindex(x)
        output[i] = ifelse(x[i], log(y[i]), y[i] * 2)
    end

    output
end
```

```
julia> foo($x,$y)
  64.067 µs (3 allocations: 781.320 KiB)
```

The `@tturbo` macro also applies to broadcast operations. Offering this functionality is particularly valuable, as no built-in macro currently exists to parallelize broadcast expressions.

While applying `@tturbo` in a for-loop form often yields higher performance, the broadcast variant offers a much simpler and more concise syntax. The example below illustrates the improvement in readability stemming from this approach.

@TTURBO (FOR-LOOP)

```
x      = rand(1_000_000)

function foo(x)
    output = similar(x)

    @tturbo for i in eachindex(x)
        output[i] = log(x[i]) / x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  500.463 µs (3 allocations: 7.629 MiB)
```

@TTURBO (BROADCASTING)

```
x      = rand(1_000_000)

foo(x) = @tturbo log.(x) ./ x
```

```
julia> @btime foo($x)
  499.715 µs (3 allocations: 7.629 MiB)
```

FLOOPS: PARALLEL FOR-LOOPS (*OPTIONAL*)

> ⚠ **Warning!**
> All the scripts below assume you've executed `using FLoops` to load the package

We conclude this section with a brief overview of the package `FLoops`. The presentation is labeled as optional since usage beyond simple applications may require some workarounds. In addition, the package doesn't appear to be actively maintained.

The primary macro offered by the package is `@floop`, exclusively designed for use with for-loops. An example of its application is provided below.

FOR-LOOP (SINGLE-THREADED)

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.450 ms (3 allocations: 7.629 MiB)
```

@FLOOP

```
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @floop for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  439.872 µs (158 allocations: 7.645 MiB)
```

`@floop` can also be used for reductions. This requires placing `@reduce` at the beginning of the line containing the reduction operation. The macro addresses the inherent **data race** associated with parallelization and additionally prevents false sharing.

FOR-LOOP (SINGLE-THREADED)

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.414 ms (0 allocations: 0 bytes)
```

FALSE SHARING

```
x = rand(1_000_000)

function foo(x)
    chunk_ranges    = index_chunks(x, n=nthreads())
    partial_outputs = zeros(length(chunk_ranges))

    @threads for (i,chunk) in enumerate(chunk_ranges)
        for j in chunk
            partial_outputs[i] += log(x[j])
        end
    end

    return sum(partial_outputs)
end
```

```
julia> @btime foo($x)
  1.468 ms (124 allocations: 13.250 KiB)
```

@FLOOP

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @floop for i in eachindex(x)
        @reduce output += log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  416.137 µs (252 allocations: 17.516 KiB)
```

FOOTNOTES

[1] For example, `5 ÷ 3` would return `1`.